

Master Machine Learning with scikit-learn

A Practical Guide to Building Better Models with Python

Kevin Markham

Copyright © 2026 by Kevin Markham. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except as permitted by U.S. copyright law.

For permission requests, please contact kevin@dataschool.io.

Chapter list

About this book	9
1 Introduction	12
2 Review of the Machine Learning workflow	20
3 Encoding categorical features	38
4 Improving your workflow with ColumnTransformer and Pipeline	58
5 Workflow review #1	79
6 Encoding text data	85
7 Handling missing values	98
8 Fixing common workflow problems	115
9 Workflow review #2	130
10 Evaluating and tuning a Pipeline	137
11 Comparing linear and non-linear models	170
12 Ensembling multiple models	183
13 Feature selection	194
14 Feature standardization	212
15 Feature engineering with custom transformers	219
16 Workflow review #3	244
17 High-cardinality categorical features	248
18 Class imbalance	260
19 Class imbalance walkthrough	277
20 Going further	291

Table of contents

About this book	9
About the author	9
Prerequisite skills	9
Book outline	10
Topics covered	10
Functions and classes covered	11
1 Introduction	12
1.1 Book overview	12
1.2 scikit-learn vs Deep Learning	13
1.3 Prerequisite skills	14
1.4 Setup and software versions	14
1.5 Book outline	16
1.6 Datasets	18
1.7 Meet the author	19
2 Review of the Machine Learning workflow	20
2.1 Loading and exploring a dataset	20
2.2 Building and evaluating a model	23
2.3 Using the model to make predictions	24
2.4 Q&A: How do I adapt this workflow to a regression problem?	27
2.5 Q&A: How do I adapt this workflow to a multiclass problem?	27
2.6 Q&A: Why should I select a Series for the target?	28
2.7 Q&A: How do I add the model's predictions to a DataFrame?	30
2.8 Q&A: How do I determine the confidence level of each prediction?	31
2.9 Q&A: How do I check the accuracy of the model's predictions?	32
2.10 Q&A: What do the "solver" and "random_state" parameters do?	33
2.11 Q&A: How do I show all of the model parameters?	34
2.12 Q&A: Should I shuffle the samples when using cross-validation?	35
3 Encoding categorical features	38
3.1 Introduction to one-hot encoding	38
3.2 Transformer methods: fit, transform, fit_transform	42
3.3 One-hot encoding of multiple features	42
3.4 Q&A: When should I use transform instead of fit_transform?	44
3.5 Q&A: What happens if the testing data includes a new category?	46
3.6 Q&A: Should I drop one of the one-hot encoded categories?	49
3.7 Q&A: How do I encode an ordinal feature?	51
3.8 Q&A: What's the difference between OrdinalEncoder and LabelEncoder?	54
3.9 Q&A: Should I encode numeric features as ordinal features?	55

4	Improving your workflow with ColumnTransformer and Pipeline	58
4.1	Preprocessing features with ColumnTransformer	58
4.2	Chaining steps with Pipeline	62
4.3	Using the Pipeline to make predictions	64
4.4	Q&A: How do I drop some columns and passthrough others?	66
4.5	Q&A: How do I transform the unspecified columns?	67
4.6	Q&A: How do I select columns from a NumPy array?	68
4.7	Q&A: How do I select columns by data type?	70
4.8	Q&A: How do I select columns by column name pattern?	72
4.9	Q&A: Should I use ColumnTransformer or make_column_transformer?	73
4.10	Q&A: Should I use Pipeline or make_pipeline?	74
4.11	Q&A: How do I examine the steps of a Pipeline?	76
5	Workflow review #1	79
5.1	Recap of our workflow	79
5.2	Comparing ColumnTransformer and Pipeline	80
5.3	Creating a Pipeline diagram	82
6	Encoding text data	85
6.1	Vectorizing text	85
6.2	Including text data in the model	88
6.3	Q&A: Why is the document-term matrix stored as a sparse matrix?	91
6.4	Q&A: What happens if the testing data includes new words?	92
6.5	Q&A: How do I vectorize multiple columns of text?	93
6.6	Q&A: Should I one-hot encode or vectorize categorical features?	95
7	Handling missing values	98
7.1	Introduction to missing values	98
7.2	Three ways to handle missing values	100
7.3	Missing value imputation	102
7.4	Using “missingness” as a feature	105
7.5	Q&A: How do I perform multivariate imputation?	106
7.6	Q&A: What are the best practices for missing value imputation?	109
7.7	Q&A: What’s the difference between ColumnTransformer and FeatureUnion?	111
8	Fixing common workflow problems	115
8.1	Two new problems	115
8.2	Problem 1: Missing values in a categorical feature	116
8.3	Problem 2: Missing values in the new data	119
8.4	Q&A: How do I see the feature names output by the ColumnTransformer?	121
8.5	Q&A: Why did we create a Pipeline inside of the ColumnTransformer?	123
8.6	Q&A: Which imputation strategy should I use with categorical features?	125
8.7	Q&A: Should I impute missing values before all other transformations?	126
8.8	Q&A: What methods can I use with a Pipeline?	128
9	Workflow review #2	130
9.1	Recap of our workflow	130
9.2	Comparing ColumnTransformer and Pipeline	132

9.3	Why not use pandas for transformations?	134
9.4	Preventing data leakage	135
10	Evaluating and tuning a Pipeline	137
10.1	Evaluating a Pipeline with cross-validation	137
10.2	Tuning a Pipeline with grid search	139
10.3	Tuning the model	140
10.4	Tuning the transformers	143
10.5	Using the best Pipeline to make predictions	150
10.6	Q&A: How do I save the best Pipeline for future use?	152
10.7	Q&A: How do I speed up a grid search?	154
10.8	Q&A: How do I tune a Pipeline with randomized search?	156
10.9	Q&A: What's the target accuracy we are trying to achieve?	160
10.10	Q&A: Is it okay that our model includes thousands of features?	161
10.11	Q&A: How do I examine the coefficients of a Pipeline?	164
10.12	Q&A: Should I split the dataset before tuning the Pipeline?	165
10.13	Q&A: What is regularization?	168
11	Comparing linear and non-linear models	170
11.1	Trying a random forest model	170
11.2	Tuning random forests with randomized search	171
11.3	Further tuning with grid search	175
11.4	Q&A: How do I tune two models with a single grid search?	178
11.5	Q&A: How do I tune two models with a single randomized search?	181
12	Ensembling multiple models	183
12.1	Introduction to ensembling	183
12.2	Ensembling logistic regression and random forests	183
12.3	Combining predicted probabilities	185
12.4	Combining class predictions	187
12.5	Choosing a voting strategy	188
12.6	Tuning an ensemble with grid search	188
12.7	Q&A: When should I use ensembling?	191
12.8	Q&A: How do I apply different weights to the models in an ensemble?	192
13	Feature selection	194
13.1	Introduction to feature selection	194
13.2	Intrinsic methods: L1 regularization	195
13.3	Filter methods: Statistical test-based scoring	197
13.4	Filter methods: Model-based scoring	200
13.5	Filter methods: Summary	205
13.6	Wrapper methods: Recursive feature elimination	206
13.7	Q&A: How do I see which features were selected?	209
13.8	Q&A: Are the selected features the “most important” features?	210
13.9	Q&A: Is it okay for feature selection to remove one-hot encoded categories?	211
14	Feature standardization	212
14.1	Standardizing numerical features	212

14.2	Standardizing all features	214
14.3	Q&A: How do I see what scaling was applied to each feature?	215
14.4	Q&A: How do I turn off feature standardization within a grid search?	216
14.5	Q&A: Which models benefit from standardization?	217
15	Feature engineering with custom transformers	219
15.1	Why not use pandas for feature engineering?	219
15.2	Transformer 1: Rounding numerical values	219
15.3	Transformer 2: Clipping numerical values	221
15.4	Transformer 3: Extracting string values	223
15.5	Rules for transformer functions	226
15.6	Transformer 4: Combining two features	226
15.7	Revising the transformers	229
15.8	Q&A: How do I fix incorrect data types within a Pipeline?	233
15.9	Q&A: How do I create features from datetime data?	236
15.10	Q&A: How do I create feature interactions?	238
15.11	Q&A: How do I save a Pipeline with custom transformers?	240
15.12	Q&A: Can FunctionTransformer be used with any transformation?	243
16	Workflow review #3	244
16.1	Recap of our workflow	244
16.2	What's the role of pandas?	246
17	High-cardinality categorical features	248
17.1	Recap of nominal and ordinal features	248
17.2	Preparing the census dataset	249
17.3	Setting up the encoders	250
17.4	Encoding nominal features for a linear model	254
17.5	Encoding nominal features for a non-linear model	255
17.6	Combining the encodings	256
17.7	Best practices for encoding	258
18	Class imbalance	260
18.1	Introduction to class imbalance	260
18.2	Preparing the mammography dataset	261
18.3	Evaluating a model with train/test split	262
18.4	Exploring the results with a confusion matrix	264
18.5	Calculating rates from a confusion matrix	265
18.6	Using AUC as the evaluation metric	267
18.7	Cost-sensitive learning	271
18.8	Tuning the decision threshold	273
19	Class imbalance walkthrough	277
19.1	Best practices for class imbalance	277
19.2	Step 1: Splitting the dataset	278
19.3	Step 2: Optimizing the model on the training set	278
19.4	Step 3: Evaluating the model on the testing set	280
19.5	Step 4: Tuning the decision threshold	281

19.6	Step 5: Retraining the model and making predictions	282
19.7	Q&A: Should I use an ROC curve or a precision-recall curve?	283
19.8	Q&A: Can I use a different metric such as F1 score?	288
19.9	Q&A: Should I use resampling to fix class imbalance?	289
20	Going further	291
20.1	Q&A: How do I read the scikit-learn documentation?	291
20.2	Q&A: How do I stay up-to-date with new scikit-learn features?	292
20.3	Q&A: How do I improve my Machine Learning skills?	293
20.4	Q&A: How do I learn Deep Learning?	294

About this book

This is a practical guide to help you transform from **Machine Learning novice** to **skilled Machine Learning practitioner**.

Throughout the book, you'll learn the best practices for proper Machine Learning and how to apply those practices to your own Machine Learning problems.

By the end of this book, you'll be more confident when tackling new Machine Learning problems because you'll understand what steps you need to take, why you need to take them, and how to correctly execute those steps using scikit-learn.

You'll know what problems you might run into, and you'll know exactly how to solve them.

Because you're learning a better way to work in scikit-learn, your code will be easier to write and to read, and you'll get better Machine Learning results faster than before!

About the author

Hi! My name is Kevin Markham. I'm the founder of [Data School](#), an online school for learning Data Science with Python.

I've been teaching Machine Learning in the classroom and online for more than 10 years, and I'm passionate about teaching people who are new to the field, regardless of their educational and professional backgrounds.

This book was adapted from my [7.5-hour video course](#) of the same name, and is a compilation of nearly everything I know about effective Machine Learning with scikit-learn!

Personally, I have a degree in Computer Engineering from Vanderbilt University and I live in Asheville, North Carolina.

Prerequisite skills

This is an intermediate-level book about scikit-learn, though it also includes many advanced topics.

You're ready for this book if you can use scikit-learn to solve simple classification or regression problems, including loading a dataset, defining the features and target, training and evaluating a model, and making predictions with new data.

If you're brand new to scikit-learn, I recommend first taking my free introductory course, [Introduction to Machine Learning with scikit-learn](#). Once you've completed lessons 1 through 7, you'll know the Machine Learning and scikit-learn fundamentals that you'll need for this book.

If you've used scikit-learn before but you just need a refresher, there's no need to take my introductory course because I'll be reviewing the Machine Learning workflow in chapter 2.

Book outline

In **chapter 1**, I'll give you an overview of the book and help you to get set up. Then in **chapter 2**, we'll move on to a review of the Machine Learning workflow in order to establish a foundation for the rest of the book.

In **chapters 3 through 9**, we'll explore how to handle the preprocessing of categorical features, text data, and missing values, and also how to integrate these steps into an efficient workflow. Then in **chapter 10**, we'll cover how to properly evaluate and tune your entire workflow for maximum performance.

In **chapters 11 through 16**, we'll walk through a variety of advanced techniques that can help to further improve your model's performance, including ensembling, feature selection, feature standardization, and feature engineering. In **chapters 17 through 19**, we'll dive deep into two common issues you'll run into during real-world Machine Learning, namely high-cardinality categorical features and class imbalance.

Finally, in **chapter 20**, I'll end the book with my advice for how you can continue to make progress with your Machine Learning education and skill development!

Topics covered

- Review of the basic Machine Learning workflow
- Encoding categorical features
- Encoding text data
- Handling missing values
- Preparing complex datasets
- Creating an efficient workflow for preprocessing and model building
- Tuning your workflow for maximum performance
- Avoiding data leakage
- Proper model evaluation
- Automatic feature selection
- Feature standardization
- Feature engineering using custom transformers
- Linear and non-linear models
- Model ensembling
- Model persistence
- Handling high-cardinality categorical features
- Handling class imbalance

Functions and classes covered

- **Workflow composition:** Pipeline, ColumnTransformer, make_pipeline, make_column_transformer, make_column_selector, make_union
- **Categorical encoding:** OneHotEncoder, OrdinalEncoder
- **Numerical encoding:** KBinsDiscretizer
- **Text encoding:** CountVectorizer
- **Missing value imputation:** SimpleImputer, KNNImputer, IterativeImputer, MissingIndicator
- **Model building:** LogisticRegression, RandomForestClassifier, ExtraTreesClassifier
- **Model ensembling:** VotingClassifier
- **Model selection:** StratifiedKFold, cross_val_score, train_test_split
- **Model evaluation:** accuracy_score, classification_report, confusion_matrix, roc_auc_score, average_precision_score, plot_confusion_matrix, plot_roc_curve, plot_precision_recall_curve
- **Hyperparameter tuning:** GridSearchCV, RandomizedSearchCV
- **Feature selection:** RFE, SelectPercentile, SelectFromModel, chi2
- **Feature standardization:** StandardScaler, MaxAbsScaler
- **Feature engineering:** FunctionTransformer, PolynomialFeatures
- **Configuration:** set_config
- **Model persistence:** joblib, pickle, cloudpickle (these are external libraries)

1 Introduction

1.1 Book overview

Hello and welcome to *Master Machine Learning with scikit-learn*. In this book, you're going to learn how to build an effective Machine Learning workflow using the latest scikit-learn techniques so that you can solve almost any supervised Machine Learning problem.

If you have just a bit of experience with scikit-learn, you've probably spent most of your time building models using artificially clean training data. But in this book, you'll learn how to build models using more complex datasets. We'll cover:

- How to handle common scenarios such as missing values, text data, categorical data, and class imbalance
- How to build a reusable workflow that starts with a pandas DataFrame and ends with a trained scikit-learn model
- How to integrate feature engineering, selection, and standardization into your workflow
- How to avoid data leakage so that you can correctly estimate model performance
- How to tune your entire workflow for maximum performance

...and so much more!

High-level topics:

- Handling missing values, text data, categorical data, and class imbalance
- Building a reusable workflow
- Feature engineering, selection, and standardization
- Avoiding data leakage
- Tuning your entire workflow

Throughout the book, you'll learn the best practices for proper Machine Learning and how to apply those practices to your own Machine Learning problems.

By the end of this book, you'll be more confident when tackling new Machine Learning problems because you'll understand what steps you need to take, why you need to take them, and how to correctly execute those steps using scikit-learn.

You'll know what problems you might run into, and you'll know exactly how to solve them.

Because you're learning a better way to work in scikit-learn, your code will be easier to write and to read, and you'll get better Machine Learning results faster than before!

How you will benefit from this book:

- Knowledge of best practices
- Confidence when tackling new Machine Learning problems
- Ability to anticipate and solve problems
- Improved code quality
- Better, faster results

1.2 scikit-learn vs Deep Learning

If you want to solve any Machine Learning problem using Python, then I always recommend starting with scikit-learn. Here's why:

- It provides a consistent interface to a huge number of Machine Learning models.
- It offers many options and tuning parameters but uses sensible defaults.
- It includes a rich set of functionality to support the entire Machine Learning workflow.
- It has exceptional documentation.
- There is an active community of researchers and developers who continue to improve and support the library.

Benefits of scikit-learn:

- Consistent interface to many models
- Many tuning parameters (but sensible defaults)
- Workflow-related functionality
- Exceptional documentation
- Active community support

In fact, scikit-learn was the most popular Machine Learning tool in Kaggle's most recent "State of Machine Learning" report, with more than 80% of Data Scientists using it.

For many Machine Learning problems, scikit-learn will be the only library you need. However, there are some specialized problems for which a Deep Learning library such as TensorFlow, PyTorch, or Keras will provide superior results. That being said, Deep Learning does have some significant drawbacks:

- Deep Learning requires more computational resources.
- Deep Learning libraries have a higher learning curve.
- Deep Learning models are less interpretable than other models.

Drawbacks of Deep Learning:

- More computational resources
- Higher learning curve
- Less interpretable models

In other words, I only recommend using Deep Learning if you already know that you need it to solve your particular problem. But for the majority of Machine Learning problems, you are likely to get similar results using scikit-learn, and you will get those results much faster and more easily with scikit-learn.

1.3 Prerequisite skills

This is an intermediate-level book about scikit-learn, though it also includes many advanced topics.

You're ready for this book if you can use scikit-learn to solve simple classification or regression problems, including loading a dataset, defining the features and target, training and evaluating a model, and making predictions with new data.

scikit-learn prerequisites:

- Loading a dataset
- Defining the features and target
- Training and evaluating a model
- Making predictions with new data

If you're brand new to scikit-learn, I recommend first taking my free introductory course, [Introduction to Machine Learning with scikit-learn](#). Once you've completed lessons 1 through 7, you'll know the Machine Learning and scikit-learn fundamentals that you'll need for this book.

New to scikit-learn?

- Enroll in *Introduction to Machine Learning with scikit-learn* (free course)
- Complete lessons 1 through 7

If you've used scikit-learn before but you just need a refresher, there's no need to take my introductory course because I'll be reviewing the Machine Learning workflow in the next chapter.

Finally, I should note that we will perform a few basic pandas operations in the book, including reading a CSV file and selecting columns from a DataFrame. However, it's not a problem if you're new to pandas because I will explain that code as we go.

1.4 Setup and software versions

To follow along with this book, you'll need to have access to scikit-learn and pandas, both of which are open source Python libraries. The easiest way to install scikit-learn and pandas (and their dependencies) is to download the [Anaconda distribution](#) of Python.

For other installation options, visit the [scikit-learn](#) and [pandas](#) websites.

How to install scikit-learn and pandas:

- **Option 1:** Install together
 - Anaconda distribution
- **Option 2:** Install separately
 - scikit-learn
 - pandas

Because we'll be using some of the latest scikit-learn functionality, it's important that you are running a modern version of scikit-learn. To check your version, just open your code editor and run this code.

```
import sklearn
sklearn.__version__
```

'0.23.2'

Note that this book uses version 0.23.2. To follow along with the book, it's important that you are using at least version 0.20.2. There are some scikit-learn features that I'll use in the book that were released in 0.21 or 0.22 or 0.23, but none of those features are essential to the core content of the book. Throughout the book, I'll specifically mention any features that require scikit-learn 0.21 or above.

scikit-learn version:

- **Book version:** 0.23.2
- **Minimum version:** 0.20.2

You are also welcome to use a later version of scikit-learn, such as 0.24, 1.0, or beyond. Throughout the book, I'll mention any changes to scikit-learn in those versions that are relevant to the book.

Regarding the pandas library, this book uses version 1.2.4, but any version should work just fine.

```
import pandas
pandas.__version__
```

'1.2.4'

One final option for following along with the book, especially if you aren't able to install Python packages on your local machine, is to use [Google Colab](#). Colab provides you with an interface similar to the Jupyter Notebook. It's free and runs entirely in your browser, though it does require that you have a Google account.

Using Google Colab with the book:

- Similar to the Jupyter Notebook
- Runs in your browser
- Free (but requires a Google account)

1.5 Book outline

Let's talk about what we're going to cover in each chapter.

- In chapter 1, which is this chapter, we're getting you ready for the book.
- In chapter 2, we'll walk through the basic Machine Learning workflow, from loading a dataset to building a model to making predictions.
- In chapter 3, we'll focus on one of the most important data preprocessing steps, namely the encoding of categorical features.
- In chapter 4, we'll see how to use `ColumnTransformer` and `Pipeline` to make our workflow more powerful and efficient.
- In chapter 5, we'll review the workflow we've built so far to make sure you understand the key concepts before we start adding additional complexity.
- In chapter 6, we'll learn how to create features from unstructured text data.
- In chapter 7, we'll discuss missing values and explore a few different ways to handle them.
- In chapter 8, we'll see what problems arise when we expand the size of our dataset, and then we'll figure out how to handle those problems.
- In chapter 9, we'll review our workflow again and discuss how it's helping us to prevent data leakage.
- In chapter 10, we'll take a deep dive into how to efficiently tune our `Pipeline` for maximum performance.
- In chapter 11, we'll try out a non-linear model called "random forests" and figure out how to tune it without overextending our computing resources.
- In chapter 12, we'll learn how to ensemble our different models two different ways and how to tune the ensemble for even better performance.
- In chapter 13, we'll discuss the benefits of feature selection and then try out a handful of automated methods for selecting features.
- In chapter 14, we'll experiment with standardizing our features to see if that improves our model performance.
- In chapter 15, we'll create a variety of new features within our `Pipeline` and discuss why you might want to do all of your feature engineering using `scikit-learn` rather than `pandas`.
- In chapter 16, we'll do one final review of the workflow that we created throughout the book.
- In chapter 17, we'll experiment with different ways of handling categorical features with lots of unique values.
- In chapter 18, we'll thoroughly explore the problem of class imbalance and the processes you can use to work around it.
- In chapter 19, we'll walk through my complete workflow for handling class imbalance so that you can see a demonstration of the best practices.
- Finally, in chapter 20, we'll discuss how you can keep learning and improving your skills on your own.

Chapters:

1. Introduction
2. Review of the Machine Learning workflow
3. Encoding categorical features
4. Improving your workflow with ColumnTransformer and Pipeline
5. Workflow review #1
6. Encoding text data
7. Handling missing values
8. Fixing common workflow problems
9. Workflow review #2
10. Evaluating and tuning a Pipeline
11. Comparing linear and non-linear models
12. Ensembling multiple models
13. Feature selection
14. Feature standardization
15. Feature engineering with custom transformers
16. Workflow review #3
17. High-cardinality categorical features
18. Class imbalance
19. Class imbalance walkthrough
20. Going further

I recommend reading the chapters in order, because each chapter builds on the material from previous chapters.

Also, you might have noticed that most of the chapters end with a series of lessons marked “Q&A”. These lessons answer common questions that may have come up in your mind while reading the rest of the lessons in that chapter. Thus, the Q&A lessons should help you to understand the core material from each chapter in greater depth.

Lesson types:

- Core lessons
- Q&A lessons

Finally, I wanted to mention that this book won’t be focusing on high-level algorithm selection, such as whether you should use a logistic regression model or a random forests model for your particular problem. That’s because I’ve found that the workflow is more important and will have a greater impact on your overall Machine Learning results than your ability to pick between algorithms.

In fact, once you’ve mastered the workflow, you can iterate through different algorithms quickly even if you don’t deeply understand them. Even if you did understand all of the algorithms, it’s hard to know in advance which one will work best for a given problem, which is why it’s so important to build a reusable workflow that enables you to switch between algorithms easily.

The bottom line is that understanding the algorithms is still useful, but the workflow is even more important, and so that’s the focus of this book.

Why not focus on algorithms?

- Workflow will have a greater impact on your results
- Reusable workflow enables you to try many different algorithms
- Hard to know (in advance) which algorithm will work best

1.6 Datasets

We'll be using three different datasets in the book:

- The first is the famous Titanic dataset.
- The second is a dataset of US census data.
- The third is a dataset of mammography scans used to detect the presence of cancer.

Datasets:

- Titanic
- US census
- Mammography scans

I chose these datasets for three main reasons:

- First, I wanted to use small-to-medium-sized datasets that could be read via URL so that you aren't required to download any files for this book, which is more convenient for everyone.
- Second, using smaller datasets means that computationally intense operations (such as grid search) won't take many hours to run. This will save you a lot of time, especially if you're working through the book without the latest and greatest hardware.
- Third, I believe you'll actually understand the lessons better because we're not using huge datasets. From years of teaching experience, I've found that you will never master complex processes until you understand the simple building blocks in great detail.

Why use smaller datasets?

- Easier and faster access to files
- Reduced computational time
- Greater understanding of the lessons

In this book, we're going to be examining the inputs and outputs from different workflow steps in detail, and that's not practical when you have thousands of features or hundreds of thousands of samples.

In fact, we're going to be spending the next few chapters with just 10 rows of data, which might sound crazy, but is actually the ideal way to truly understand how the different scikit-learn components work together.

The great news is that once you master the workflow you'll learn in this book, you'll be able to handle datasets of almost any size without changing a single step. In other words, the knowledge you gain

here will 100% transfer to far more complex datasets, and yet you will still understand in detail what is actually going on inside your code.

1.7 Meet the author

Finally, before we wrap up this chapter, I just want to introduce myself. My name is Kevin Markham and I'm the author of this book. I'm the founder of [Data School](#), an online school for learning Data Science with Python.

I've been teaching Machine Learning in the classroom and online for more than 10 years, and I'm passionate about teaching people who are new to the field, regardless of their educational and professional backgrounds.

Personally, I have a degree in Computer Engineering from Vanderbilt University and I live in Asheville, North Carolina.

About me:

- Founder of Data School
- Teaching Machine Learning for 10+ years
- Passionate about teaching people who are new to the field
- Degree in Computer Engineering
- Living in Asheville, North Carolina

2 Review of the Machine Learning workflow

2.1 Loading and exploring a dataset

In this chapter, we're going to do a quick review of the basic Machine Learning workflow, from loading and exploring a dataset to building and evaluating a model, and then finally using that model to make predictions.

We'll start by importing the pandas library and using its `read_csv` function to read a dataset directly from a URL.

We specified that only the first 10 rows should be read in. We'll work with the entire dataset later in the book, but by starting with just 10 rows, we can more easily examine the input and output of each step.

```
import pandas as pd
df = pd.read_csv('http://bit.ly/MLtrain', nrows=10)
```

Let's print out these 10 rows.

This is the famous Titanic dataset. There's a lot we can learn by working with this dataset because it contains a diversity of feature types, lots of missing values, and other problems we'll need to solve.

The Titanic ship sank in 1912, and each row in this dataset represents one passenger. Some of the passengers survived, represented by a 1 in the Survived column, whereas others did not, represented by a 0.

df

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	3	Bra...	male	22.0	1	0	A/5...	7.2500	NaN	S
1	1	1	Cum...	female	38.0	1	0	PC ...	71....	C85	C
2	1	3	Hei...	female	26.0	0	0	STO...	7.9250	NaN	S
3	1	1	Fut...	female	35.0	1	0	113803	53....	C123	S
4	0	3	All...	male	35.0	0	0	373450	8.0500	NaN	S
5	0	3	Mor...	male	NaN	0	0	330877	8.4583	NaN	Q
6	0	1	McC...	male	54.0	0	0	17463	51....	E46	S
7	0	3	Pal...	male	2.0	3	1	349909	21....	NaN	S
8	1	3	Joh...	female	27.0	0	2	347742	11....	NaN	S
9	1	2	Nas...	female	14.0	1	0	237736	30....	NaN	C

Let's review some basic terminology:

- The Survived column is the target column, which is what we are trying to predict.

- The target column is categorical, and thus this is a classification problem. More specifically, it's called binary classification because there are only two classes, 0 and 1.
- All other columns are possible features, meaning inputs to the model.
- Each row is known as a sample or observation, and represents a passenger. At the moment, we have 10 samples.
- This is known as our training data because we know the actual target values.

Machine Learning terminology:

- **Target:** Goal of prediction
- **Classification:** Problem with a categorical target
- **Feature:** Input to the model (column)
- **Sample:** Single observation (row)
- **Training data:** Data with known target values

Next, let's select some features. There are many valid methods for feature selection, such as:

- Human intuition, meaning what makes sense as a feature
- Domain knowledge, meaning what you know because of your expertise in that field
- Data exploration, meaning relationships in the data that you discover
- Automated methods, which we'll cover later in the book

Feature selection methods:

- Human intuition
- Domain knowledge
- Data exploration
- Automated methods

To start, we're going to use intuition and start with Parch and Fare as our features:

- Parch is the number of parents or children aboard with that passenger. Our intuition is that families may have gotten priority access to lifeboats, and thus Parch may be predictive of who survived, which is our target.
- Fare is the amount the passenger paid. Our intuition is that people who paid more may have gotten priority access to lifeboats, and again it may be predictive of our target.

As you saw above, both Parch and Fare are numeric features.

Currently selected features:

- **Parch:** Number of parents or children aboard with that passenger
- **Fare:** Amount the passenger paid

Let's now define our X and our y:

- X is the feature matrix, and we'll use double brackets to select those two columns as a DataFrame.
- y is the target, and we'll use single brackets to select this column as a Series.

```
X = df[['Parch', 'Fare']]
X
```

	Parch	Fare
0	0	7.2500
1	0	71.2833
2	0	7.9250
3	0	53.1000
4	0	8.0500
5	0	8.4583
6	0	51.8625
7	1	21.0750
8	2	11.1333
9	0	30.0708

```
y = df['Survived']
y
```

	Survived
0	0
1	1
2	1
3	1
4	0
5	0
6	0
7	0
8	1
9	1

Object shape is important in scikit-learn, and so we'll check the shapes of both X and y:

- X is a pandas DataFrame, thus it has 2 dimensions.
- y is a pandas Series, thus it has 1 dimension.

By convention, you use an uppercase X because it has 2 dimensions, and you use a lowercase y because it has 1 dimension.

```
X.shape
```

```
(10, 2)
```

```
y.shape
```

```
(10,)
```

2.2 Building and evaluating a model

Now that we've defined X and y , our next step is to build and evaluate a model.

To start, we're going to use logistic regression as our model. It's a good default choice for classification problems because it's both fast and interpretable.

We import it from the `linear_model` module, and then we create an instance called `logreg`. This is our model object.

The default solver for logistic regression has changed between different scikit-learn versions, but in this book I'm going to set the solver to `liblinear`. I'm specifying the `solver` explicitly and setting a value for `random_state` so that if you run the same code at home, you will most likely get the same results as me.

```
from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression(solver='liblinear', random_state=1)
```

Let's now talk about model evaluation. The goal of model evaluation is to simulate how a model will perform on future data so that we can choose between models today. To do model evaluation, we need both an evaluation procedure and an evaluation metric.

The procedure we will use is K-fold cross-validation. Another option is to use train/test split, but cross-validation is generally superior because it gives a lower variance estimate of model performance.

The metric we will use is classification accuracy. There are many other classification metrics we could have chosen, but accuracy is suitable in this case for two reasons:

- First, there is not significant class imbalance.
- Second, predicting the positive class correctly is just as important to us as predicting the negative class correctly.

That being said, I will cover other classification metrics in the chapters on class imbalance.

Requirements for model evaluation:

- **Procedure:** K-fold cross-validation
- **Metric:** Classification accuracy

With such a small dataset, we're going to use 3-fold cross-validation, rather than 5 or 10 folds which is more typical. Let me briefly review what happens during 3-fold cross-validation:

- The rows are split into 3 subsets, which we'll call A, B, and C.
- First, A and B together become the training set, and C becomes the testing set. The model is trained on the training set, the trained model makes predictions for the testing set, and those predictions are evaluated.
- Next, A and C together become the training set, and B becomes the testing set. Again, the model is trained, it makes predictions, and the predictions are evaluated.
- Finally, B and C together become the training set, and A becomes the testing set. The training, predicting, and evaluation process happens one final time.

- Because the evaluation process occurred 3 times, it returns 3 scores, and we usually take the mean of those scores.

Steps of 3-fold cross-validation:

1. Split rows into 3 subsets (A, B, C)
2. A & B is training set, C is testing set
 - Train model on training set
 - Make predictions on testing set
 - Evaluate predictions
3. Repeat with A & C as training set, B as testing set
4. Repeat with B & C as training set, A as testing set
5. Calculate the mean of the scores

Let's go ahead and use cross-validation to evaluate our model:

- First we import the `cross_val_score` function from the `model_selection` module.
- Then we pass it the model object, `X` and `y`, the number of cross-validation folds, and the evaluation metric. Although the default metric for classification problems is accuracy, I recommend specifying it explicitly so that there's no ambiguity.
- When we run `cross_val_score`, it does the dataset splitting, training, predicting, and evaluation. Three accuracy scores are returned, and the mean of those scores is 69%.

```
from sklearn.model_selection import cross_val_score
cross_val_score(logreg, X, y, cv=3, scoring='accuracy').mean()
```

0.6944444444444443

If you received a different result, that's not a problem. The results can vary among different scikit-learn versions due to changes in the default parameters, algorithm changes, bug fixes, and so on.

Unfortunately, we can't take these results seriously because the dataset is so small. There's actually no reliable evaluation procedure when your training data only contains 10 rows, but I did want to demonstrate it anyway to emphasize that model evaluation is a normal part of the Machine Learning workflow.

2.3 Using the model to make predictions

At this point in the workflow, we would typically try making changes in order to achieve a higher accuracy, such as:

- Tuning the model's hyperparameters
- Adding or removing features
- Trying a different classification model

Ways to improve the model:

- Hyperparameter tuning
- Adding or removing features
- Trying a different model

We'll cover these topics in detail later in the book, but for now, let's assume that we're happy with the model as-is. Thus, our next steps are to train the model and then use it to make predictions on new data.

We'll use the model's `fit` method, which instructs the model to try to learn the relationship between `X` and `y`.

```
logreg.fit(X, y)
```

```
LogisticRegression(random_state=1, solver='liblinear')
```

There are four important points I want to note here:

- First, you should train your model on the entire dataset before using it to make predictions, otherwise you are throwing away valuable training data. In truth, we do have more than 10 rows, but for now we are considering our entire training dataset to be these 10 rows.
- Second, the model object is modified in-place when you run the `fit` method, and so there's no need to overwrite the `logreg` object using an assignment statement.
- Third, scikit-learn understands how to work with pandas objects, and so we can pass `X` and `y` directly to the `fit` method.
- Finally, if you're using scikit-learn 0.23 or later, you will only see the parameters that have changed from the defaults when you print or fit a model. That's why it only displays the `random_state` and `solver` parameters, whereas in previous versions of scikit-learn, all model parameters would have been displayed.

Important points about model fitting:

- Train your model on the entire dataset before making predictions
- Assignment statement is unnecessary
- Passing pandas objects is fine
- Only prints parameters that have changed (version 0.23 or later)

Now, let's read in a new dataset for which we don't know the target values. Again, we are only going to keep the first 10 rows.

You'll notice that it has the same columns as the `df` DataFrame, except that there's no `Survived` column, which is the column we're going to predict.

```
df_new = pd.read_csv('http://bit.ly/MLnewdata', nrows=10)
df_new
```

	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	3	Kel...	male	34.5	0	0	330911	7.8292	NaN	Q
1	3	Wil...	female	47.0	1	0	363272	7.0000	NaN	S
2	2	Myl...	male	62.0	0	0	240276	9.6875	NaN	Q
3	3	Wir...	male	27.0	0	0	315154	8.6625	NaN	S
4	3	Hir...	female	22.0	1	1	310...	12....	NaN	S
5	3	Sve...	male	14.0	0	0	7538	9.2250	NaN	S
6	3	Con...	female	30.0	0	0	330972	7.6292	NaN	Q
7	2	Cal...	male	26.0	1	1	248738	29....	NaN	S
8	3	Abr...	female	18.0	0	0	2657	7.2292	NaN	C
9	3	Dav...	male	21.0	2	0	A/4...	24....	NaN	S

Before we make predictions, we have to define `X_new`. It has to have the same columns as `X`, and those columns have to be in the same order.

```
X_new = df_new[['Parch', 'Fare']]
X_new
```

	Parch	Fare
0	0	7.8292
1	0	7.0000
2	0	9.6875
3	0	8.6625
4	1	12.2875
5	0	9.2250
6	0	7.6292
7	1	29.0000
8	0	7.2292
9	0	24.1500

Finally, we'll use the trained model to make predictions by passing `X_new` to the `predict` method, which outputs a NumPy array. There are 10 predictions because it makes 1 prediction for each sample in `X_new`.

The predictions are in the same order as the samples in `X_new`, meaning the first prediction is for the first row in `X_new`, the second prediction is for the second row in `X_new`, and so on.

```
logreg.predict(X_new)
```

```
array([0, 0, 0, 0, 1, 0, 0, 1, 0, 1])
```

Note that we can't actually evaluate the accuracy of these predictions because we don't know the true target values for the samples in `X_new`.

2.4 Q&A: How do I adapt this workflow to a regression problem?

In this book, we're going to be focusing on classification problems, which means that the target you're trying to predict is categorical. The other main type of prediction problem is regression, in which your target value is continuous.

If you're planning to work on a regression problem, the good news is that the workflow I'm teaching will work just as well to solve classification or regression problems. There are only two changes you will need to make to adapt this workflow for regression:

- First, you will need to choose a different Machine Learning model. For example, you might choose linear regression instead of logistic regression, since linear regression predicts continuous values whereas logistic regression predicts class values.
- Second, you will need to choose a different model evaluation metric. For example, you might choose mean squared error instead of accuracy, since mean squared error is appropriate for continuous values whereas accuracy is only appropriate for categorical values.

Adapting this workflow for regression:

1. Choose a different model
2. Choose a different evaluation metric

2.5 Q&A: How do I adapt this workflow to a multiclass problem?

In this chapter, we worked on a binary classification problem, which means there are only two possible output classes.

Multiclass problems are ones in which there are more than two output classes. The classic example of this is the iris dataset, in which each iris plant can be classified as one of three possible species.

Thankfully, in scikit-learn, all classifiers automatically handle multiclass problems with no changes to the workflow. It automatically detects the number of classes from the data, thus you don't even have to inform scikit-learn that you're working on a multiclass problem.

Types of classification problems:

- **Binary:** Two output classes
- **Multiclass:** More than two output classes

So how do classifiers actually handle multiclass problems?

- Many classifiers are inherently multiclass, meaning that they work exactly the same regardless of the number of classes.
- For classifiers that only work in the binary case, they can be extended to the multiclass case using the so-called "one-vs-one" or "one-vs-rest" strategies, in which multiple models are fit and the results are combined. You can research more about these strategies if you're interested, but

the key point is that this is handled for you automatically by scikit-learn without you needing to do anything special.

How classifiers handle multiclass problems:

- Many are inherently multiclass
- Others can be extended using “one-vs-one” or “one-vs-rest” strategies

2.6 Q&A: Why should I select a Series for the target?

From our DataFrame, there are two ways you could imagine selecting the target variable of Survived:

- The first way is as a pandas Series, which we can do using a single set of brackets.
- The second way is as a pandas DataFrame with one column, which we can do using two sets of brackets.

```
df['Survived']
```

Survived	
0	0
1	1
2	1
3	1
4	0
5	0
6	0
7	0
8	1
9	1

```
df[['Survived']]
```

Survived	
0	0
1	1
2	1
3	1
4	0
5	0
6	0
7	0
8	1
9	1

These two objects look the same, but they actually have different shapes. The Series is a one-dimensional object, while the DataFrame is a two-dimensional object.

```
df['Survived'].shape
```

```
(10,)
```

```
df[['Survived']].shape
```

```
(10, 1)
```

The difference between these two objects is more clear if you convert them to NumPy arrays using the `to_numpy` method.

```
df['Survived'].to_numpy()
```

```
array([0, 1, 1, 1, 0, 0, 0, 0, 1, 1])
```

```
df[['Survived']].to_numpy()
```

```
array([[0],  
       [1],  
       [1],  
       [1],  
       [0],  
       [0],  
       [0],  
       [0],  
       [1],  
       [1]])
```

Now that you've seen that these two objects are different, the question is: Why does it matter which object you use for the target?

To answer this question, I have to briefly explain multilabel classification. A multilabel classification problem is one in which each sample can simultaneously have more than one label. The classic example of this is classifying the topic of a document. For example, a document might be about politics, religion, or law, or it might fit into multiple topics at once.

This is different from multiclass classification because multiclass only allows a sample to have a single label, whereas multilabel allows a single sample to have multiple labels.

Multilabel vs multiclass problems:

- **Multilabel:** Each sample can have more than one label
- **Multiclass:** Each sample can have one label

Anyway, scikit-learn supports multilabel classification by allowing you to represent the target as a two-dimensional object. For example, if you had 10 documents and there were 3 possible labels, you would actually use a 10 by 3 DataFrame as your y value.

That is all to say that using a two-dimensional DataFrame as your y value signals to scikit-learn that you're working on a multilabel problem. Our classification problem is not multilabel, and thus we use a one-dimensional Series as our y value to signal that we're working on a single-label problem.

Multilabel vs multiclass targets:

- **Multilabel:** 2-dimensional y (DataFrame)
- **Multiclass:** 1-dimensional y (Series)

2.7 Q&A: How do I add the model's predictions to a DataFrame?

Let's say that you wanted to match up the 10 predictions output by the model with the 10 rows of the X_new DataFrame so that you can see the predictions next to the features.

To do this, you would first convert the predictions from a NumPy array to a pandas Series. Note that we are setting the Series index to match the index of X_new, and we're giving the Series a name.

```
predictions = pd.Series(logreg.predict(X_new), index=X_new.index,
                        name='Prediction')
```

Then, you use the concat function to concatenate the X_new DataFrame and the predictions Series along the 'columns' axis, which outputs a DataFrame. Notice that the name of the Series became the name of that DataFrame column.

```
pd.concat([X_new, predictions], axis='columns')
```

	Parch	Fare	Prediction
0	0	7.8292	0
1	0	7.0000	0
2	0	9.6875	0
3	0	8.6625	0
4	1	12.2875	1
5	0	9.2250	0
6	0	7.6292	0
7	1	29.0000	1
8	0	7.2292	0
9	0	24.1500	1

2.8 Q&A: How do I determine the confidence level of each prediction?

For some classification problems, you are only interested in the predicted class labels.

```
logreg.predict(X_new)
```

```
array([0, 0, 0, 0, 1, 0, 0, 1, 0, 1])
```

However, sometimes it's useful to output the predicted probabilities of class membership using the `predict_proba` method.

The output array has 10 rows because the model made predictions for 10 samples, and it has 2 columns because there are 2 possible classes. The left column represents class 0, and the right column represents class 1.

```
logreg.predict_proba(X_new)
```

```
array([[0.57804075, 0.42195925],
       [0.58275546, 0.41724454],
       [0.56742414, 0.43257586],
       [0.57328835, 0.42671165],
       [0.48357081, 0.51642919],
       [0.57007262, 0.42992738],
       [0.57917926, 0.42082074],
       [0.38795132, 0.61204868],
       [0.58145374, 0.41854626],
       [0.48342837, 0.51657163]])
```

Array of predicted probabilities:

- One row for each sample
- One column for each class

Let's talk about how to interpret this array, using the first row as an example:

- The model calculated a likelihood of 58% that the first sample in `X_new` was class 0 and a 42% likelihood that it was class 1.
- Because class 0 had a higher likelihood, the model predicted class 0 for this sample.
- As you might imagine, the values in every row will always add up to 1.

If you need just the second column, meaning the predicted probabilities of class 1, then you can extract it using NumPy's slicing notation. The `:` means select all rows, and the `1` means select the column in the 1 position, which is the second column.

```
logreg.predict_proba(X_new)[: , 1]
```

```
array([0.42195925, 0.41724454, 0.43257586, 0.42671165, 0.51642919,  
       0.42992738, 0.42082074, 0.61204868, 0.41854626, 0.51657163])
```

Some classifiers, such as logistic regression, are known as well-calibrated classifiers, which means that their predicted probabilities can be directly interpreted as the model's confidence level in that prediction. So for example, it's more confident that the 8th sample is class 1 than it is that the 10th sample is class 1.

Knowing these confidence levels can be useful if you're most interested in the samples with the highest predicted probabilities. For example, if you were trying to predict who might be interested in purchasing a specific product, you might focus all of your marketing budget on reaching those customers with the highest predicted probabilities of purchase.

Keep in mind that there are other classifiers which are not as well-calibrated, such as Naive Bayes. In those cases, it's less appropriate to interpret their predicted probabilities as confidence levels. Thus if you know you're going to be interested in these confidence levels, it's best to use a well-calibrated classifier like logistic regression.

2.9 Q&A: How do I check the accuracy of the model's predictions?

At the end of this chapter, we made predictions for the 10 samples in `X_new`, though we couldn't check the accuracy of these predictions because we don't know the true target values for those samples. That will sometimes be the case in the real world.

For example, if you built a model to predict what medical conditions someone might develop based on their genetic information, you may not ever find out whether the model's predictions were correct, either because that data is not being collected or because that data is protected by privacy laws.

In other cases, you can actually check the accuracy of your predictions. For example, if you built a model to predict the outcome of all US Supreme Court cases, you would make those predictions before those cases were decided, and then you could check the model's accuracy once the court's rulings were publicly announced.

Checking model accuracy:

- **Not possible:** Target value is unknown or is private data
- **Possible:** Target value is known

Ideally, the actual accuracy of your model will be close to the accuracy that you estimated using your training data during model evaluation. If it's not close, that could indicate a problem with your model evaluation procedure, or it could indicate that there are some important differences between your training data and the new data.

In all cases, you can incorporate this new data into your training data since you now know the true target values, which should help the model to make better predictions in the future.

2.10 Q&A: What do the “solver” and “random_state” parameters do?

In this chapter, when creating the model object, I set the logistic regression’s solver to liblinear, and I set the random_state to 1. I set these values so that if you ran the same code at home, you would most likely get the same results as me. In this lesson, I’ll explain what these two parameters actually do.

```
logreg = LogisticRegression(solver='liblinear', random_state=1)
```

The solver is the algorithm used to solve the optimization problem of calculating the logistic regression’s coefficients. In other words, given the features and the target, the solver figures out the coefficients. The solvers have different strengths and weaknesses, different properties, and ultimately may come up with different results. Here’s a comparison chart from the scikit-learn documentation:

Solvers					
Penalties	'liblinear'	'lbfgs'	'newton-cg'	'sag'	'saga'
Multinomial + L2 penalty	no	yes	yes	yes	yes
OVR + L2 penalty	yes	yes	yes	yes	yes
Multinomial + L1 penalty	no	no	no	no	yes
OVR + L1 penalty	yes	no	no	no	yes
Elastic-Net	no	no	no	no	yes
No penalty ('none')	no	yes	yes	yes	yes
Behaviors					
Penalize the intercept (bad)	yes	no	no	no	no
Faster for large datasets	no	no	no	yes	yes
Robust to unscaled datasets	yes	yes	yes	no	no

I recommend reviewing this chart and reading the documentation to decide which solver to use for your particular problem, but ultimately it’s fine to just try each one and see what happens.

liblinear used to be the default solver for logistic regression, but in scikit-learn version 0.22, they changed the default to lbfgs instead. I’m having all of us use liblinear in this book so that we will tend to get the same results, regardless of scikit-learn version. Keep in mind that we still aren’t guaranteed to get the exact same results, because with each new version, bugs are fixed and other algorithm parameters are sometimes changed.

Default solver for logistic regression:

- **Before version 0.22:** liblinear
- **Starting in version 0.22:** lbfgs

One final note about the solver is that if you ever get a convergence warning when using logistic regression, the best solution is usually just to use a different solver.

Next, let’s talk about the random_state parameter. It happens that there is some randomness involved in three of the solvers, including liblinear. That means you may get different results each time you fit

the model. By setting the `random_state`, you ensure that your model will output the same results every time.

More generally, any time you're running a scikit-learn function that involves a random process, I recommend setting the `random_state` parameter to any integer. That allows your code to be reproducible, both by you and others. Keep in mind that the only way to know whether a given function involves a random process is by reading the documentation.

Advice for `random_state`:

- Set `random_state` to any integer when a random process is involved
- Allows your code to be reproducible

2.11 Q&A: How do I show all of the model parameters?

Starting in scikit-learn version 0.23, when you print any estimator (such as a model, a transformer, or a Pipeline), it only shows you the parameters that are not set to their default values. For example, when we print out the `logreg` model object, it only shows the `random_state` and `solver` parameters because we set those explicitly.

```
logreg
```

```
LogisticRegression(random_state=1, solver='liblinear')
```

However, you can still see all parameters by running the `get_params` method.

```
logreg.get_params()
```

```
{'C': 1.0,
 'class_weight': None,
 'dual': False,
 'fit_intercept': True,
 'intercept_scaling': 1,
 'l1_ratio': None,
 'max_iter': 100,
 'multi_class': 'auto',
 'n_jobs': None,
 'penalty': 'l2',
 'random_state': 1,
 'solver': 'liblinear',
 'tol': 0.0001,
 'verbose': 0,
 'warm_start': False}
```

If you like, you can restore the behavior from previous scikit-learn versions by importing the `set_config` function and then setting the `print_changed_only` parameter to `False`.

```
from sklearn import set_config
set_config(print_changed_only=False)
```

Now, all parameters will be printed, regardless of whether you've changed them.

```
logreg
```

```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                    intercept_scaling=1, l1_ratio=None, max_iter=100,
                    multi_class='auto', n_jobs=None, penalty='l2',
                    random_state=1, solver='liblinear', tol=0.0001, verbose=0,
                    warm_start=False)
```

I prefer the new behavior, so I'm going to set `print_changed_only` back to `True`.

```
set_config(print_changed_only=True)
```

2.12 Q&A: Should I shuffle the samples when using cross-validation?

When I ran `cross_val_score` earlier in this chapter, I passed an integer (3) that specified the number of cross-validation folds.

```
cross_val_score(logreg, X, y, cv=3, scoring='accuracy')
```

```
array([0.75      , 0.66666667, 0.66666667])
```

This code shows you what happens “under the hood” when you specify `cv=3` for a classification problem. I'm going to walk through the code so you understand what's happening and you can modify it when needed.

First, you'll notice that we're importing a class called `StratifiedKFold`. It's known as a cross-validation splitter, which means that its role is to split datasets. We'll create an instance of this class and pass it a 3 so that it will create 3 folds. Then we can pass this instance to `cross_val_score` instead of an integer.

```
from sklearn.model_selection import StratifiedKFold
kf = StratifiedKFold(3)
cross_val_score(logreg, X, y, cv=kf, scoring='accuracy')
```

```
array([0.75      , 0.66666667, 0.66666667])
```

It's called `StratifiedKFold` because it uses stratified sampling to ensure that the class proportions are approximately equal in each fold. For example, if 40% of the passengers in the dataset survived, then stratified sampling ensures that about 40% of each fold is passengers who survived.

In other words, it ensures that each fold is representative of the entire dataset. Stratified sampling is desirable because it produces more reliable cross-validation scores, and again, scikit-learn will do this for you by default.

Stratified sampling:

- Ensures that each fold is representative of the dataset
- Produces more reliable cross-validation scores

Another good thing to know about `StratifiedKFold` is that by default, it does not shuffle the samples before splitting. As such, there is nothing random about this process and thus you'll get the same results every time you run `cross_val_score`.

In most cases, it doesn't matter whether you shuffle the samples before splitting. However, if the order of the samples in your dataset is not arbitrary, then it's important to randomly shuffle the samples when cross-validating.

For example, you could imagine that if your dataset was sorted by one of the features, then some folds would only have high values of that feature and other folds would only have low values of that feature, which could result in unreliable cross-validation scores.

If you do need to shuffle the samples, you simply modify the cross-validation splitter by setting `shuffle` to `True`, and then pass that splitter object to `cross_val_score`. Note that because you are introducing randomness into the process by shuffling, you should also set a `random_state` to ensure reproducibility.

```
kf = StratifiedKFold(3, shuffle=True, random_state=1)
cross_val_score(logreg, X, y, cv=kf, scoring='accuracy')
```

```
array([0.75      , 0.33333333, 0.66666667])
```

In summary, if you have a classification problem and the samples are in an arbitrary order, you can just pass an integer to the `cv` parameter of `cross_val_score`, and it will use stratified sampling without shuffling.

If your samples are not in an arbitrary order, you should use `StratifiedKFold` as your splitter and set `shuffle` to `True`, and then pass the splitter object to the `cv` parameter of `cross_val_score`, as I did above.

When to shuffle your samples:

- **Unordered samples:** Shuffling not needed
- **Ordered samples:** Shuffling needed

Finally, if you're working on a regression problem and you need to shuffle the samples, you should use the `KFold` class instead of the `StratifiedKFold` class, since stratified sampling does not apply to regression problems.

How to shuffle your samples:

- **Classification:** `StratifiedKFold`
- **Regression:** `KFold`

3 Encoding categorical features

3.1 Introduction to one-hot encoding

In this chapter, we're going to focus on one of the most important data preprocessing steps, which is the encoding of categorical features.

Let's take a look at our Titanic DataFrame. In the last chapter, the only features we used were Parch and Fare. In this chapter, we want to add Embarked and Sex as additional features, in case they improve our model.

df

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	3	Bra...	male	22.0	1	0	A/5...	7.2500	NaN	S
1	1	1	Cum...	female	38.0	1	0	PC ...	71....	C85	C
2	1	3	Hei...	female	26.0	0	0	STO...	7.9250	NaN	S
3	1	1	Fut...	female	35.0	1	0	113803	53....	C123	S
4	0	3	All...	male	35.0	0	0	373450	8.0500	NaN	S
5	0	3	Mor...	male	NaN	0	0	330877	8.4583	NaN	Q
6	0	1	McC...	male	54.0	0	0	17463	51....	E46	S
7	0	3	Pal...	male	2.0	3	1	349909	21....	NaN	S
8	1	3	Joh...	female	27.0	0	2	347742	11....	NaN	S
9	1	2	Nas...	female	14.0	1	0	237736	30....	NaN	C

As a reminder, Parch is the number of parents or children aboard with that passenger, and Fare is the amount the passenger paid. Our first new feature, Embarked, is the port that each passenger embarked from, and the possible values are C, Q, or S. Our other new feature, Sex, is simply male or female.

Currently selected features:

- **Parch:** Number of parents or children aboard with that passenger
- **Fare:** Amount the passenger paid
- **Embarked:** Port the passenger embarked from
- **Sex:** Male or female

Both Embarked and Sex are known as unordered categorical features because there are distinct categories and there's no inherent logical ordering to the categories. This type of data is also known as nominal data.

Unordered categorical data:

- Contains distinct categories
- No inherent logical ordering to the categories
- Also called “nominal data”

All scikit-learn models expect features to be numeric, and so Embarked and Sex can’t actually be passed directly to a model. Instead, we’re going to encode them using a process called one-hot encoding, also known as dummy encoding.

Let’s look at the code for one-hot encoding. First, we import the `OneHotEncoder` class from the preprocessing module. Then, we create an instance of it and set `sparse` to `False`.

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse=False)
```

By default, `OneHotEncoder` will output a sparse matrix, which is the most efficient and performant data structure for this type of data. By setting `sparse` to `False`, it will instead output a dense matrix, which is just the normal (human-readable) way of representing a matrix. This representation will allow us to examine the output so that we can understand the encoding scheme.

Matrix representations:

- **Sparse:** More efficient and performant
- **Dense:** More readable

Next, we’ll encode the Embarked column by passing it to the `fit_transform` method of the `OneHotEncoder`. We’ll talk about the `fit_transform` method in the next lesson, but for now I just want to highlight that we’ll use double brackets around Embarked to pass it as a single-column DataFrame instead of using single brackets to pass it as a Series.

This is important because `OneHotEncoder` expects to receive a two-dimensional object (such as a DataFrame) since a one-dimensional object is considered ambiguous. A one-dimensional Series could be interpreted either as a single feature or a single sample, whereas our two-dimensional DataFrame signals to scikit-learn that this is indeed a single feature.

Why use double brackets?

- **Single brackets:**
 - Outputs a Series
 - Could be interpreted as a single feature or a single sample
- **Double brackets:**
 - Outputs a single-column DataFrame
 - Interpreted as a single feature

Running the `fit_transform` method outputs this 10 by 3 array. This is the encoded version of the Embarked column, and it is exactly what we will pass to the model instead of the strings C, Q, and S.

```
ohe.fit_transform(df[['Embarked']])
```

```
array([[0., 0., 1.],
       [1., 0., 0.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [1., 0., 0.]])
```

Let's talk about how we interpret this output. There are 3 columns because there were 3 unique values in Embarked. Each row contains a single 1, and the rest of the values in the row are 0. 100 means "C", 010 means "Q", and 001 means "S", which you can confirm by comparing it to the Embarked column in our DataFrame.

As an aside, this is called one-hot encoding because in each row there is one "hot" level, meaning one non-zero level.

This is also the same output you would get by using the `get_dummies` function in pandas, though we'll talk later in the book about why it's best to do all of your preprocessing in scikit-learn instead of pandas.

Output of OneHotEncoder:

- One column for each unique value
- One non-zero value in each row:
 - 1, 0, 0 means "C"
 - 0, 1, 0 means "Q"
 - 0, 0, 1 means "S"

Let's now look at the `categories_` attribute of the `OneHotEncoder`. You can think of it as the column header for our 10 by 3 array. In other words, the `categories_` attribute tells you that the first column represents "C", the second column represents "Q", and the third column represents "S". Because the categories are always in alphabetical order from left to right, I didn't actually have to examine the `categories_` attribute in order to know how to interpret the array.

As an aside, you'll notice a lot of attributes in scikit-learn end in an `_`. This is scikit-learn's convention for any attribute that is learned or estimated from the data during the `fit` step.


```
ohe.categories_
```

```
[array(['C', 'Q', 'S'], dtype=object)]
```

We've now seen how the `OneHotEncoder` encodes the `Embarked` feature. But why is this a reasonable way to encode a categorical feature?

You can think of it this way: `OneHotEncoder` creates a feature from each level so that the model can learn the relationship between each level and the target value. In this case, the model can learn the relationship between the target value of `Survived` and whether or not a passenger embarked at a given port.

For example, the model might learn from the first feature that passengers who embarked at C have a higher survival rate than passengers who didn't embark at C. This is similar to how a model might learn from a numeric feature like `Fare` that passengers with a higher `Fare` have a higher survival rate than passengers with a lower `Fare`.

Why use one-hot encoding?

- Model can learn the relationship between each level and the target value
- **Example:** Model might learn that “C” passengers have a higher survival rate than “not C” passengers

At this point, you might be wondering whether we could have instead encoded `Embarked` as a single numeric feature with the values 0, 1, and 2 representing C, Q, and S. The answer is “yes”, we can do this, but it's generally not a good idea to do this with unordered categories because it would imply an ordering that doesn't inherently exist.

To see why it's not a good idea, let's pretend that passengers who embarked at C and S had high survival rates, and passengers who embarked at Q had low survival rates. There would be no way for a linear model (like logistic regression) to learn that relationship if `Embarked` is encoded as a single feature. That's because a single feature can't be assigned both a negative coefficient to represent the impact of Q (with respect to C) and a positive coefficient to represent the impact of S (with respect to Q).

Why not encode as a single feature?

- **Pretend:**
 - C: high survival rate
 - Q: low survival rate
 - S: high survival rate
- **Single feature would need two coefficients:**
 - Negative coefficient for impact of Q (with respect to C)
 - Positive coefficient for impact of S (with respect to Q)

In summary, encoding Embarked as a single feature would prohibit a linear model from learning a non-linear relationship in the data, which is why encoding it as multiple features is generally the better choice.

3.2 Transformer methods: fit, transform, fit_transform

Let's discuss the `fit_transform` method, since that's the method we used with `OneHotEncoder` to encode the Embarked feature.

`OneHotEncoder` is known as a transformer, meaning its role is to perform data transformations. Transformers usually have a `fit` method and always have a `transform` method. The `fit` method is when the transformer learns something, and the `transform` method is when it uses what it learned to do the data transformation.

Generic transformer methods:

- **fit:** Transformer learns something
- **transform:** Transformer uses what it learned to do the data transformation

Using `OneHotEncoder` as an example, the `fit` method is when it learns the categories from the data in alphabetical order, and the `transform` method is when it creates the feature matrix using those categories.

OneHotEncoder methods:

- **fit:** Learn the categories
- **transform:** Create the feature matrix using those categories

The `fit_transform` method, which is what we used above, just combines those two steps into a single method call. You can actually do those steps as two separate calls of `fit` then `transform`, but the single method call of `fit_transform` is better because it's more computationally efficient and also more readable (in my opinion).

3.3 One-hot encoding of multiple features

We saw how to use `OneHotEncoder` to encode the Embarked column, but we actually need to encode both Embarked and Sex. Thankfully, `OneHotEncoder` can be applied to multiple features at once.

To do this, we simply pass a two-column `DataFrame` to the `fit_transform` method, whereas previously we had passed a one-column `DataFrame`. It outputs 5 columns, in which the first 3 columns represent Embarked and the last 2 columns represent Sex.

```
ohe.fit_transform(df[['Embarked', 'Sex']])
```

```
array([[0., 0., 1., 0., 1.],
       [1., 0., 0., 1., 0.],
       [0., 0., 1., 1., 0.],
       [0., 0., 1., 1., 0.],
       [0., 0., 1., 0., 1.],
       [0., 1., 0., 0., 1.],
       [0., 0., 1., 0., 1.],
       [0., 0., 1., 0., 1.],
       [0., 0., 1., 1., 0.],
       [1., 0., 0., 1., 0.]])
```

Looking at the `categories_` attribute, we first see the 3 categories that were learned from Embarked in alphabetical order, and then we see the 2 categories that were learned from Sex in alphabetical order. Thus, we know that a 10 in the last two columns means “female”, and a 01 in the last two columns means “male”.

```
ohe.categories_
```

```
[array(['C', 'Q', 'S'], dtype=object), array(['female', 'male'], dtype=object)]
```

So for example, the first sample in the output array is 00101, which means they embarked from S and they are male. The second sample in the array is 10010, which means they embarked from C and they are female. And so on.

Decoding the output array:

- **First three columns:**
 - 1, 0, 0 means “C”
 - 0, 1, 0 means “Q”
 - 0, 0, 1 means “S”
- **Last two columns:**
 - 1, 0 means “female”
 - 0, 1 means “male”
- **Example:**
 - 0, 0, 1, 0, 1 means “S, male”
 - 1, 0, 0, 1, 0 means “C, female”

Recall that our goal in this chapter was to numerically encode Embarked and Sex so we could include them in our model along with Parch and Fare. How might we do that?

One idea would be to manually stack Parch and Fare side-by-side with the 5 columns output by OneHotEncoder, and then train the model using all 7 columns. However, we would need to repeat the same exact process of encoding and stacking with the new data, since if you train a model with 7 features, you need the same 7 features in the new data in order to make predictions.

How to manually add Embarked and Sex to the model:

1. Stack Parch and Fare side-by-side with OneHotEncoder output
2. Repeat the same process with new data

This process is less than ideal, since repeating the same steps twice is both inefficient and error-prone. Not only that, but the complexity of this process will continue to increase as you preprocess additional features.

Problems with a manual approach:

- Repeating steps is inefficient and error-prone
- Complexity will increase

In the next chapter, I'll introduce you to the `ColumnTransformer` and `Pipeline` classes. We'll use these two classes to accomplish our goal of adding Embarked and Sex to our model, but we'll do it in a way that is both reliable and efficient.

3.4 Q&A: When should I use transform instead of fit_transform?

Earlier in this chapter, we used the `fit_transform` method of `OneHotEncoder` to encode two categorical features. In this lesson, I'll show you when it's appropriate to just use the `transform` method instead of `fit_transform`. The example below will use `OneHotEncoder`, but the principles I'm teaching here apply the same way to all transformers.

We'll start by creating a `DataFrame` of training data with just 1 categorical feature.

```
demo_train = pd.DataFrame({'letter': ['A', 'B', 'C', 'B']})
demo_train
```

	letter
0	A
1	B
2	C
3	B

Let's run `fit_transform` on the entire `DataFrame`.

```
ohe.fit_transform(demo_train)
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.]])
```

Recall that `fit_transform` is really 2 steps. During the first step, which is `fit`, the `OneHotEncoder` learns the 3 categories. During the second step, which is `transform`, the `OneHotEncoder` creates the feature matrix using those categories. It outputs a 4 by 3 array, since there are 4 samples and 3 categories.

Example of `fit_transform` on training data:

- **fit:** Learn 3 categories (A, B, C)
- **transform:** Create feature matrix with 3 columns

Now, we'll create a `DataFrame` of testing data. It contains the same feature, but that feature includes one fewer category.

```
demo_test = pd.DataFrame({'letter':['A', 'C', 'A']})
demo_test
```

	letter
0	A
1	C
2	A

What would happen if we ran `fit_transform` on the testing data?

```
ohe.fit_transform(demo_test)
```

```
array([[1., 0.],
       [0., 1.],
       [1., 0.]])
```

The output array only includes two columns, because the testing data only included two categories. The first column represents the A category, and the second column represents the C category.

Example of `fit_transform` on testing data:

- **fit:** Learn 2 categories (A, C)
- **transform:** Create feature matrix with 2 columns

This is problematic, because if we trained a model using the 3-column feature matrix, and then tried to make predictions on the 2-column feature matrix, it would error due to a shape mismatch. That makes sense because if you train a model such as logistic regression using 3 features, it will learn 3 coefficients, and it expects to use all 3 of those coefficients when making predictions.

The solution is to run `fit_transform` on the training data, and only run `transform` on the testing data. Let's take a look at the output arrays.

Notice that the categories are represented the same way in both arrays: The first column represents A, the second column represents B, and the third column represents C.

```
ohe.fit_transform(demo_train)
```

```
array([[1., 0., 0.],  
       [0., 1., 0.],  
       [0., 0., 1.],  
       [0., 1., 0.]])
```

```
ohe.transform(demo_test)
```

```
array([[1., 0., 0.],  
       [0., 0., 1.],  
       [1., 0., 0.]])
```

This happened because we only ran the `fit` method once, on the training data, and the `fit` method is when the `OneHotEncoder` learns the categories.

Then we ran the `transform` method twice, both on the training data and the testing data. Because we didn't run the `fit` method on the testing data, the categories learned from the training data were applied to both the training and testing data. This is critically important because it means that both our training and testing feature matrices have 3 columns, and those 3 columns mean the same thing.

Correct process:

1. Run `fit_transform` on training data:
 - **fit:** Learn 3 categories (A, B, C)
 - **transform:** Create feature matrix with 3 columns
2. Run `transform` on testing data:
 - **transform:** Create feature matrix with 3 columns

In summary, when using any transformer, you will always use the `fit_transform` method on the training data and only the `transform` method on the testing data.

3.5 Q&A: What happens if the testing data includes a new category?

In the previous lesson, we created this example `DataFrame` of training data.

```
demo_train
```

	letter
0	A
1	B
2	C
3	B

When we passed that DataFrame to `fit_transform`, the output array included three columns.

```
ohe.fit_transform(demo_train)
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.]])
```

We know from the `categories_` attribute that those columns represent the categories A, B, and C.

```
ohe.categories_
```

```
[array(['A', 'B', 'C'], dtype=object)]
```

Now we'll create a new DataFrame of testing data that includes a category, D, which was not seen in the training data.

```
demo_test_unknown = pd.DataFrame({'letter': ['A', 'C', 'D']})
demo_test_unknown
```

	letter
0	A
1	C
2	D

If you pass this new DataFrame to the `transform` method, it will throw an error because it doesn't know how to represent the D category. It only knows how to represent the A, B, and C categories because those are the ones that were seen by the `OneHotEncoder` during the `fit` step.

```
ohe.transform(demo_test_unknown)
```

```
ValueError: Found unknown categories ['D'] in column 0 during transform
```

There are two possible solutions to this problem.

The first solution is to specify the categories manually to the `OneHotEncoder` when creating an instance.

```
ohe = OneHotEncoder(sparse=False, categories=[['A', 'B', 'C', 'D']])
```

Then, when `fit_transform` is run on the training data, a column is reserved for each of the four categories.

```
ohe.fit_transform(demo_train)
```

```
array([[1., 0., 0., 0.],  
       [0., 1., 0., 0.],  
       [0., 0., 1., 0.],  
       [0., 1., 0., 0.]])
```

As a result, the transform on the testing data will no longer error.

```
ohe.transform(demo_test_unknown)
```

```
array([[1., 0., 0., 0.],  
       [0., 0., 1., 0.],  
       [0., 0., 0., 1.]])
```

However, specifying the categories manually is only a useful solution if you know all possible categories that might ever appear in your data. But in the real world, you don't always know the full set of categories ahead of time.

For example, there might be rare categories that aren't present in your set of samples, or new categories might be added in the future. For example, if one of your categorical features was medical billing codes, you could imagine that new billing codes are added over time.

Why you might not know all possible categories:

- Rare categories aren't present in your set of samples
- New categories are added later

If you don't know all possible categories, then the solution is to set the `handle_unknown` parameter of the `OneHotEncoder` to `'ignore'`, which overrides the default value of `'error'`.

```
ohe = OneHotEncoder(sparse=False, handle_unknown='ignore')
```

Let's use the `fit_transform` method on our training data one more time. The output array includes three columns representing A, B, and C.

```
ohe.fit_transform(demo_train)
```



```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.]])
```

Now, when you use the transform method on the testing data, the third sample is encoded as all zeros because D is an unknown category.

```
ohe.transform(demo_test_unknown)
```

```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 0., 0.]])
```

Although this might seem strange, this is actually quite a reasonable approach since you don't have any information from the training data about the relationship between the D category and the target value.

One limitation of this approach, however, is that all unknown categories will be encoded the same way, which means that an E value in the testing data would also be encoded as all zeros.

Here's my overall advice:

1. When starting a project, keep the `handle_unknown` parameter set to its default value of 'error' so that you'll know if you are encountering new categories in your testing data.
2. If you do find that you're encountering new categories, but you can determine the full set of categories through research, then define the categories manually when creating the `OneHotEncoder` instance.
3. If you can't determine the full set of categories, then set the `handle_unknown` parameter to 'ignore'. However, you should retrain your model as soon as possible using data that includes any new categories.

Advice for OneHotEncoder:

1. Start with `handle_unknown` set to 'error'
2. If possible, specify the categories manually
3. If necessary, set `handle_unknown` to 'ignore' and then retrain your model

3.6 Q&A: Should I drop one of the one-hot encoded categories?

Here's the example training data that we've used in the past few lessons.

```
demo_train
```

	letter
0	A
1	B
2	C
3	B

And here's the default one-hot encoding of this DataFrame.

```
ohe.fit_transform(demo_train)
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.]])
```

When one-hot encoding, it's somewhat common to drop the first column of the output array because it contains redundant information and because it avoids collinearity between features.

You can drop the first column:

- Contains redundant information
- Avoids collinearity between features

If you want to drop the first column, you can set the `OneHotEncoder`'s `drop` parameter to `'first'`, though this option only exists in scikit-learn version 0.21 and later. When you run the `fit_transform`, you can see that the output array contains one fewer column. However, the new encoding retains the same information, since each category is still represented by a unique code: 00 means "A", 10 means "B", and 01 means "C".

```
ohe = OneHotEncoder(sparse=False, drop='first')
ohe.fit_transform(demo_train)
```

```
array([[0., 0.],
       [1., 0.],
       [0., 1.],
       [1., 0.]])
```

Decoding the output array (after dropping the first column):

- 0, 0 means "A"
- 1, 0 means "B"
- 0, 1 means "C"

Dropping the first column will work regardless of the number of categories, but you're only ever allowed to drop a single column. And it doesn't actually matter which column you drop, though the convention is to drop the first column.

You've now seen that you can drop the first column, but the question is: Should you drop the first column? Here's my advice.

If you know that perfectly collinear features will cause problems, such as when feeding the resulting data into a neural network or an unregularized regression, then it's a good idea to drop the first column. However, for most scikit-learn models, perfectly collinear features will not cause any problems, and thus dropping the first column will not benefit the model.

There are also some significant downsides to dropping the first column that you need to be aware of:

- Dropping the first column is incompatible with ignoring unknown categories, which is the `handle_unknown='ignore'` option that we saw in the previous lesson, since the dropped category and unknown categories would both be encoded as all zeros. You are allowed to do this starting in scikit-learn 1.0, but I still don't recommend it.
- Dropping the first column can introduce bias into the model if you standardize your features, such as with `StandardScaler`, or if you use a regularized model, such as logistic regression, since the dropped category will be exempt from standardization and regularization.

Should you drop the first column?

- **Advantages:**
 - Useful if perfectly collinear features will cause problems (does not apply to most models)
- **Disadvantages:**
 - Incompatible with `handle_unknown='ignore'`
 - Introduces bias if you standardize features or use a regularized model

In summary, I recommend that you drop the first column only if you know that perfectly collinear features will cause problems, otherwise I don't recommend dropping the first column.

3.7 Q&A: How do I encode an ordinal feature?

Throughout this chapter, we used one-hot encoding to encode unordered categorical features, also known as nominal data. But how should you encode categorical features with an inherent logical ordering, also known as ordinal data? That's the subject of this lesson.

Types of categorical data:

- Unordered (nominal data)
- Ordered (ordinal data)

Let's take a look at our Titanic DataFrame.

Pclass, which stands for passenger class, is an ordinal feature. Although it's already numeric, the numbers 1, 2, and 3 represent the categories 1st class, 2nd class, and 3rd class. It's considered ordinal data because there's a logical ordering to the categories.

Our intuition is that there may be a relationship between Pclass values increasing and survival rate decreasing, because passengers in the lower-numbered classes may have gotten priority access to lifeboats.

df

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	3	Bra...	male	22.0	1	0	A/5...	7.2500	NaN	S
1	1	1	Cum...	female	38.0	1	0	PC ...	71....	C85	C
2	1	3	Hei...	female	26.0	0	0	STO...	7.9250	NaN	S
3	1	1	Fut...	female	35.0	1	0	113803	53....	C123	S
4	0	3	All...	male	35.0	0	0	373450	8.0500	NaN	S
5	0	3	Mor...	male	NaN	0	0	330877	8.4583	NaN	Q
6	0	1	McC...	male	54.0	0	0	17463	51....	E46	S
7	0	3	Pal...	male	2.0	3	1	349909	21....	NaN	S
8	1	3	Joh...	female	27.0	0	2	347742	11....	NaN	S
9	1	2	Nas...	female	14.0	1	0	237736	30....	NaN	C

Thus if we were going to include Pclass in the model, we would keep the existing numeric encoding so that the model can learn the relationship between Pclass and Survived with a single feature. You could use one-hot encoding with Pclass instead, but the model wouldn't be able to learn that relationship as effectively because that information would be spread out across three features.

Options for encoding Pclass:

- **Ordinal encoding:** Create 1 feature
- **One-hot encoding:** Create 3 features

Now, let's create an example DataFrame to see how to handle ordinal features that are instead stored as strings.

In this DataFrame, we have two ordinal features, Class and Size.

```
df_ordinal = pd.DataFrame({'Class': ['third', 'first', 'second',
                                     'third'],
                           'Size': ['S', 'S', 'L', 'XL']})
df_ordinal
```

	Class	Size
0	third	S
1	first	S
2	second	L
3	third	XL

If you have ordinal data, you should use the `OrdinalEncoder` class to do the encoding. First, you import it from the preprocessing module. Then, you create an instance of `OrdinalEncoder`, and when you do so, you define the logical order of the categories.

We pass a list of lists to the `categories` parameter, in which the first inner list is the categories for the `Class` feature, and the second inner list is the categories for the `Size` feature. I put the two lists in that order because that is the order in which I'll be passing the features to the `fit_transform` method.

One important note is that I included the “M” category for `Size` even though it wasn't present in this `DataFrame` because I knew that it would occur in the dataset at some point.

```
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder(categories=[['first', 'second', 'third'],
                               ['S', 'M', 'L', 'XL']])
```

Next, we pass the `DataFrame` to the `OrdinalEncoder`'s `fit_transform` method in order to do the encoding. You'll notice that each input feature became a single feature in the output array.

```
oe.fit_transform(df_ordinal)
```

```
array([[2., 0.],
       [0., 0.],
       [1., 2.],
       [2., 3.]])
```

For the `Class` feature, “first” was encoded as 0, “second” was encoded as 1, and “third” was encoded as 2.

For the `Size` feature, “S” was encoded as 0, “L” was encoded as 2, and “XL” was encoded as 3. And if “M” appears in the data at some point, it will be encoded as 1.

Again, we encoded each input feature as a single column so that the model can learn the relationship between the target and an increase or decrease in each feature.

Decoding the output array:

- **First column:**
 - 0 means “first”
 - 1 means “second”
 - 2 means “third”
- **Second column:**
 - 0 means “S”
 - 1 means “M”
 - 2 means “L”
 - 3 means “XL”
- **Example:**

– 2, 0 means “third, S”

Let’s briefly contrast this with the output you would get if you used `OneHotEncoder` with these same two features.

`OneHotEncoder` would create 7 columns in the output array, since `Class` has 3 categories and `Size` has 4 categories. These 7 columns contain the same information as the 2 columns output by `OrdinalEncoder`, but the model would have a comparatively harder time learning from the 7 columns since the information is expressed in a less compact form.

```
ohe = OneHotEncoder(sparse=False,
                    categories=[['first', 'second', 'third'],
                               ['S', 'M', 'L', 'XL']])
ohe.fit_transform(df_ordinal)
```

```
array([[0., 0., 1., 1., 0., 0., 0.],
       [1., 0., 0., 1., 0., 0., 0.],
       [0., 1., 0., 0., 0., 1., 0.],
       [0., 0., 1., 0., 0., 0., 1.]])
```

Here’s a summary of my advice on this topic:

1. If you have an ordinal feature that’s already encoded numerically, then leave it as-is.
2. If you have an ordinal feature that’s stored as strings, then encode it using `OrdinalEncoder`.
3. If you have a nominal feature, then encode it using `OneHotEncoder`.

Advice for encoding categorical data:

- **Ordinal feature stored as numbers:** Leave as-is
- **Ordinal feature stored as strings:** Use `OrdinalEncoder`
- **Nominal feature:** Use `OneHotEncoder`

In chapter 17, we’ll explore this topic further and see if there are cases in which you should diverge from this advice.

One final note about `OrdinalEncoder` is that unlike `OneHotEncoder`, it does not allow for new categories in the testing data that were not seen during training. However, that functionality is available beginning in scikit-learn version 0.24 using a `handle_unknown` parameter.

3.8 Q&A: What’s the difference between `OrdinalEncoder` and `LabelEncoder`?

There are many similarities between the `OrdinalEncoder` and `LabelEncoder` classes, so in this lesson I’ll explain how they’re different and why you should be using `OrdinalEncoder`, not `LabelEncoder`.

The first main difference is that `OrdinalEncoder` allows you to define the order of the categories, whereas `LabelEncoder` does not. `LabelEncoder` simply uses the alphabetical order of the values you pass to it to determine which value to encode as 0, which value to encode as 1, and so on.

The second main difference is that `OrdinalEncoder` can be used to encode multiple features at once, whereas `LabelEncoder` can only encode one column of data at once.

	<code>OrdinalEncoder</code>	<code>LabelEncoder</code>
Can you define the category order?	Yes	No
Can you encode multiple features?	Yes	No

Because of these differences, `OrdinalEncoder` is much better suited than `LabelEncoder` for encoding ordinal features. In fact, `LabelEncoder` is only intended for the encoding of class labels, hence its name.

You might be asking why `LabelEncoder` even exists, given its limitations. There are two reasons:

- First, in older versions of scikit-learn, some classification models were not able to handle string-based labels. `LabelEncoder` was used to encode those strings as integers so that they could be passed to the model. That limitation has since been removed, and so all scikit-learn classifiers can now handle string-based labels. Therefore, you should never need to use `LabelEncoder` for encoding your class labels.
- Second, also in older versions of scikit-learn, `OneHotEncoder` did not accept strings as input. Thus if you had categorical data stored as strings, you actually had to use `LabelEncoder` to encode the strings as integers before passing them to the `OneHotEncoder`. Again, that limitation has since been removed, and so you can pass string-based categorical data directly to `OneHotEncoder`.

Outdated uses for `LabelEncoder`:

- Encoding string-based labels for some classifiers
- Encoding string-based features for `OneHotEncoder`

Because of this legacy from older versions of scikit-learn, many people are familiar with `LabelEncoder` and thus use it to encode features. However, the best practice is to use `OrdinalEncoder` to encode ordinal features. In fact, it's rare that you will ever need to use `LabelEncoder`, which is why I'm not using it in this book.

3.9 Q&A: Should I encode numeric features as ordinal features?

Normally, when you have a continuous numeric feature such as `Fare`, you pass that feature directly to your Machine Learning model.

```
df[['Fare']]
```

	Fare
0	7.2500
1	71.2833
2	7.9250
3	53.1000
4	8.0500
5	8.4583
6	51.8625
7	21.0750
8	11.1333
9	30.0708

However, one strategy that is sometimes used with numeric features is to “discretize” or “bin” them into categorical features. In scikit-learn, we can do this using `KBinsDiscretizer`.

```
from sklearn.preprocessing import KBinsDiscretizer
```

When creating an instance of `KBinsDiscretizer`, you define the number of bins, the binning strategy, and the method used for encoding the result.

```
kb = KBinsDiscretizer(n_bins=3, strategy='quantile', encode='ordinal')
```

Here’s the output when we pass the `Fare` feature to the `fit_transform` method.

Because we specified 3 bins, every sample has been assigned to bin 0 or 1 or 2. The smallest values were assigned to bin 0, the largest values were assigned to bin 2, and the values in between were assigned to bin 1. Thus, we’ve taken a continuous numeric feature and encoded it as an ordinal feature, and this ordinal feature could be passed to the model in place of the numeric feature.

```
kb.fit_transform(df[['Fare']])
```

```
array([[0.],
       [2.],
       [0.],
       [2.],
       [0.],
       [1.],
       [2.],
       [1.],
       [1.],
       [2.]])
```

The obvious follow-up question is: Should we discretize our numeric features? Theoretically, discretization can benefit linear models by helping them to learn non-linear trends. However, my general recommendation is to not use discretization, for three main reasons:

- First, discretization removes all nuance from the data, which makes it harder for a model to learn the actual trends that are present in the data.

- Second, discretization reduces the variation in the data, which makes it easier to find trends that don't actually exist.
- Third, any possible benefits of discretization are highly dependent on the parameters used with `KBinsDiscretizer`. Making those decisions by hand creates a risk of overfitting the training data, and making those decisions during a tuning process adds both complexity and processing time, and so neither of those options is particularly attractive to me.

Why not discretize numeric features?

- Makes it harder to learn the actual trends
- Makes it easier to discover non-existent trends
- May result in overfitting

4 Improving your workflow with ColumnTransformer and Pipeline

4.1 Preprocessing features with ColumnTransformer

In the last chapter, our goal was to include two numeric features and two categorical features in our model. We saw how to numerically encode the categorical features using `OneHotEncoder`, but we lacked an efficient process for stacking those encoded features next to the numerical features, and we lacked an efficient way to apply this same preprocessing to our new data.

Problems from Chapter 3:

- Need to stack categorical features next to numerical features
- Need to apply the same preprocessing to new data

In this chapter, we're going to solve both of those problems using the `ColumnTransformer` and `Pipeline` classes:

- `ColumnTransformer` will make it easy to apply different preprocessing steps to different columns.
- `Pipeline` will make it easy to apply the same workflow to training data and new data.

How to solve those problems:

- **ColumnTransformer:** Apply different preprocessing steps to different columns
- **Pipeline:** Apply the same workflow to training data and new data

To start, we'll create a Python list of the four columns we've been working with, and use that to create our `X` object.

```
cols = ['Parch', 'Fare', 'Embarked', 'Sex']
```

```
X = df[cols]  
X
```

	Parch	Fare	Embarked	Sex
0	0	7.2500	S	male
1	0	71.2833	C	female
2	0	7.9250	S	female
3	0	53.1000	S	female
4	0	8.0500	S	male
5	0	8.4583	Q	male
6	0	51.8625	S	male
7	1	21.0750	S	male
8	2	11.1333	S	female
9	0	30.0708	C	female

We're still going to be one-hot encoding the Embarked and Sex columns, so we'll create an instance of `OneHotEncoder`. We're using the default options for `OneHotEncoder`, which means it will output a sparse matrix, but that's fine because we're not going to examine the output directly.

```
ohe = OneHotEncoder()
```

Now it's time to create our first `ColumnTransformer`, which will take care of any data transformations that we specify. We'll start by importing the `make_column_transformer` function from the `compose` module.

In general, you use `make_column_transformer` by passing it one or more tuples, and each tuple should have two elements:

1. The first element is a transformer.
2. The second element is a list of columns to which that transformer should be applied. Note that in most cases, this element should be a list even if you are only specifying a single column.

In our case, we'll pass it a single tuple in which the first element is our `OneHotEncoder` object and the second element is a list of the two columns we want to one-hot encode.

After all tuples, we'll set the `remainder` parameter to `'drop'`, which means that all columns which are not explicitly mentioned in the `ColumnTransformer` should be dropped. `'drop'` is actually the default value for `remainder`, but I'm including it here just for clarity.

Note that I could have defined the `ColumnTransformer` on a single line, but I prefer breaking the lines in this way for readability.

When we run this code, the `make_column_transformer` code returns a `ColumnTransformer` object, which we'll save as `ct`.

```
from sklearn.compose import make_column_transformer
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    remainder='drop')
```

Tuple elements for `make_column_transformer`:

1. Transformer object
2. List of columns to which the transformer should be applied

Next, we'll perform the transformation by passing `X`, which is our four-column DataFrame, to the `fit_transform` method of the `ct` object. It outputs a 10 by 5 array that represents the one-hot encoding of the Embarked and Sex columns. The first three columns represent Embarked and the other two columns represent Sex, and they're in that order because that's the order in which they were listed in the `ColumnTransformer`.

Note that even though the `Parch` and `Fare` columns are part of `X`, they're excluded from the output array because we told the `ColumnTransformer` to drop all unspecified columns.

```
ct.fit_transform(X)
```

```
array([[0., 0., 1., 0., 1.],
       [1., 0., 0., 1., 0.],
       [0., 0., 1., 1., 0.],
       [0., 0., 1., 1., 0.],
       [0., 0., 1., 0., 1.],
       [0., 1., 0., 0., 1.],
       [0., 0., 1., 0., 1.],
       [0., 0., 1., 0., 1.],
       [0., 0., 1., 1., 0.],
       [1., 0., 0., 1., 0.]])
```

Output columns:

- **Columns 1-3:** Embarked
- **Columns 4-5:** Sex

This is nice, but our actual goal was to create a matrix that includes the `Parch` and `Fare` columns alongside the encoded versions of Embarked and Sex. To accomplish that, we'll simply change the value of `remainder` from `'drop'` to `'passthrough'`. This means that all columns which are not mentioned in the `ColumnTransformer` should be passed through to the output unmodified. In other words, include the `Parch` and `Fare` columns in the output, but don't transform them in any way.

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    remainder='passthrough')
```

When we run the `fit_transform` method this time, it outputs a 10 by 7 array. The first five columns represent the encoded Embarked and Sex columns, and the sixth and seventh columns are the `Parch` and `Fare` columns. The column order is based on the order in which the columns were listed in the `ColumnTransformer`, followed by any you pass through.

```
ct.fit_transform(X)
```

```
array([[ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  0.      ,  7.25   ],
       [ 1.      ,  0.      ,  0.      ,  1.      ,  0.      ,  0.      , 71.2833 ],
```

```
[ 0.      ,  0.      ,  1.      ,  1.      ,  0.      ,  0.      ,  7.925  ],
[ 0.      ,  0.      ,  1.      ,  1.      ,  0.      ,  0.      ,  53.1   ],
[ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  0.      ,  8.05   ],
[ 0.      ,  1.      ,  0.      ,  0.      ,  1.      ,  0.      ,  8.4583 ],
[ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  0.      ,  51.8625 ],
[ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  1.      ,  21.075  ],
[ 0.      ,  0.      ,  1.      ,  1.      ,  0.      ,  2.      ,  11.1333 ],
[ 1.      ,  0.      ,  0.      ,  1.      ,  0.      ,  0.      ,  30.0708 ]])
```

Output columns:

- **Columns 1-3:** Embarked
- **Columns 4-5:** Sex
- **Column 6:** Parch
- **Column 7:** Fare

We were able to figure out on our own what each column represents, but you can also use the `ColumnTransformer`'s `get_feature_names` method to confirm the meanings of these 7 features. The `x0` simply means feature 0 that was passed to the `OneHotEncoder`, and the `x1` means feature 1.

```
ct.get_feature_names()
```

```
['onehotencoder__x0_C',
 'onehotencoder__x0_Q',
 'onehotencoder__x0_S',
 'onehotencoder__x1_female',
 'onehotencoder__x1_male',
 'Parch',
 'Fare']
```

Before we move on, I have two quick asides about the `get_feature_names` method:

- First, the `get_feature_names` method didn't work with passthrough columns prior to scikit-learn version 0.23, so you'll get an error if you run the code with previous versions.
- Second, the `get_feature_names` method has been replaced with a similar method called `get_feature_names_out` starting in scikit-learn 1.0.

Notes about `get_feature_names`:

- **Before version 0.23:** Didn't work with passthrough columns
- **Starting in version 1.0:** Has been replaced with `get_feature_names_out`

To wrap up this lesson, I want to show you one other way to specify this same `ColumnTransformer`.

As I mentioned before, `make_column_transformer` accepts tuples, and the first element of each tuple is usually a transformer object (like our `ohe` object). However, the first element of the tuple can also be the special strings `'drop'` or `'passthrough'`, which tells the `ColumnTransformer` to drop or pass through specific columns.

Tuple elements for `make_column_transformer` (revised):

1. Transformer object or 'drop' or 'passthrough'
2. List of columns to which the transformer should be applied

So, we're going to add a second tuple in which the transformer is the string 'passthrough', and we want to apply this passthrough transformer to the columns Parch and Fare. This `ColumnTransformer` will do the exact same thing as the previous one, but I actually prefer this notation any time I have a small number of passthrough columns, since it reminds me of which columns I'm passing through.

It's still important to remember that the default value for the `remainder` parameter is 'drop', which means that any unspecified columns will be dropped, though we don't have any unspecified columns in this case.

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    ('passthrough', ['Parch', 'Fare']))
```

We'll run the `fit_transform` method one more time, and you can see that it outputs the same 7 columns as before. And to be clear, this is the feature matrix that we will pass to our model.

```
ct.fit_transform(X)
```

```
array([[ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  0.      ,  7.25   ],
       [ 1.      ,  0.      ,  0.      ,  1.      ,  0.      ,  0.      , 71.2833 ],
       [ 0.      ,  0.      ,  1.      ,  1.      ,  0.      ,  0.      ,  7.925  ],
       [ 0.      ,  0.      ,  1.      ,  1.      ,  0.      ,  0.      , 53.1    ],
       [ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  0.      ,  8.05   ],
       [ 0.      ,  1.      ,  0.      ,  0.      ,  1.      ,  0.      ,  8.4583 ],
       [ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  0.      , 51.8625 ],
       [ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  1.      , 21.075  ],
       [ 0.      ,  0.      ,  1.      ,  1.      ,  0.      ,  2.      , 11.1333 ],
       [ 1.      ,  0.      ,  0.      ,  1.      ,  0.      ,  0.      , 30.0708 ]])
```

4.2 Chaining steps with Pipeline

In the previous lesson, we accomplished our first goal, which was to apply different preprocessing to different columns using `ColumnTransformer`. In this lesson, we're moving on to our second goal, which is to apply the same workflow to training data and new data using the `Pipeline` class.

A `Pipeline` is used to chain together sequential steps. In this case, we want to chain together two steps, namely data preprocessing followed by model building.

We'll start by importing the `make_pipeline` function. Then, we can create a `Pipeline` instance by passing it two objects: our `ColumnTransformer` instance for data preprocessing, and our `LogisticRegression` instance for model building. We'll save it as an object called `pipe`, which is a 2-step `Pipeline`.

```
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(ct, logreg)
```

Pipeline steps:

1. Data preprocessing with ColumnTransformer
2. Model building with LogisticRegression

You might remember that back in chapter 2, we used cross-validation to evaluate our model when it only included the Parch and Fare features. Now that we've added the Embarked and Sex features, it would normally make sense to cross-validate the updated model to see whether adding those features made our model better or worse. And in fact, you can (and should) cross-validate an entire Pipeline.

However, any model evaluation procedure is highly unreliable with only 10 rows of data, and so any change in the cross-validated accuracy would be misleading. Thus we're going to skip the cross-validation step for the moment, though we'll return to it in chapter 8 once we're using the full dataset.

Since we're skipping cross-validation, our next step is just to run the `fit` method on the Pipeline, and pass it `X` and `y`. Here's what happens when we fit the Pipeline:

1. First, it runs the ColumnTransformer step, meaning that it takes `X`, which is a 4-column DataFrame that contains both numbers and strings, and transforms it into the 7-column feature matrix that only includes numbers.
2. Second, it runs the LogisticRegression step, meaning that the model is fit to this 7-column feature matrix. In other words, it learns the relationship between those 7 features and the `y` values.

Note that when you fit a Pipeline, it will actually print out the steps. You can see that step 1 is a ColumnTransformer that includes a OneHotEncoder and a passthrough transformer, and step 2 is a LogisticRegression model.

```
pipe.fit(X, y)
```

```
Pipeline(steps=[('columntransformer',
                  ColumnTransformer(transformers=[('onehotencoder',
                                                    OneHotEncoder(),
                                                    ['Embarked', 'Sex']),
                                                    ('passthrough', 'passthrough',
                                                    ['Parch', 'Fare'])])),
                 ('logisticregression',
                  LogisticRegression(random_state=1, solver='liblinear'))])
```

Fitting the Pipeline:

1. ColumnTransformer converts `X` (4 columns) into a numeric feature matrix (7 columns)

2. LogisticRegression model is fit to the feature matrix

In case it helps you to understand the Pipeline better, I'm going to show you what happens “under the hood” when you fit this Pipeline. To be clear, you should not actually write the following code, rather it is just for teaching purposes:

1. First, X is transformed by the ColumnTransformer into X_t, which stands for “X transformed”.
2. Second, the LogisticRegression model is fit on X_t and y.

```
X_t = ct.fit_transform(X)
logreg.fit(X_t, y)
```

```
LogisticRegression(random_state=1, solver='liblinear')
```

As you would expect, X has the shape 10 by 4, and X_t has the shape 10 by 7.

```
print(X.shape)
print(X_t.shape)
```

```
(10, 4)
(10, 7)
```

4.3 Using the Pipeline to make predictions

Now that we've fit our Pipeline, we want to use it to make predictions on new data.

The first step is to update the X_new DataFrame so that it contains the same columns as X. Recall that the cols object contains the names of our four columns, and so we can use it to select those four columns from the df_new DataFrame.

```
X_new = df_new[cols]
X_new
```

	Parch	Fare	Embarked	Sex
0	0	7.8292	Q	male
1	0	7.0000	S	female
2	0	9.6875	Q	male
3	0	8.6625	S	male
4	1	12.2875	S	female
5	0	9.2250	S	male
6	0	7.6292	Q	female
7	1	29.0000	S	male
8	0	7.2292	C	female
9	0	24.1500	S	male

Now, we can pass `X_new` to the Pipeline's `predict` method to make predictions for these ten samples. When we run it, the Pipeline applies the same transformations to `X_new` that it applied to `X`, and the transformed version of `X_new` is passed to the fitted logistic regression model so that it can make predictions.

In other words, the Pipeline enabled us to accomplish our second goal, which is to apply the same workflow to training data and new data.

As a reminder, we can't evaluate the accuracy of these ten predictions because we don't know the true target values for `X_new`.

```
pipe.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0])
```

Predicting with the Pipeline:

1. `ColumnTransformer` applies the same transformations to `X_new`
2. Fitted `LogisticRegression` model makes predictions on the transformed version of `X_new`

Just like before, I'm going to show you what happens "under the hood" when you make predictions using this Pipeline. Again, you should not actually write the following code, rather it is just for teaching purposes:

1. First, `X_new` is transformed by the `ColumnTransformer` into `X_new_t`, which stands for "X_new transformed".
2. Second, the fitted `LogisticRegression` model makes predictions for the samples in `X_new_t`.

```
X_new_t = ct.transform(X_new)
logreg.predict(X_new_t)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0])
```

As you would expect, `X_new` has the shape 10 by 4, and `X_new_t` has the shape 10 by 7.

```
print(X_new.shape)
print(X_new_t.shape)
```

```
(10, 4)
(10, 7)
```

One important point I want to highlight is that the Pipeline's `predict` method called the `ColumnTransformer`'s `transform` method, not its `fit_transform` method. Why would that be?

Recall that the `fit` step is when a transformer learns something, and the `transform` step is when it uses what it learned to do the transformation. Thus you fit on `X` to learn an encoding, and you transform on `X` and `X_new` to apply that encoding.

This is critically important. Our logistic regression model was fit on 7 columns, and so it learned 7 coefficients. To make predictions, you need to pass 7 columns to the `predict` method, and those 7 columns need to mean the same thing as the 7 columns you used when fitting the model. Thus, the `predict` method only runs `transform` so that the exact same encoding will be applied to the training data and the new data.

ColumnTransformer methods:

1. Run `fit_transform` on `X`:
 - **fit**: Learn the encoding
 - **transform**: Apply the encoding to create 7 columns
2. Run `transform` on `X_new`:
 - **transform**: Apply the encoding to create 7 columns

It's okay if you're still a bit fuzzy on the difference between `fit` and `transform`, because the `Pipeline` object will just do the right thing for you when you run `fit` or `predict`. However, understanding the difference will ultimately help you to go further with `scikit-learn`.

4.4 Q&A: How do I drop some columns and passthrough others?

Currently we only have 4 columns in `X`, namely `Parch`, `Fare`, `Embarked`, and `Sex`. But imagine that we had many more columns, and we wanted to drop a few columns and pass through the rest. How would we do that efficiently?

We can use the special string `'drop'` to tell the `ColumnTransformer` which columns to drop, and also tell it to pass through all remaining columns. So in this example, we're one-hot encoding `Embarked` and `Sex`, which creates 5 columns, dropping `Fare`, and passing through `Parch`, which adds 1 more column.

We could use this same pattern to drop a few columns and pass through hundreds of columns without having to list the `passthrough` columns one-by-one.

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    ('drop', ['Fare']),
    remainder='passthrough')
ct.fit_transform(X)
```

```
array([[0., 0., 1., 0., 1., 0.],
       [1., 0., 0., 1., 0., 0.],
       [0., 0., 1., 1., 0., 0.],
       [0., 0., 1., 1., 0., 0.],
       [0., 0., 1., 0., 1., 0.],
       [0., 1., 0., 0., 1., 0.],
       [0., 0., 1., 0., 1., 0.]])
```

```
[0., 0., 1., 0., 1., 1.],
[0., 0., 1., 1., 0., 2.],
[1., 0., 0., 1., 0., 0.]])
```

Conversely, we might want to pass through a few columns and drop the rest. We can use the special string 'passthrough' to tell the ColumnTransformer which columns to pass through, and also tell it to drop all remaining columns. So in this example, we're one-hot encoding Embarked and Sex, which creates 5 columns, passing through Parch, which adds 1 more column, and dropping Fare.

Again, we can use this pattern to pass through a few columns and drop hundreds of columns without listing them all.

Finally, just a reminder that 'drop' is the default value for remainder, so you aren't actually required to specify it here.

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    ('passthrough', ['Parch']),
    remainder='drop')
ct.fit_transform(X)
```

```
array([[0., 0., 1., 0., 1., 0.],
       [1., 0., 0., 1., 0., 0.],
       [0., 0., 1., 1., 0., 0.],
       [0., 0., 1., 1., 0., 0.],
       [0., 0., 1., 0., 1., 0.],
       [0., 1., 0., 0., 1., 0.],
       [0., 0., 1., 0., 1., 0.],
       [0., 0., 1., 0., 1., 1.],
       [0., 0., 1., 1., 0., 2.],
       [1., 0., 0., 1., 0., 0.]])
```

4.5 Q&A: How do I transform the unspecified columns?

We know how to drop or pass through the unspecified columns in a ColumnTransformer, but let's pretend we wanted to apply a transformation to all of the unspecified columns. This is actually simple to do by passing a transformer to the remainder parameter.

For example, we might want to scale all of the unspecified columns. One option is MaxAbsScaler, which divides each feature by its maximum value and thus scales it to the range -1 to 1. We'll import it from the preprocessing module and then create an instance.

```
from sklearn.preprocessing import MaxAbsScaler
scaler = MaxAbsScaler()
```

Then, we can pass the scaler to the remainder parameter.

When we run the `fit_transform` method, you can see that the first 5 columns were created from Embarked and Sex, and the final column is the scaled version of the Parch column. (We'll talk more about feature scaling in chapter 14.)

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    ('drop', ['Fare']),
    remainder=scaler)
ct.fit_transform(X)
```

```
array([[0. , 0. , 1. , 0. , 1. , 0. ],
       [1. , 0. , 0. , 1. , 0. , 0. ],
       [0. , 0. , 1. , 1. , 0. , 0. ],
       [0. , 0. , 1. , 1. , 0. , 0. ],
       [0. , 0. , 1. , 0. , 1. , 0. ],
       [0. , 1. , 0. , 0. , 1. , 0. ],
       [0. , 0. , 1. , 0. , 1. , 0. ],
       [0. , 0. , 1. , 0. , 1. , 0.5],
       [0. , 0. , 1. , 1. , 0. , 1. ],
       [1. , 0. , 0. , 1. , 0. , 0. ]])
```

4.6 Q&A: How do I select columns from a NumPy array?

Throughout this book, we've been using a pandas DataFrame as our input. But what if your input data was a NumPy array instead? Let's see how that affects our workflow.

We'll start by converting the `X` and `X_new` DataFrames into NumPy arrays called `X_array` and `X_new_array`.

```
X_array = X.to_numpy()
X_new_array = X_new.to_numpy()
```

Here's what `X_array` looks like.

```
X_array
```

```
array([[0, 7.25, 'S', 'male'],
       [0, 71.2833, 'C', 'female'],
       [0, 7.925, 'S', 'female'],
       [0, 53.1, 'S', 'female'],
       [0, 8.05, 'S', 'male'],
       [0, 8.4583, 'Q', 'male'],
       [0, 51.8625, 'S', 'male'],
```

```
[1, 21.075, 'S', 'male'],
[2, 11.1333, 'S', 'female'],
[0, 30.0708, 'C', 'female']], dtype=object)
```

If this was our input data, and we wanted to use a `ColumnTransformer`, we wouldn't be able to specify the columns by name because columns of a NumPy array don't have names. However, we do have a couple of other options.

First, we could specify the columns by integer position. Embarked and Sex are columns 2 and 3, so in this example, we're one-hot encoding Embarked and Sex and passing through the remainder. Notice that we're passing `X_array`, not `X`, to the `fit_transform` method.

```
ct = make_column_transformer(
    (ohe, [2, 3]),
    remainder='passthrough')
ct.fit_transform(X_array)
```

```
array([[0.0, 0.0, 1.0, 0.0, 1.0, 0, 7.25],
       [1.0, 0.0, 0.0, 1.0, 0.0, 0, 71.2833],
       [0.0, 0.0, 1.0, 1.0, 0.0, 0, 7.925],
       [0.0, 0.0, 1.0, 1.0, 0.0, 0, 53.1],
       [0.0, 0.0, 1.0, 0.0, 1.0, 0, 8.05],
       [0.0, 1.0, 0.0, 0.0, 1.0, 0, 8.4583],
       [0.0, 0.0, 1.0, 0.0, 1.0, 0, 51.8625],
       [0.0, 0.0, 1.0, 0.0, 1.0, 1, 21.075],
       [0.0, 0.0, 1.0, 1.0, 0.0, 2, 11.1333],
       [1.0, 0.0, 0.0, 1.0, 0.0, 0, 30.0708]], dtype=object)
```

Another option is to specify the columns using slices, which is useful for large ranges of columns next to one another. In this case, we're selecting columns 2 through 3 for one-hot encoding, and passing through the remainder. Remember that Python slices are inclusive of the starting value, which is 2 in this case, and exclusive of the ending value, which is 4 in this case.

```
ct = make_column_transformer(
    (ohe, slice(2, 4)),
    remainder='passthrough')
ct.fit_transform(X_array)
```

```
array([[0.0, 0.0, 1.0, 0.0, 1.0, 0, 7.25],
       [1.0, 0.0, 0.0, 1.0, 0.0, 0, 71.2833],
       [0.0, 0.0, 1.0, 1.0, 0.0, 0, 7.925],
       [0.0, 0.0, 1.0, 1.0, 0.0, 0, 53.1],
       [0.0, 0.0, 1.0, 0.0, 1.0, 0, 8.05],
       [0.0, 1.0, 0.0, 0.0, 1.0, 0, 8.4583],
       [0.0, 0.0, 1.0, 0.0, 1.0, 0, 51.8625],
       [0.0, 0.0, 1.0, 0.0, 1.0, 1, 21.075],
       [0.0, 0.0, 1.0, 1.0, 0.0, 2, 11.1333],
       [1.0, 0.0, 0.0, 1.0, 0.0, 0, 30.0708]], dtype=object)
```

One final option is to specify the columns using a boolean mask. Normally you would create the mask using some sort of condition, but in this case I'm just writing out a mask to select columns 2 and 3 for one-hot encoding, and passing through the remainder.

```
ct = make_column_transformer(
    (ohe, [False, False, True, True]),
    remainder='passthrough')
ct.fit_transform(X_array)

array([[0.0, 0.0, 1.0, 0.0, 1.0, 0, 7.25],
       [1.0, 0.0, 0.0, 1.0, 0.0, 0, 71.2833],
       [0.0, 0.0, 1.0, 1.0, 0.0, 0, 7.925],
       [0.0, 0.0, 1.0, 1.0, 0.0, 0, 53.1],
       [0.0, 0.0, 1.0, 0.0, 1.0, 0, 8.05],
       [0.0, 1.0, 0.0, 0.0, 1.0, 0, 8.4583],
       [0.0, 0.0, 1.0, 0.0, 1.0, 0, 51.8625],
       [0.0, 0.0, 1.0, 0.0, 1.0, 1, 21.075],
       [0.0, 0.0, 1.0, 1.0, 0.0, 2, 11.1333],
       [1.0, 0.0, 0.0, 1.0, 0.0, 0, 30.0708]], dtype=object)
```

Options for selecting columns from a NumPy array:

- Integer position
- Slice
- Boolean mask

Other than that, the rest of our workflow remains the same. We'll just update the Pipeline to use our new ColumnTransformer.

```
pipe = make_pipeline(ct, logreg)
```

Then we can fit the Pipeline with `X_array` and `y`, and make predictions for `X_new_array`.

```
pipe.fit(X_array, y)
pipe.predict(X_new_array)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0])
```

4.7 Q&A: How do I select columns by data type?

So far in this book, we've been selecting columns one-by-one. But let's say that we had many more columns, and we simply wanted to one-hot encode all object columns and passthrough all numeric columns without listing all of them out. How would we do that?

The easiest way to do this is with the `make_column_selector` function, which was new in scikit-learn version 0.22.

```
from sklearn.compose import make_column_selector
```

We're going to create two column selectors called `select_object` and `select_number`. To do this, we just set the `dtype_include` parameter to the data type we want to include, and it outputs a callable.

```
select_object = make_column_selector(dtype_include=object)
select_number = make_column_selector(dtype_include='number')
```

Then, we pass the callables to `make_column_transformer` instead of the column names, and the callables select the columns for us.

When we run `fit_transform`, you can see that once again, the object columns have been one-hot encoded and the numeric columns have been passed through.

```
ct = make_column_transformer(
    (ohe, select_object),
    ('passthrough', select_number))
ct.fit_transform(X)
```

```
array([[ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  0.      ,  7.25   ],
       [ 1.      ,  0.      ,  0.      ,  1.      ,  0.      ,  0.      , 71.2833],
       [ 0.      ,  0.      ,  1.      ,  1.      ,  0.      ,  0.      ,  7.925  ],
       [ 0.      ,  0.      ,  1.      ,  1.      ,  0.      ,  0.      , 53.1    ],
       [ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  0.      ,  8.05   ],
       [ 0.      ,  1.      ,  0.      ,  0.      ,  1.      ,  0.      ,  8.4583],
       [ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  0.      , 51.8625],
       [ 0.      ,  0.      ,  1.      ,  0.      ,  1.      ,  1.      , 21.075  ],
       [ 0.      ,  0.      ,  1.      ,  1.      ,  0.      ,  2.      , 11.1333],
       [ 1.      ,  0.      ,  0.      ,  1.      ,  0.      ,  0.      , 30.0708]])
```

One slight variation of this is that you can tell `make_column_selector` to exclude (instead of include) a specific data type. In this example, we're using the `dtype_exclude` parameter to create a column selector that excludes the object data type.

```
exclude_object = make_column_selector(dtype_exclude=object)
```

This time, we'll tell the `ColumnTransformer` to one-hot encode all object columns and pass through all non-object columns, which has the same effect as before.

```
ct = make_column_transformer(
    (ohe, select_object),
    ('passthrough', exclude_object))
ct.fit_transform(X)
```

```
array([[ 0.    ,  0.    ,  1.    ,  0.    ,  1.    ,  0.    ,  7.25   ],
       [ 1.    ,  0.    ,  0.    ,  1.    ,  0.    ,  0.    , 71.2833],
       [ 0.    ,  0.    ,  1.    ,  1.    ,  0.    ,  0.    ,  7.925   ],
       [ 0.    ,  0.    ,  1.    ,  1.    ,  0.    ,  0.    , 53.1    ],
       [ 0.    ,  0.    ,  1.    ,  0.    ,  1.    ,  0.    ,  8.05   ],
       [ 0.    ,  1.    ,  0.    ,  0.    ,  1.    ,  0.    ,  8.4583],
       [ 0.    ,  0.    ,  1.    ,  0.    ,  1.    ,  0.    , 51.8625],
       [ 0.    ,  0.    ,  1.    ,  0.    ,  1.    ,  1.    , 21.075   ],
       [ 0.    ,  0.    ,  1.    ,  1.    ,  0.    ,  2.    , 11.1333],
       [ 1.    ,  0.    ,  0.    ,  1.    ,  0.    ,  0.    , 30.0708]])
```

There are also other data type options you can use, such as the datetime data type or the pandas category data type.

```
select_datetime = make_column_selector(dtype_include='datetime')
select_category = make_column_selector(dtype_include='category')
```

Finally, it's worth noting that you can also pass a list of multiple data types to `make_column_selector`.

```
select_multiple = make_column_selector(dtype_include=[object,
                                                    'category'])
```

4.8 Q&A: How do I select columns by column name pattern?

Let's say that we had a lot of columns, and all of the columns that we wanted to select for a particular transformation had the same pattern in their names. For example, perhaps the desired columns all started with the same word.

Once again, we can use the `make_column_selector` function, which also allows us to select columns by regular expression pattern. Here's a silly example in which we select columns that include the capital letters E or S.

```
select_ES = make_column_selector(pattern='E|S')
```

When we run the `fit_transform` method, Embarked and Sex have been one-hot encoded, and the remaining columns have been passed through.

```
ct = make_column_transformer(
    (ohe, select_ES),
    remainder='passthrough')
ct.fit_transform(X)
```



```
array([[ 0.    ,  0.    ,  1.    ,  0.    ,  1.    ,  0.    ,  7.25   ],
       [ 1.    ,  0.    ,  0.    ,  1.    ,  0.    ,  0.    , 71.2833],
       [ 0.    ,  0.    ,  1.    ,  1.    ,  0.    ,  0.    ,  7.925  ],
       [ 0.    ,  0.    ,  1.    ,  1.    ,  0.    ,  0.    , 53.1    ],
       [ 0.    ,  0.    ,  1.    ,  0.    ,  1.    ,  0.    ,  8.05   ],
       [ 0.    ,  1.    ,  0.    ,  0.    ,  1.    ,  0.    ,  8.4583],
       [ 0.    ,  0.    ,  1.    ,  0.    ,  1.    ,  0.    , 51.8625],
       [ 0.    ,  0.    ,  1.    ,  0.    ,  1.    ,  1.    , 21.075  ],
       [ 0.    ,  0.    ,  1.    ,  1.    ,  0.    ,  2.    , 11.1333],
       [ 1.    ,  0.    ,  0.    ,  1.    ,  0.    ,  0.    , 30.0708]])
```

Again, this is only useful if your column names follow a particular pattern and you know how to write regular expressions.

4.9 Q&A: Should I use ColumnTransformer or make_column_transformer?

So far in this book, we’ve been creating ColumnTransformers using the `make_column_transformer` function. In this lesson, I’ll show you how to use the `ColumnTransformer` class and then compare it to `make_column_transformer` so that you can decide which one you want to use.

To start, we’ll import the `ColumnTransformer` class from the `compose` module, and then we’ll create an instance.

When creating an instance, the first difference you might notice is that the tuples have three elements rather than two. The first element of each tuple is a name (of your choosing) that you assign to the transformer.

In this case, the first tuple is our one-hot encoding of Embarked and Sex, and we’re assigning it the name “OHE”. The second tuple is our special passthrough transformer for Parch and Fare, and we’re assigning it the name “pass”. We can see these names when we print out the `ColumnTransformer`.

You might also notice that the tuples are in a list, which is a requirement of the `ColumnTransformer` class.

```
from sklearn.compose import ColumnTransformer
ct = ColumnTransformer(
    [('OHE', ohe, ['Embarked', 'Sex']),
     ('pass', 'passthrough', ['Parch', 'Fare'])])
ct
```

```
ColumnTransformer(transformers=[('OHE', OneHotEncoder(), ['Embarked', 'Sex']),
                               ('pass', 'passthrough', ['Parch', 'Fare'])])
```

Tuple elements for ColumnTransformer:

1. Transformer name

2. Transformer object or 'drop' or 'passthrough'
3. List of columns to which the transformer should be applied

Now let's create the same `ColumnTransformer` using the `make_column_transformer` function. When using `make_column_transformer`, we don't define names for the transformers. Instead, each transformer is assigned a default name, which is the lowercase version of the transformer's class name.

As you can see when we print it out, the one-hot encoder is assigned the name "onehotencoder", and the passthrough transformer is assigned the name "passthrough".

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    ('passthrough', ['Parch', 'Fare']))
ct
```

```
ColumnTransformer(transformers=[('onehotencoder', OneHotEncoder(),
                                ['Embarked', 'Sex']),
                                ('passthrough', 'passthrough',
                                ['Parch', 'Fare'])])
```

All of that being said, which one should you use?

I prefer `make_column_transformer`, because I find the code both easier to read and easier to write, so that's what I'll use in this book. I usually don't mind the default transformer names, and in fact I like that I don't have to come up with a name for each transformer.

However, there are times when defining names for the transformers is useful. Custom names can be clearer if you're performing a grid search of transformer parameters, or if you're using the same type of transformer multiple times in the same `ColumnTransformer` instance. We'll see examples of this later in the book.

One final note is that the `ColumnTransformer` class enables transformer weights, meaning you can emphasize the output of some transformers more than others. The specific use case of this is not yet clear to me, but if you do decide to use transformer weights, then you can't use the `make_column_transformer` function and you must use the `ColumnTransformer` class.

	<code>ColumnTransformer</code>	<code>make_column_transformer</code>
Allows custom names?	Yes	No
Allows transformer weights?	Yes	No

4.10 Q&A: Should I use Pipeline or make_pipeline?

So far in this book, we've been creating Pipelines using the `make_pipeline` function. In this lesson, I'll show you how to use the `Pipeline` class and then compare it to `make_pipeline` so that you can

decide which one you want to use.

To start, we'll import the Pipeline class from the pipeline module, and then we'll create an instance.

When creating an instance, the main difference you might notice is that we're passing in a list of tuples to the Pipeline constructor. Each tuple has two elements, in which the first element is the name you're assigning to the Pipeline step, and the second element is the model or transformer you're including in the Pipeline.

In this case, the first tuple is our preprocessing step using ColumnTransformer, and we're assigning it the name "preprocessor". The second tuple is our model building step using logistic regression, and we're assigning it the name "classifier". We can see these names when we print out the Pipeline.

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([('preprocessor', ct), ('classifier', logreg)])
pipe
```

```
Pipeline(steps=[('preprocessor',
                  ColumnTransformer(transformers=[('onehotencoder',
                                                    OneHotEncoder(),
                                                    ['Embarked', 'Sex']),
                                                    ('passthrough', 'passthrough',
                                                    ['Parch', 'Fare'])])),
                ('classifier',
                 LogisticRegression(random_state=1, solver='liblinear'))])
```

Tuple elements for Pipeline:

1. Step name
2. Model or transformer object

We can also see the step names by accessing the named_steps attribute of the Pipeline and running the keys method.

```
pipe.named_steps.keys()
```

```
dict_keys(['preprocessor', 'classifier'])
```

Now let's create the same Pipeline using the make_pipeline function. When using make_pipeline, we don't define names for the steps. Instead, each step is assigned a default name, which is the lowercase version of the step's class name.

As you can see when we print it out, the first step is assigned the name "columntransformer", and the second step is assigned the name "logisticregression".

```
pipe = make_pipeline(ct, logreg)
pipe
```

```
Pipeline(steps=[('columntransformer',
                  ColumnTransformer(transformers=[('onehotencoder',
                                                    OneHotEncoder(),
                                                    ['Embarked', 'Sex']),
                                                    ('passthrough', 'passthrough',
                                                    ['Parch', 'Fare'])])),
                ('logisticregression',
                  LogisticRegression(random_state=1, solver='liblinear'))])
```

Again, we can also see the step names by accessing the `named_steps` attribute.

```
pipe.named_steps.keys()
```

```
dict_keys(['columntransformer', 'logisticregression'])
```

All of that being said, which one should you use?

I prefer `make_pipeline`, because I find the code both easier to read and easier to write, so that's what I'll use in this book. I usually don't mind the default step names, and in fact I like that I don't have to come up with a name for each step.

However, custom step names can be useful for clarity, especially if you're performing a grid search of a Pipeline. We'll see many examples of this later in the book.

	Pipeline	make_pipeline
Allows custom names?	Yes	No

4.11 Q&A: How do I examine the steps of a Pipeline?

Sometimes you might want to examine the steps of a fitted Pipeline so that you can understand what's happening within each step. In this lesson, I'll show you how to do that.

We'll start by fitting the Pipeline, which prints out the two steps.

```
pipe.fit(X, y)
```

```
Pipeline(steps=[('columntransformer',
                  ColumnTransformer(transformers=[('onehotencoder',
                                                    OneHotEncoder(),
                                                    ['Embarked', 'Sex']),
                                                    ('passthrough', 'passthrough',
                                                    ['Parch', 'Fare'])])),
                ('logisticregression',
                  LogisticRegression(random_state=1, solver='liblinear'))])
```

As I mentioned in the previous lesson, `make_pipeline` assigned a name to each step, which is the lowercase version of the step's class name. In this case, our step names are “columntransformer” and “logisticregression”.

```
pipe.named_steps.keys()
```

```
dict_keys(['columntransformer', 'logisticregression'])
```

To examine an individual step, you select the `named_steps` attribute and pass the step name in brackets. (Note that if we had assigned custom step names such as “preprocessor” and “classifier”, we would be using those here instead.)

```
pipe.named_steps['columntransformer']
```

```
ColumnTransformer(transformers=[('onehotencoder', OneHotEncoder(),
                                  ['Embarked', 'Sex']),
                                  ('passthrough', 'passthrough',
                                  ['Parch', 'Fare'])])
```

```
pipe.named_steps['logisticregression']
```

```
LogisticRegression(random_state=1, solver='liblinear')
```

Once you've accessed a step, you can examine its attributes or run its methods. For example, we can run the `get_feature_names` method from the “columntransformer” step to learn the names of each feature. As a reminder, the `x0` means feature 0 that was passed to the `OneHotEncoder`, and the `x1` means feature 1.

```
pipe.named_steps['columntransformer'].get_feature_names()
```

```
['onehotencoder__x0_C',
 'onehotencoder__x0_Q',
 'onehotencoder__x0_S',
 'onehotencoder__x1_female',
 'onehotencoder__x1_male',
 'Parch',
 'Fare']
```

We can also see the coefficient values of the 7 features by examining the `coef_` attribute of the “logisticregression” step. These coefficients are listed in the same order as the features, though the intercept is stored in a separate attribute.

By finding the 4 positive coefficients, you can determine that embarking at port C, being female, and having a higher Parch and Fare are all associated with a greater likelihood of survival. Note that these are just associations the model learned from 10 rows of training data. They are not necessarily statistically significant associations, and in fact scikit-learn does not provide p-values.

```
pipe.named_steps['logisticregression'].coef_
```

```
array([[ 0.26491287, -0.19848033, -0.22907928,  1.0075062 , -1.17015293,
         0.20056557,  0.01597307]])
```

Finally, it’s worth noting that there are three other ways that you can examine the steps of a Pipeline:

- First, you can use `named_steps` with periods.
- Second, you can exclude the `named_steps` attribute entirely.
- Third, you can reference the step by position rather than by name.

```
pipe.named_steps.logisticregression.coef_
```

```
array([[ 0.26491287, -0.19848033, -0.22907928,  1.0075062 , -1.17015293,
         0.20056557,  0.01597307]])
```

```
pipe['logisticregression'].coef_
```

```
array([[ 0.26491287, -0.19848033, -0.22907928,  1.0075062 , -1.17015293,
         0.20056557,  0.01597307]])
```

```
pipe[1].coef_
```

```
array([[ 0.26491287, -0.19848033, -0.22907928,  1.0075062 , -1.17015293,
         0.20056557,  0.01597307]])
```

Personally, I like the initial bracket notation because I think it’s the most readable, even though it requires the most typing. However, using `named_steps` with the periods seems to be the only option that supports the autocompletion of both the step name and the attribute, which is a nice benefit.

5 Workflow review #1

5.1 Recap of our workflow

In this chapter, we're going to review the workflow that we've built so far to make sure you understand the key concepts before we start adding additional complexity.

To start, we're going to walk through all of the code that is necessary to recreate our workflow up to this point. We begin by importing pandas, the OneHotEncoder and LogisticRegression classes, and the make_column_transformer and make_pipeline functions.

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline
```

Next, we create a list of the four columns we're going to select from our data.

```
cols = ['Parch', 'Fare', 'Embarked', 'Sex']
```

Then, we read in 10 rows of training data and use it to define our X and y.

```
df = pd.read_csv('http://bit.ly/MLtrain', nrows=10)
X = df[cols]
y = df['Survived']
```

We read in 10 rows of new data and use it to define X_new.

```
df_new = pd.read_csv('http://bit.ly/MLnewdata', nrows=10)
X_new = df_new[cols]
```

We create an instance of OneHotEncoder, which is our only transformer at this point.

```
ohe = OneHotEncoder()
```

Then we build the ColumnTransformer, which one-hot encodes Embarked and Sex and passes through Parch and Fare.

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    ('passthrough', ['Parch', 'Fare']))
```

We also create an instance of `LogisticRegression`.

```
logreg = LogisticRegression(solver='liblinear', random_state=1)
```

Finally, we create a two-step Pipeline, fit the Pipeline to `X` and `y`, and use the fitted Pipeline to make predictions on `X_new`.

```
pipe = make_pipeline(ct, logreg)
pipe.fit(X, y)
pipe.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0])
```

That's really all the code we need to recreate our entire workflow from the last few chapters. You'll notice that there are no calls to `fit_transform` or `transform` because all of that functionality is encapsulated by the Pipeline.

I did exclude cross-validation from this recap because, as mentioned previously, any model evaluation procedure is highly unreliable with only 10 rows of data. However, we will thoroughly explore the topic of model evaluation later in the book, once we're using the full dataset.

5.2 Comparing ColumnTransformer and Pipeline

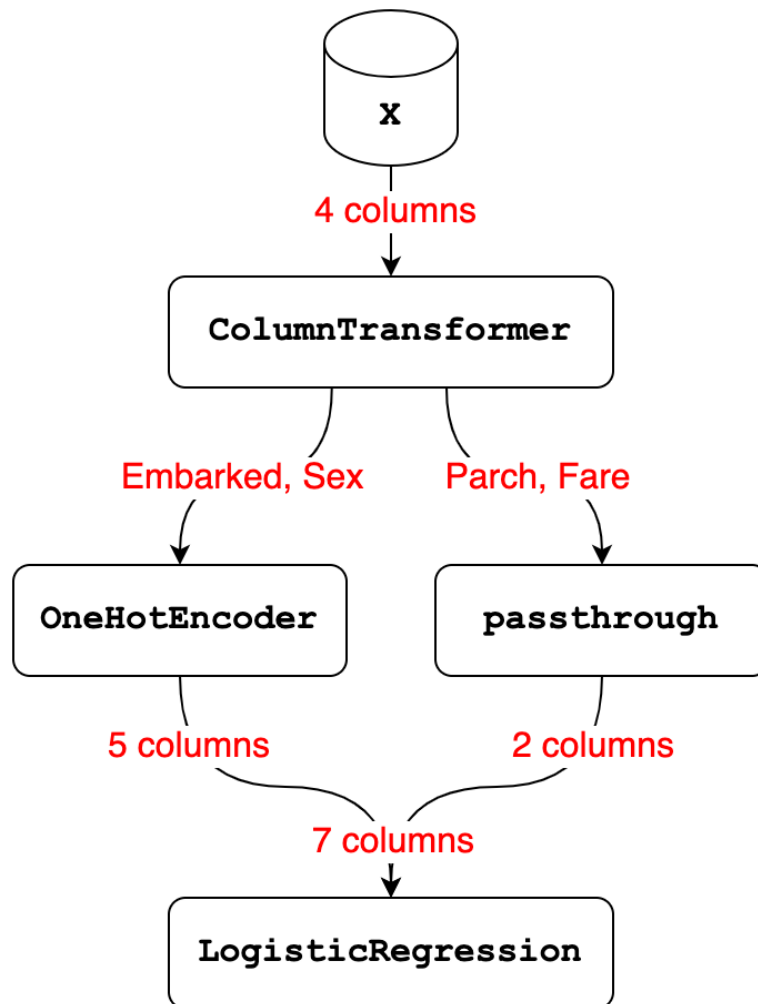
In order to be successful with the rest of this book, it's very important that you clearly understand the differences between a `ColumnTransformer` and a `Pipeline`. In this lesson, I'm going to explain and illustrate those differences.

Let's start with the `ColumnTransformer`, which received 4 columns of input from the `X` DataFrame:

- It selected 2 of those columns, namely `Embarked` and `Sex`, and used the `OneHotEncoder` to transform them into 5 columns.
- It selected the other 2 columns, namely `Parch` and `Fare`, and did nothing to them, which resulted in 2 columns.
- Finally, it stacked the 5 columns output by `OneHotEncoder` and the 2 columns output by the passthrough transformer side-by-side, resulting in a total of 7 columns.

Now let's talk about the `Pipeline`, which has 2 steps:

- Step 1 is a `ColumnTransformer` that received 4 columns of input and transformed them into 7 columns.
- Step 2 is a `LogisticRegression` model that received 7 columns of input and used those 7 columns either for fitting or predicting.



With those examples in mind, we can step back and summarize the differences between a `ColumnTransformer` and a `Pipeline`:

- A `ColumnTransformer` pulls out subsets of columns and transforms them independently, and then stacks the results side-by-side.
- It only ever does data transformations, meaning your `ColumnTransformer` will never include a model.
- It does not have steps, because each subset of columns is transformed independently. In other words, data does not flow from one transformer to the next.

In contrast:

- A `Pipeline` is a series of steps that occur in order, and the output of each step becomes the input to the next step.
- Thus if you had a 3-step `Pipeline`, the output of step 1 becomes the input to step 2, and the output of step 2 becomes the input to step 3.
- The last step of a `Pipeline` can be a model or a transformer, whereas all other steps must be transformers.

ColumnTransformer vs Pipeline:

- **ColumnTransformer:**
 - Selects subsets of columns, transforms them independently, stacks the results side-by-side
 - Only includes transformers
 - Does not have steps (transformers operate in parallel)
- **Pipeline:**
 - Series of steps that occur in order
 - Output of each step becomes the input to the next step
 - Last step is a model or transformer, all other steps are transformers

In short, a Pipeline contains steps that operate in sequence, whereas a ColumnTransformer contains transformers that operate in parallel. In later chapters, you'll see why this difference is so important and how it guides the structure of our workflow.

5.3 Creating a Pipeline diagram

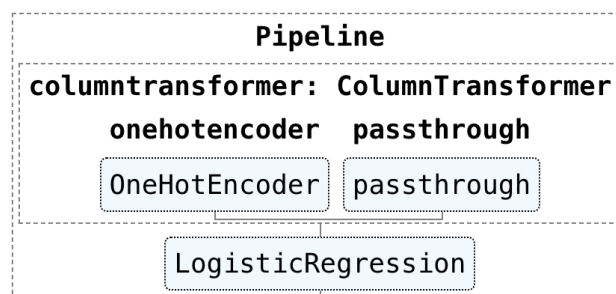
To wrap up this chapter, I want to show you a feature that's new in scikit-learn version 0.23 that can help you to visualize and thus better understand your Pipeline.

To start, we'll import the `set_config` function. Then we run it and set the `display` parameter to `'diagram'`.

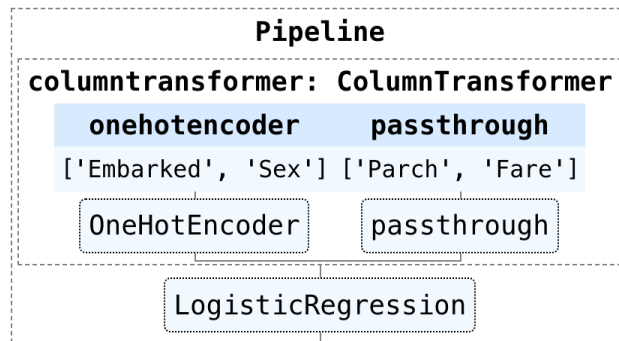
```
from sklearn import set_config
set_config(display='diagram')
```

With that configuration, you'll see a diagram any time you print out a Pipeline (or any other estimator) within Jupyter.

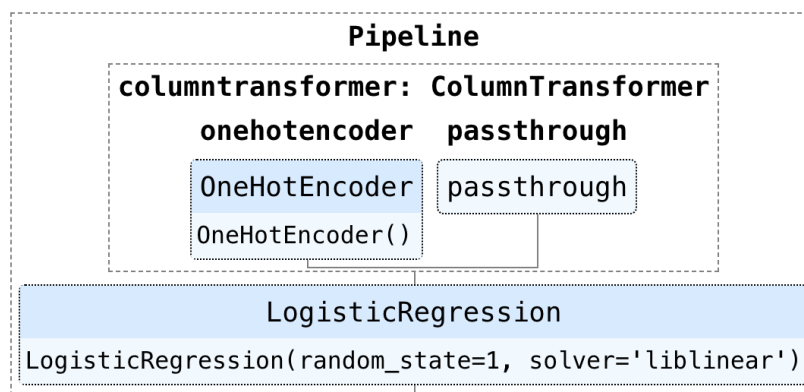
```
pipe
```



This is similar to the diagram I created in the previous lesson, except you can click on any element in order to see more details. For example, if you click on the transformer names, you can see the columns they're transforming.



If you click on the class names, you can see any parameters that have been changed from their default values.



If you use this configuration but you ever need to see the regular text output, you can just use the print function with your Pipeline.

```
print(pipe)
```

```
Pipeline(steps=[('columntransformer',
                  ColumnTransformer(transformers=[('onehotencoder',
                                                    OneHotEncoder(),
                                                    ['Embarked', 'Sex']),
                                                    ('passthrough', 'passthrough',
                                                    ['Parch', 'Fare'])])),
                ('logisticregression',
                  LogisticRegression(random_state=1, solver='liblinear'))])
```

Finally, it's worth noting that displaying diagrams is the default starting in scikit-learn version 1.1. If you'd prefer to always see the text output, you can change the configuration by setting the display parameter to 'text'.

```
set_config(display='text')
pipe
```

```

Pipeline(steps=[('columntransformer',
                  ColumnTransformer(transformers=[('onehotencoder',
                                                    OneHotEncoder(),
                                                    ['Embarked', 'Sex']),
                                                    ('passthrough', 'passthrough',
                                                    ['Parch', 'Fare'])])),
                ('logisticregression',
                  LogisticRegression(random_state=1, solver='liblinear'))])

```

6 Encoding text data

6.1 Vectorizing text

So far in this book, we've only focused on numerical and categorical features. In this chapter, we'll learn how to create features from unstructured text data.

Let's take another look at our Titanic DataFrame, this time focusing on the Name column.

We want to include Name in our model in case it contains predictive information about the likelihood of survival. For example, maybe their last name is predictive of survival if they're part of an important family, and maybe certain titles (such as "Mrs.") are also predictive.

The Name column can't be passed directly to the model because it's not numeric. In this lesson, we'll learn how to encode it numerically, and in the next lesson we'll add it to the `ColumnTransformer` and `Pipeline`.

```
df[['Name']]
```

	Name
0	Braund, Mr. Owen Harris
1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)
2	Heikkinen, Miss. Laina
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)
4	Allen, Mr. William Henry
5	Moran, Mr. James
6	McCarthy, Mr. Timothy J
7	Palsson, Master. Gosta Leonard
8	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
9	Nasser, Mrs. Nicholas (Adele Achem)

One idea for encoding would be to simply one-hot encode the Name column. However, `OneHotEncoder` would treat each full name as its own category. This is unlikely to be useful, since we don't expect to see any full name repeated more than once.

Instead, we want to consider each word in their name independently, so that we can learn different things from each part of their name. The `CountVectorizer` class was built for this purpose, so that's what we'll use. In brief, `CountVectorizer` converts text into a matrix of token counts, and shortly you'll see exactly what that means.

Ideas for encoding the Name column:

- **OneHotEncoder:** Each full name is treated as a category (not recommended)
- **CountVectorizer:** Each word in a name is treated independently (recommended)

To start, we'll import `CountVectorizer` from the `feature_extraction` module and create an instance called `vect`.

```
from sklearn.feature_extraction.text import CountVectorizer
vect = CountVectorizer()
```

Then we'll pass the `Name` to the `fit_transform` method of `vect`. Notice that we're using single brackets around `Name` to pass it as a `Series`, which is because `CountVectorizer` expects 1-dimensional input. This is unlike `OneHotEncoder` and most other transformers, which expect 2-dimensional input.

The `fit_transform` method outputs what is called a document-term matrix, which we save as `dtm`. When we print it out, you can see that it's a sparse matrix containing 10 rows and 40 columns. There's one row for each of the 10 names, and one column for each of the 40 features it created.

```
dtm = vect.fit_transform(df['Name'])
dtm
```

```
<10x40 sparse matrix of type '<class 'numpy.int64'>'
  with 46 stored elements in Compressed Sparse Row format>
```

CountVectorizer vs other transformers:

- **CountVectorizer:** 1-dimensional input (`Series`)
- **Other transformers:** 2-dimensional input (`DataFrame`)

Let's examine the names of the 40 feature columns it created by running the `get_feature_names` method. (Note that this has been replaced by the `get_feature_names_out` method starting in scikit-learn 1.0.)

These are the 40 unique words that were found in the `Name` column after making them lowercase, removing all punctuation, and excluding words that were only 1 character long (per the default settings for `CountVectorizer`).

Note that the features are sorted alphabetically, which is the same convention used by the `OneHotEncoder` when learning category names.

```
print(vect.get_feature_names())
```

```
[achem, adele, allen, berg, ..., thayer, timothy, vilhelmina, william]
```

Default settings for CountVectorizer:

- Convert all words to lowercase
- Remove all punctuation
- Exclude one-character words

Let's quickly summarize what we know about the document-term matrix it created:

- There are 10 rows and 40 columns.
- Each row represents a row from the training data, and each column represents a word.
- The rows are known as documents, and the feature names are known as terms, which is why it's called a document-term matrix.
- It's stored as a sparse matrix.

About the document-term matrix:

- 10 rows and 40 columns
- Rows represent rows from training data, columns represent words
- Rows are “documents”, feature names are “terms”
- Sparse matrix

We want to examine the document-term matrix to better understand it. Unlike `OneHotEncoder`, `CountVectorizer` does not have a `sparse=False` argument that we can set in order to view the dense representation. Instead, we'll use the `toarray` method to make it dense and then convert that into a `DataFrame`, using the feature names as the column headings.

How to examine a document-term matrix:

1. Use `toarray` method to make it dense
2. Convert dense matrix into a `DataFrame`
3. Use feature names as column headings

```
pd.DataFrame(dtm.toarray(), columns=vect.get_feature_names())
```

	achem	adele	allen	berg	bradley	braund	briggs	cummings	elisabeth	florence
0	0	0	0	0	0	1	0	0	0	0
1	0	0	0	0	1	0	1	1	0	1
2	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0
4	0	0	1	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0
8	0	0	0	1	0	0	0	0	1	0
9	1	1	0	0	0	0	0	0	0	0

What can we learn by examining this? If we compare the document-term matrix to the `Name` column, we see that in each row, `CountVectorizer` counted how many times each word appeared.

Let's take the first row as an example. Using the `head` method, we can see that the name in the first row of training data was “Braund, Mr. Owen Harris”. Thus the first row of the document-term matrix contains 4 ones (under “braund”, “mr”, “owen”, and “harris”), and the other 36 entries are all zeros.

```
df[['Name']].head(1)
```

	Name
0	Braund, Mr. Owen Harris

This encoding is known as the “Bag of Words” representation of text data. It’s known as a “bag” because this encoding doesn’t capture the word ordering present in each name, rather it only captures the count of how many times a word appears in each name.

“Bag of Words” representation:

- Ignores word order
- Only counts how many times a word appears

This representation is the feature matrix that will get passed to the model. Just like the `OneHotEncoder` created a 10 by 3 matrix from `Embarked`, `CountVectorizer` created this 10 by 40 matrix from `Name`.

From each of the 40 features, the model can learn the relationship between the target value and how many times that word appeared in each passenger’s name. Thus if a particular word is predictive of survival, such as “Braund”, the model will be able to learn that from the matrix.

6.2 Including text data in the model

Now that we know how to encode text data, we’re ready to include it in our model.

We’ll start by updating the `cols` list to include the `Name` column, and then use `cols` to update the `X` `DataFrame`.

```
cols = ['Parch', 'Fare', 'Embarked', 'Sex', 'Name']
X = df[cols]
X
```

	Parch	Fare	Embarked	Sex	Name
0	0	7.2500	S	male	Braund, Mr. Owen Harris
1	0	71.2833	C	female	Cumings, Mrs. John Bradley (Flo...
2	0	7.9250	S	female	Heikkinen, Miss. Laina
3	0	53.1000	S	female	Futrelle, Mrs. Jacques Heath (L...
4	0	8.0500	S	male	Allen, Mr. William Henry
5	0	8.4583	Q	male	Moran, Mr. James
6	0	51.8625	S	male	McCarthy, Mr. Timothy J
7	1	21.0750	S	male	Palsson, Master. Gosta Leonard
8	2	11.1333	S	female	Johnson, Mrs. Oscar W (Elisabet...
9	0	30.0708	C	female	Nasser, Mrs. Nicholas (Adele Ac...

Next, we’ll update the `ColumnTransformer` by adding one more tuple to specify that the `CountVectorizer` should be applied to `Name`.

Notice that there are no brackets around `'Name'`. This is not because there’s only one column being passed to `CountVectorizer`. Instead, it’s because `CountVectorizer` expects 1-dimensional input, and brackets would signal 2-dimensional input to the `ColumnTransformer`.


```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    ('passthrough', ['Parch', 'Fare']))
```

Next, we'll use the `fit_transform` method to try out the transformation. The output contains 47 columns.

```
ct.fit_transform(X)
```

```
<10x47 sparse matrix of type '<class 'numpy.float64'>'
  with 78 stored elements in Compressed Sparse Row format>
```

We can confirm with `get_feature_names` the meaning of these 47 columns: 3 columns for Embarked, 2 columns for Sex, 40 columns for Name, 1 column for Parch, and 1 column for Fare. Again, the features are in this order because that's the order in which they were passed to the `ColumnTransformer`.

```
ct.get_feature_names()
```

```
['onehotencoder__x0_C',
 'onehotencoder__x0_Q',
 'onehotencoder__x0_S',
 'onehotencoder__x1_female',
 'onehotencoder__x1_male',
 'countvectorizer__achem',
 'countvectorizer__adele',
 'countvectorizer__allen',
 'countvectorizer__berg',
 'countvectorizer__bradley',
 'countvectorizer__braund',
 'countvectorizer__briggs',
 'countvectorizer__cumings',
 'countvectorizer__elisabeth',
 'countvectorizer__florence',
 'countvectorizer__futrelle',
 'countvectorizer__gosta',
 'countvectorizer__harris',
 'countvectorizer__heath',
 'countvectorizer__heikkinen',
 'countvectorizer__henry',
 'countvectorizer__jacques',
 'countvectorizer__james',
 'countvectorizer__john',
 'countvectorizer__johnson',
```

```

'countvectorizer__laina',
'countvectorizer__leonard',
'countvectorizer__lily',
'countvectorizer__master',
'countvectorizer__may',
'countvectorizer__mccarthy',
'countvectorizer__miss',
'countvectorizer__moran',
'countvectorizer__mr',
'countvectorizer__mrs',
'countvectorizer__nasser',
'countvectorizer__nicholas',
'countvectorizer__oscar',
'countvectorizer__owen',
'countvectorizer__palsson',
'countvectorizer__peel',
'countvectorizer__thayer',
'countvectorizer__timothy',
'countvectorizer__vilhelmina',
'countvectorizer__william',
'Parch',
'Fare']

```

Output columns:

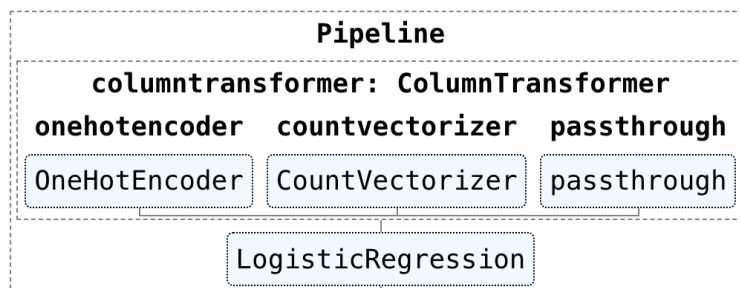
- **Columns 1-3:** Embarked
- **Columns 4-5:** Sex
- **Columns 6-45:** Name
- **Column 46:** Parch
- **Column 47:** Fare

We'll update the Pipeline to use the modified ColumnTransformer.

```
pipe = make_pipeline(ct, logreg)
```

Then we can fit the Pipeline, which displays our Pipeline diagram.

```
pipe.fit(X, y)
```



Our last step before prediction is to update `X_new` to include the `Name` column.

```
X_new = df_new[cols]
```

Finally, we'll use the fitted Pipeline to make predictions for `X_new`.

```
pipe.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0])
```

We've now accomplished our goal, which is to include the `Name` column in our model.

6.3 Q&A: Why is the document-term matrix stored as a sparse matrix?

Just like `OneHotEncoder`, `CountVectorizer` outputs a sparse matrix by default. To explore why it does this, let's create a Python list of two short text documents and call it `text`.

```
text = ['Machine Learning is fun', 'I am learning Machine Learning']
```

We'll pass those documents to the `fit_transform` method of `CountVectorizer`, make it dense with the `toarray` method, and then convert it to a `DataFrame`. As you can see, the word "I" was ignored because it's only one character, and there's a 2 under "learning" because that word appears twice in the second document.

```
pd.DataFrame(vect.fit_transform(text).toarray(),
             columns=vect.get_feature_names())
```

	am	fun	is	learning	machine
0	0	1	1	1	1
1	1	0	0	2	1

Now let's use `CountVectorizer` on the same text to output a sparse matrix instead, which is the default representation. The matrix is 2 rows by 5 columns, and there are 7 stored elements, meaning 7 non-zero values in the matrix.

```
dtm = vect.fit_transform(text)
dtm
```

```
<2x5 sparse matrix of type '<class 'numpy.int64'>'
  with 7 stored elements in Compressed Sparse Row format>
```

We can actually see those 7 elements by using the `print` function. It turns out that a sparse matrix only stores the positions of the non-zero values and the values at those positions. In contrast, a dense matrix stores every value, whether or not it's zero.

```
print(dtm)
```

```
(0, 4)    1
(0, 3)    1
(0, 2)    1
(0, 1)    1
(1, 4)    1
(1, 3)    2
(1, 0)    1
```

As you might imagine, most elements in a typical document-term matrix are zero. This is because a collection of documents tends to have a large number of unique words, whereas any given document in that collection only contains a small fraction of those words.

When most elements in a matrix are zero, a sparse representation requires far less storage space than a dense representation and is also more performant, which is why it's the default for `CountVectorizer`. The output of `OneHotEncoder` also tends to be mostly zeros, and thus a sparse matrix is its default as well.

Preferred matrix representation:

- **Most elements are zero:** Sparse matrix
- **Most elements are non-zero:** Dense matrix

6.4 Q&A: What happens if the testing data includes new words?

In the previous lesson, we created two short text documents.

```
text
```

```
['Machine Learning is fun', 'I am learning Machine Learning']
```

Let's pretend that those documents were our training data. If we passed those documents to the `fit_transform` method of `CountVectorizer`, there are 5 features that would be learned.

```
dtm = vect.fit_transform(text)
vect.get_feature_names()
```

```
['am', 'fun', 'is', 'learning', 'machine']
```

Now, let's create another short text document to act as our testing data. It includes two words, “is” and “fun”, that were in the training data, and two words, “data” and “science”, that were not in the training data.

```
text_new = ['Data Science is FUN!']
```

As we've discussed throughout this book, your testing data needs to have the same columns as your training data. In other words, we need to create a document-term matrix from the testing data that has the same 5 columns as the training data.

To do this, we'll pass the testing data to the `transform` method, and `CountVectorizer` will build a matrix using the features it learned during the fit step. In other words, the vectorizer learned its vocabulary from the training data, and it used that same vocabulary when creating the document-term matrix for the testing data.

```
vect.transform(text_new).toarray()
```

```
array([[0, 1, 1, 0, 0]])
```

CountVectorizer methods:

- **fit:** Learn the vocabulary
- **transform:** Create the document-term matrix using that vocabulary

As you can see by comparing the output to the feature names, the vectorizer only learned the words “fun” and “is” from the testing data. The words “data” and “science” were ignored because they were not seen in the training data.

Ignoring unknown words actually makes intuitive sense, because if a word wasn't seen during training, then you don't know anything about the relationship between that word and the target variable. This is similar to setting the `OneHotEncoder`'s `handle_unknown` parameter to 'ignore', since that ignores unknown categories encountered during the transform step by encoding them as all zeros.

6.5 Q&A: How do I vectorize multiple columns of text?

Let's take a look at the Name and Ticket columns from the Titanic DataFrame. Even though some of the Ticket values are entirely numeric, they're actually all stored as strings.

```
df[['Name', 'Ticket']]
```

	Name	Ticket
0	Braund, Mr. Owen Harris	A/5 21171
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	PC 17599
2	Heikkinen, Miss. Laina	STON/O2. 3101282
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	113803
4	Allen, Mr. William Henry	373450
5	Moran, Mr. James	330877
6	McCarthy, Mr. Timothy J	17463
7	Palsson, Master. Gosta Leonard	349909
8	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	347742
9	Nasser, Mrs. Nicholas (Adele Achem)	237736

If we wanted to apply `CountVectorizer` to both columns in order to include both in our model, how would we do it?

First, let's try applying `CountVectorizer` to the columns separately. Vectorizing `Name` creates a 10 by 40 matrix, and vectorizing `Ticket` creates a 10 by 13 matrix.

```
vect.fit_transform(df['Name'])
```

```
<10x40 sparse matrix of type '<class 'numpy.int64'>'
  with 46 stored elements in Compressed Sparse Row format>
```

```
vect.fit_transform(df['Ticket'])
```

```
<10x13 sparse matrix of type '<class 'numpy.int64'>'
  with 13 stored elements in Compressed Sparse Row format>
```

What we want is to stack these matrices side-by-side as a 10 by 53 matrix.

One idea would be to pass both columns as a `DataFrame` to `CountVectorizer`, which is how we would transform multiple columns with `OneHotEncoder`. However, you'll see that the output is a 2 by 2 matrix, which is not what we had hoped for. This is because `CountVectorizer` expects 1-dimensional input, and we passed it 2-dimensional input instead.

```
vect.fit_transform(df[['Name', 'Ticket']])
```

```
<2x2 sparse matrix of type '<class 'numpy.int64'>'
  with 2 stored elements in Compressed Sparse Row format>
```

To actually get the result we're looking for, we have to pass each column separately to `make_column_transformer`. You'll see that it does indeed output a 10 by 53 matrix.

To be clear, it's not problematic to use the same `CountVectorizer` object twice, since it will still learn two separate vocabularies.

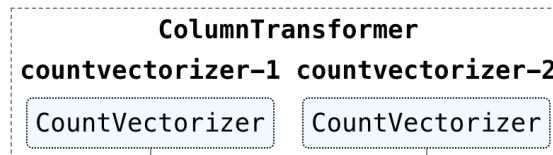
```
ct = make_column_transformer(
    (vect, 'Name'),
    (vect, 'Ticket'))
```

```
ct.fit_transform(df)
```

```
<10x53 sparse matrix of type '<class 'numpy.int64'>'
  with 59 stored elements in Compressed Sparse Row format>
```

Recall that `make_column_transformer` assigns names to all transformers. Normally the assigned name would be “countvectorizer”, but it can’t give both transformers the same name. Instead, it appends numbers at the end, as you can see in the diagram.

```
ct
```



You can also see these names by running the `keys` method on the `named_transformers_` attribute.

```
ct.named_transformers_.keys()
```

```
dict_keys(['countvectorizer-1', 'countvectorizer-2', 'remainder'])
```

If you wanted to avoid these names, you could instead create the `ColumnTransformer` using the `ColumnTransformer` class, since that requires you to assign a custom name to each transformer.

6.6 Q&A: Should I one-hot encode or vectorize categorical features?

Let’s say you have categorical features which only contain one word, such as `Embarked` and `Sex`. We’ve been using `OneHotEncoder` to encode them, but should we use `CountVectorizer` instead? Let’s try it out and see what happens.

```
df[['Embarked', 'Sex']]
```

	Embarked	Sex
0	S	male
1	C	female
2	S	female
3	S	female
4	S	male
5	Q	male
6	S	male
7	S	male
8	S	female
9	C	female

If we use `CountVectorizer` on the `Sex` column, it produces the exact same output as the `OneHotEncoder`.

```
vect.fit_transform(df['Sex']).toarray()
```

```
array([[0, 1],
       [1, 0],
       [1, 0],
       [1, 0],
       [0, 1],
       [0, 1],
       [0, 1],
       [0, 1],
       [0, 1],
       [1, 0],
       [1, 0]])
```

But as we saw in the previous lesson, `CountVectorizer` won't do what you expect if you try to encode multiple columns at once, since it expects 1-dimensional input.

```
vect.fit_transform(df[['Embarked', 'Sex']]).toarray()
```

```
array([[1, 0],
       [0, 1]])
```

In addition, the default settings for `CountVectorizer` don't allow for one-character tokens, so you would have to modify those settings if you wanted to use it with `Embarked`.

```
vect.fit_transform(df['Embarked']).toarray()
```

```
ValueError: empty vocabulary; perhaps the documents only contain stop words
```


Finally, `OneHotEncoder` lets you decide how you want to handle categories that weren't seen during training (using the `handle_unknown` parameter), whereas `CountVectorizer` will always ignore words that it didn't see during training.

In summary, `OneHotEncoder` is the better encoding mechanism for any data that you would consider categorical, since it can encode multiple columns at once, it allows one-character category names by default, and it provides more options for handling unknown categories.

Advantages of `OneHotEncoder` for categorical data:

- Encodes multiple columns at once
- Allows one-character category names
- Gives more options for handling unknown categories

7 Handling missing values

7.1 Introduction to missing values

In this chapter, we’re going to deal with an issue that is common in real datasets, namely missing values.

A missing value is simply a value that does not exist in the dataset. It might be missing because that value purposefully wasn’t collected for that sample, or it might be missing due to an error in the data collection process.

Common sources of missing values:

- Value purposefully wasn’t collected
- Error in the data collection process

Let’s see an example in the Titanic dataset. We want to use Age as a feature, but notice that it has a missing value in row 5, encoded as NaN. This stands for “not a number”, and it’s how missing values are typically encoded in a pandas DataFrame.

df

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	3	Bra...	male	22.0	1	0	A/5...	7.2500	NaN	S
1	1	1	Cum...	female	38.0	1	0	PC ...	71....	C85	C
2	1	3	Hei...	female	26.0	0	0	STO...	7.9250	NaN	S
3	1	1	Fut...	female	35.0	1	0	113803	53....	C123	S
4	0	3	All...	male	35.0	0	0	373450	8.0500	NaN	S
5	0	3	Mor...	male	NaN	0	0	330877	8.4583	NaN	Q
6	0	1	McC...	male	54.0	0	0	17463	51....	E46	S
7	0	3	Pal...	male	2.0	3	1	349909	21....	NaN	S
8	1	3	Joh...	female	27.0	0	2	347742	11....	NaN	S
9	1	2	Nas...	female	14.0	1	0	237736	30....	NaN	C

When we use the phrase “missing values”, we’re talking about NaNs only. It does not refer to categories or words in new data that weren’t seen during training. For example, if our new data contained the value “Z” in the Embarked column, that is not called a “missing value”, rather that’s called “an unknown category” or “a category you didn’t see during training”.

Missing values vs unknown categories:

- **Missing value:** Value encoded as NaN
- **Unknown category:** Category not seen in the training data

To start our exploration of this topic, let's see what happens if we try to add the Age column to our model.

First, we'll add Age to the cols list and use that to modify the X DataFrame. Again, notice the missing value in Age.

```
cols = ['Parch', 'Fare', 'Embarked', 'Sex', 'Name', 'Age']
X = df[cols]
X
```

	Parch	Fare	Embarked	Sex	Name	Age
0	0	7.2500	S	male	Braund, Mr. Owen Harris	22.0
1	0	71.2833	C	female	Cumings, Mrs. John Bradley (Flo...	38.0
2	0	7.9250	S	female	Heikkinen, Miss. Laina	26.0
3	0	53.1000	S	female	Futrelle, Mrs. Jacques Heath (L...	35.0
4	0	8.0500	S	male	Allen, Mr. William Henry	35.0
5	0	8.4583	Q	male	Moran, Mr. James	NaN
6	0	51.8625	S	male	McCarthy, Mr. Timothy J	54.0
7	1	21.0750	S	male	Palsson, Master. Gosta Leonard	2.0
8	2	11.1333	S	female	Johnson, Mrs. Oscar W (Elisabet...	27.0
9	0	30.0708	C	female	Nasser, Mrs. Nicholas (Adele Ac...	14.0

Then, we'll add Age to the ColumnTransformer as a passthrough column.

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    ('passthrough', ['Parch', 'Fare', 'Age']))
```

We'll update the Pipeline to include the modified ColumnTransformer.

```
pipe = make_pipeline(ct, logreg)
```

Finally, we'll attempt to fit the Pipeline, but it throws an error due to the presence of a missing value.

```
pipe.fit(X, y)
```

ValueError: Input contains NaN

In general, scikit-learn models don't accept data with missing values, with the exception of histogram-based gradient boosting trees. As such, we'll need to figure out a way to handle the missing value if we want to include Age as a feature in our model.

7.2 Three ways to handle missing values

Let's talk about three different ways we can handle missing values in our dataset.

The first way is to drop any rows from the DataFrame that have missing values. We can use the `dropna` method in pandas to experiment with this, and you'll notice that row 5 is now gone. (Note that you would also need to drop the same row from `y`.)

```
X.dropna()
```

	Parch	Fare	Embarked	Sex	Name	Age
0	0	7.2500	S	male	Braund, Mr. Owen Harris	22.0
1	0	71.2833	C	female	Cumings, Mrs. John Bradley (Flo...	38.0
2	0	7.9250	S	female	Heikkinen, Miss. Laina	26.0
3	0	53.1000	S	female	Futrelle, Mrs. Jacques Heath (L...	35.0
4	0	8.0500	S	male	Allen, Mr. William Henry	35.0
6	0	51.8625	S	male	McCarthy, Mr. Timothy J	54.0
7	1	21.0750	S	male	Palsson, Master. Gosta Leonard	2.0
8	2	11.1333	S	female	Johnson, Mrs. Oscar W (Elisabet...	27.0
9	0	30.0708	C	female	Nasser, Mrs. Nicholas (Adele Ac...	14.0

However, there are three main problems with this approach:

- First, if a high proportion of your rows have missing values, then this approach is impractical because it will discard too much of your training data.
- Second, if there's a useful pattern in the "missingness", then dropping the rows will obscure this pattern.
- Third, this takes care of the training data, but it doesn't help you deal with missing values that might appear in new data.

Approach 1: Drop rows with missing values

- May discard too much training data
- May obscure a pattern in the "missingness"
- Doesn't help you with new data

A second option for handling missing values is to drop any columns that have missing values. Again, we can use the `dropna` method to do this if we set the `axis` parameter to `'columns'`. You can see that row 5 is back, but the Age column is now gone.

```
X.dropna(axis='columns')
```

	Parch	Fare	Embarked	Sex	Name
0	0	7.2500	S	male	Braund, Mr. Owen Harris
1	0	71.2833	C	female	Cumings, Mrs. John Bradley (Flo...
2	0	7.9250	S	female	Heikkinen, Miss. Laina
3	0	53.1000	S	female	Futrelle, Mrs. Jacques Heath (L...
4	0	8.0500	S	male	Allen, Mr. William Henry
5	0	8.4583	Q	male	Moran, Mr. James
6	0	51.8625	S	male	McCarthy, Mr. Timothy J
7	1	21.0750	S	male	Palsson, Master. Gosta Leonard
8	2	11.1333	S	female	Johnson, Mrs. Oscar W (Elisabet...
9	0	30.0708	C	female	Nasser, Mrs. Nicholas (Adele Ac...

The main problem with this approach is that you're discarding a feature that may be useful to your model.

Approach 2: Drop columns with missing values

- May discard useful features

A third option for handling missing values is to impute missing values. Imputation means that you're filling in missing values based on what you know from the non-missing data. Before proceeding with imputation, it's important to carefully consider its costs and benefits:

- The benefit is that you're able to keep more samples and features in your dataset, which may help to improve your model.
- The cost is that you're inserting values that may not match the true, unknown values, which may make your model less reliable.

Approach 3: Impute missing values

- **Benefit:** Keeps more samples and features
- **Cost:** Imputed values may not match the true values

When making this decision, here are some useful factors to consider:

- How important are the particular samples that imputation would allow you to keep?
- How important are the particular features that imputation would allow you to keep?
- What percentage of the values in a feature would need to be imputed?
- Are there samples or features without missing values that are highly correlated with the samples or features with missing values, such that the model wouldn't be negatively affected by dropping those samples or features?
- Is the missingness random, or is there a useful pattern in the missingness that would be lost if those samples or features were dropped?

Factors to consider before imputing:

- How important are the samples?
- How important are the features?
- What percentage of values would need to be imputed?

- Are there other samples or features that contain the same information?
- Is the missingness random?

Imputation is a complex topic, and ultimately I can't tell you whether you should impute missing values in your particular situation. Instead, I'm going to show you how to do imputation in scikit-learn, and you can decide whether or not to do it. In addition, I'll provide some advice for imputation in lesson 7.6.

7.3 Missing value imputation

In this lesson, we're going to perform missing value imputation on the Age column so that we can include it in our model. There are three different imputers that we can use in scikit-learn, but we're going to start with `SimpleImputer` and then I'll show you the others later in the chapter.

First, we'll import `SimpleImputer` from the `impute` module and create an instance called `imp`.

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer()
```

Then, we'll pass the Age column to the `fit_transform` method to perform the imputation. Note that `SimpleImputer`, like most transformers, requires 2-dimensional input, which is why there are double brackets around Age.

```
imp.fit_transform(X[['Age']])
```

```
array([[22.      ],
       [38.      ],
       [26.      ],
       [35.      ],
       [35.      ],
       [28.11111111],
       [54.      ],
       [ 2.      ],
       [27.      ],
       [14.      ]])
```

By default, `SimpleImputer` fills any missing values with the mean of the non-missing values, which is 28.11 in this case. It also supports other imputation strategies, namely the median value, the most frequent value, and a user-defined value. Note that all of these strategies can be applied to numeric features, but only the most frequent and user-defined strategies can be applied to categorical features.

Simple imputation strategies:

- Mean value

- Median value
- Most frequent value
- User-defined value

In order to check what value was imputed, we can examine the `statistics_` attribute, which was learned from the data during the fit step.

```
imp.statistics_
```

```
array([28.11111111])
```

Now that we know how `SimpleImputer` works, we'll update the `ColumnTransformer` to include imputation of the Age column. As a reminder, brackets are required around Age because `SimpleImputer` expects 2-dimensional input, whereas brackets are not allowed around Name because `CountVectorizer` expects 1-dimensional input.

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp, ['Age']),
    ('passthrough', ['Parch', 'Fare']))
```

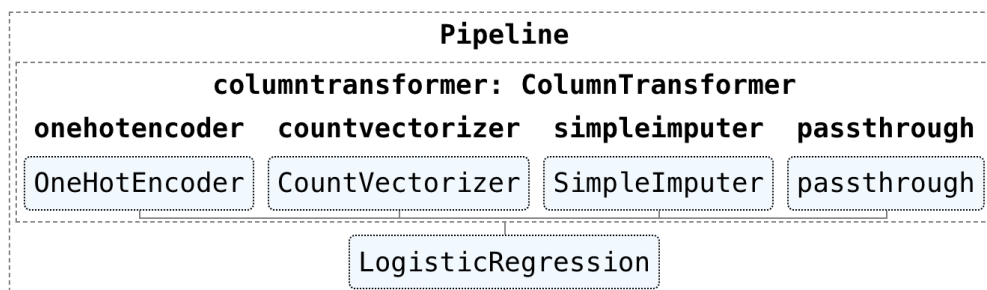
We'll run the `fit_transform` method, and you can see that there are now 48 columns in the feature matrix, whereas in the last chapter there were 47.

```
ct.fit_transform(X)
```

```
<10x48 sparse matrix of type '<class 'numpy.float64'>'
  with 88 stored elements in Compressed Sparse Row format>
```

Next, we'll update the Pipeline to include the revised `ColumnTransformer`, and fit it on X and y. There's no error this time because the one missing value in X was imputed.

```
pipe = make_pipeline(ct, logreg)
pipe.fit(X, y)
```



Before we make predictions, we're going to examine the Pipeline to confirm the imputation value for Age that was learned from X:

- First, we access the `columntransformer` step of the Pipeline using the `named_steps` attribute.
- Then, from that Pipeline step, we access the `simpleimputer` transformer using the `named_transformers_` attribute.
- Finally, from that transformer, we access the `statistics_` attribute.

This confirms what we saw previously, which is that the imputer learned a value of 28.11 from the Age column.

```
(pipe.named_steps['columntransformer']  
    .named_transformers_['simpleimputer']  
    .statistics_)
```

```
array([28.11111111])
```

Next, we'll update the `X_new` DataFrame to use the same columns as X.

```
X_new = df_new[cols]  
X_new
```

	Parch	Fare	Embarked	Sex	Name	Age
0	0	7.8292	Q	male	Kelly, Mr. James	34.5
1	0	7.0000	S	female	Wilkes, Mrs. James (Ellen Needs)	47.0
2	0	9.6875	Q	male	Myles, Mr. Thomas Francis	62.0
3	0	8.6625	S	male	Wirz, Mr. Albert	27.0
4	1	12.2875	S	female	Hirvonen, Mrs. Alexander (Helga...	22.0
5	0	9.2250	S	male	Svensson, Mr. Johan Cervin	14.0
6	0	7.6292	Q	female	Connolly, Miss. Kate	30.0
7	1	29.0000	S	male	Caldwell, Mr. Albert Francis	26.0
8	0	7.2292	C	female	Abraham, Mrs. Joseph (Sophie Ha...	18.0
9	0	24.1500	S	male	Davies, Mr. John Samuel	21.0

Finally, we'll use the fitted Pipeline to make predictions for `X_new`.

```
pipe.predict(X_new)
```

```
array([0, 0, 0, 0, 1, 0, 1, 0, 1, 0])
```

With respect to imputation, it's worth talking about exactly what happens during the predict step. Since `X_new` didn't have any missing values in the Age column, then nothing got imputed during prediction.

But let's pretend that `X_new` did have a missing value in Age. If that was the case, then the imputed value would have been the mean of Age in X, which is 28.11, not the mean of Age in `X_new`. This is critically important because a transformer is only allowed to learn from the training data, and then apply what it learned to both the training and new data.

What would have been imputed for Age in X_new?

- Imputed value would be the mean of Age in X, not the mean of Age in X_new
- Transformer is only allowed to learn from the training data

As we've already seen in this book, the `OneHotEncoder` learns its categories from the training data, the `CountVectorizer` learns its vocabulary from the training data, and similarly, the `SimpleImputer` learns its imputation value from the training data. This is the reason why we run `fit_transform` on the training data and `transform` (only) on the new data.

What do transformers learn from the training data?

- **OneHotEncoder:** Learns categories
- **CountVectorizer:** Learns vocabulary
- **SimpleImputer:** Learns imputation value

If you're struggling with this concept, here's another way of looking at it that might be helpful to you. Pretend that X_new only contained a single sample, and that sample had a missing Age value. If you passed that X_new to the `predict` method, it's clear that scikit-learn has to look to the training data for the imputation value, since there would be no other Age values in X_new for it to examine.

7.4 Using “missingness” as a feature

When imputing missing values, it's also possible to use “missingness” as a feature of its own.

Starting in scikit-learn version 0.21, we can set the `add_indicator` parameter to `True` when creating a `SimpleImputer`.

```
imp_indicator = SimpleImputer(add_indicator=True)
```

Then when we use `fit_transform`, a separate column known as a missing indicator is included in the output matrix, and it indicates the presence of missing values.

```
imp_indicator.fit_transform(X[['Age']])
```

```
array([[22.      ,  0.      ],
       [38.      ,  0.      ],
       [26.      ,  0.      ],
       [35.      ,  0.      ],
       [35.      ,  0.      ],
       [28.11111111,  1.      ],
       [54.      ,  0.      ],
       [ 2.      ,  0.      ],
       [27.      ,  0.      ],
       [14.      ,  0.      ]])
```

Adding a missing indicator is useful when the data is not missing at random, since there might be a relationship between the “missingness” and the target value.

Why add a missing indicator?

- Useful when the data is not missing at random
- Can encode the relationship between “missingness” and the target value

For example, if Age was missing for some samples because older passengers declined to give their ages, and older passengers are more likely to have survived, then there is a relationship between Age being missing and the likelihood of survival. Thus the missingness itself can be a useful feature, and we can include that feature in the model using a missing indicator.

We’re not going to modify our Pipeline to include a missing indicator at this time, but we’ll return to this concept later in the book.

7.5 Q&A: How do I perform multivariate imputation?

So far in this chapter, we’ve used `SimpleImputer` for imputation. `SimpleImputer` does univariate imputation, which means that it only looks at the feature being imputed when deciding what values to impute. Thus when imputing missing values for Age, `SimpleImputer` only considers the values in the Age column.

However, there’s another type of imputation called multivariate imputation. The intuition of multivariate imputation is that it can be useful to take other features into account when deciding what values to impute.

For example, maybe a high Parch and a low Fare is common for kids. Thus if Age is missing for a row which has a high Parch and a low Fare, then you should impute a low Age rather than the mean of Age, which is what `SimpleImputer` would do.

Multivariate imputation is available in scikit-learn via the `IterativeImputer` and `KNNImputer` classes, and in this lesson I’ll explain how both of them work.

Types of imputation:

- **Univariate imputation:** Only examines the feature being imputed
 - `SimpleImputer`
- **Multivariate imputation:** Takes multiple features into account
 - `IterativeImputer`
 - `KNNImputer`

`IterativeImputer` was introduced in scikit-learn version 0.21. It’s considered experimental, which means that the API and predictions may change in future versions. As such, scikit-learn will only allow you to import the `IterativeImputer` class from the `impute` module if you first import the

`enable_iterative_imputer` function from the experimental module. This is scikit-learn's way of requiring you to acknowledge that you're using experimental functionality.

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
```

Then, we'll create an instance of `IterativeImputer` called `imp_iterative`, and pass three columns from `X` to its `fit_transform` method: `Parch`, `Fare`, and `Age`. `Parch` and `Fare` don't have any missing values, and `Age` has 1 missing value. You can see that 24.24 was imputed for the missing value of `Age`.

```
imp_iterative = IterativeImputer()
imp_iterative.fit_transform(X[['Parch', 'Fare', 'Age']])
```

```
array([[ 0.          ,  7.25         , 22.          ],
       [ 0.          , 71.2833        , 38.          ],
       [ 0.          ,  7.925        , 26.          ],
       [ 0.          , 53.1          , 35.          ],
       [ 0.          ,  8.05         , 35.          ],
       [ 0.          ,  8.4583        , 24.23702669],
       [ 0.          , 51.8625        , 54.          ],
       [ 1.          , 21.075         ,  2.          ],
       [ 2.          , 11.1333        , 27.          ],
       [ 0.          , 30.0708        , 14.          ]])
```

Here's how the `IterativeImputer` works:

1. For the 9 rows in which `Age` is not missing, scikit-learn trains a regression model in which `Parch` and `Fare` are the features and `Age` is the target.
2. For the 1 row in which `Age` is missing, scikit-learn passes the `Parch` and `Fare` values to the trained model. The model makes a prediction for `Age`, and that value is used for imputation.

In summary, `IterativeImputer` turned this into a regression problem with 2 features, 9 samples of training data, and 1 sample of new data.

How `IterativeImputer` works:

1. **Age not missing:** Train a regression model to predict `Age` using `Parch` and `Fare`
2. **Age missing:** Predict `Age` using trained model

There are four things I want to comment on about `IterativeImputer`:

- First, it only works with numerical features. This is unlike `SimpleImputer`, which also works with categorical features.
- Second, you have to decide which features to pass to `IterativeImputer`. My advice is to use features that you believe are highly correlated with one another.

- Third, you are allowed to pass it multiple features with missing values. In other words, Parch, Fare, and Age could all have missing values. IterativeImputer will just do three different regression problems, and each column will take a turn being the target.
- Fourth, you can actually choose which model IterativeImputer uses for the regression problem. By default, it uses Bayesian ridge regression.

Notes about IterativeImputer:

- Only works with numerical features
- You have to decide which features to include
- You can include multiple features with missing values
- You can choose the regression model

Anyway, if you decided you wanted to include it in the ColumnTransformer, you would just specify that Parch, Fare, and Age should be transformed by `imp_iterative`, and thus there would no longer be any passthrough columns.

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp_iterative, ['Parch', 'Fare', 'Age']))
```

Our other option for multivariate imputation is `KNNImputer`, which was introduced in scikit-learn version 0.22 and is not considered experimental.

To use it, we'll import it from the `impute` module, create an instance called `imp_knn`, and then pass the same three columns from `X` to the `fit_transform` method. This time, you'll see that 30.5 was imputed for the missing value of Age.

```
from sklearn.impute import KNNImputer
imp_knn = KNNImputer(n_neighbors=2)
imp_knn.fit_transform(X[['Parch', 'Fare', 'Age']])
```

```
array([[ 0.      ,  7.25   , 22.    ],
       [ 0.      , 71.2833, 38.    ],
       [ 0.      ,  7.925  , 26.    ],
       [ 0.      , 53.1    , 35.    ],
       [ 0.      ,  8.05   , 35.    ],
       [ 0.      ,  8.4583, 30.5   ],
       [ 0.      , 51.8625, 54.    ],
       [ 1.      , 21.075  ,  2.    ],
       [ 2.      , 11.1333, 27.    ],
       [ 0.      , 30.0708, 14.    ]])
```

Here's how the `KNNImputer` works:

1. First, `KNNImputer` finds the row in which Age is missing.

2. Because I set the `n_neighbors` parameter to 2, scikit-learn finds the 2 nearest rows in which Age is not missing, measured by how close the Parch and Fare values are to this row. In this case, the third and the fifth rows are the two nearest rows.
3. Finally, scikit-learn calculates the mean of the Age values from those 2 rows. In this case, it takes the mean of 26 and 35 which is 30.5, and that value is used as the imputation value.

How KNNImputer works:

1. Find the row in which Age is missing
2. Find the `n_neighbors` nearest rows in which Age is not missing
3. Calculate the mean of Age from the nearest rows

You might be wondering how you should choose the value for `n_neighbors`. As you'll see later in the book, transformer hyperparameters like this should be chosen through a tuning process in the same way that you would tune the hyperparameters for a model.

7.6 Q&A: What are the best practices for missing value imputation?

Missing value imputation may be considered dubious from a statistics point of view, but if your primary goal is prediction, then imputation has been shown experimentally to work well under certain conditions.

In this lesson, I'm going to summarize what I've learned from research papers about the best practices for effective missing value imputation.

Types of missing data:

- **Missing Completely At Random (MCAR):**
 - No relationship between missingness and underlying data
 - **Example:** Booking agent forgot to gather Age
- **Missing Not At Random (MNAR):**
 - Relationship between missingness and underlying data
 - **Example:** Older passengers declined to give their Age
- **Missing due to a structural deficiency:**
 - Data omitted for a specific purpose
 - **Example:** Staff members did not pay a Fare

The best practices actually differ depending upon the type of missing data you have, and so I'm going to start with the first type of missing data, which is known as Missing Completely At Random or "MCAR". Missing data is designated as MCAR if there's no relationship between the missingness and the underlying data.

An example of this would be if Age was missing because the booking agent forgot to gather that information from a few of the passengers. In other words, the data is just as likely to be missing for an older person as a younger person, and thus missingness is not meaningfully related to Age.

If you determine that your data is MCAR, here's what the research indicates:

- If you have a small dataset, then `IterativeImputer` tends to be more effective than mean imputation.
- If you have a large dataset, `IterativeImputer` and mean imputation tend to work equally well, though `IterativeImputer` will have a much higher computational cost.
- There's no benefit to adding a missing indicator since the missingness is random.

Advice for MCAR imputation:

- **Small dataset:** `IterativeImputer` is more effective than mean imputation
- **Large dataset:** `IterativeImputer` and mean imputation work equally well
- No benefit to adding a missing indicator

The next type of missing data is known as Missing Not At Random or "MNAR". Missing data is designated as MNAR if there is a relationship between the missingness and the underlying data, which is commonly the case in real-world datasets.

I mentioned an example of this in a previous lesson, namely if Age was missing because some of the older passengers declined to give their ages. In other words, there is a relationship between missingness and Age.

If your data is MNAR, here's what the research indicates:

- Mean imputation is more effective than `IterativeImputer` because `IterativeImputer` actually obscures the pattern that the model might otherwise be able to learn.
- It's important that you add a missing indicator so that the model can learn from the pattern of missingness.
- It's recommended to use a powerful, non-linear model for prediction, since mean imputation tends not to work as well in combination with a linear model.

Advice for MNAR imputation:

- Mean imputation is more effective than `IterativeImputer`
- Add a missing indicator
- Use a powerful, non-linear model for prediction

The final type of missing data I'll mention is data that's missing due to a structural deficiency. This means that a value is missing because it was omitted for a specific purpose. An example of this would be if the passenger list included staff members, and their values for the Fare column were listed as missing to indicate that they did not pay a fare.

In the case of a structural deficiency, my advice is to impute the most logical and reasonable user-defined value, and also add a missing indicator. For the example I mentioned, it would make the most sense to insert a Fare value of 0 for all staff members.

Advice for structural deficiency imputation:

- Impute a logical and reasonable user-defined value
- Add a missing indicator

All of these examples should make clear that if you're going to impute missing values, it's important to thoroughly understand your data before choosing an imputation strategy.

Finally, it's worth reiterating that histogram-based gradient boosting trees in scikit-learn have built-in support for missing values. In other words, you can pass it data with missing values, and it will handle them internally. This is significant because it has a lower computation cost than complex imputation strategies like `IterativeImputer`, and yet it can perform just as well or better than imputation across a variety of missing value scenarios.

Advantages of histogram-based gradient boosting trees:

- Built-in support for missing values
- Lower computational cost than `IterativeImputer`
- Performs well across many missing value scenarios

Therefore, if you have a large dataset with a lot of missing values, it's worth trying out a histogram-based gradient boosting tree as your prediction model and excluding the imputation step, and comparing its performance with any other model that does require an imputation step.

7.7 Q&A: What's the difference between `ColumnTransformer` and `FeatureUnion`?

As we saw earlier in this chapter, you can add a missing indicator to the output of `SimpleImputer` by setting the `add_indicator` parameter to `True`.

```
imp_indicator = SimpleImputer(add_indicator=True)
imp_indicator.fit_transform(X[['Age']])
```

```
array([[22.      ,  0.      ],
       [38.      ,  0.      ],
       [26.      ,  0.      ],
       [35.      ,  0.      ],
       [35.      ,  0.      ],
       [28.11111111,  1.      ],
       [54.      ,  0.      ],
       [ 2.      ,  0.      ],
       [27.      ,  0.      ],
       [14.      ,  0.      ]])
```

In this lesson, we're going to explore a different way to create this same matrix as a way of learning about the `FeatureUnion` class.

To create the left column of this matrix, we can use the `imp` object, which is a `SimpleImputer` without a missing indicator.

```
imp.fit_transform(X[['Age']])
```

```
array([[22.      ],
       [38.      ],
       [26.      ],
       [35.      ],
       [35.      ],
       [28.11111111],
       [54.      ],
       [ 2.      ],
       [27.      ],
       [14.      ]])
```

To create the right column of the matrix, we can use the `MissingIndicator` class. We'll import the class from the `impute` module and create an instance called `indicator`.

```
from sklearn.impute import MissingIndicator
indicator = MissingIndicator()
```

When we pass `Age` to the `fit_transform` method, it outputs `False` and `True` instead of 0 and 1, but otherwise the results are identical to the right column of the matrix.

```
indicator.fit_transform(X[['Age']])
```

```
array([[False],
       [False],
       [False],
       [False],
       [False],
       [ True],
       [False],
       [False],
       [False],
       [False]])
```

The final step in recreating our original matrix is to stack these two columns side-by-side, which we can do by using the `FeatureUnion` class. A `FeatureUnion` applies multiple transformations to a single input column and stacks the results side-by-side.

We'll import the `make_union` function from the `pipeline` module, and then create an instance called `union` that contains both the `imp` and `indicator` objects.


```
from sklearn.pipeline import make_union
union = make_union(imp, indicator)
```

When we pass Age to the union's `fit_transform` method, it runs both the `imp` and `indicator` transformations and stacks the results side-by-side. Note that `False` and `True` are converted to 0 and 1 when included in a numeric array, and thus we've recreated our original matrix.

```
union.fit_transform(X[['Age']])
```

```
array([[22.      ,  0.      ],
       [38.      ,  0.      ],
       [26.      ,  0.      ],
       [35.      ,  0.      ],
       [35.      ,  0.      ],
       [28.11111111,  1.      ],
       [54.      ,  0.      ],
       [ 2.      ,  0.      ],
       [27.      ,  0.      ],
       [14.      ,  0.      ]])
```

Alternatively, we could have achieved the same results using a `ColumnTransformer`. Specifically, we could have passed the Age column to two separate transformations.

```
ct = make_column_transformer(
    (imp, ['Age']),
    (indicator, ['Age']))
ct.fit_transform(X)
```

```
array([[22.      ,  0.      ],
       [38.      ,  0.      ],
       [26.      ,  0.      ],
       [35.      ,  0.      ],
       [35.      ,  0.      ],
       [28.11111111,  1.      ],
       [54.      ,  0.      ],
       [ 2.      ,  0.      ],
       [27.      ,  0.      ],
       [14.      ,  0.      ]])
```

Stepping back, here's a quick comparison between `FeatureUnion` and `ColumnTransformer`:

- A `FeatureUnion` works on a single input column, and applies multiple different transformations to that one column in parallel.
- A `ColumnTransformer` works on multiple input columns, and applies a different transformation to each input column in parallel.

FeatureUnion vs ColumnTransformer:

- **FeatureUnion:**
 - Single input column
 - Applies multiple different transformations to that column in parallel
- **ColumnTransformer:**
 - Multiple input columns
 - Applies a different transformation to each column in parallel

You can see that ColumnTransformer is far more flexible than FeatureUnion, and so my recommendation is that you use ColumnTransformer to do all of your transformations. For the rare case in which you need to apply multiple different transformations in parallel to the same column, you can use either a FeatureUnion or a ColumnTransformer, depending upon which solution makes more sense to you.

8 Fixing common workflow problems

8.1 Two new problems

Up to now, we've only been working with the first 10 rows of the Titanic dataset to make it easy to examine the input and output of each workflow step. In this chapter, we'll begin using the full Titanic dataset. This will create a few new problems that are common with real datasets, and we'll figure out how to handle those problems appropriately.

We'll start by reading the training data into `df` and reading the new data into `df_new`, overwriting the existing objects.

When examining the shapes, you can see that `df_new` has one fewer column than `df` because it doesn't contain the target column of `Survived`.

```
df = pd.read_csv('http://bit.ly/MLtrain')
df.shape
```

```
(891, 11)
```

```
df_new = pd.read_csv('http://bit.ly/MLnewdata')
df_new.shape
```

```
(418, 10)
```

We'll check for missing values in these two DataFrames by chaining together the `isna` and `sum` methods. The results tell us how many missing values are present in each column.

This reveals two problems we'll have to handle that weren't present in our 10-row datasets:

- First, `Embarked` contains missing values in `df`.
- Second, `Fare` contains a missing value in `df_new`.

We'll spend the rest of this chapter addressing these two problems.

Note that we don't have to worry about missing values in `Cabin` because we're not yet using that as a feature, and our existing workflow already accounts for the missing values in `Age`.

```
df.isna().sum()
```

	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

```
df_new.isna().sum()
```

	0
Pclass	0
Name	0
Sex	0
Age	86
SibSp	0
Parch	0
Ticket	0
Fare	1
Cabin	327
Embarked	0

Features with missing values:

- Problematic:
 - **Embarked:** Missing values in df
 - **Fare:** Missing value in df_new
- Not problematic:
 - **Cabin:** Not currently using
 - **Age:** Already being imputed

8.2 Problem 1: Missing values in a categorical feature

In this lesson, we're going to figure out how to handle the missing values in the Embarked column. We'll start with a reminder of the six feature columns we're using.

```
cols = ['Parch', 'Fare', 'Embarked', 'Sex', 'Name', 'Age']
```

We'll redefine X and y to use the full dataset rather than the 10-row dataset.

```
X = df[cols]
y = df['Survived']
```

Here's a reminder of the ColumnTransformer we created in the previous chapter.

```
ct = make_column_transformer(
    (ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp, ['Age']),
    ('passthrough', ['Parch', 'Fare']))
```

Normally we would pass `X` to the `fit_transform` method, but in this case it errors because the `Embarked` column contains missing values.

```
ct.fit_transform(X)
```

`ValueError: Input contains NaN`

Our solution will be to impute missing values for `Embarked` before one-hot encoding it.

Starting in scikit-learn version 0.24, `OneHotEncoder` automatically handles missing values by treating them as a new category. I'm using version 0.23, and so I'll be writing the code to manually treat missing values as a new category. Even if you're using version 0.24 or later, I still recommend following my code because what I'll teach you will enable you to solve similar problems that scikit-learn does not automatically handle.

How OneHotEncoder handles missing values:

- **Before version 0.24:** Errors if the input contains missing values
- **Starting in version 0.24:** Treats missing values as a new category

The first step of this solution is to create a new instance of `SimpleImputer`, which we'll call `imp_constant`. For categorical features, you can either impute the most frequent value or a constant user-defined value. We'll choose the latter by setting the `strategy` parameter to `'constant'`, and the constant value we'll impute is the string `'missing'`.

```
imp_constant = SimpleImputer(strategy='constant', fill_value='missing')
```

Imputation strategies for categorical features:

- Most frequent value
- User-defined value

Next, we'll create a two-step `Pipeline` that only contains transformers. The first step is imputation using our new imputer, and the second step is one-hot encoding. We'll call this `Pipeline` `imp_ohe` to remind us of the two steps it contains.

```
imp_ohe = make_pipeline(imp_constant, ohe)
```

We can test out the `imp_ohe` Pipeline by passing the `Embarked` column to its `fit_transform` method. It outputs four columns because missing values are essentially being treated as a fourth category in addition to C, Q, and S.

```
imp_ohe.fit_transform(X[['Embarked']])
```

```
<891x4 sparse matrix of type '<class 'numpy.float64'>'
  with 891 stored elements in Compressed Sparse Row format>
```

We can confirm this by accessing the second step of the Pipeline, which is the `OneHotEncoder`, and then examining the `categories_` attribute.

```
imp_ohe[1].categories_
```

```
[array(['C', 'Q', 'S', 'missing'], dtype=object)]
```

In case it helps you to understand `imp_ohe` better, I'm going to show you what happens “under the hood” when you `fit_transform` this Pipeline. To be clear, you should not actually write the following code, rather it's just for teaching purposes:

1. First, the `imp_constant` object imputes a string value of “missing” for any missing values in the `Embarked` column.
2. Second, the output of the imputer is one-hot encoded by the `ohe` object, which outputs four columns.

```
ohe.fit_transform(imp_constant.fit_transform(X[['Embarked']]))
```

```
<891x4 sparse matrix of type '<class 'numpy.float64'>'
  with 891 stored elements in Compressed Sparse Row format>
```

Now that we've created a transformer-only Pipeline to handle the missing values in `Embarked`, we'll simply replace the `ohe` transformer in our `ColumnTransformer` with the `imp_ohe` Pipeline.

```
ct = make_column_transformer(
    (imp_ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp, ['Age']),
    ('passthrough', ['Parch', 'Fare']))
```

There are two things I want to note about the `imp_ohe` Pipeline:

- First, you're only allowed to include transformer objects in a ColumnTransformer, but `imp_ohe` is eligible because all of its steps are transformers.
- Second, it's completely fine to apply `imp_ohe` to the Sex column as well as Embarked. There are no missing values in the Sex column, so the imputation step won't affect it, and it will simply get passed along to the one-hot encoding step.

Notes about the `imp_ohe` Pipeline:

- Treated like a transformer because all of its steps are transformers
- Imputation step won't affect the Sex column

By replacing `ohe` with `imp_ohe`, we have now solved the problem of missing values in the Embarked column. Thus, we can pass `X` to the ColumnTransformer's `fit_transform` method, and it no longer throws an error.

As an aside, the output matrix is now much wider than before because the Name column of `X` contains a large number of unique words.

```
ct.fit_transform(X)
```

```
<891x1518 sparse matrix of type '<class 'numpy.float64'>'
  with 7328 stored elements in Compressed Sparse Row format>
```

8.3 Problem 2: Missing values in the new data

Now that we've solved our first problem, we're going to move on to the second problem, which is the missing values in the Fare column. Recall that Fare has missing values in `X_new` but not in `X`, and thus our modeling Pipeline would error when making predictions for `X_new` if we don't account for these missing values.

Our solution to this problem is to impute missing values for Fare. The ColumnTransformer already contains an imputer (`imp`) that does mean imputation on the Age column, so we'll just apply it to Fare as well. This is actually all that's required to solve our problem.

```
ct = make_column_transformer(
    (imp_ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp, ['Age', 'Fare']),
    ('passthrough', ['Parch']))
```

Now, we'll pass `X` to the `fit_transform` method of the ColumnTransformer. It will output the same number of columns as it did before, since Fare just moved from a passthrough column to a transformed column.

```
ct.fit_transform(X)
```

```
<891x1518 sparse matrix of type '<class 'numpy.float64'>'
  with 7328 stored elements in Compressed Sparse Row format>
```

To be clear, the Fare column does not have any missing values in `X`, thus the imputer did not impute any values for Fare during the `fit_transform`. However, it did learn the mean of Fare in `X`, which is the imputation value that will be applied to Fare in `X_new` during prediction.

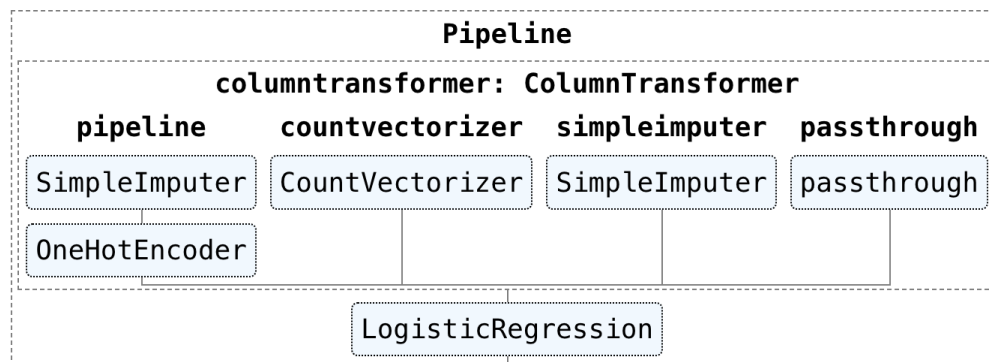
What will be imputed for Fare?

- **X**: No missing Fare values, thus no imputation of Fare
- **X_new**: Missing Fare value, thus impute the mean of Fare in X during prediction

- **X**: No missing Fare values, thus no imputation of Fare
- **X_new**: Missing Fare value, thus impute the mean of Fare in X during prediction

Next, we'll update our modeling Pipeline to include the revised ColumnTransformer, and fit it on X and y. You can see from the diagram that there's now a transformer Pipeline within the ColumnTransformer, which is within the modeling Pipeline.

```
pipe = make_pipeline(ct, logreg)
pipe.fit(X, y)
```



Finally, we'll redefine `X_new` to use the full dataset, and then use the fitted `Pipeline` to make predictions for `X_new`. We know that we've solved our second problem because the `Pipeline` did not throw any errors during the predict step.

```
X_new = df_new[cols]
pipe.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1,
       1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1,
       1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0])
```



```

0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,
0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0,
1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0,
0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1,
0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1,
0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1])

```

8.4 Q&A: How do I see the feature names output by the ColumnTransformer?

When we pass `X` to the `ColumnTransformer`'s `fit_transform` method, it outputs a matrix with 1518 columns. How can we find out the names of these columns?

```
ct.fit_transform(X)
```

```

<891x1518 sparse matrix of type '<class 'numpy.float64'>'
  with 7328 stored elements in Compressed Sparse Row format>

```

Earlier in the book, we used the `get_feature_names` method for this purpose, which, as I mentioned previously, has been replaced by `get_feature_names_out` starting in `scikit-learn` 1.0. However, `get_feature_names` will only work if all of the underlying transformers have a `get_feature_names` method. In this case, it errors because neither `Pipeline` transformers nor `SimpleImputer` transformers have a `get_feature_names` method.

```
ct.get_feature_names()
```

```
AttributeError: Transformer pipeline does not provide get_feature_names
```

The good news is that starting in `scikit-learn` 1.1, the `get_feature_names_out` method is available for all transformers, which means that retrieving the feature names will no longer error.

Changes to `get_feature_names`:

- **Starting in version 1.0:** `get_feature_names` replaced with `get_feature_names_out`
- **Starting in version 1.1:** `get_feature_names_out` available for all transformers

In the meantime, our only solution for figuring out the column names is to inspect the transformers one-by-one.

When we print out the `transformers_` attribute, we can see that there are 4 transformers.

```
ct.transformers_  
  
[('pipeline',  
  Pipeline(steps=[('simpleimputer',  
                   SimpleImputer(fill_value='missing', strategy='constant')),  
                   ('onehotencoder', OneHotEncoder())]),  
  ['Embarked', 'Sex']),  
 ('countvectorizer', CountVectorizer(), 'Name'),  
 ('simpleimputer', SimpleImputer(), ['Age', 'Fare']),  
 ('passthrough', 'passthrough', ['Parch'])]
```

The first transformer is a Pipeline of SimpleImputer and OneHotEncoder. OneHotEncoder has a `get_feature_names` method, which we can access by selecting the pipeline transformer and then its onehotencoder step. `get_feature_names` outputs 6 features, which we know are the first 6 features in the matrix because this is the first transformer in the ColumnTransformer.

```
(ct.named_transformers_['pipeline']  
 .named_steps['onehotencoder']  
 .get_feature_names())  
  
array(['x0_C', 'x0_Q', 'x0_S', 'x0_missing', 'x1_female', 'x1_male'],  
      dtype=object)
```

The second transformer is a CountVectorizer. It also has a `get_feature_names` method, which we can access by selecting the countvectorizer transformer. We could print out all of the feature names, but instead we'll pass it to the `len` function, which indicates that the next 1509 features in the matrix came from CountVectorizer.

```
len(ct.named_transformers_['countvectorizer'].get_feature_names())
```

1509

The third transformer is a SimpleImputer, which doesn't change the number of columns since we're not adding a missing indicator, so we know that the next two features in the matrix are Age and Fare.

The fourth transformer is a passthrough transformer, which also doesn't change the number of columns, so we know that the final feature in the matrix is Parch.

We've now accounted for all 1518 features: 6 from the Pipeline transformer, 1509 from the CountVectorizer, 2 from the SimpleImputer, and 1 from the passthrough transformer.

Features output by each transformer:

- Pipeline: 6 features (Embarked and Sex)
- CountVectorizer: 1509 features (Name)
- SimpleImputer: 2 features (Age and Fare)
- passthrough: 1 feature (Parch)

8.5 Q&A: Why did we create a Pipeline inside of the ColumnTransformer?

Earlier in this chapter, since the Embarked column contained missing values and needed one-hot encoding, we created a two-step Pipeline called `imp_ohe`. The first step of this Pipeline is imputation of a constant value, and the second step is one-hot encoding.

```
imp_ohe = make_pipeline(imp_constant, ohe)
```

We included the `imp_ohe` Pipeline in the `ColumnTransformer`, and applied it to both the Embarked and Sex columns. Here's what it would look like if the `ColumnTransformer` only contained the `imp_ohe` Pipeline.

```
ct = make_column_transformer(  
    (imp_ohe, ['Embarked', 'Sex']))
```

When you run the `fit_transform` method, Embarked turns into 4 columns and Sex turns into 2 columns, and the results are stacked side-by-side.

```
ct.fit_transform(X)
```

```
array([[0., 0., 1., 0., 0., 1.],  
       [1., 0., 0., 0., 1., 0.],  
       [0., 0., 1., 0., 1., 0.],  
       ...,  
       [0., 0., 1., 0., 1., 0.],  
       [1., 0., 0., 0., 0., 1.],  
       [0., 1., 0., 0., 0., 1.]])
```

Because the Sex column didn't contain any missing values and only needed one-hot encoding, we could have achieved the exact same result by applying `imp_ohe` to Embarked and separately applying `ohe` to Sex.

```
ct = make_column_transformer(  
    (imp_ohe, ['Embarked']),  
    (ohe, ['Sex']))
```

The `fit_transform` does indeed output the same results as above, though I personally prefer the first `ColumnTransformer`.

```
ct.fit_transform(X)
```

```
array([[0., 0., 1., 0., 0., 1.],
       [1., 0., 0., 0., 1., 0.],
       [0., 0., 1., 0., 1., 0.],
       ...,
       [0., 0., 1., 0., 1., 0.],
       [1., 0., 0., 0., 0., 1.],
       [0., 1., 0., 0., 0., 1.]])
```

One common question is whether you can avoid using the `imp_ohe` Pipeline entirely by making a `ColumnTransformer` like this instead, in which the imputation of a constant value is applied to Embarked, and one-hot encoding is applied to both Embarked and Sex.

```
ct = make_column_transformer(
    (imp_constant, ['Embarked']),
    (ohe, ['Embarked', 'Sex']))
```

The answer is no, you cannot. The `fit_transform` method throws an error because Embarked contains missing values, and the ohe transformer is not able to handle missing values.

```
ct.fit_transform(X)
```

```
ValueError: Input contains NaN
```

The key insight here is that there's no interaction between the transformers in a `ColumnTransformer`. In other words, there's no flow of data from one transformer to the next, meaning the output of the `imp_constant` transformer does not become the input to the ohe transformer. Thus, the ohe transformer is operating on the original Embarked column, not a transformed Embarked column in which missing values have been imputed.

If that's confusing, it might be useful to recall the key differences between a Pipeline and a `ColumnTransformer`:

- In a Pipeline, the output of one step becomes the input to the next step. This is precisely why we created the `imp_ohe` Pipeline: We needed the output of the imputer to become the input to the one-hot encoder.
- In contrast, a `ColumnTransformer` does not have steps. Instead, it has transformers that operate in parallel, and the output of each transformer is stacked beside the other transformer outputs.

Pipeline vs ColumnTransformer:

- Pipeline:

- Output of one step becomes the input to the next step
- `imp_ohe`: Output of `imp_constant` becomes the input to `ohe`
- **ColumnTransformer:**
 - Transformers operate in parallel
 - `ct`: Output of each transformer is stacked beside the other transformer outputs

8.6 Q&A: Which imputation strategy should I use with categorical features?

When imputing missing values for a categorical feature, you can either impute the most frequent value or a constant user-defined value. In this lesson, I'm going to discuss how you might choose between these two strategies.

Imputing a constant value essentially treats the missing values as a new category, which I believe is the better choice regardless of whether the values are missing at random or not at random. Imputing a constant value is especially important if the majority of values are missing, since imputing the most frequent value in that case would more than double the size of the category that was imputed, which would be quite misleading to the model.

That being said, imputing the most frequent value is much more acceptable when you have only a small number of missing values for a given feature, since the imputation won't have much of an impact on the model anyway.

Imputation strategies for categorical features:

- **Constant user-defined value:**
 - Treats missing values as a new category (recommended)
 - Important if the majority of values are missing
- **Most frequent value:**
 - Acceptable if only a small number of values are missing

It's important to note that if you impute a constant value for a categorical feature, and that feature has missing values in the new data but not the training data, then you'll need to set the `OneHotEncoder`'s `handle_unknown` parameter to `'ignore'`. That's because the missing values category won't be learned during the `OneHotEncoder`'s fit step, and thus unknown values seen during the transform step need to be ignored in order to avoid an error.

The alternative here is to impute the most frequent value for that feature, in which case you can leave the `handle_unknown` parameter set to its default value of `'error'`.

Possible problem with imputing a constant value:

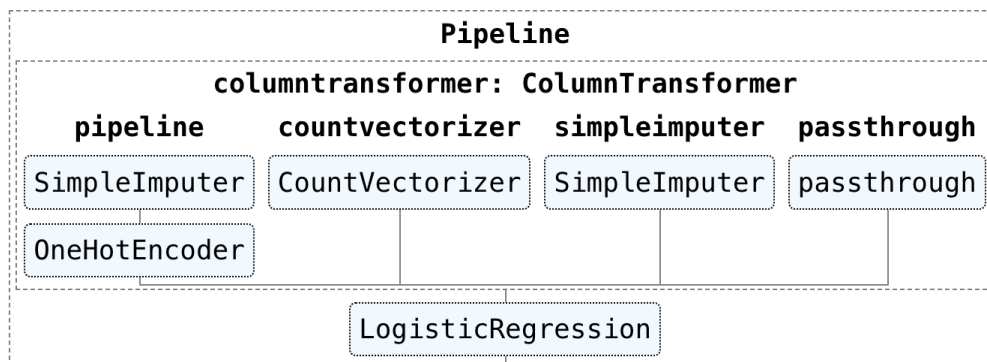
- **Condition:** The feature only has missing values in the new data

- **Solution:** Set `handle_unknown` to 'ignore' for the `OneHotEncoder`
- **Alternative:** Impute the most frequent value, and leave `handle_unknown` set to 'error'

8.7 Q&A: Should I impute missing values before all other transformations?

Here's the Pipeline that we've built throughout the book. The strategy I've used so far is to include all data transformations within a single `ColumnTransformer`, including any missing value imputation, and then use that `ColumnTransformer` as the first step in a two-step Pipeline.

pipe



However, an alternative approach would be to create a three-step Pipeline in which the first step is missing value imputation, the second step includes all other data transformations, and the third step is the model. Let's try it out to see if this is a better approach.

Impute missing values as a first step?

- **Current Pipeline:**
 - **Step 1:** All data transformations
 - **Step 2:** Model
- **Alternative Pipeline:**
 - **Step 1:** Missing value imputation
 - **Step 2:** All other data transformations
 - **Step 3:** Model

This would be the first `ColumnTransformer`, which only does missing value imputation. Constant value imputation is applied to `Embarked`, mean imputation is applied to `Age` and `Fare`, and the other columns are passed through because they don't contain any missing values in the training or new data. It would be the first step in the Pipeline.

```
ct1 = make_column_transformer(
    (imp_constant, ['Embarked']),
    (imp, ['Age', 'Fare']),
    ('passthrough', ['Sex', 'Name', 'Parch']))
```

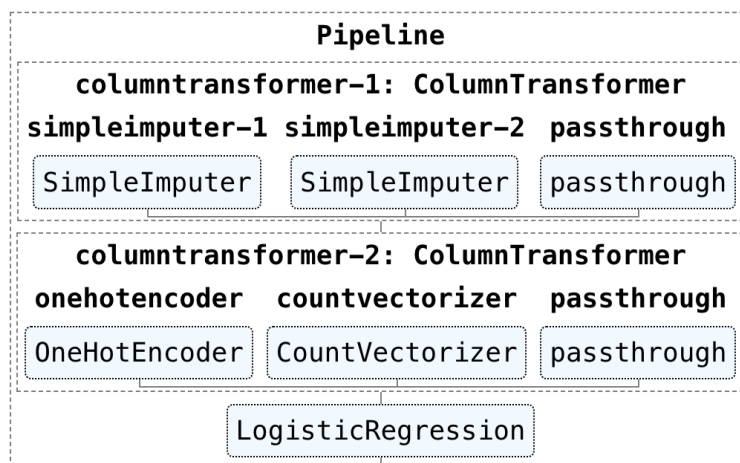
This would be the second ColumnTransformer, which handles all other data transformations. It would be the second step in the Pipeline, and thus it would operate on the output of the first ColumnTransformer. However, a ColumnTransformer outputs a NumPy array, not a DataFrame, and thus in this ColumnTransformer we would have to reference the columns by position instead of by name.

We know the order of the columns from the first ColumnTransformer, and thus Embarked and Sex would be columns 0 and 3 and Name would be column 4. Embarked and Sex are one-hot encoded, Name is vectorized, and the other columns are passed through.

```
ct2 = make_column_transformer(
    (ohe, [0, 3]),
    (vect, 4),
    ('passthrough', [1, 2, 5]))
```

Now that we've created the ColumnTransformers, we can include them in a three-step Pipeline and fit the Pipeline to X and y.

```
pipe = make_pipeline(ct1, ct2, logreg)
pipe.fit(X, y)
```



Finally, we can use this three-step Pipeline to make predictions, and it does indeed make the same predictions as our original two-step Pipeline.

```
pipe.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1,
       1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1,
       1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,
       0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0,
       1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
       1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
       0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1,
       0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
       0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
       1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1])
```

Using a three-step Pipeline like this is certainly a valid approach. However, I find the original two-step Pipeline easier to write and to read, and thus I prefer the two-step approach.

8.8 Q&A: What methods can I use with a Pipeline?

The rules for Pipelines are that all steps other than the final step must be a transformer, and the final step can be a model or a transformer.

Rules for Pipeline steps:

- All steps other than the final step must be a transformer
- Final step can be a model or a transformer

If a Pipeline ends in a model, such as our pipe object, you can use the Pipeline's `fit` and `predict` methods:

- If you run the `fit` method, all steps before the final one run `fit_transform`, and the final step runs `fit`.
- If you run the `predict` method, all steps before the final one run `transform`, and the final step runs `predict`.

Pipeline ends in a model:

- `fit`:
 - All steps before the final step run `fit_transform`
 - Final step runs `fit`

- **predict:**
 - All steps before the final step run transform
 - Final step runs predict

If a Pipeline ends in a transformer, such as our `imp_ohe` object, you generally use the Pipeline's `fit_transform` and `transform` methods, but you can also use the `fit` method:

- If you run the `fit_transform` method, all steps run `fit_transform`.
- If you run the `transform` method, all steps run `transform`.
- If you run the `fit` method, all steps before the final one run `fit_transform`, and the final step runs `fit`.

Pipeline ends in a transformer:

- **fit_transform:**
 - All steps run `fit_transform`
- **transform:**
 - All steps run `transform`
- **fit:**
 - All steps before the final step run `fit_transform`
 - Final step runs `fit`

Although this is a lot of information to take in, developing this level of understanding will definitely make it easier for you to test and debug your future Pipelines.

9 Workflow review #2

9.1 Recap of our workflow

In this chapter, we're going to review the workflow that we've built so far and also discuss the concept of data leakage.

To start, we're going to walk through all of the code that's necessary to recreate our workflow up to this point. We begin by importing pandas, the three transformer classes we're using, one modeling class, and two composition functions.

```
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline
```

Next, we create a list of the six columns we're going to select from our data.

```
cols = ['Parch', 'Fare', 'Embarked', 'Sex', 'Name', 'Age']
```

Then, we read in all of the training data and use it to define our X and y.

```
df = pd.read_csv('http://bit.ly/MLtrain')
X = df[cols]
y = df['Survived']
```

We read in all of the new data and use it to define X_new.

```
df_new = pd.read_csv('http://bit.ly/MLnewdata')
X_new = df_new[cols]
```

We create four instances of our transformers, namely two different instances of SimpleImputer and one instance each of OneHotEncoder and CountVectorizer.

```
imp = SimpleImputer()
imp_constant = SimpleImputer(strategy='constant', fill_value='missing')
ohe = OneHotEncoder()
vect = CountVectorizer()
```

We create a two-step transformer Pipeline of constant value imputation and one-hot encoding.

```
imp_ohe = make_pipeline(imp_constant, ohe)
```

Then we build the ColumnTransformer, which imputes and one-hot encodes Embarked and Sex, vectorizes Name, imputes Age and Fare, and passes through Parch.

```
ct = make_column_transformer(
    (imp_ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp, ['Age', 'Fare']),
    ('passthrough', ['Parch']))
```

We also create an instance of LogisticRegression.

```
logreg = LogisticRegression(solver='liblinear', random_state=1)
```

Finally, we create a two-step modeling Pipeline, fit the Pipeline to X and y, and use the fitted Pipeline to make predictions on X_new.

```
pipe = make_pipeline(ct, logreg)
pipe.fit(X, y)
pipe.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1,
       1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0,
       1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0,
       0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1,
       1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,
       0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1,
       1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0,
       1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0,
       1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1,
       0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0,
       0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0,
       1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0,
       0, 1])
```

That's really all the code we need to recreate our workflow from the book so far. As a reminder, there are no calls to `fit_transform` or `transform` because all of that functionality is encapsulated by the Pipeline.

9.2 Comparing ColumnTransformer and Pipeline

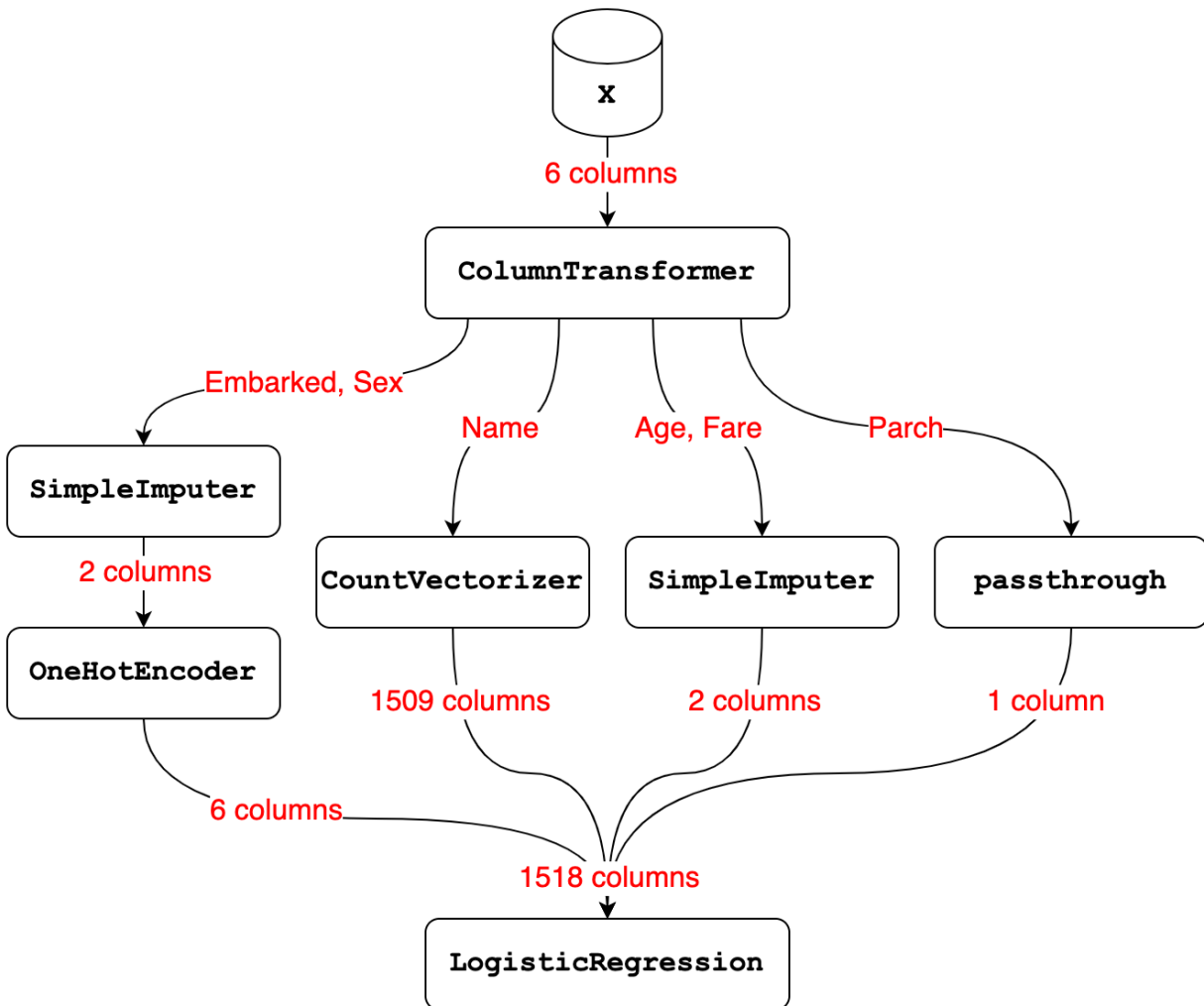
Here's a diagram that illustrates our workflow so far, which we will briefly review.

Let's start with the `ColumnTransformer`, which received 6 columns of input from the `X DataFrame`:

- It selected the `Embarked` and `Sex` columns and passed them to the transformer Pipeline of `SimpleImputer` and `OneHotEncoder`. The `SimpleImputer` step of that Pipeline imputed a constant value and output 2 columns, and the `OneHotEncoder` step transformed those 2 columns into 6 columns.
- It selected the `Name` column and passed it to the `CountVectorizer`, which output 1509 columns.
- It selected the `Age` and `Fare` columns and passed them to the `SimpleImputer`, which imputed the mean and output 2 columns.
- It selected the `Parch` column and passed it through unmodified.
- Finally, it stacked the $6 + 1509 + 2 + 1$ columns side-by-side, resulting in a total of 1518 columns.

Now let's talk about the modeling Pipeline, which has 2 steps:

- Step 1 is a `ColumnTransformer` that received 6 columns of input and transformed them into 1518 columns.
- Step 2 is a `LogisticRegression` model that received 1518 columns of input and used those columns either for fitting or predicting.



With those examples in mind, we can step back and briefly review the differences between a ColumnTransformer and a Pipeline:

- A ColumnTransformer pulls out subsets of columns and transforms them independently, and then stacks the results side-by-side.
- It only ever does data transformations.
- It does not have steps, because each subset of columns is transformed independently.

In contrast:

- A Pipeline is a series of steps that occur in order, and the output of each step becomes the input to the next step.
- The last step of a Pipeline can be a model or a transformer, whereas all other steps must be transformers.

ColumnTransformer vs Pipeline:

- ColumnTransformer:

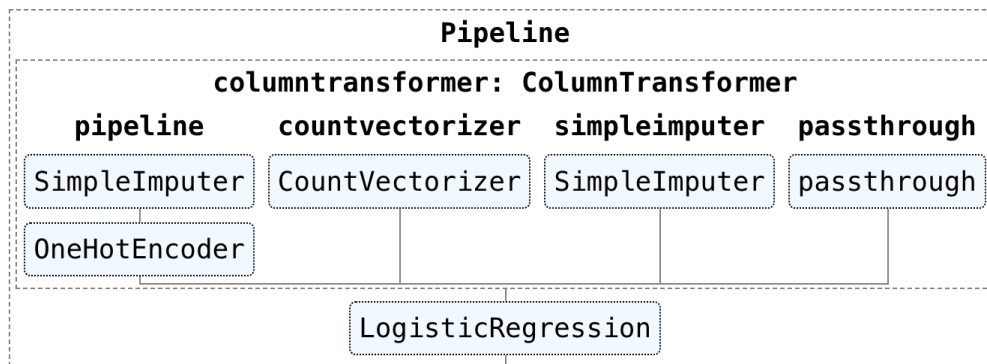
- Selects subsets of columns, transforms them independently, stacks the results side-by-side
- Only includes transformers
- Does not have steps (transformers operate in parallel)

- **Pipeline:**

- Series of steps that occur in order
- Output of each step becomes the input to the next step
- Last step is a model or transformer, all other steps are transformers

Finally, we can print out the Pipeline to see a similar diagram. You can see that our workflow includes a transformer Pipeline within a ColumnTransformer, which itself is within a modeling Pipeline.

```
pipe
```



9.3 Why not use pandas for transformations?

You might be wondering why we did all of our data transformations in scikit-learn, when instead we could have done some of them in pandas and then passed the transformed data to scikit-learn. There are four main reasons:

- First, `CountVectorizer` is not available within pandas, and it's one of the most useful techniques for encoding text data. You could try doing most of your transformations in pandas and then just use `CountVectorizer` in scikit-learn, but that adds additional complexity to your workflow, especially if you have to combine the dense matrix output by pandas with the sparse matrix output by `CountVectorizer`. Thus, your workflow is more efficient when you do all of your transformations within scikit-learn.
- Second, one-hot encoding is available within pandas using the `get_dummies` function, but you would probably end up adding the one-hot encoded columns to your `DataFrame`, which makes the `DataFrame` wider and more difficult to navigate. By using scikit-learn for one-hot encoding instead, you can leave the source `DataFrame` in its original format.

- Third, missing value imputation is available within pandas using the `fillna` function, but that will actually result in data leakage, which I'll explain in detail in the next lesson. By using scikit-learn for imputation instead, you avoid the problem of data leakage.
- Fourth, doing all of your transformations within scikit-learn allows you to cross-validate and tune your entire Pipeline rather than just your model. This will often lead to a better-performing model, which I'll demonstrate in the next chapter.

	scikit-learn	pandas
Encoding text data	CountVectorizer	Not available
One-hot encoding	OneHotEncoder	<code>get_dummies</code>
Imputing missing values	SimpleImputer & others	<code>fillna</code>
Cross-validation and tuning	Your entire Pipeline	Just your model

9.4 Preventing data leakage

In the previous lesson, I mentioned that imputing missing values in pandas would lead to something called “data leakage”. Data leakage occurs when you inadvertently include knowledge from the testing data when training a model.

What is data leakage?

- Inadvertently including knowledge from the testing data when training a model

First, I'll explain why this is problematic, and then I'll explain why imputing missing values in pandas would cause data leakage.

In short, data leakage is problematic because it will cause your model evaluation scores to be less reliable. This may lead you to make bad decisions when tuning hyperparameters, and it will lead you to overestimate how well your model will perform on new data. It's hard to know whether data leakage will skew your evaluation scores by a negligible amount or a huge amount, so it's best to just avoid data leakage entirely.

Why is data leakage problematic?

- Your model evaluation scores will be less reliable
- You might make bad decisions when tuning hyperparameters
- You will overestimate how well your model will perform on new data

So why would imputing missing values in pandas cause data leakage? Your model evaluation procedure, such as cross-validation, is supposed to simulate the future, so that you can accurately estimate right now how well your model will perform on new data. But if you impute missing values on your whole dataset in pandas and then pass it to scikit-learn, your model evaluation procedure will no longer be an accurate simulation of reality. That's because the imputation values will be based on your entire dataset, meaning both the training portion and the testing portion, whereas the imputation values should just be based on the training portion.

In other words, imputation based on the entire dataset is like peeking into the future and then using what you learned from the future during model training, which is definitely not allowed.

You might think that one way around this problem would be to split your dataset into training and testing sets and then impute missing values using pandas. That could work if you're only ever planning to use train/test split for model evaluation, but it would not work if you're planning to use cross-validation. That's because during cross-validation, the rows contained in the training set will change multiple times, and thus it would be quite impractical to write the pandas code to avoid data leakage in this scenario.

Imputation on the full dataset can cause data leakage:

- Your model evaluation procedure is supposed to simulate the future
- Imputation based on your full dataset “leaks” information about the future into model training

So far, we've only been talking about data leakage in the context of missing value imputation. But there are other transformations that if done in pandas on the full dataset will also cause data leakage.

For example, feature scaling in pandas would lead to data leakage, and even one-hot encoding in pandas would lead to data leakage unless there's a known, fixed set of categories. More generally, any transformation which incorporates information about other rows when transforming a row will lead to data leakage if done in pandas.

Other transformations on the full dataset can also cause data leakage:

- Feature scaling
- One-hot encoding
- Any transformation which incorporates information about other rows

Now that you've learned how data transformations in pandas can cause data leakage, I'll briefly mention three ways in which scikit-learn prevents data leakage:

- First, scikit-learn transformers have separate fit and transform steps, which allow you to base your data transformations on the training set only, and then apply those transformations to both the training set and the testing set.
- Second, the `fit` and `predict` methods of a `Pipeline` encapsulate all calls to `fit_transform` and `transform` so that they're called at the appropriate times.
- Third, `cross_val_score` splits the data prior to performing data transformations, which I'll explain in detail in the next chapter.

How does scikit-learn prevent data leakage?

- Transformers have separate fit and transform steps
- Pipeline methods call `fit_transform` and `transform` at the appropriate times
- `cross_val_score` splits the data prior to performing transformations

10 Evaluating and tuning a Pipeline

10.1 Evaluating a Pipeline with cross-validation

In this chapter, we're going to take a deep dive into how to efficiently tune our Pipeline for maximum accuracy.

Let's first return to the topic of model evaluation. As you might recall, we used cross-validation back in chapter 2 to evaluate our most basic model. Since that chapter, we've added many more features without re-running cross-validation. That's because any model evaluation procedure is highly unreliable with only 10 rows of data, thus it would have been misleading to run cross-validation and compare the results.

Now that we're using the full dataset, cross-validation can once again be used. Here's how we'll use it to evaluate our Pipeline:

- First, we'll import the `cross_val_score` function from the `model_selection` module.
- Instead of passing a model to `cross_val_score`, we can actually pass it the entire Pipeline.
- We'll also pass it `X` and `y`.
- Then we'll specify the number of cross-validation folds. Using 5 or 10 folds has been shown to be a reasonable default choice, and so we'll choose 5 in order to minimize the amount of computation. (5 has actually been the default for `cross_val_score` since version 0.22, but I like to include it anyway for clarity.)
- Finally, we'll specify the evaluation metric of classification accuracy.

When we run it, `cross_val_score` outputs a mean accuracy of 0.811, which we'll use as the baseline accuracy against which our future Pipelines can be compared.

```
from sklearn.model_selection import cross_val_score
cross_val_score(pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8114619295712762

Pipeline accuracy scores:

- **Baseline (no tuning):** 0.811

Let's talk about what actually happens “under the hood” when we run the `cross_val_score` function on a Pipeline:

- In step 1, `cross_val_score` splits the data into 5 folds. 4 out of 5 folds (meaning 80% of the data) is set aside for training, and the remaining fold (meaning 20% of the data) is set aside for testing.
- In step 2, the Pipeline's `fit` method is run on the training portion. Thus the transformations specified in the `ColumnTransformer` are performed on the training portion, and the transformed training data is used to fit the model.
- In step 3, the Pipeline's `predict` method is run on the testing portion. Thus the transformations learned during step 2 are applied to the testing portion, and the fitted model makes predictions on the transformed testing data.
- In step 4, the accuracy of those predictions is calculated.
- Steps 1 through 4 are then repeated 4 more times, and each time a different fold is set aside as the testing portion.
- `cross_val_score` outputs 5 accuracy scores, and we take the mean of those scores.

Steps of 5-fold cross-validation on a Pipeline:

1. Split data into 5 folds (A, B, C, D, E)
 - ABCD is training set
 - E is testing set
2. Pipeline is fit on training set
 - ABCD is transformed
 - Model is fit on transformed data
3. Pipeline makes predictions on testing set
 - E is transformed (using step 2 transformations)
 - Model makes predictions on transformed data
4. Calculate accuracy of those predictions
5. Repeat the steps above 4 more times, with a different testing set each time
6. Calculate the mean of the 5 scores

One thing you might have noticed is that `cross_val_score` splits the data in step 1 before performing the transformations in steps 2 and 3. As a result, the imputation values for Age and Fare and the vocabulary for `CountVectorizer` are all computed 5 different times. Each time, these values are computed using the training set only, and then applied to both the training and testing sets.

Alternatively, you could imagine performing all of the transformations first, and then splitting the data. This would be much faster, since the imputation values and the vocabulary would be computed only once on the full dataset.

So why does `cross_val_score` split the data first? Because splitting the data before performing the transformations prevents data leakage, whereas performing the transformations on the full dataset before splitting the data would cause data leakage, since information about the testing set would be “leaked” into the model training process.

As we discussed in the previous chapter, this is one way that scikit-learn helps to shield you from data leakage.

Why does `cross_val_score` split the data first?

- **Proper cross-validation:**
 - Data is split (step 1) before transformations (steps 2 and 3)
 - Imputation values and vocabulary are computed using training set only
 - Prevents data leakage
- **Improper cross-validation:**
 - Transformations are performed before data is split
 - Imputation values and vocabulary are computed using full dataset
 - Causes data leakage

10.2 Tuning a Pipeline with grid search

Now that we've calculated the baseline accuracy for our Pipeline, the next step is to tune the hyperparameters for both the model and the transformers. Recall that we've been using the default parameters for most objects in the Pipeline, and so tuning those parameters is likely to result in a more accurate model.

Before proceeding, I'll briefly explain some terminology. In the field of statistics, "hyperparameters" are values that you set, whereas "parameters" are values learned from the data by the estimator during the fitting process.

For example, the C value of logistic regression is called a hyperparameter because it's something you can set and optimize, whereas the coefficients of a logistic regression model are called parameters because they're learned from the data.

Statistics terminology:

- **Hyperparameters:** Values that you set
 - **Example:** C value of logistic regression
- **Parameters:** Values learned from the data
 - **Example:** Coefficients of logistic regression model

In this book, I'm generally going to follow scikit-learn's conventions as I understand them:

- I'll use the phrase "hyperparameter tuning" to refer to the process of tuning a model or tuning a Pipeline containing a model and transformers.
- I'll use the phrase "parameter" to refer to anything passed to a class. For example, this includes the C and `random_state` values passed to the LogisticRegression class, and the `strategy` value passed to the SimpleImputer class.

scikit-learn terminology:

- **Hyperparameter tuning:** Tuning a model or a Pipeline
- **Parameter:** Anything passed to a class
 - **LogisticRegression:** C, random_state
 - **SimpleImputer:** strategy

With that being said, we're going to use a scikit-learn class called `GridSearchCV` to perform the hyperparameter tuning. In a grid search, you define which values you want to try for each parameter, and it cross-validates every possible combination of those values.

Hyperparameter tuning with `GridSearchCV`:

- You define which values to try for each parameter
- It cross-validates every combination of those values

We can actually use `GridSearchCV` to tune the entire Pipeline at once, including both the model and the transformers. This has two huge benefits over just tuning a model:

- First, it enables you to tune the model and transformer parameters simultaneously, which is important because the best-performing combination might be when none of the parameters are set to their default values.
- Second, this prevents data leakage because the data splitting and transformations will occur within the cross-validation done by `GridSearchCV`.

Benefits of tuning a Pipeline:

- Tunes the model and transformers simultaneously
- Prevents data leakage

Keep in mind that if we had instead done the data transformations in pandas, we would have missed out on both of these benefits.

10.3 Tuning the model

In this lesson, we're going to tune the model, and then in the next lesson, we'll also tune the transformers.

For the `LogisticRegression` model, we're going to tune two parameters:

- The first parameter is `penalty`, which is the type of regularization. For this parameter, the default value is 'l2', and we're going to try the values 'l1' and 'l2'. (To be clear, the first character of each of those values is a lowercase "l".)
- The second parameter is `C`, which is the amount of regularization. For this parameter, the default value is 1, and we're going to try the values 0.1, 1, and 10.

LogisticRegression tuning parameters:

- **penalty:** Type of regularization
 - 'l1'
 - 'l2' (default)
- **C:** Amount of regularization
 - 0.1
 - 1 (default)
 - 10

Deciding which parameters to tune and what values to try requires both research and experience, and unfortunately, it's different for every type of model.

In order to tune a Pipeline with GridSearchCV, we need to get the names of the Pipeline steps from the `named_steps` attribute. We'll tune the `logisticregression` step in this lesson, and we'll tune the `columntransformer` step in the next lesson.

```
pipe.named_steps.keys()
```

```
dict_keys(['columntransformer', 'logisticregression'])
```

To use GridSearchCV, we need to create a dictionary in which each entry represents a parameter and the values we want to try for that parameter:

- The key for each entry is the Pipeline step name, followed by two underscores, followed by the parameter name. Thus the key for the first entry is 'logisticregression__penalty', and the key for the second entry is 'logisticregression__C'. Using two underscores is what allows GridSearchCV to distinguish between the step name and the parameter name. Using a single underscore would be ambiguous, since a step name or parameter name can have an underscore within it.
- The value for each entry is a list of the values you want to try for that parameter. Thus the value for the first entry is a list of 'l1' and 'l2', and the value for the second entry is a list of 0.1, 1, and 10.

Parameter dictionary for GridSearchCV:

- **Key:** `step__parameter`
 - 'logisticregression__penalty'
 - 'logisticregression__C'
- **Value:** List of values to try
 - ['l1', 'l2']
 - [0.1, 1, 10]

We'll create an empty dictionary called `params`, add these two entries, and then print it out just to make sure that it looks correct.

```
params = {}
params['logisticregression__penalty'] = ['l1', 'l2']
params['logisticregression__C'] = [0.1, 1, 10]
params
```

```
{'logisticregression__penalty': ['l1', 'l2'],
 'logisticregression__C': [0.1, 1, 10]}
```

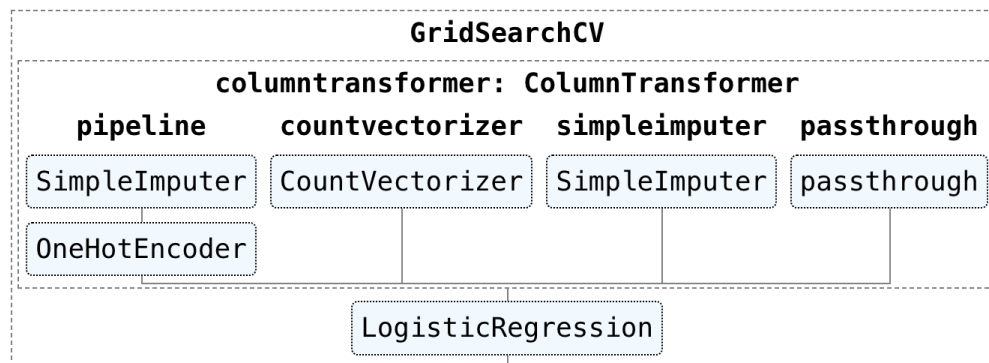
Now that we've created the parameter dictionary, we can set up the grid search. We'll import the GridSearchCV class from the model_selection module.

```
from sklearn.model_selection import GridSearchCV
```

Next, we'll create an instance of GridSearchCV called grid. We'll pass it the Pipeline, the parameter dictionary, the number of folds, and the evaluation metric.

Finally, we'll run the grid search by fitting the grid object to X and y. Because our scikit-learn configuration is set to display diagrams, it outputs a diagram of the Pipeline once the grid search is complete.

```
grid = GridSearchCV(pipe, params, cv=5, scoring='accuracy')
grid.fit(X, y)
```



The results of the grid search are stored in an attribute called cv_results_, which we'll convert to a DataFrame. We'll use a filter to only keep the columns we need, and rename the parameter columns to make them easier to read.

The DataFrame contains 6 rows because cross-validation ran 6 times, which is every possible combination of the 2 values of penalty and the 3 values of C that we specified.

```
results = (pd.DataFrame(grid.cv_results_)
           .filter(regex='param_|mean_test|rank'))
results.columns = results.columns.str.split('__').str[-1]
results
```

	C	penalty	mean_test_score	rank_test_score
0	0.1	l1	0.783385	6
1	0.1	l2	0.788990	5
2	1	l1	0.814814	2
3	1	l2	0.811462	3
4	10	l1	0.818166	1
5	10	l2	0.809234	4

Notice the `rank_test_score` column. We'll use the `DataFrame`'s `sort_values` method to sort the rows by that column in ascending order.

By examining the `mean_test_score` column, we can see that the best parameter combination resulted in a cross-validated accuracy of 0.818, which is higher than our baseline accuracy of 0.811.

We also see that the best accuracy occurred when `C` was 10 and `penalty` was 'l1', neither of which was the default value for that parameter.

```
results.sort_values('rank_test_score')
```

	C	penalty	mean_test_score	rank_test_score
4	10	l1	0.818166	1
2	1	l1	0.814814	2
3	1	l2	0.811462	3
5	10	l2	0.809234	4
1	0.1	l2	0.788990	5
0	0.1	l1	0.783385	6

Pipeline accuracy scores:

- **Grid search (2 parameters):** 0.818
- **Baseline (no tuning):** 0.811

10.4 Tuning the transformers

In the previous lesson, we built a grid search for tuning model parameters and found that the best accuracy occurred when `C` was 10 and `penalty` was 'l1'. In this lesson, we're going to expand the search to also include transformer parameters.

When expanding the search, you might first think that we should set `C` to 10 and `penalty` to 'l1', and then only search the transformer parameters, since that would be the most computationally efficient approach.

However, the better approach is actually to consider all of the values for `C` and `penalty` in combination with all of the transformer parameters. That's because we're searching for the best combination of all parameters, and since each parameter can influence what is optimal for the other parameters, the best combination might use a `C` value other than 10 or a `penalty` value other than 'l1'.

Options for expanding the grid search:

- **Initial idea:** Set `C=10` and `penalty='l1'`, then only search transformer parameters
- **Better approach:** Search for the best combination of `C`, `penalty`, and transformer parameters

All of that is to say that we're going to expand the existing params dictionary to include transformer parameters. But to include transformer parameters, we first need to figure out the transformer names.

From the previous lesson, you might recall that the first step in the Pipeline is named `columntransformer`. We'll access that step using the `named_steps` attribute, which then allows us to examine the `named_transformers_` attribute of the `ColumnTransformer`.

As an aside, `named_transformers_` ends with an underscore because it's set during the fit step, whereas `named_steps` does not end with an underscore because it's set when the Pipeline instance is created.

Anyway, we can now see the transformer names. We're going to tune a single parameter from three of the transformers. Normally I might tune more parameters, but for the sake of brevity I'm only going to tune three.

```
pipe.named_steps['columntransformer'].named_transformers_
```

```
{'pipeline': Pipeline(steps=[('simpleimputer',
                              SimpleImputer(fill_value='missing', strategy='constant')),
                              ('onehotencoder', OneHotEncoder())]),
 'countvectorizer': CountVectorizer(),
 'simpleimputer': SimpleImputer(),
 'passthrough': 'passthrough'}
```

The first parameter we're going to tune is the `drop` parameter of `OneHotEncoder`, which was added to scikit-learn in version 0.21 and which I discussed in [lesson 3.6](#).

The parameter values we're going to try are `None` and `'first'`. `None` is the default, and it means don't drop any columns, whereas `'first'` means drop the first column of each feature after encoding.

OneHotEncoder tuning parameter:

- **drop:** Method for dropping a column of each feature
 - `None` (default)
 - `'first'`

To add it to the params dictionary, we specify the Pipeline step name, which is `columntransformer`. Then we specify the transformer name, which is `pipeline`. Then we specify the step name of the inner Pipeline, which is `onehotencoder`. Finally we specify the parameter name, which is `drop`. All of these components are separated by two underscores.


```
params['columntransformer__pipeline__onehotencoder__drop'] = [None,
                                                                'first']
```

If you're ever unsure how to specify a parameter for a grid search, you can see all of the Pipeline's parameters by using the `get_params` method followed by the `keys` method. I'm converting the output to a list for easier readability. This list is also useful if you prefer to copy and paste the parameter names rather than typing them.

As you can see, there are many transformer and model parameters that we're not tuning, many of which could be useful to tune given enough time and computational resources.

```
list(pipe.get_params().keys())
```

```
['memory',
 'steps',
 'verbose',
 'columntransformer',
 'logisticregression',
 'columntransformer__n_jobs',
 'columntransformer__remainder',
 'columntransformer__sparse_threshold',
 'columntransformer__transformer_weights',
 'columntransformer__transformers',
 'columntransformer__verbose',
 'columntransformer__pipeline',
 'columntransformer__countvectorizer',
 'columntransformer__simpleimputer',
 'columntransformer__passthrough',
 'columntransformer__pipeline__memory',
 'columntransformer__pipeline__steps',
 'columntransformer__pipeline__verbose',
 'columntransformer__pipeline__simpleimputer',
 'columntransformer__pipeline__onehotencoder',
 'columntransformer__pipeline__simpleimputer__add_indicator',
 'columntransformer__pipeline__simpleimputer__copy',
 'columntransformer__pipeline__simpleimputer__fill_value',
 'columntransformer__pipeline__simpleimputer__missing_values',
 'columntransformer__pipeline__simpleimputer__strategy',
 'columntransformer__pipeline__simpleimputer__verbose',
 'columntransformer__pipeline__onehotencoder__categories',
 'columntransformer__pipeline__onehotencoder__drop',
 'columntransformer__pipeline__onehotencoder__dtype',
 'columntransformer__pipeline__onehotencoder__handle_unknown',
 'columntransformer__pipeline__onehotencoder__sparse',
 'columntransformer__countvectorizer__analyzer',
 'columntransformer__countvectorizer__binary',
 'columntransformer__countvectorizer__decode_error',
```

```

'columntransformer__countvectorizer__dtype',
'columntransformer__countvectorizer__encoding',
'columntransformer__countvectorizer__input',
'columntransformer__countvectorizer__lowercase',
'columntransformer__countvectorizer__max_df',
'columntransformer__countvectorizer__max_features',
'columntransformer__countvectorizer__min_df',
'columntransformer__countvectorizer__ngram_range',
'columntransformer__countvectorizer__preprocessor',
'columntransformer__countvectorizer__stop_words',
'columntransformer__countvectorizer__strip_accents',
'columntransformer__countvectorizer__token_pattern',
'columntransformer__countvectorizer__tokenizer',
'columntransformer__countvectorizer__vocabulary',
'columntransformer__simpleimputer__add_indicator',
'columntransformer__simpleimputer__copy',
'columntransformer__simpleimputer__fill_value',
'columntransformer__simpleimputer__missing_values',
'columntransformer__simpleimputer__strategy',
'columntransformer__simpleimputer__verbose',
'logisticregression__C',
'logisticregression__class_weight',
'logisticregression__dual',
'logisticregression__fit_intercept',
'logisticregression__intercept_scaling',
'logisticregression__l1_ratio',
'logisticregression__max_iter',
'logisticregression__multi_class',
'logisticregression__n_jobs',
'logisticregression__penalty',
'logisticregression__random_state',
'logisticregression__solver',
'logisticregression__tol',
'logisticregression__verbose',
'logisticregression__warm_start']

```

Moving along, the second parameter we're going to tune is the `ngram_range` parameter of `CountVectorizer`.

The parameter values we're going to try are the tuples (1, 1) and (1, 2). (1, 1) is the default, and it creates a single feature from each word. (1, 2) creates features from both single words, known as unigrams, and word pairs, known as bigrams.

CountVectorizer tuning parameter:

- **ngram_range:** Selection of word n-grams to be extracted as features
 - (1, 1) (default)

– (1, 2)

Again, we specify the Pipeline step name, then the transformer name, and then the parameter name. Note that these three components are separated by double underscores, but there's just a single underscore within `ngram_range` because that's part of the parameter name.

```
params['columntransformer__countvectorizer__ngram_range'] = [(1, 1),  
                                                                (1, 2)]
```

The third parameter we're going to tune is the `add_indicator` parameter of `SimpleImputer`, which was added to scikit-learn in version 0.21 and which I discussed in [lesson 7.4](#).

The parameter values we're going to try are `False` and `True`. `False` is the default, and it does not add a missing indicator column, whereas `True` does add a missing indicator column.

SimpleImputer tuning parameter:

- `add_indicator`: Option to add a missing indicator column
 - `False` (default)
 - `True`

Once again, we specify the Pipeline step name, then the transformer name, and then the parameter name.

```
params['columntransformer__simpleimputer__add_indicator'] = [False, True]
```

Before running the grid search, we'll print out the `params` dictionary. By multiplying $2 \times 3 \times 2 \times 2 \times 2$, we can calculate that there are now 48 parameter combinations, and thus the grid search will take about 8 times longer than the previous search.

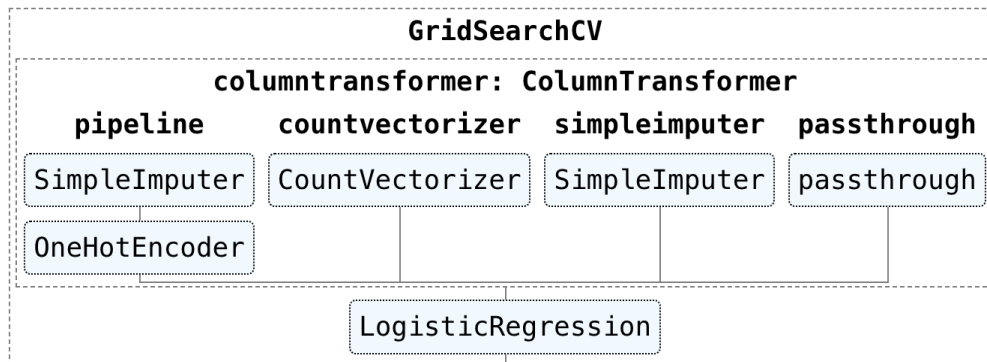
As an aside, if we had used the `Pipeline` and `ColumnTransformer` classes instead of the `make_pipeline` and `make_column_transformer` functions, we could have customized the step names and transformer names, which would have made these parameter specifications a bit easier to read and write. You can read [lessons 4.9](#) and [4.10](#) for a review of that topic.

```
params
```

```
{'logisticregression__penalty': ['l1', 'l2'],  
 'logisticregression__C': [0.1, 1, 10],  
 'columntransformer__pipeline__onehotencoder__drop': [None, 'first'],  
 'columntransformer__countvectorizer__ngram_range': [(1, 1), (1, 2)],  
 'columntransformer__simpleimputer__add_indicator': [False, True]}
```

Next we'll recreate the grid object with the new `params` dictionary, and then we'll run the grid search.

```
grid = GridSearchCV(pipe, params, cv=5, scoring='accuracy')
grid.fit(X, y)
```



Just like last time, we'll convert the search results into a DataFrame and sort it by the `rank_test_score` column.

As you can see from the `mean_test_score` column, the best accuracy of 0.828 is an improvement over the previous grid search, which had an accuracy of 0.818. Keep in mind that your exact results may differ based on your scikit-learn version along with other factors. However, there's no randomness involved when you set `cv` to an integer, and so your results will be the same every time you run this particular grid search.

```
results = (pd.DataFrame(grid.cv_results_)
           .filter(regex='param_|mean_test|rank'))
results.columns = results.columns.str.split('__').str[-1]
results.sort_values('rank_test_score')
```

	ngram_range	drop	add_indicator	C	penalty	mean_test_score	rank_test_score
34	(1, 2)	None	True	10	l1	0.828253	1
28	(1, 2)	None	False	10	l1	0.824889	2
40	(1, 2)	first	False	10	l1	0.824889	2
46	(1, 2)	first	True	10	l1	0.822648	4
16	(1, 1)	first	False	10	l1	0.820407	5
22	(1, 1)	first	True	10	l1	0.819296	6
4	(1, 1)	None	False	10	l1	0.818166	7
10	(1, 1)	None	True	10	l1	0.817061	8
20	(1, 1)	first	True	1	l1	0.814820	9
2	(1, 1)	None	False	1	l1	0.814814	10
44	(1, 2)	first	True	1	l1	0.813703	11
47	(1, 2)	first	True	10	l2	0.812598	12
8	(1, 1)	None	True	1	l1	0.812579	13
38	(1, 2)	first	False	1	l1	0.812579	14
14	(1, 1)	first	False	1	l1	0.812579	14
26	(1, 2)	None	False	1	l1	0.812567	16
11	(1, 1)	None	True	10	l2	0.811481	17
21	(1, 1)	first	True	1	l2	0.811468	18
3	(1, 1)	None	False	1	l2	0.811462	19
23	(1, 1)	first	True	10	l2	0.810363	20
9	(1, 1)	None	True	1	l2	0.810345	21
15	(1, 1)	first	False	1	l2	0.810332	22
32	(1, 2)	None	True	1	l1	0.810332	22
17	(1, 1)	first	False	10	l2	0.809234	24
35	(1, 2)	None	True	10	l2	0.809234	24
5	(1, 1)	None	False	10	l2	0.809234	24
29	(1, 2)	None	False	10	l2	0.808104	27
45	(1, 2)	first	True	1	l2	0.808097	28
41	(1, 2)	first	False	10	l2	0.806980	29
39	(1, 2)	first	False	1	l2	0.805844	30
27	(1, 2)	None	False	1	l2	0.805844	30
33	(1, 2)	None	True	1	l2	0.804739	32
31	(1, 2)	None	True	0.1	l2	0.793491	33
7	(1, 1)	None	True	0.1	l2	0.793484	34
19	(1, 1)	first	True	0.1	l2	0.791243	35
43	(1, 2)	first	True	0.1	l2	0.790114	36
37	(1, 2)	first	False	0.1	l2	0.789003	37
25	(1, 2)	None	False	0.1	l2	0.788996	38
1	(1, 1)	None	False	0.1	l2	0.788990	39
13	(1, 1)	first	False	0.1	l2	0.787885	40
0	(1, 1)	None	False	0.1	l1	0.783385	41
30	(1, 2)	None	True	0.1	l1	0.783385	41
24	(1, 2)	None	False	0.1	l1	0.783385	41
6	(1, 1)	None	True	0.1	l1	0.783385	41
36	(1, 2)	first	False	0.1	l1	0.777785	45
42	(1, 2)	first	True	0.1	l1	0.777785	45
12	(1, 1)	first	False	0.1	l1	0.777785	45
18	(1, 1)	first	True	0.1	l1	0.777785	45

Pipeline accuracy scores:

- **Grid search (5 parameters):** 0.828
- **Grid search (2 parameters):** 0.818
- **Baseline (no tuning):** 0.811

Rather than always examining the results DataFrame, we can actually just access the single best score and the set of parameters that resulted in that score via attributes of the grid object.

```
grid.best_score_
```

```
0.828253091456908
```

```
grid.best_params_
```

```
{'columntransformer__countvectorizer__ngram_range': (1, 2),  
  'columntransformer__pipeline__onehotencoder__drop': None,  
  'columntransformer__simpleimputer__add_indicator': True,  
  'logisticregression__C': 10,  
  'logisticregression__penalty': 'l1'}
```

It's worth noting that only the drop parameter is using its default value, whereas the other four parameters are not using their default values.

It's hard to say whether this truly is the best set of parameters, because some of the differences in accuracy between parameter combinations may be due to chance, based on which samples happened to appear in each fold. That's just a limitation of basic cross-validation, and so all we can say with confidence is that this is a good combination of parameters.

10.5 Using the best Pipeline to make predictions

Now that we've tuned both the model parameters and the transformer parameters, we want to use those parameters with the Pipeline when making predictions.

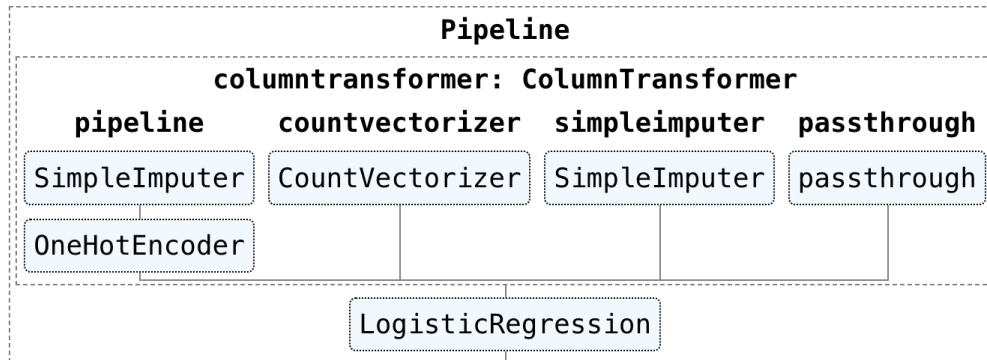
GridSearchCV actually makes this very easy. After locating the best set of parameters, it automatically refits the Pipeline on X and y using the best set of parameters, and it stores that fitted Pipeline as an attribute called `best_estimator_`. As you can see, that attribute is indeed a Pipeline object.

```
type(grid.best_estimator_)
```

```
sklearn.pipeline.Pipeline
```

If we printed out the `best_estimator_` attribute and clicked on the components, we would see that the parameters of this Pipeline match the best parameter set we located in the previous lesson.

```
grid.best_estimator_
```



In order to make predictions using this Pipeline, all we have to do is run the grid object's predict method, which calls the predict method of the best_estimator_, and pass it X_new.

```
grid.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1,
       1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1,
       1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
       1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1,
       1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,
       0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0,
       1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1,
       1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
       0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1,
       0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
       0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
       1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1])
```

I want to emphasize that this Pipeline, with the best set of parameters, was automatically refit to the entire dataset. You should always train your model on the entire dataset, meaning all samples for which you know the target value, before using it to make predictions on new data. Otherwise, you're throwing away valuable training data.

10.6 Q&A: How do I save the best Pipeline for future use?

After completing a grid search, you may want to save the Pipeline with the best set of parameters so that you can use it to make predictions later.

As we saw in the previous lesson, the Pipeline with the best set of parameters is stored as an attribute of the GridSearchCV object called `best_estimator_`, so this is the object that we want to save.

```
type(grid.best_estimator_)
```

```
sklearn.pipeline.Pipeline
```

You can save a Pipeline to a file using pickle, which is part of the Python standard library.

```
import pickle
```

We'll use pickle's `dump` method to save the Pipeline to a file called "pipe.pickle".

```
with open('pipe.pickle', 'wb') as f:
    pickle.dump(grid.best_estimator_, f)
```

Then we can use pickle's `load` method to load the Pipeline from the file into an object called `pipe_from_pickle`.

```
with open('pipe.pickle', 'rb') as f:
    pipe_from_pickle = pickle.load(f)
```

`pipe_from_pickle` is identical to `grid.best_estimator_`, and so when we use `pipe_from_pickle` to make predictions, these predictions are identical to the predictions made by the grid object.

```
pipe_from_pickle.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1,
        1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
        1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
        1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1,
        1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0,
        0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0,
        0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
        1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1,
        1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,
        0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1,
        1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1,
        1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
```



```

0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0,
0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1,
0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0,
1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1,
0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1])

```

One alternative to pickle is joblib, which is usually more efficient than pickle for scikit-learn objects. Although it's not part of the Python standard library, joblib has been a dependency of scikit-learn since version 0.21.

```
import joblib
```

Just like pickle, you use joblib's dump method to save the Pipeline to a file, which we'll call "pipe.joblib".

```
joblib.dump(grid.best_estimator_, 'pipe.joblib')
```

```
['pipe.joblib']
```

Then, we'll use the load method to load the Pipeline from the file into an object called pipe_from_joblib.

```
pipe_from_joblib = joblib.load('pipe.joblib')
```

Finally, we'll use pipe_from_joblib to make predictions.

```
pipe_from_joblib.predict(X_new)
```

```

array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1,
       1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1,
       1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
       1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1,
       1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,
       0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0,
       1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1,
       1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
       0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1,
       0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0,

```

```
1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1,
0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1])
```

To be clear, pickle and joblib are not limited to Pipelines and can be used with other scikit-learn objects, such as a standalone model object that is not inside a Pipeline.

There are a couple of warnings to keep in mind when working with pickle and joblib objects:

- First, the objects may be version-specific and architecture-specific. As such, you should only load them into an identical environment, meaning the same versions of scikit-learn and its dependencies, using the identical computing architecture.
- Second, these objects can be poisoned with malicious code, and so you should only load objects from a trusted source.

Warnings for pickle and joblib objects:

- May be version-specific and architecture-specific
- Can be poisoned with malicious code

Finally, it's worth mentioning that there are alternatives to pickle and joblib such as ONNX and PMML. These formats don't capture the full model object, but instead save a representation that can be used to make predictions. One major benefit of these formats is that they are neither environment-specific nor architecture-specific.

Alternatives to pickle and joblib:

- **Examples:** ONNX, PMML
- Save a model representation for making predictions
- Work across environments and architectures

10.7 Q&A: How do I speed up a grid search?

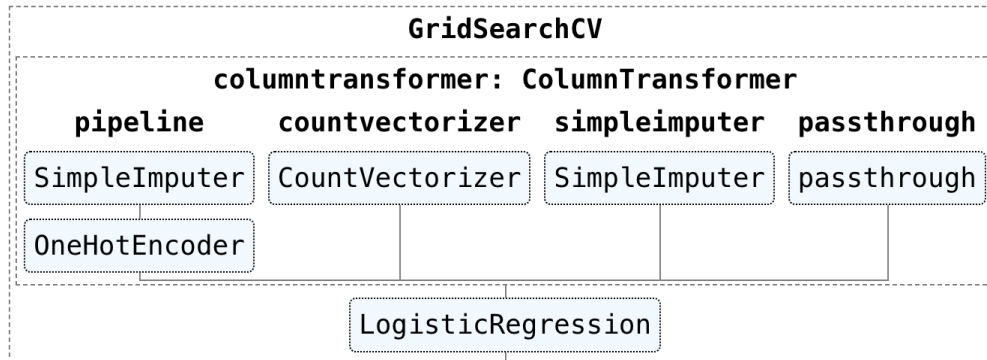
Let's recreate the GridSearchCV object, but this time we'll add the `verbose` parameter and set it to 1. When we run the search, this parameter will result in two changes to the output:

- First, it will calculate the number of parameter combinations for you, which is 48. Since this is 5-fold cross-validation, that means the Pipeline will be fit 240 times.
- Second, it will report back how long the search took, and sometimes it will give you progress updates along the way.

```
grid = GridSearchCV(pipe, params, cv=5, scoring='accuracy', verbose=1)
grid.fit(X, y)
```

Fitting 5 folds for each of 48 candidates, totalling 240 fits

```
[Parallel(n_jobs=1)]: Done 49 tasks      | elapsed:    0.5s
[Parallel(n_jobs=1)]: Done 199 tasks    | elapsed:    2.4s
```

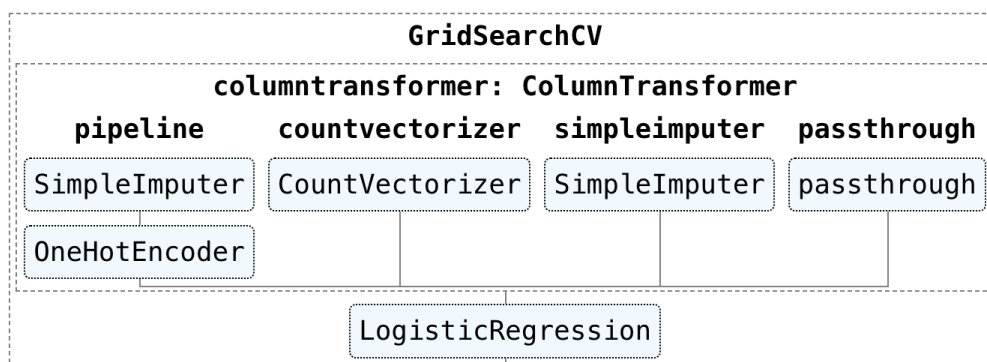


Now, let's also add the `n_jobs` parameter, set it to -1, and re-run the grid search. This instructs scikit-learn to use parallel processing with all of your CPUs to perform the search. If your machine has multiple processors, this will generally be faster, though in this case it took about the same amount of time.

```
grid = GridSearchCV(pipe, params, cv=5, scoring='accuracy', verbose=1,
                    n_jobs=-1)
grid.fit(X, y)
```

Fitting 5 folds for each of 48 candidates, totalling 240 fits

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 34 tasks      | elapsed:    1.8s
[Parallel(n_jobs=-1)]: Done 240 out of 240 | elapsed:    2.3s finished
```



If you find it useful to know how long a search takes, but verbose mode is a bit too verbose for you, another option is to remove the `verbose` parameter and instead prefix the second line with `%time`.

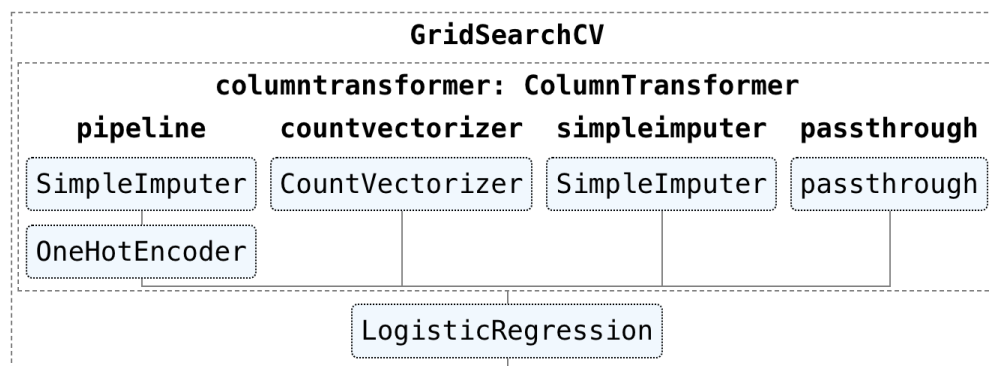
This is known as an IPython line magic, and it will work as long as you're using the Jupyter Notebook or the IPython interpreter.

All this command does is tell you how long a particular line of code took to run. The number to focus on is the wall time.

```
grid = GridSearchCV(pipe, params, cv=5, scoring='accuracy', n_jobs=-1)
%time grid.fit(X, y)
```

CPU times: user 246 ms, sys: 11.7 ms, total: 258 ms

Wall time: 752 ms



My general recommendation is to set `n_jobs` to -1 any time you're running a grid search, which is what I'll do for the rest of the book. However, it's still a good idea to use `%time` or verbose mode to confirm that parallel processing is actually reducing the search time on your particular machine.

10.8 Q&A: How do I tune a Pipeline with randomized search?

When you provide a set of parameter values to `GridSearchCV`, it will cross-validate every possible combination of those parameters. For example, we know that with this set of parameters, cross-validation will run 48 times.

```
params
```

```
{'logisticregression__penalty': ['l1', 'l2'],
 'logisticregression__C': [0.1, 1, 10],
 'columntransformer__pipeline__onehotencoder__drop': [None, 'first'],
 'columntransformer__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'columntransformer__simpleimputer__add_indicator': [False, True]}
```

Let's say that we wanted to try additional `C` values for logistic regression. I'll make a copy of the `params` dictionary called `more_params`, and then modify the `C` parameter in this dictionary to have 6 possible values instead of 3.

```
more_params = params.copy()
more_params['logisticregression__C'] = [0.01, 0.1, 1, 10, 100, 1000]
```

Since there are twice as many C values, we know that a grid search will take twice as long, meaning it will run cross-validation 96 times. But what if that grid search takes more time than we have available?

An alternative method we can use is called randomized search, which is implemented in the `RandomizedSearchCV` class. We'll import it from the `model_selection` module and then create an instance.

The API is very similar to `GridSearchCV`, except that you also specify the number of times it should run using the `n_iter` parameter. In this case, we'll set the number of iterations to 10.

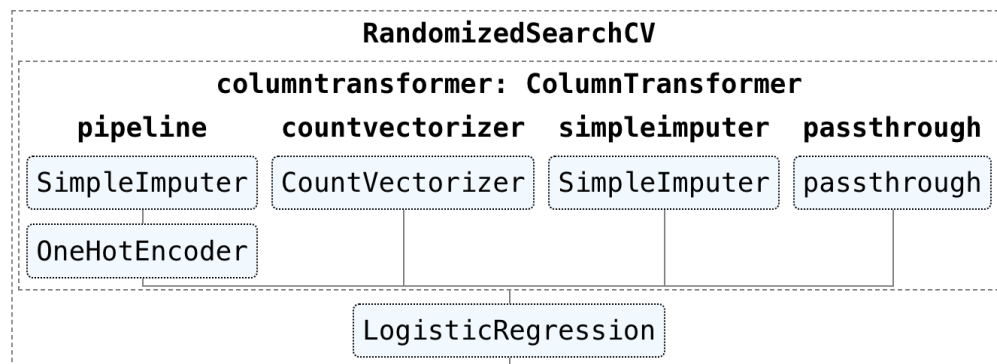
During each iteration, it picks out a set of parameters at random and cross-validates that parameter set. In other words, it does the same thing as `GridSearchCV`, except that it picks out random combinations of parameters from the parameter dictionary rather than trying every single combination. Because there's an element of randomness, we'll also set the `random_state` parameter to 1 for reproducibility.

We'll use the `fit` method to run the search, and because it will only try 10 combinations instead of 96 combinations, it will run about 10 times faster than a grid search would.

How to use `RandomizedSearchCV`:

- **n_iter**: Specify the number of randomly-chosen parameter combinations to cross-validate
- **random_state**: Set to any integer for reproducibility

```
from sklearn.model_selection import RandomizedSearchCV
rand = RandomizedSearchCV(pipe, more_params, cv=5, scoring='accuracy',
                          n_iter=10, random_state=1, n_jobs=-1)
rand.fit(X, y)
```



By printing out the results of the search, you can see that it ran 10 times using random combinations of all of those parameters.

```
results = (pd.DataFrame(rand.cv_results_)
            .filter(regex='param_|mean_test|rank'))
results.columns = results.columns.str.split('__').str[-1]
results
```

	penalty	C	add_indicator	drop	ngram_range	mean_test_score	rank_test_score
0	l1	1	True	first	(1, 1)	0.814820	3
1	l2	10	False	first	(1, 1)	0.809234	7
2	l1	1000	True	first	(1, 1)	0.811437	5
3	l2	1000	False	None	(1, 2)	0.810345	6
4	l1	10	False	first	(1, 2)	0.824889	2
5	l1	0.1	False	first	(1, 2)	0.777785	9
6	l2	1	True	None	(1, 2)	0.804739	8
7	l1	100	True	first	(1, 1)	0.813684	4
8	l1	100	False	first	(1, 2)	0.827129	1
9	l2	0.01	True	first	(1, 2)	0.744184	10

You might be surprised to know that the best score it found, 0.827, is almost as high as the best score found by our grid search earlier in the chapter, which was 0.828. That being said, we did try additional C values in our randomized search, so the comparison isn't entirely fair.

```
rand.best_score_
```

```
0.8271294959512898
```

Pipeline accuracy scores:

- **Grid search (5 parameters):** 0.828
- **Randomized search (more C values):** 0.827
- **Grid search (2 parameters):** 0.818
- **Baseline (no tuning):** 0.811

Here's the set of parameters that produced that score.

```
rand.best_params_
```

```
{'logisticregression__penalty': 'l1',
 'logisticregression__C': 100,
 'columntransformer__simpleimputer__add_indicator': False,
 'columntransformer__pipeline__onehotencoder__drop': 'first',
 'columntransformer__countvectorizer__ngram_range': (1, 2)}
```

There are four things I especially like about using a randomized search instead of a grid search:

- First, randomized search will usually find the best result (or almost the best result) in far less time than grid search, which is what we saw above. This is because there are often a lot of parameter combinations that will produce similar results to one another.

- Second, it's easier to control the computational budget of a randomized search. You can test how long a small number of searches takes, and then if you have a certain amount of time available for a search, you can simply choose the number of iterations that can be completed within that time period.
- Third, randomized search gives you the freedom to tune many more model and transformer parameters without worrying that it will take forever. You can try out a ton of different parameters for a short amount of time, and then narrow down which parameters to focus on based on what seems to be working. (We'll see this in practice in the next chapter.)
- Fourth, randomized search will sometimes produce even better results than grid search because you can try a finer grid. For example, let's say you were tuning a parameter that allowed continuous values from 0 to 1. If you were using a grid search, you might try the values 0, 0.5, and 1. But if you were using randomized search, you might try the values 0, 0.01, 0.02, and so on. It may turn out that the best value for this parameter is around 0.3, and randomized search could help you to find that out, whereas this grid search would have no chance of finding that out.

Why use RandomizedSearchCV instead of GridSearchCV?

- Similar results in far less time
- Easier to control the computational budget
- Freedom to tune many more parameters
- Can use a much finer grid

If you do need to create a fine grid of numbers for a randomized search, one useful function is NumPy's `linspace`. For example, this code specifies that I want 101 equally spaced values, starting with 0 and ending with 1.

```
import numpy as np
np.linspace(0, 1, 101)
```

```
array([0.    , 0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1 ,
       0.11, 0.12, 0.13, 0.14, 0.15, 0.16, 0.17, 0.18, 0.19, 0.2 , 0.21,
       0.22, 0.23, 0.24, 0.25, 0.26, 0.27, 0.28, 0.29, 0.3 , 0.31, 0.32,
       0.33, 0.34, 0.35, 0.36, 0.37, 0.38, 0.39, 0.4 , 0.41, 0.42, 0.43,
       0.44, 0.45, 0.46, 0.47, 0.48, 0.49, 0.5 , 0.51, 0.52, 0.53, 0.54,
       0.55, 0.56, 0.57, 0.58, 0.59, 0.6 , 0.61, 0.62, 0.63, 0.64, 0.65,
       0.66, 0.67, 0.68, 0.69, 0.7 , 0.71, 0.72, 0.73, 0.74, 0.75, 0.76,
       0.77, 0.78, 0.79, 0.8 , 0.81, 0.82, 0.83, 0.84, 0.85, 0.86, 0.87,
       0.88, 0.89, 0.9 , 0.91, 0.92, 0.93, 0.94, 0.95, 0.96, 0.97, 0.98,
       0.99, 1.   ])
```

Another similar function is NumPy's `logspace`. This code specifies that I want 6 values, from 10 to the negative 2nd power through 10 to the 3rd power.

```
np.logspace(-2, 3, 6)
```

```
array([1.e-02, 1.e-01, 1.e+00, 1.e+01, 1.e+02, 1.e+03])
```

If you're comfortable using the SciPy library, you can instead specify continuous parameters for a randomized search using SciPy distributions. However, I find it much easier to just use NumPy's `linspace` and `logspace` functions.

10.9 Q&A: What's the target accuracy we are trying to achieve?

When you're building and tuning a modeling Pipeline, it's natural to wonder how you'll know when you're done. In other words, how good of a model is "good enough"? There are three ways that I tend to think about this question.

The first way is to ask the question: What is the minimum accuracy that we need to achieve for our model to be considered useful? In most cases, you want your model to at least outperform null accuracy, which is the accuracy you could achieve by always predicting the most frequent class.

To calculate the null accuracy for our training data, we use the `value_counts` method on `y`, and set `normalize` to `True` in order to display the counts as a percentage. From the results, we can see that class 0 is the most frequent class, and about 61.6% of the `y` values are class 0.

```
y.value_counts(normalize=True)
```

	Survived
0	0.616162
1	0.383838

Thus the null accuracy for this problem is 61.6%, since an uninformed model, also known as the null model, could achieve that accuracy simply by predicting class 0 in all cases. In other words, this is the accuracy level that we want to outperform, otherwise the model is not providing any value. Thankfully, all of our Pipelines are outperforming null accuracy by a considerable amount.

Pipeline accuracy scores:

- **Grid search (5 parameters):** 0.828
- **Randomized search (more C values):** 0.827
- **Grid search (2 parameters):** 0.818
- **Baseline (no tuning):** 0.811
- **Null model:** 0.616

The second way to think about this question is to ask: What is the maximum accuracy we could eventually reach? For most real problems, it's impossible to know how accurate your model could be if you did enough tuning and tried enough models. It's also impossible to know how accurate your model could be if you gathered more samples or more features. The main exception to this is if you're working on a well-studied research problem, because in that case there may be a state-of-the-art benchmark that everyone is trying to surpass.

Thus in most practical circumstances, you don't set a target accuracy. Instead, you work to improve the model until you run out of time, money, or ideas.

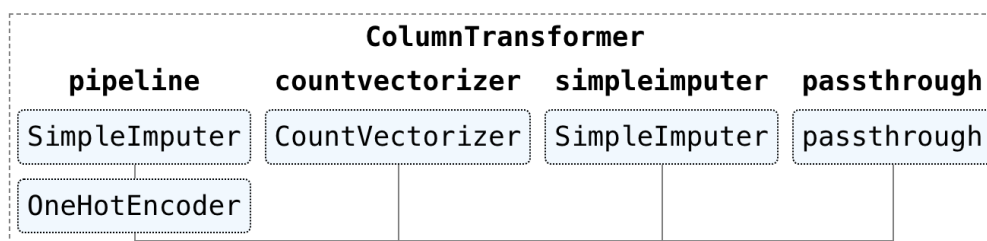
When is a model “good enough”?

- **Useful model:** Outperforms null accuracy
- **Best possible model:** Usually impossible to know the theoretical maximum accuracy
- **Practical model:** Continue improving until you run out of resources

10.10 Q&A: Is it okay that our model includes thousands of features?

The pipe object is our Pipeline that hasn’t been tuned by grid search. Recall that you can examine an individual Pipeline step by using the `named_steps` attribute. In this case, we’ll select the first step, which is our ColumnTransformer.

```
pipe.named_steps['columntransformer']
```



By passing `X` to its `fit_transform` method, we can see that the `ColumnTransformer` outputs 1518 feature columns. As we saw in lesson 8.4, all except 9 of these features were created from the `Name` column by `CountVectorizer`.

```
pipe.named_steps['columntransformer'].fit_transform(X)
```

```
<891x1518 sparse matrix of type '<class 'numpy.float64'>'
  with 7328 stored elements in Compressed Sparse Row format>
```

The cross-validated accuracy of this Pipeline is 0.811, which we’ve been calling the baseline accuracy against which other Pipelines can be compared.

```
cross_val_score(pipe, X, y, cv=5, scoring='accuracy').mean()
```

```
0.8114619295712762
```

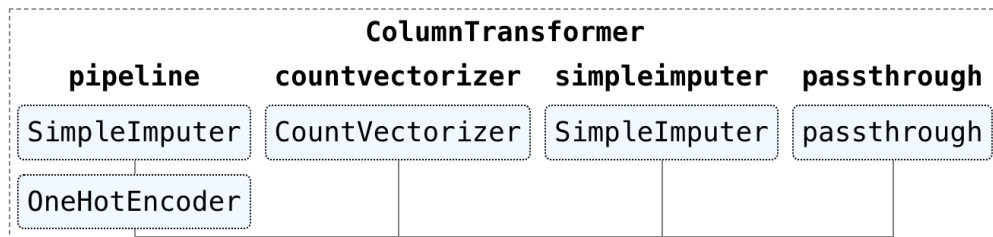
Pipeline accuracy scores:

- Grid search (5 parameters): 0.828

- **Randomized search (more C values):** 0.827
- **Grid search (2 parameters):** 0.818
- **Baseline (no tuning):** 0.811
- **Null model:** 0.616

Similarly, we can select the ColumnTransformer from our Pipeline that was tuned by grid search. If we clicked on CountVectorizer in the diagram, we would see that ngram_range is (1, 2), meaning CountVectorizer will create features from both unigrams and bigrams in the Name column.

```
grid.best_estimator_.named_steps['columntransformer']
```



By passing X to its fit_transform method, we can see that this ColumnTransformer outputs 3671 feature columns. Again, all except 9 of these features were created from the Name column.

```
grid.best_estimator_.named_steps['columntransformer'].fit_transform(X)
```

```
<891x3671 sparse matrix of type '<class 'numpy.float64'>'
  with 10191 stored elements in Compressed Sparse Row format>
```

The cross-validated accuracy of this Pipeline is 0.828.

```
grid.best_score_
```

```
0.828253091456908
```

Pipeline accuracy scores:

- **Grid search (5 parameters):** 0.828
- **Randomized search (more C values):** 0.827
- **Grid search (2 parameters):** 0.818
- **Baseline (no tuning):** 0.811
- **Null model:** 0.616

Finally, let's compare these two Pipelines to a Pipeline that doesn't include the Name column. First, we'll create a ColumnTransformer called no_name_ct that excludes Name.

```
no_name_ct = make_column_transformer(
    (imp_ohe, ['Embarked', 'Sex']),
    (imp, ['Age', 'Fare']),
    ('passthrough', ['Parch']))
```

As you can see, this ColumnTransformer only outputs 9 feature columns.

```
no_name_ct.fit_transform(X).shape
```

(891, 9)

Then, we'll add `no_name_ct` to a Pipeline called `no_name_pipe` and cross-validate it. The accuracy is 0.783, which is significantly lower than the Pipelines that included the Name column.

To be fair, this Pipeline hasn't been tuned, though honestly there is no hyperparameter tuning we could do to make it perform as well as the Pipelines that included the Name column.

```
no_name_pipe = make_pipeline(no_name_ct, logreg)
cross_val_score(no_name_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.7833908731404181

Pipeline accuracy scores:

- Grid search (5 parameters): 0.828
- Randomized search (more C values): 0.827
- Grid search (2 parameters): 0.818
- Baseline (no tuning): 0.811
- Baseline excluding Name (no tuning): 0.783
- Null model: 0.616

Here are some conclusions that we can draw from this experiment:

- First, including the Name column in the Pipeline significantly increased the cross-validated accuracy, which means that adding those thousands of feature columns did not result in overfitting. Instead, it tells us that the Name column contains more predictive signal than noise with respect to the target.
- More generally, this experiment tells us that having more features than samples does not necessarily result in overfitting.

What did we learn?

- Name column contains more predictive signal than noise
- More features than samples does not necessarily result in overfitting

It's worth noting that there is additional tuning we could do to CountVectorizer to reduce the number of features it creates. However, there's no way to know whether that would increase or decrease the Pipeline's accuracy without actually trying it.

10.11 Q&A: How do I examine the coefficients of a Pipeline?

Recall that once a grid search is complete, `GridSearchCV` automatically refits the Pipeline on `X` and `y` and stores it as an attribute called `best_estimator_`. Therefore, we can access the model coefficients by first selecting the `logisticregression` step and then selecting the `coef_` attribute.

```
grid.best_estimator_.named_steps['logisticregression'].coef_  
  
array([[ 0.56431161,  0.          , -0.08767203, ...,  0.01408723,  
        -0.43713268, -0.46358519]])
```

Ideally, we would also be able to get the names of the features that correspond to these coefficients by running the `get_feature_names` method on the `ColumnTransformer` step. However, `get_feature_names` only works if all of the underlying transformers have a `get_feature_names` method, and that is not the case here.

```
grid.best_estimator_.named_steps['columntransformer'].get_feature_names()
```

`AttributeError: Transformer pipeline does not provide get_feature_names`

Instead, as we saw previously in lesson [8.4](#), you would have to inspect the transformers one-by-one in order to determine the feature names.

```
grid.best_estimator_.named_steps['columntransformer'].transformers_  
  
[('pipeline',  
  Pipeline(steps=[('simpleimputer',  
                   SimpleImputer(fill_value='missing', strategy='constant')),  
                   ('onehotencoder', OneHotEncoder()))),  
  ['Embarked', 'Sex']),  
 ('countvectorizer', CountVectorizer(ngram_range=(1, 2)), 'Name'),  
 ('simpleimputer', SimpleImputer(add_indicator=True), ['Age', 'Fare']),  
 ('passthrough', 'passthrough', ['Parch'])]
```

Note that starting in scikit-learn version 1.1, the `get_feature_names_out` method should work on this `ColumnTransformer`, since the `get_feature_names_out` method will be available for all transformers.

10.12 Q&A: Should I split the dataset before tuning the Pipeline?

When we perform a grid search, we're trying to find the parameters that maximize the cross-validation score on a dataset. Thus, we're using the same data to accomplish two separate goals:

- First, to choose the best parameters for the Pipeline, which are stored in the `best_params_` attribute.
- Second, to estimate the future performance of the Pipeline on new data when using these parameters, which is stored in the `best_score_` attribute.

Goals of a grid search:

- Choose the best parameters for the Pipeline
- Estimate its performance on new data when using these parameters

```
grid.best_params_
```

```
{'columntransformer__countvectorizer__ngram_range': (1, 2),  
 'columntransformer__pipeline__onehotencoder__drop': None,  
 'columntransformer__simpleimputer__add_indicator': True,  
 'logisticregression__C': 10,  
 'logisticregression__penalty': 'l1'}
```

```
grid.best_score_
```

```
0.828253091456908
```

Using the same data for these two separate goals actually biases the Pipeline to this dataset and can result in overly optimistic scores.

If your main objective is to choose the best parameters, then this process is totally fine. You'll just have to accept that its actual performance on new data may be lower than the performance estimated by grid search.

But if you also need a realistic estimate of the Pipeline's performance on new data, then there's an alternative process you can use, which I'll walk you through in this lesson.

Is it okay to use the same data for both goals?

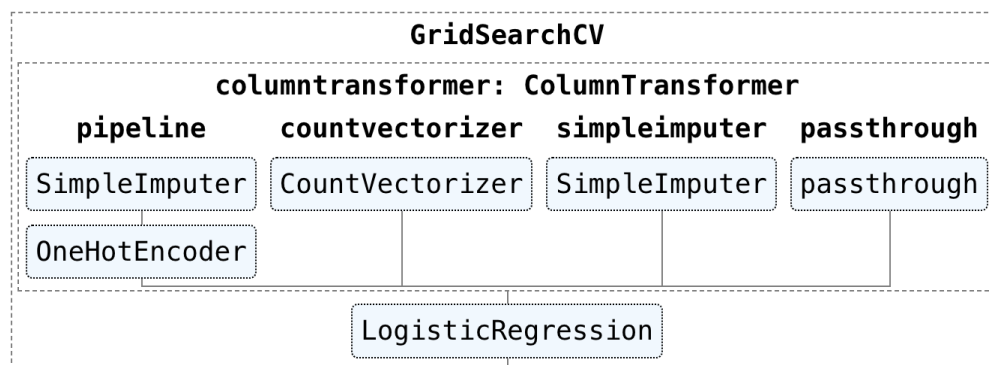
- **Yes:** If your main objective is to choose the best parameters
- **No:** If you need a realistic estimate of performance on new data

To start, we'll import the `train_test_split` function from the `model_selection` module and use it to split the data, with 75% of the data as training and 25% of the data as testing. Note that I set the `stratify` parameter to `y` so that the class proportions will be approximately equal in the training and testing sets.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
                                                    random_state=1,
                                                    stratify=y)
```

Next, we'll create a new GridSearchCV object called training_grid. When we run the grid search, we'll only pass it the training set so that the tuning process only takes the training set into account.

```
training_grid = GridSearchCV(pipe, params, cv=5, scoring='accuracy',
                             n_jobs=-1)
training_grid.fit(X_train, y_train)
```



Here are the best parameters found by grid search on the training set.

```
training_grid.best_params_
```

```
{'columntransformer__countvectorizer__ngram_range': (1, 2),
 'columntransformer__pipeline__onehotencoder__drop': 'first',
 'columntransformer__simpleimputer__add_indicator': False,
 'logisticregression__C': 10,
 'logisticregression__penalty': 'l2'}
```

We're not actually interested in the best score found during the grid search. Instead, we're going to use the best parameters found by the grid search to make predictions for the testing set, and then evaluate the accuracy of those predictions. We can do this by passing the testing set to the training_grid's score method.

The accuracy it outputs is 0.816, which is a more realistic estimate of how the Pipeline will perform on new data, since the testing set is brand new data that the Pipeline has never seen. However, it's still just a single realization of this model, and so it's impossible to know how precise this value is.

```
training_grid.score(X_test, y_test)
```

```
0.8161434977578476
```

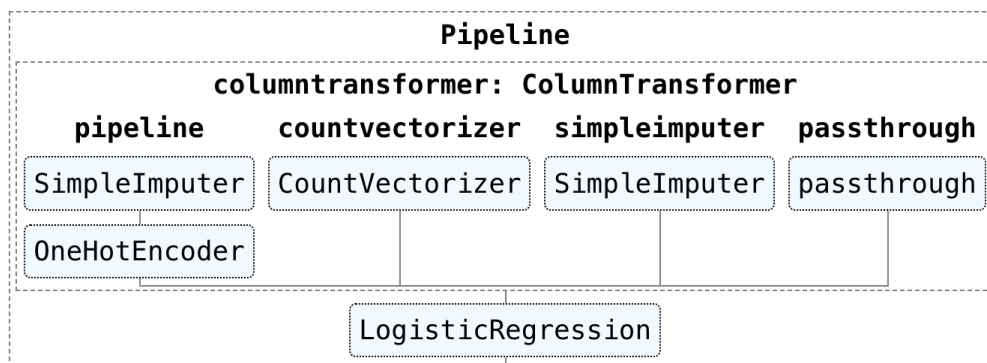
Pipeline accuracy scores:

- Grid search (5 parameters): 0.828
- Randomized search (more C values): 0.827
- Grid search (2 parameters): 0.818
- Grid search (estimate for new data): 0.816
- Baseline (no tuning): 0.811
- Baseline excluding Name (no tuning): 0.783
- Null model: 0.616

Now that we've found the best parameters for the Pipeline and estimated its likely performance on new data, our final step is to actually make predictions on new data. Before making predictions, it's critical that we train the Pipeline on all of our data, meaning the entirety of X and y, otherwise we're throwing away valuable data.

In other words, we can't simply use the `training_grid`'s `predict` method since it was only refit on `X_train` and `y_train`. Instead, we need to save the Pipeline with the best parameters, which we'll call `best_pipe`, and fit it to X and y.

```
best_pipe = training_grid.best_estimator_  
best_pipe.fit(X, y)
```



Now we can make predictions on new data.

```
best_pipe.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1,  
       1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,  
       1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,  
       1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1,  
       1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,  
       0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,  
       1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,  
       1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1])
```

```
0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0,
1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0,
0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1,
0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0,
1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1,
0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1])
```

If you decide that you're going to follow the process that I've just outlined, then there are two important guidelines to follow:

First, you should only use the testing set for evaluating Pipeline performance one time. If you keep tuning the Pipeline again and again, each time checking its performance on the testing set, you're essentially tuning the Pipeline to the particulars of the testing set. At that point, it no longer functions as an independent data source and thus its performance estimates will become less reliable.

Second, it's important that you have enough data in order for the training and testing sets to both be sufficiently large once the dataset has been split:

- If the training set is too small, then the grid search won't have enough data to find the optimal parameters.
- If the testing set is too small, then it won't be able to provide a reliable estimate of Pipeline performance.

Both of these situations would defeat the purpose of splitting the dataset, and thus this approach is best when you have a large enough dataset. Unfortunately, it's difficult to say in the abstract how much data is "enough", since that depends on the particulars of the dataset and the problem.

Guidelines for using this process:

- **Only use the testing set once:**
 - If used multiple times, performance estimates will become less reliable
- **You must have enough data:**
 - If training set is too small, grid search won't find the optimal parameters
 - If testing set is too small, it won't provide a reliable performance estimate

10.13 Q&A: What is regularization?

Earlier in this chapter, we tuned the regularization parameters of logistic regression. In this lesson, I'll briefly explain what regularization actually is.

Regularization is a process that constrains the size of a model's coefficients in order to minimize overfitting. Overfitting is when your model fits too closely to patterns in the training data, which causes your model not to perform well when it makes predictions on new data.

Regularization minimizes overfitting by reducing the variance of the model. Thus if you believe a model is too complex, regularization will reduce the error due to variance more than it increases the error due to bias, resulting in a model that is more likely to generalize to new data.

In simpler terms, regularization makes your model a bit less flexible so that it's more likely to follow the true patterns in the data and less likely to follow the noise. Regularization is especially useful when you have outliers in the training data, because regularization decreases the influence that outliers have on the model.

Brief explanation of regularization:

- Constrains the size of model coefficients to minimize overfitting
- Reduces the variance of an overly complex model to help the model generalize
- Decreases model flexibility so that it follows the true patterns in the data

11 Comparing linear and non-linear models

11.1 Trying a random forest model

So far, the only model we've used in this book is logistic regression. But what if you wanted to try a different model?

One great thing about the scikit-learn API is that once you've built a workflow, you can easily swap in a different model, usually without making any other changes to your workflow. This is a huge benefit of scikit-learn, since it's not possible to know ahead of time which model is going to work best for a given problem and dataset. This is also known as the “no free lunch” theorem.

In this chapter, we're going to try out the random forest model, which is one of the most well-known models in Machine Learning. Whereas logistic regression is a linear model, random forests is a non-linear model based on decision trees. These two types of models have different overall properties, thus it may turn out that one type is better suited to this particular problem than the other type.

Random forest model:

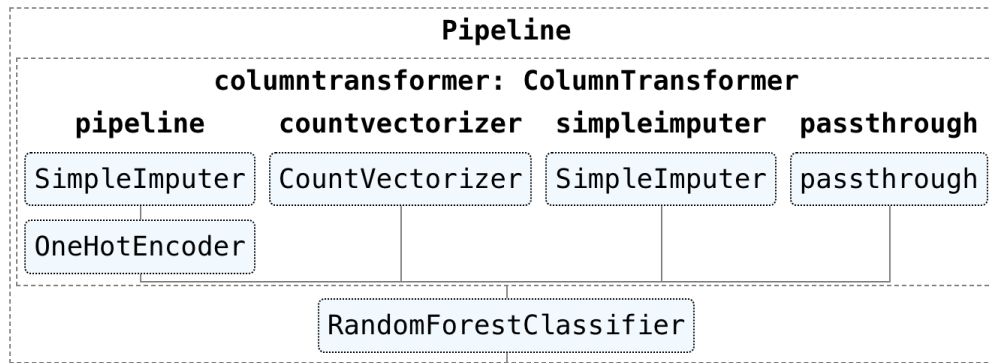
- Non-linear model
- Based on decision trees
- Different properties from logistic regression

We'll start out by importing the `RandomForestClassifier` class from the `ensemble` module, and creating an instance called `rf`. Because there's randomness involved in a random forest, we'll set `random_state` for reproducibility. And because building a random forest can be computationally expensive, it has its own `n_jobs` parameter (just like grid search and randomized search), which we'll set to `-1` to enable parallel processing.

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(random_state=1, n_jobs=-1)
```

We'll create a new `Pipeline` object called `rf_pipe` that uses random forests instead of logistic regression.

```
rf_pipe = make_pipeline(ct, rf)
rf_pipe
```



Finally, we can cross-validate it to generate a baseline accuracy, which is 0.811. This accuracy is nearly identical to the baseline accuracy of our logistic regression Pipeline, but it's likely that we can improve it through hyperparameter tuning.

```
cross_val_score(rf_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.811436821291821

Pipeline accuracy scores:

- **Grid search (LR):** 0.828
- **Baseline (LR):** 0.811
- **Baseline (RF):** 0.811

As an aside, I've simplified the Pipeline accuracy scores table to only include the most important scores from the previous chapter. As you might guess, "LR" stands for logistic regression and "RF" stands for random forests. And to be clear, I'll always use the term "baseline" in this table to describe a Pipeline that has not undergone any hyperparameter tuning via grid search or randomized search.

11.2 Tuning random forests with randomized search

When tuning random forests, we'll try tuning the same transformer parameters as we did in the previous chapter, but different parameters for the model. It's still important to tune the transformers and the model at the same time, because it may turn out that the best data transformations for a random forest model are different than the best data transformations for a logistic regression model.

Rather than typing out a parameter dictionary from scratch, we can start by creating a copy of the params dictionary called `rf_params`.

```
rf_params = params.copy()
rf_params
```

```
{'logisticregression__penalty': ['l1', 'l2'],
 'logisticregression__C': [0.1, 1, 10],
 'columntransformer__pipeline__onehotencoder__drop': [None, 'first'],
 'columntransformer__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'columntransformer__simpleimputer__add_indicator': [False, True]}
```

Then we'll delete the entries from the `rf_params` dictionary that only apply to logistic regression.

```
del rf_params['logisticregression__penalty']
del rf_params['logisticregression__C']
rf_params
```

```
{'columntransformer__pipeline__onehotencoder__drop': [None, 'first'],
 'columntransformer__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'columntransformer__simpleimputer__add_indicator': [False, True]}
```

Alternatively, we could have created the `rf_params` dictionary using a dictionary comprehension that only keeps the entries in `params` that start with the letters “col”.

```
rf_params = {k:v for k, v in params.items() if k.startswith('col')}
rf_params
```

```
{'columntransformer__pipeline__onehotencoder__drop': [None, 'first'],
 'columntransformer__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'columntransformer__simpleimputer__add_indicator': [False, True]}
```

Now we're ready to add tuning parameters for the `RandomForestClassifier`. Random forests has a lot of parameters you can tune, which can make for a computationally expensive grid search if you try to tune all of them. This is compounded by the fact that random forests is comparatively slower to train than logistic regression.

When you're not quite sure which parameters to tune or which values to try for those parameters, I would suggest a two-step approach, which is what we'll use in this chapter:

1. First, use a randomized search with a variety of parameters and values. This allows you to test out a lot of different combinations while still controlling the computational budget. Examine the results of the search to look for trends of what's working and what's not.
2. Second, use a grid search with a more optimized set of parameters and values, based on what you learned from the randomized search.

Two-step approach to hyperparameter tuning:

1. **Randomized search:** Test a variety of parameters and values, then examine the results for trends
2. **Grid search:** Use an optimized set of parameters and values based on what you learned from step 1

We'll start by trying out four parameters from the `RandomForestClassifier`, which I selected based on research and experience.

First, we'll confirm that the Pipeline step name is `randomforestclassifier`.

```
rf_pipe.named_steps.keys()
```

```
dict_keys(['columntransformer', 'randomforestclassifier'])
```

Then, we'll add the four parameters I selected and some reasonable values for those parameters to the `rf_params` dictionary:

- `n_estimators` is the number of decision trees in the random forest.
- `min_samples_leaf` is a way to control overfitting, just like regularization is used to control overfitting in a logistic regression model.
- `max_features` and `bootstrap` affect certain properties of the random forest algorithm.

RandomForestClassifier tuning parameters:

- **`n_estimators`:** Number of decision trees in the forest
 - 100 (default)
 - 300
 - 500
 - 700
- **`min_samples_leaf`:** Minimum number of samples at a leaf node
 - 1 (default)
 - 2
 - 3
- **`max_features`:** Number of features to consider when choosing a split
 - 'sqrt' (default)
 - None
- **`bootstrap`:** Whether bootstrap samples are used when building trees
 - True (default)
 - False

```
rf_params['randomforestclassifier__n_estimators'] = [100, 300, 500, 700]
rf_params['randomforestclassifier__min_samples_leaf'] = [1, 2, 3]
rf_params['randomforestclassifier__max_features'] = ['sqrt', None]
rf_params['randomforestclassifier__bootstrap'] = [True, False]
rf_params
```

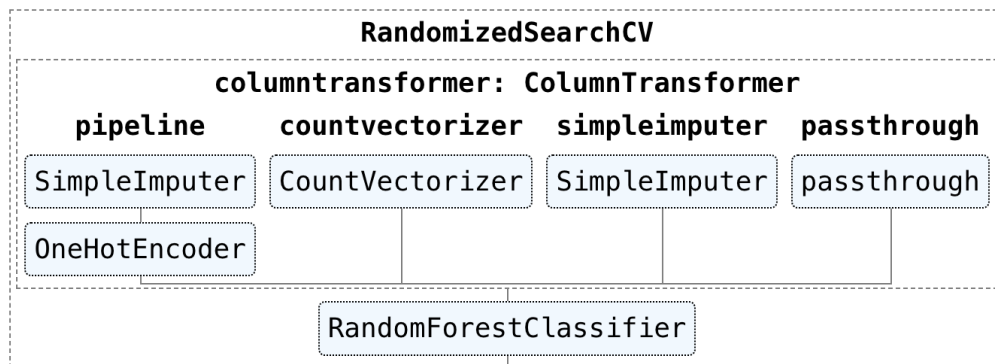
```
{'columntransformer__pipeline__onehotencoder__drop': [None, 'first'],
```

```
'columntransformer__countvectorizer__ngram_range': [(1, 1), (1, 2)],
'columntransformer__simpleimputer__add_indicator': [False, True],
'randomforestclassifier__n_estimators': [100, 300, 500, 700],
'randomforestclassifier__min_samples_leaf': [1, 2, 3],
'randomforestclassifier__max_features': ['sqrt', None],
'randomforestclassifier__bootstrap': [True, False]}
```

Finally, we'll create an instance of `RandomizedSearchCV` called `rf_rand`, making sure to use the `rf_pipe` and `rf_params` objects, and we'll run 100 iterations of the randomized search.

```
rf_rand = RandomizedSearchCV(rf_pipe, rf_params, cv=5,
                             scoring='accuracy', n_iter=100,
                             random_state=1, n_jobs=-1)
%time rf_rand.fit(X, y)
```

CPU times: user 2.91 s, sys: 347 ms, total: 3.25 s
Wall time: 1min 33s



We see that the best score found during the randomized search is 0.825, which is better than our baseline score of 0.811, but not quite as good as the 0.828 score of our best logistic regression Pipeline.

```
rf_rand.best_score_
```

0.8249262444291003

Pipeline accuracy scores:

- Grid search (LR): 0.828
- Randomized search (RF): 0.825
- Baseline (LR): 0.811
- Baseline (RF): 0.811

11.3 Further tuning with grid search

Let's now examine the results from the randomized search to look for trends, specifically focusing on the top 20 results. We'll convert the results to a DataFrame, filter to only keep the columns we need, rename the parameter columns, and then sort by `rank_test_score`. Keep in mind that it's hard to draw any definitive conclusions since this is a randomized search, so what we're looking for is just any obvious trends.

```
results = (pd.DataFrame(rf_rand.cv_results_)
            .filter(regex='param_|mean_test|rank'))
results.columns = results.columns.str.split('__').str[-1]
results.sort_values('rank_test_score').head(20)
```

	n_est...	min_sam...	max_fea...	boot...	add_ind...	drop	ngram...	mean...	rank...
13	700	3	None	True	False	first	(1, 1)	0.824926	1
70	700	3	None	True	False	None	(1, 1)	0.822685	2
31	500	3	None	True	True	first	(1, 2)	0.822685	2
45	300	2	None	True	False	None	(1, 1)	0.822679	4
33	300	3	None	True	False	first	(1, 1)	0.821562	5
54	300	2	None	True	True	None	(1, 1)	0.821549	6
68	500	3	None	True	False	None	(1, 2)	0.820444	7
81	100	3	None	True	True	first	(1, 2)	0.820432	8
15	300	2	None	True	True	first	(1, 1)	0.819315	9
94	500	2	None	True	True	first	(1, 1)	0.819308	10
98	100	2	None	True	True	first	(1, 1)	0.819308	10
63	700	1	None	True	True	first	(1, 1)	0.819283	12
18	700	2	None	True	True	first	(1, 1)	0.818191	13
57	500	2	None	True	False	None	(1, 1)	0.818185	14
12	100	3	None	True	True	None	(1, 2)	0.818185	14
72	300	1	None	True	False	None	(1, 1)	0.818178	16
2	500	1	sqrt	False	False	first	(1, 1)	0.818160	17
41	700	2	None	True	True	None	(1, 1)	0.817067	18
10	500	2	None	True	False	first	(1, 2)	0.817061	19
8	500	2	None	True	True	None	(1, 2)	0.817061	19

Starting with `n_estimators`, we see that higher numbers are performing better, which is typical for `n_estimators`. It seems unlikely that 100 will produce the best result, so we'll exclude that value from our grid search. And since the current best result is at 700, it seems useful to add a value of 900 to our grid search, in case increasing it further is even better.

Do keep in mind that increasing `n_estimators` also increases the time needed to train the model. You could consider just setting a single large value for `n_estimators` rather than searching through multiple values, since larger values will generally produce better results up to a certain point, but I prefer to tune this value when computational resources allow for it.

The next parameter to examine is `min_samples_leaf`. Similar to `n_estimators`, the lowest value of 1 seems unlikely to produce the best result, so we'll remove it. The current best result is 3, so we'll also try the values 4 and 5 in the grid search.

For `max_features`, it's clear that `None` is performing better, so we're no longer going to try `'sqrt'`.

For bootstrap, it's clear that True is performing better, so we're no longer going to try False. Finally, there aren't any clear trends for the transformer parameters, so we'll leave those as-is.

Trends in the randomized search results:

- **n_estimators:**
 - Higher numbers are performing better
 - Remove 100, add 900
- **min_samples_leaf:**
 - Higher numbers are performing better
 - Remove 1, add 4 and 5
- **max_features:**
 - None is performing better
 - Remove 'sqrt'
- **bootstrap:**
 - True is performing better
 - Remove False
- **Transformer parameters:**
 - No clear trends
 - Leave as-is

Here are the updated values we're going to try. For max_features and bootstrap, you'll see that we can just pass a list with a single value so that this parameter value will always get set during the search.

```
rf_params['randomforestclassifier__n_estimators'] = [300, 500, 700, 900]
rf_params['randomforestclassifier__min_samples_leaf'] = [2, 3, 4, 5]
rf_params['randomforestclassifier__max_features'] = [None]
rf_params['randomforestclassifier__bootstrap'] = [True]
rf_params
```

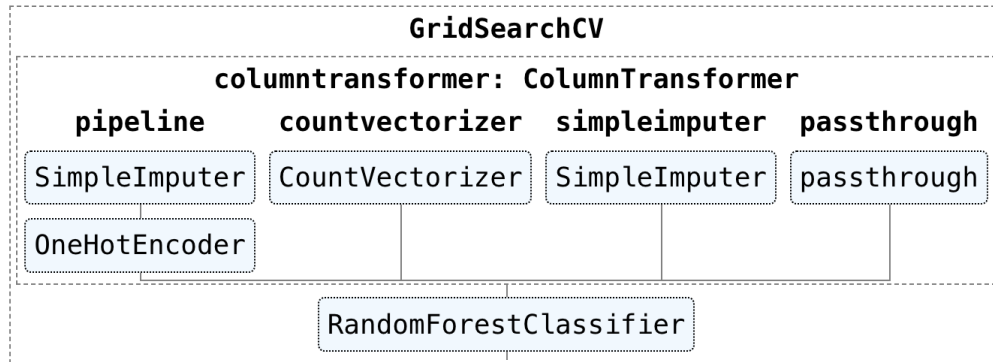
```
{'columntransformer__pipeline__onehotencoder__drop': [None, 'first'],
 'columntransformer__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'columntransformer__simpleimputer__add_indicator': [False, True],
 'randomforestclassifier__n_estimators': [300, 500, 700, 900],
 'randomforestclassifier__min_samples_leaf': [2, 3, 4, 5],
 'randomforestclassifier__max_features': [None],
 'randomforestclassifier__bootstrap': [True]}
```

At this point, you could continue to run additional randomized searches in order to study the trends further, but we're just going to move on to the grid search.

We'll create an instance of GridSearchCV called `rf_grid`, making sure to use the `rf_pipe` and `rf_params` objects, and then run the search.

```
rf_grid = GridSearchCV(rf_pipe, rf_params, cv=5, scoring='accuracy',
                      n_jobs=-1)
%time rf_grid.fit(X, y)
```

CPU times: user 2.21 s, sys: 245 ms, total: 2.45 s
Wall time: 3min 11s



The best score from our grid search is 0.829, which is just a tiny bit higher than the 0.828 score of our best logistic regression Pipeline.

```
rf_grid.best_score_
```

0.8294143493817087

Pipeline accuracy scores:

- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Randomized search (RF): 0.825
- Baseline (LR): 0.811
- Baseline (RF): 0.811

These are the best parameters that were found during the grid search. Again, it's hard to say whether this is truly the best set of parameters, but we at least know that it's a good set of parameters.

```
rf_grid.best_params_
```

```
{'columntransformer__countvectorizer__ngram_range': (1, 1),
 'columntransformer__pipeline__onehotencoder__drop': 'first',
 'columntransformer__simpleimputer__add_indicator': True,
 'randomforestclassifier__bootstrap': True,
 'randomforestclassifier__max_features': None,
 'randomforestclassifier__min_samples_leaf': 4,
 'randomforestclassifier__n_estimators': 300}
```

11.4 Q&A: How do I tune two models with a single grid search?

So far, we've set up two separate Pipelines called `pipe` and `rf_pipe` that each end in a different model. That made it easy to grid search each Pipeline with its own set of relevant parameters. However, you can actually tune two different models using a single grid search if you like.

To do this, the first step is to create a new Pipeline using the Pipeline class instead of the `make_pipeline` function. The reason we're doing this is so that we can provide custom names for the steps. In this case, we'll call the Pipeline object `both_pipe`, and we'll call the step names 'preprocessor' and 'classifier'. We'll set the classifier to be logistic regression, though this is just a placeholder as you'll see shortly.

```
both_pipe = Pipeline([('preprocessor', ct), ('classifier', logreg)])
```

Next, we'll create a new parameter dictionary called `params1`. For the simplicity of this example, we're only going to tune one parameter from the preprocessor step and two parameters from the classifier step.

Additionally, we're going to add one more entry to the dictionary to indicate that the classifier we want to use with this parameter set is logistic regression. Notice that this is a logistic regression object, not a string, and also notice that we put it in brackets to make it a list. This might seem strange, but it will make more sense momentarily.

```
params1 = {}
params1['preprocessor__countvectorizer__ngram_range'] = [(1, 1), (1, 2)]
params1['classifier__penalty'] = ['l1', 'l2']
params1['classifier__C'] = [0.1, 1, 10]
params1['classifier'] = [logreg]
params1
```

```
{'preprocessor__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'classifier__penalty': ['l1', 'l2'],
 'classifier__C': [0.1, 1, 10],
 'classifier': [LogisticRegression(random_state=1, solver='liblinear')]}
```

Next, we'll create another parameter dictionary called `params2`. Again, we'll tune one parameter from the preprocessor step and two parameters from the classifier step. You'll notice that the classifier parameters are random forest parameters, not logistic regression parameters.

Just like above, we'll add one more entry to the dictionary to indicate that the classifier we want to use with this parameter set is random forests. During the grid search, this will override the logistic regression classifier we specified when creating the Pipeline.

```
params2 = {}
params2['preprocessor__countvectorizer__ngram_range'] = [(1, 1), (1, 2)]
params2['classifier__n_estimators'] = [300, 500]
params2['classifier__min_samples_leaf'] = [3, 4]
```

```
params2['classifier'] = [rf]
params2
```

```
{'preprocessor__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'classifier__n_estimators': [300, 500],
 'classifier__min_samples_leaf': [3, 4],
 'classifier': [RandomForestClassifier(n_jobs=-1, random_state=1)]}
```

Next, we'll create a list called `both_params` that includes both of these parameter sets.

```
both_params = [params1, params2]
both_params
```

```
[{'preprocessor__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'classifier__penalty': ['l1', 'l2'],
 'classifier__C': [0.1, 1, 10],
 'classifier': [LogisticRegression(random_state=1, solver='liblinear')]},
 {'preprocessor__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'classifier__n_estimators': [300, 500],
 'classifier__min_samples_leaf': [3, 4],
 'classifier': [RandomForestClassifier(n_jobs=-1, random_state=1)]}]
```

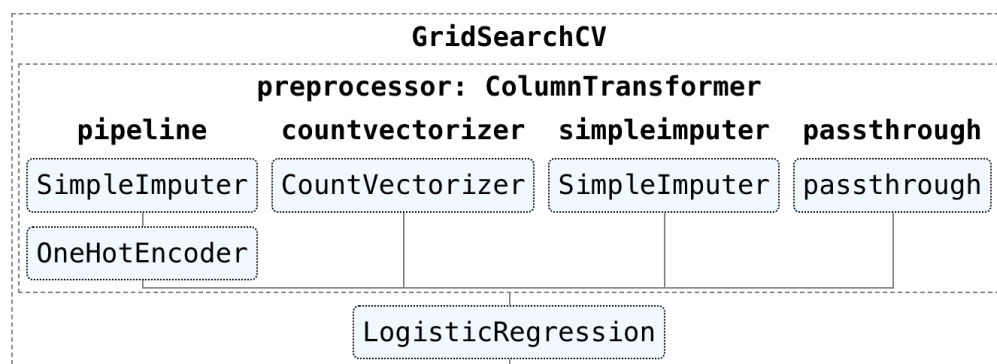
Finally, we'll create an instance of `GridSearchCV` called `both_grid`, making sure to pass it the `both_pipe` and `both_params` objects.

When we run the search, here's what will happen:

1. First, it will try every combination of parameters from `params1`, which is $2 \times 2 \times 3 \times 1$, or 12 combinations.
2. Second, it will try every combination of parameters from `params2`, which is $2 \times 2 \times 2 \times 1$, or 8 combinations.

Thus, it will run a total of 20 times.

```
both_grid = GridSearchCV(both_pipe, both_params, cv=5,
                        scoring='accuracy', n_jobs=-1)
both_grid.fit(X, y)
```



Let's take a look at the results. You can see that it ran 12 times with a logistic regression model and 8 times with a random forest model. Also notice that when the logistic regression model runs, the random forest-related parameters are listed as NaN, and vice versa when the random forest model runs.

```
results = (pd.DataFrame(both_grid.cv_results_)
           .filter(regex='param_.+_|mean_test'))
results.columns = results.columns.str.split('__').str[-1]
results
```

	C	penalty	ngram_range	min_samples_leaf	n_estimators	mean_test_score
0	0.1	l1	(1, 1)	NaN	NaN	0.783385
1	0.1	l1	(1, 2)	NaN	NaN	0.783385
2	0.1	l2	(1, 1)	NaN	NaN	0.788990
3	0.1	l2	(1, 2)	NaN	NaN	0.788996
4	1	l1	(1, 1)	NaN	NaN	0.814814
5	1	l1	(1, 2)	NaN	NaN	0.812567
6	1	l2	(1, 1)	NaN	NaN	0.811462
7	1	l2	(1, 2)	NaN	NaN	0.805844
8	10	l1	(1, 1)	NaN	NaN	0.818166
9	10	l1	(1, 2)	NaN	NaN	0.824889
10	10	l2	(1, 1)	NaN	NaN	0.809234
11	10	l2	(1, 2)	NaN	NaN	0.808104
12	NaN	NaN	(1, 1)	3	300	0.783378
13	NaN	NaN	(1, 2)	3	300	0.723859
14	NaN	NaN	(1, 1)	3	500	0.784502
15	NaN	NaN	(1, 2)	3	500	0.738478
16	NaN	NaN	(1, 1)	4	300	0.701469
17	NaN	NaN	(1, 2)	4	300	0.616163
18	NaN	NaN	(1, 1)	4	500	0.686900
19	NaN	NaN	(1, 2)	4	500	0.616163

As usual, the `best_score_` and `best_params_` attributes are still available.

```
both_grid.best_score_
```

```
0.8248885820099178
```

```
both_grid.best_params_
```

```
{'classifier': LogisticRegression(C=10, penalty='l1', random_state=1,
                                  solver='liblinear'),
 'classifier__C': 10,
 'classifier__penalty': 'l1',
 'preprocessor__countvectorizer__ngram_range': (1, 2)}
```

Finally, here are two neat extensions to what we've done in this lesson that you could try on your own:

- First, since the two models have separate parameter dictionaries, you could theoretically tune different preprocessing parameters for each model. For example, you could tune different CountVectorizer parameters for logistic regression and random forests.
- Taking it one step further, you could even create two different preprocessor objects and tune them using the same grid search, just like we tuned two different models using the same grid search. That would allow you, for example, to use different encoders when preparing data for your logistic regression and random forest models.

Extensions of this approach:

- Tune different preprocessing parameters for each model
- Tune two different preprocessor objects

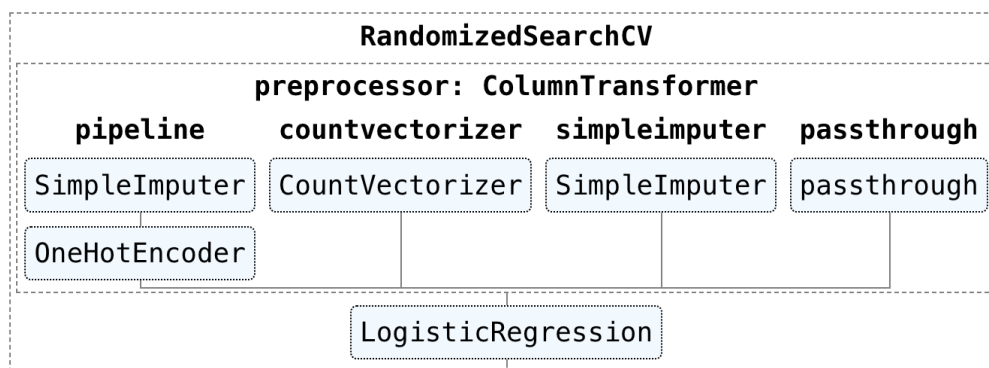
11.5 Q&A: How do I tune two models with a single randomized search?

Starting in scikit-learn version 0.22, RandomizedSearchCV can search multiple parameter dictionaries. This allows you to do a randomized search of multiple models, in the same exact way that we did a grid search of multiple models.

Here's an example in which we pass both_pipe and both_params to RandomizedSearchCV and run it for 10 iterations.

```
both_rand = RandomizedSearchCV(both_pipe, both_params, cv=5,
                               scoring='accuracy', n_iter=10,
                               random_state=1, n_jobs=-1)

both_rand.fit(X, y)
```



If you examine the results, you'll find that logistic regression will be chosen more often because we defined more parameter combinations for the logistic regression model. This behavior may change in a future version, such that each model is equally likely to be chosen, regardless of the number of parameter combinations.

```

results = (pd.DataFrame(both_rand.cv_results_)
           .filter(regex='param_.+_|mean_test'))
results.columns = results.columns.str.split('__').str[-1]
results

```

	ngram_range	penalty	C	n_estimators	min_samples_leaf	mean_test_score
0	(1, 2)	l2	0.1	NaN	NaN	0.788996
1	(1, 1)	NaN	NaN	300	4	0.701469
2	(1, 1)	l2	1	NaN	NaN	0.811462
3	(1, 1)	l2	10	NaN	NaN	0.809234
4	(1, 1)	l2	0.1	NaN	NaN	0.788990
5	(1, 1)	NaN	NaN	500	3	0.784502
6	(1, 1)	l1	1	NaN	NaN	0.814814
7	(1, 2)	NaN	NaN	300	4	0.616163
8	(1, 2)	l2	1	NaN	NaN	0.805844
9	(1, 2)	l1	0.1	NaN	NaN	0.783385

12 Ensembling multiple models

12.1 Introduction to ensembling

So far, we've been trying to choose between two different models, namely logistic regression and random forests. However, you can actually use a process called ensembling to combine multiple models. The goal of ensembling is to produce a combined model, known as an ensemble, that is more accurate than any of the individual models.

The process for ensembling is simple:

- For a regression problem, you calculate the average of the predictions made by the individual regressors and use that as your prediction.
- For a classification problem, you can either average the predicted probabilities output by the classifiers, or you can let the classifiers vote on which class to predict.

How to create an ensemble:

- **Regression:** Average the predictions
- **Classification:** Average the predicted probabilities, or let the classifiers vote on the class

The idea behind ensembling is that if you have a collection of individually imperfect models, the “one-off” errors made by each model are probably not going to be made by the rest of the models. Thus, the errors will be discarded (or at least reduced) when ensembling the models. Another way to say this is that ensembling produces better predictions because the ensemble has a lower variance than any of the individual models.

Why does ensembling work?

- “One-off” errors made by each model will be discarded when ensembling
- Ensemble has a lower variance than any individual model

In this chapter, we'll ensemble our classification models two different ways, and then we'll tune the ensemble to try to achieve even better performance.

12.2 Ensembling logistic regression and random forests

In this lesson, we're going to ensemble logistic regression and random forests. Because their predictions are generated using completely different processes, they're likely to make different types of errors and thus they're good candidates for ensembling.

Let's see a reminder of their cross-validation scores. Both the logistic regression and random forest Pipelines have an accuracy of 0.811, so our goal with ensembling is to increase this score.

```
logreg = LogisticRegression(solver='liblinear', random_state=1)
pipe = make_pipeline(ct, logreg)
cross_val_score(pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8114619295712762

```
rf = RandomForestClassifier(random_state=1, n_jobs=-1)
rf_pipe = make_pipeline(ct, rf)
cross_val_score(rf_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.811436821291821

Pipeline accuracy scores:

- **Grid search (RF):** 0.829
- **Grid search (LR):** 0.828
- **Baseline (LR):** 0.811
- **Baseline (RF):** 0.811

We'll create the ensemble using the `VotingClassifier` class, which we'll import from the `ensemble` module. We'll create an instance called `vc` and pass it a list of tuples, in which the first element of the tuple is a name and the second element is a classifier object.

The options for the `voting` parameter are `'soft'`, in which predicted probabilities are averaged, and `'hard'`, in which only class predictions are taken into account. We'll try soft voting first. Also, we'll set `n_jobs` to `-1` to enable parallel processing.

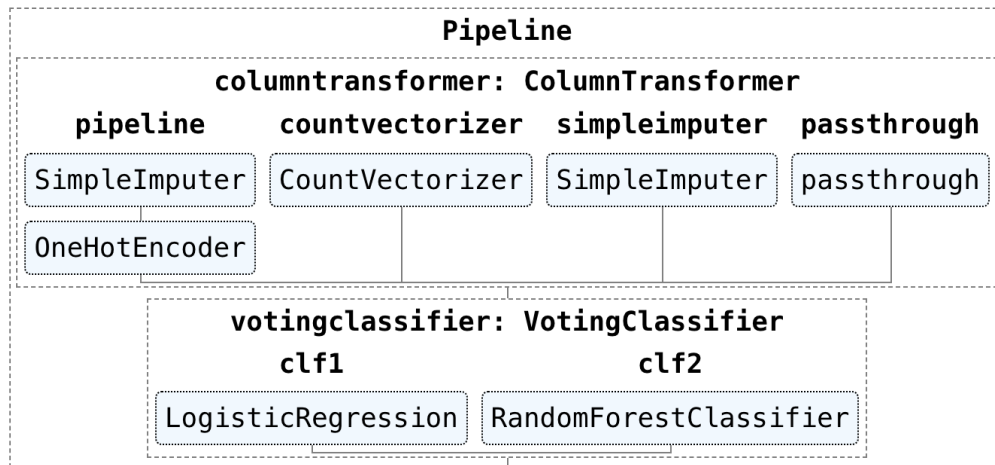
```
from sklearn.ensemble import VotingClassifier
vc = VotingClassifier([('clf1', logreg), ('clf2', rf)], voting='soft',
                      n_jobs=-1)
```

Voting options for VotingClassifier:

- `'soft'`: Average the predicted probabilities
- `'hard'`: Majority vote using class predictions

Then, we'll create a new Pipeline called `vc_pipe` in which the `VotingClassifier` is the second step instead of a model.

```
vc_pipe = make_pipeline(ct, vc)
vc_pipe
```

12.3 Combining predicted probabilities

Let's examine how the `VotingClassifier` makes predictions when using soft voting. We'll do this by examining the predicted probabilities output by the logistic regression and random forest `Pipelines` for the first 3 samples in `X_new`.

As a reminder, the left column is the predicted probability of class 0 (for each sample), and the right column is the predicted probability of class 1.

```
pipe.fit(X, y)
pipe.predict_proba(X_new)[:3]
```

```
array([[0.88916549, 0.11083451],
       [0.14200691, 0.85799309],
       [0.9190551 , 0.0809449 ]])
```

```
rf_pipe.fit(X, y)
rf_pipe.predict_proba(X_new)[:3]
```

```
array([[0.99, 0.01],
       [0.24, 0.76],
       [0.97, 0.03]])
```

Now we'll use the `VotingClassifier` to output predicted probabilities. It uses the `fit` and `predict_proba` methods, just like any other classifier.

If you examine its predicted probabilities, you'll see that it's simply averaging the two sets of probabilities from logistic regression and random forests. For example, the 0.19 in the left column of the second sample is the average of 0.14 (from logistic regression) and 0.24 (from random forests).

```
vc_pipe.fit(X, y)
vc_pipe.predict_proba(X_new)[:3]
```

```
array([[0.93958275, 0.06041725],
       [0.19100345, 0.80899655],
       [0.94452755, 0.05547245]])
```

In order to make class predictions for `X_new`, you use the `predict` method, which simply chooses whichever class has the higher predicted probability. In this case, it predicted 0, 1, and 0, because those classes had the higher predicted probability.

```
vc_pipe.predict(X_new)[:3]
```

```
array([0, 1, 0])
```

In the 3 cases we just examined, logistic regression and random forests agreed on the class predictions. Let's now examine a case in which the two models disagree.

For sample 80, logistic regression predicts class 0 but without much confidence, whereas random forests predicts class 1 with more confidence.

```
pipe.predict_proba(X_new)[80]
```

```
array([0.51799634, 0.48200366])
```

```
rf_pipe.predict_proba(X_new)[80]
```

```
array([0.29, 0.71])
```

When `VotingClassifier` averages the predicted probabilities for this sample, the class 1 value is higher than the class 0 value, thus it will predict class 1.

```
vc_pipe.predict_proba(X_new)[80]
```

```
array([0.40399817, 0.59600183])
```

Let's move on to cross-validation to see how the `VotingClassifier` Pipeline with soft voting performs. Its score is 0.818, which is better than either model alone.

```
cross_val_score(vc_pipe, X, y, cv=5, scoring='accuracy').mean()
```

```
0.8181846713953927
```

Pipeline accuracy scores:

- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (VC soft voting): 0.818
- Baseline (LR): 0.811
- Baseline (RF): 0.811

12.4 Combining class predictions

Now let's modify the `VotingClassifier` to use hard voting, which means that it ignores predicted probabilities and just takes a majority vote based on class predictions.

```
vc = VotingClassifier([('clf1', logreg), ('clf2', rf)], voting='hard',  
                      n_jobs=-1)  
vc_pipe = make_pipeline(ct, vc)
```

When we cross-validate the Pipeline with hard voting, it performs a bit better, scoring 0.820.

```
cross_val_score(vc_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8204255853367648

Pipeline accuracy scores:

- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (VC hard voting): 0.820
- Baseline (VC soft voting): 0.818
- Baseline (LR): 0.811
- Baseline (RF): 0.811

However, this result is actually misleading:

- When you use hard voting with `VotingClassifier` and there's a tie, it always chooses the lowest numbered class. In other words, every time the two models disagreed, it chose class 0.
- It follows that hard voting is performing better than soft voting purely by chance. If the tiebreaking algorithm instead chose the highest numbered class, hard voting would be performing worse than soft voting.

Why is this result misleading?

- In the case of a tie, hard voting always chooses class 0
- Thus hard voting is performing better than soft voting by chance

12.5 Choosing a voting strategy

The previous lesson brings up the obvious question: When should you use soft voting, and when should you use hard voting?

- Soft voting is preferred if you have an even number of models in the ensemble, and especially if you only have two models.
- Soft voting is preferred if all of your models output well-calibrated predicted probabilities, whereas hard voting is preferred otherwise.
- Hard voting will always work, whereas soft voting will only work if all of the models in the ensemble include the `predict_proba` method. For example, `LinearSVC` and `Perceptron` don't include the `predict_proba` method.

Soft voting vs hard voting:

- **Soft voting:**
 - Preferred if you have an even number of models (especially two)
 - Preferred if all models are well-calibrated
 - Only works if all models have the `predict_proba` method
- **Hard voting:**
 - Preferred if some models are not well-calibrated
 - Does not require the `predict_proba` method

Ultimately, you can just try both soft and hard voting and see which works better, keeping in mind that hard voting results can be misleading if you have an even number of classifiers.

Because we're using an even number of classifiers, we'll change back to soft voting.

```
vc = VotingClassifier([('clf1', logreg), ('clf2', rf)], voting='soft',
                      n_jobs=-1)
vc_pipe = make_pipeline(ct, vc)
```

12.6 Tuning an ensemble with grid search

Like with any model, we can tune the `VotingClassifier`'s hyperparameters using a grid search to try to improve its accuracy. Keep in mind that the best parameters for the `VotingClassifier Pipeline` might be different than the parameters for either model when they were tuned separately.

We'll start by creating a `vc_params` dictionary that only includes the `ColumnTransformer` parameters.

```
vc_params = {k:v for k, v in params.items() if k.startswith('col')}
vc_params
```

```
{'columntransformer__pipeline__onehotencoder__drop': [None, 'first'],
 'columntransformer__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'columntransformer__simpleimputer__add_indicator': [False, True]}
```

We can look at the `named_steps` attribute of `vc_pipe` to confirm that the second step name is `votingclassifier`.

```
vc_pipe.named_steps.keys()
```

```
dict_keys(['columntransformer', 'votingclassifier'])
```

And you may recall that we assigned the names `clf1` and `clf2` to our two models within the `VotingClassifier`.

```
vc_pipe.named_steps['votingclassifier'].named_estimators
```

```
{'clf1': LogisticRegression(random_state=1, solver='liblinear'),
 'clf2': RandomForestClassifier(n_jobs=-1, random_state=1)}
```

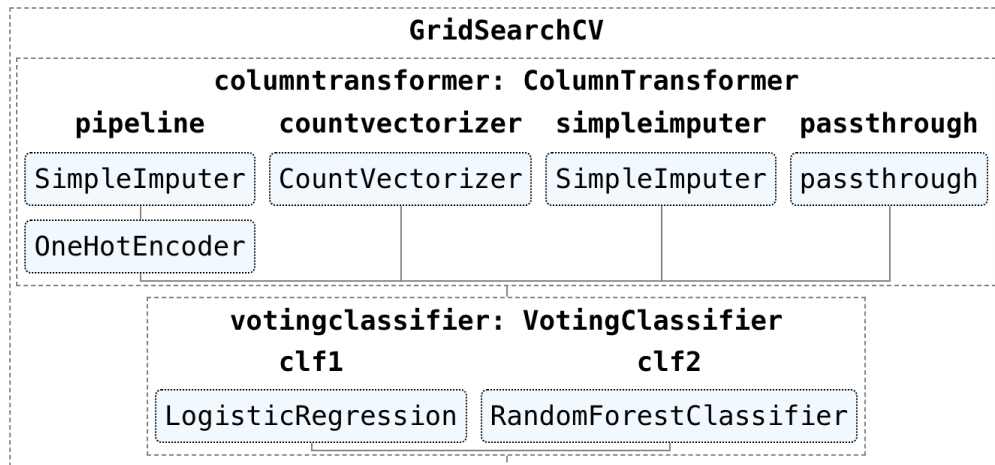
Knowing those names, we can now add some model parameters to `vc_params`. For simplicity and speed, we'll just tune a smaller selection of parameters and values.

```
vc_params['votingclassifier__clf1__penalty'] = ['l1', 'l2']
vc_params['votingclassifier__clf1__C'] = [1, 10]
vc_params['votingclassifier__clf2__n_estimators'] = [100, 300]
vc_params['votingclassifier__clf2__min_samples_leaf'] = [2, 3]
vc_params
```

```
{'columntransformer__pipeline__onehotencoder__drop': [None, 'first'],
 'columntransformer__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'columntransformer__simpleimputer__add_indicator': [False, True],
 'votingclassifier__clf1__penalty': ['l1', 'l2'],
 'votingclassifier__clf1__C': [1, 10],
 'votingclassifier__clf2__n_estimators': [100, 300],
 'votingclassifier__clf2__min_samples_leaf': [2, 3]}
```

Finally, we'll create a `GridSearchCV` object called `vc_grid`, making sure to use the `vc_pipe` and `vc_params` objects, and then run the search.

```
vc_grid = GridSearchCV(vc_pipe, vc_params, cv=5, scoring='accuracy',
                       n_jobs=-1)
vc_grid.fit(X, y)
```



You can see that the best score has improved significantly, to 0.834.

```
vc_grid.best_score_
```

```
0.833864791915134
```

Pipeline accuracy scores:

- Grid search (VC soft voting): 0.834
- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (VC hard voting): 0.820
- Baseline (VC soft voting): 0.818
- Baseline (LR): 0.811
- Baseline (RF): 0.811

Here's the best set of parameters that it found.

```
vc_grid.best_params_
```

```
{'columntransformer__countvectorizer__ngram_range': (1, 2),
 'columntransformer__pipeline__onehotencoder__drop': None,
 'columntransformer__simpleimputer__add_indicator': True,
 'votingclassifier__clf1__C': 10,
 'votingclassifier__clf1__penalty': 'l1',
 'votingclassifier__clf2__min_samples_leaf': 3,
 'votingclassifier__clf2__n_estimators': 100}
```

Finally, using the tuned grid to make predictions for new data is the same as always.

```
vc_grid.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1,
       1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1,
       1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
       1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1,
       1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,
       0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0,
       1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1,
       1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
       0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1,
       0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
       0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
       1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1])
```

12.7 Q&A: When should I use ensembling?

Ensembling generally improves the performance of your model, and so it's a useful technique any time that performance is your highest priority. Keep in mind, however, that ensembling adds more complexity to your process, and the ensemble is also less interpretable than a single model.

Should you ensemble?

- **Advantages:**
 - Improves model performance
- **Disadvantages:**
 - Adds more complexity
 - Decreases interpretability

If you do decide to use ensembling, my advice is to include at least 3 models in the ensemble (when possible). It's important that all models you include are performing reasonably well on their own. And as mentioned before, it's ideal if they generate their predictions using different processes.

Advice for ensembling:

- Include at least 3 models

- Models should be performing well on their own
- Ideal if they generate predictions using different processes

12.8 Q&A: How do I apply different weights to the models in an ensemble?

By default, each model within an ensemble is given equal weight. However, you can try weighting certain models more than others to give them more “voting power” when determining the predicted class labels or predicted probabilities.

For example, we could give the logistic regression model double the voting power of the random forest model by setting the `weights` parameter of the `VotingClassifier`.

```
vc = VotingClassifier([('clf1', logreg), ('clf2', rf)], voting='soft',
                      weights=[2, 1], n_jobs=-1)
vc_pipe = make_pipeline(ct, vc)
```

Let’s see how that affects the predicted probabilities. Once again, here are the predicted probabilities output by the logistic regression and random forest Pipelines for the first 3 samples in `X_new`.

```
pipe.predict_proba(X_new)[:3]
```

```
array([[0.88916549, 0.11083451],
       [0.14200691, 0.85799309],
       [0.9190551 , 0.0809449 ]])
```

```
rf_pipe.predict_proba(X_new)[:3]
```

```
array([[0.99, 0.01],
       [0.24, 0.76],
       [0.97, 0.03]])
```

And here are the predicted probabilities output by the `VotingClassifier`. As you can see, the predicted probabilities are closer to the ones output by logistic regression because we gave that model twice the weight. For example, the 0.17 in the left column of the second sample is closer to 0.14 (from logistic regression) than it is to 0.24 (from random forests).

```
vc_pipe.fit(X, y)
vc_pipe.predict_proba(X_new)[:3]
```

```
array([[0.92277699, 0.07722301],
       [0.17467127, 0.82532873],
       [0.93603673, 0.06396327]])
```


You can confirm whether the weights are helping or hurting the ensemble by using cross-validation. In this case, the score is 0.816, which is slightly worse than our baseline `VotingClassifier`.

```
cross_val_score(vc_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8159437574540205

Pipeline accuracy scores:

- Grid search (VC soft voting): 0.834
- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (VC hard voting): 0.820
- Baseline (VC soft voting): 0.818
- Baseline (VC soft voting with LR weighted): 0.816
- Baseline (LR): 0.811
- Baseline (RF): 0.811

You can also search for the optimal weights using a grid search. Here's how you might add that to the `vc_params` dictionary.

```
vc_params['votingclassifier__weights'] = [(1, 1), (2, 1), (1, 2)]
```

13 Feature selection

13.1 Introduction to feature selection

Feature selection is the process of removing uninformative features from your model. These are features that are not helping your model to make better predictions. In other words, uninformative features are adding “noise” to your model, rather than “signal”.

There are a few reasons you might want to add feature selection to your workflow:

- Model accuracy can often be improved by removing uninformative features.
- Models are generally easier to interpret when they include fewer features.
- When you have fewer features, models will take less time to train and it may cost less to gather the data that is required to train them.

Potential benefits of feature selection:

- Higher accuracy
- Greater interpretability
- Faster training
- Lower costs

As I mentioned at the start of the book, there are many valid methods for feature selection, including human intuition, domain knowledge, and data exploration. In this chapter, we’re going to do feature selection using automated methods that we can include in our Pipeline.

Feature selection methods:

- Human intuition
- Domain knowledge
- Data exploration
- Automated methods

There are three types of automated methods that we’ll cover in this chapter: intrinsic methods, filter methods, and wrapper methods.

Methods for automated feature selection:

- Intrinsic methods
- Filter methods
- Wrapper methods

For the purposes of simplicity and training speed, we'll use our logistic regression Pipeline as the starting point for the next few chapters. However, everything you're learning could also be applied to the random forest Pipeline or the ensemble Pipeline.

13.2 Intrinsic methods: L1 regularization

An intrinsic feature selection method is one in which feature selection happens automatically as part of the model building process. These are also known as implicit methods or embedded methods.

What are intrinsic methods?

- Feature selection happens automatically during model building
- Also called “implicit methods” or “embedded methods”

We've actually already used an intrinsic feature selection method in this book. Recall that in chapter 10, we tuned our logistic regression Pipeline using a grid search. Here were the best parameters.

```
grid.best_params_
```

```
{'columntransformer__countvectorizer__ngram_range': (1, 2),  
 'columntransformer__pipeline__onehotencoder__drop': None,  
 'columntransformer__simpleimputer__add_indicator': True,  
 'logisticregression__C': 10,  
 'logisticregression__penalty': 'l1'}
```

Notice the C and penalty parameters of logistic regression. The L1 penalty is the type of regularization that was used, and the C value indicates the amount of regularization.

LogisticRegression tuning parameters:

- **penalty:** Type of regularization
- **C:** Amount of regularization

In general, regularization shrinks model coefficients in order to minimize overfitting to the training data and improve the model's ability to generalize to new data.

One notable aspect of L1 regularization is that as the amount of regularization increases, some coefficients will be shrunk all the way to zero, which means they will be excluded from the model. In other words, L1 regularization automatically does feature selection.

How does L1 regularization do feature selection?

- Regularization shrinks model coefficients to help the model to generalize
- L1 regularization shrinks some coefficients to zero, which removes those features

To see an example of this, let's take a look at the coefficients of the best model found by grid search, which is stored in the `best_estimator_` attribute. Notice that the second coefficient is zero, which means that the L1 regularization caused that feature to be removed from the model.

```
grid.best_estimator_.named_steps['logisticregression'].coef_  
  
array([[ 0.56431161,  0.          , -0.08767203, ...,  0.01408723,  
        -0.43713268, -0.46358519]])
```

By checking the shape of the inner array, we can see that there are 3671 features.

```
grid.best_estimator_.named_steps['logisticregression'].coef_[0].shape
```

```
(3671,)
```

We can then check how many of the coefficients are zero. It turns out that 3103 coefficients were set to zero, which means that L1 regularization removed those features, leaving only 568 features.

Note that as the amount of regularization increases, more coefficients will be shrunk to zero and thus more features will be removed from the model. In the case of logistic regression, you increase the amount of regularization by decreasing the value of `C`.

```
sum(grid.best_estimator_.named_steps['logisticregression'].coef_[0] == 0)
```

```
3103
```

Let's compare this to our logistic regression Pipeline that was not tuned by grid search. You can see that it uses L2 regularization.

```
pipe.named_steps['logisticregression'].get_params()
```

```
{'C': 1.0,  
 'class_weight': None,  
 'dual': False,  
 'fit_intercept': True,  
 'intercept_scaling': 1,  
 'l1_ratio': None,  
 'max_iter': 100,  
 'multi_class': 'auto',  
 'n_jobs': None,  
 'penalty': 'l2',  
 'random_state': 1,  
 'solver': 'liblinear',  
 'tol': 0.0001,  
 'verbose': 0,  
 'warm_start': False}
```

Although L2 regularization does shrink coefficients, we can confirm that it does not shrink them all the way to zero, and thus it does not perform feature selection.

```
sum(pipe.named_steps['logisticregression'].coef_[0] == 0)
```

0

Keep in mind that although L1 regularization produced a better-performing model in this situation, that will not always be the case. It's a good idea to try both types of regularization and see which one works better.

To wrap up this section, let's talk about some advantages and disadvantages of intrinsic feature selection methods:

- The main advantages are speed and simplicity: Since feature selection is implicitly performed during model fitting, no additional feature selection process needs to be added to the workflow, which tends to save a lot of computational time.
- The main disadvantage is that it's model-dependent: The model that is best for your particular problem may not perform intrinsic feature selection.

Advantages and disadvantages of intrinsic methods:

- **Advantages:**
 - No added computation
 - No added steps
- **Disadvantages:**
 - Model-dependent

13.3 Filter methods: Statistical test-based scoring

The next type of feature selection we'll cover is filter methods. Here's how they work:

1. First, every single feature is scored to quantify its potential relationship with the target column.
2. Then, the features are ranked by their scores, and only the top scoring features are provided to the model.

Thus, they're called filter methods because they filter out what they believe to be the least informative features and then pass on the more informative features to the model.

As you'll see in this section, filter methods vary in terms of the processes they use to score the features.

How filter methods work:

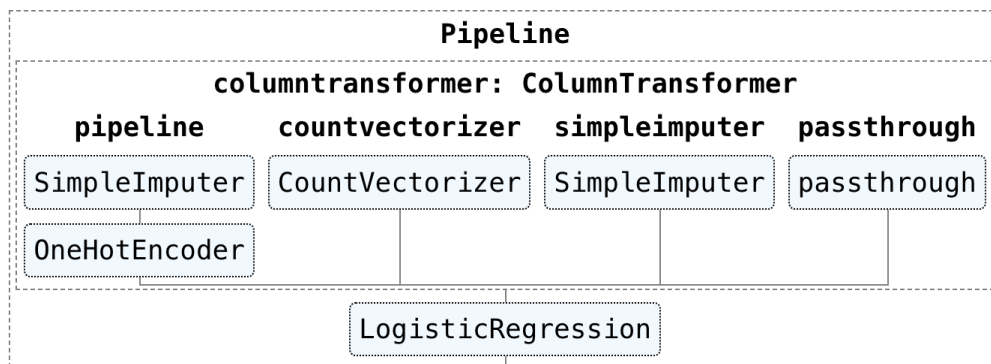
1. Each feature is scored by its relationship to the target

2. Top scoring features (most informative ones) are provided to the model

Our starting point for this section will be the logistic regression Pipeline that has not been tuned by grid search. The reason for this is because we want to tune all of the Pipeline steps simultaneously, rather than tuning the transformers and model first and then adding feature selection.

In other words, the presence of a feature selection process may alter the optimal parameters for the transformers and the model, and thus we need to tune all three steps at once. Right now it's a two-step Pipeline, but there will be three steps once we add feature selection to the Pipeline.

```
pipe
```



Let's cross-validate this Pipeline to generate the baseline accuracy that we want to improve upon, which is 0.811.

```
cross_val_score(pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8114619295712762

Pipeline accuracy scores:

- Grid search (VC): 0.834
- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (VC): 0.818
- Baseline (LR): 0.811
- Baseline (RF): 0.811

The first filter method we'll use is SelectPercentile. SelectPercentile scores features using univariate statistical tests:

- You specify a statistical test, and it uses that test to score each feature independently.
- Then, it passes on to the model a certain percentage (that you specify) of the best scoring features.

Thus, the assumption behind `SelectPercentile` is that a statistical test can assess the strength of the relationship between a feature and the target, and that if a feature appears to be independent of the target, then it is uninformative for the purpose of classification.

How `SelectPercentile` works:

1. Scores each feature using the statistical test you specify
2. Passes to the model the percentage of features you specify

Let's see how `SelectPercentile` works. After importing `SelectPercentile` and `chi2` from the `feature_selection` module, we'll create an instance of `SelectPercentile` called `selection`.

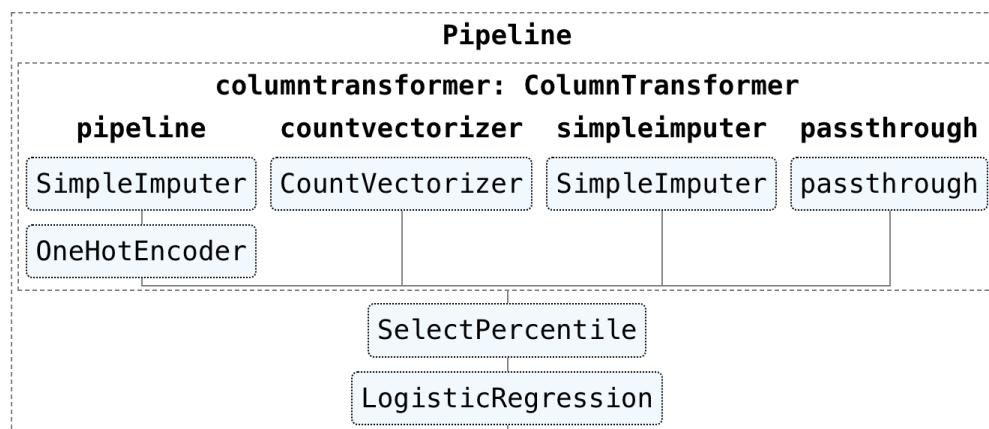
First, we'll pass it the statistical test. In this case we're using chi-squared, but other tests are available in scikit-learn.

Then, we'll pass it the percentile. We're arbitrarily using 50 to keep 50% of the features, but this is a parameter you should tune. And to be clear, lower values for this parameter keep fewer features, so a value of 10 would only keep 10% percent of the features.

```
from sklearn.feature_selection import SelectPercentile, chi2
selection = SelectPercentile(chi2, percentile=50)
```

Next, we'll create a `Pipeline` called `fs_pipe` in which feature selection is after the `ColumnTransformer` but before the model. Thus, it will perform feature selection on the transformed features, not the original features.

```
fs_pipe = make_pipeline(ct, selection, logreg)
fs_pipe
```



Because we've included feature selection within the `Pipeline`, we can continue to cross-validate the entire process to see the impact of feature selection on model accuracy. When we run cross-validation on the new `Pipeline`, the score has improved to 0.819.

```
cross_val_score(fs_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8193019898311469

Pipeline accuracy scores:

- Grid search (VC): 0.834
- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (LR with SelectPercentile): 0.819
- Baseline (VC): 0.818
- Baseline (LR): 0.811
- Baseline (RF): 0.811

It's worth noting that there's an alternative to SelectPercentile called SelectKBest. SelectKBest is nearly identical, except that you specify a number of features to keep rather than a percentage.

SelectPercentile vs SelectKBest:

- SelectPercentile: Specify percentage of features to keep
- SelectKBest: Specify number of features to keep

13.4 Filter methods: Model-based scoring

The other filter method we'll use is called SelectFromModel. Whereas SelectPercentile scores features using a statistical test, SelectFromModel actually uses a model to score features:

1. First, you specify a model to use only for feature selection. That model is fit on all of the features, and the `coef_` or `feature_importances_` attribute of the model is used as the scores.
2. Then, it passes on to your prediction model all of the features that score above a certain threshold (that you specify).

How SelectFromModel works:

1. Scores each feature using the model you specify
 - Model is fit on all features
 - Coefficients or feature importances are used as scores
2. Passes to the prediction model features that score above a threshold you specify

Thus for a model to be used by SelectFromModel, it has to calculate either coefficients or feature importances. Models that can be used by SelectFromModel include logistic regression, linear SVC, and tree-based models.

Models that can be used by SelectFromModel:

- Logistic regression
- Linear SVC
- Tree-based models
- Any other model with coefficients or feature importances

To be clear, `SelectFromModel` is a filter method (not an intrinsic method) because it's filtering which features are passed to your separate prediction model.

Let's see how all of this fits together. We're going to start by using logistic regression for feature selection. We'll create a new instance of logistic regression called `logreg_selection` that's only going to be used for feature selection. It's completely separate from the logistic regression model we're using to make predictions.

```
logreg_selection = LogisticRegression(solver='liblinear', penalty='l1',
                                     random_state=1)
```

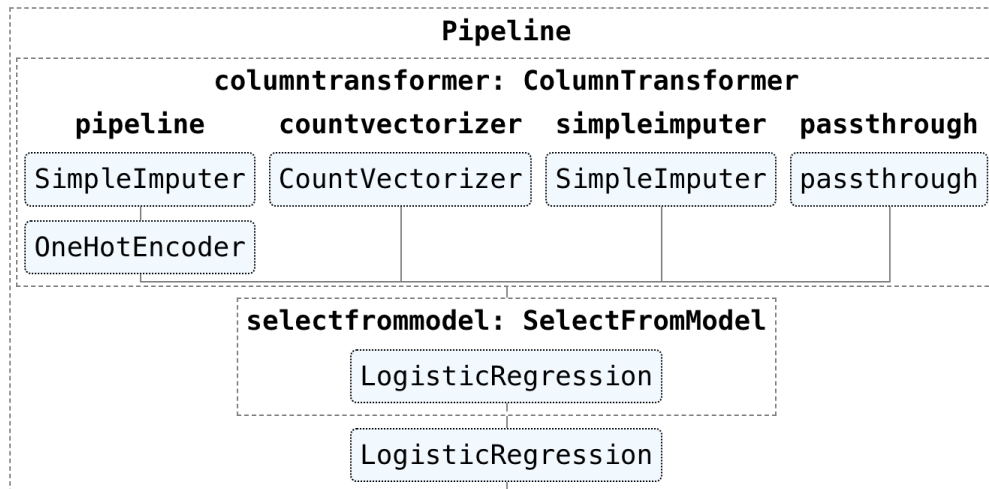
Then, we'll import `SelectFromModel` from the `feature_selection` module and create an instance called `selection`.

First, we pass it the model we're using for selection. Second, we pass it a threshold. This can be the mean or median of the scores, though you can optionally include a scaling factor (such as $1.5 \times$ mean). All features above this threshold will be passed to the prediction model, thus setting a higher threshold means fewer features will be kept.

```
from sklearn.feature_selection import SelectFromModel
selection = SelectFromModel(logreg_selection, threshold='mean')
```

Then, we'll update `fs_pipe` to use the new feature selection object. Notice that logistic regression appears twice: one instance is being used only for feature selection, and the other instance is being used only for prediction.

```
fs_pipe = make_pipeline(ct, selection, logreg)
fs_pipe
```



When we cross-validate the updated Pipeline, the score has improved again, to 0.826.

```
cross_val_score(fs_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8260121775155358

Pipeline accuracy scores:

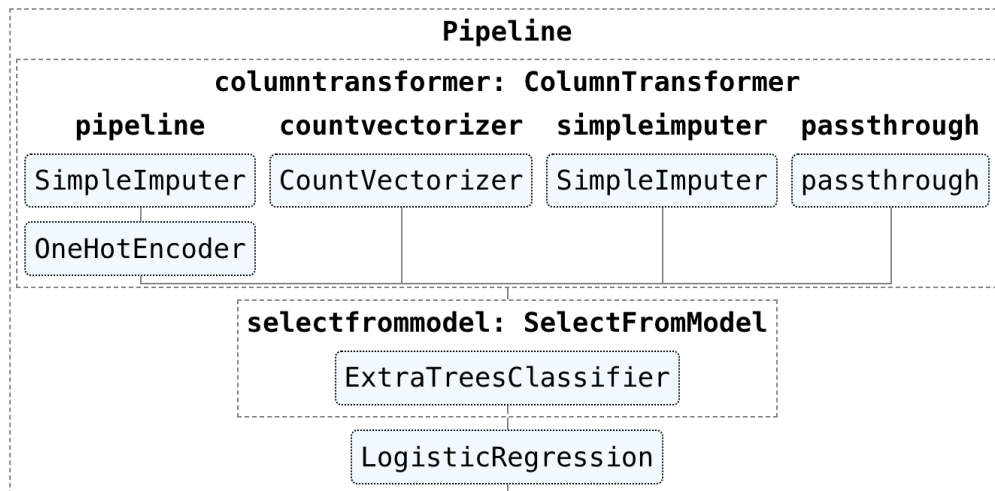
- Grid search (VC): 0.834
- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (LR with SelectFromModel LR): 0.826
- Baseline (LR with SelectPercentile): 0.819
- Baseline (VC): 0.818
- Baseline (LR): 0.811
- Baseline (RF): 0.811

Now let's try using a tree-based model with SelectFromModel. We'll use ExtraTreesClassifier, which is an ensemble of decision trees similar to random forests. After importing it from the ensemble module, we'll create an instance to use for feature selection called et_selection.

```
from sklearn.ensemble import ExtraTreesClassifier
et_selection = ExtraTreesClassifier(n_estimators=100, random_state=1)
```

Then, we'll update both the feature selection object and the Pipeline. Notice that ExtraTreesClassifier has replaced logistic regression as the second step in the Pipeline.

```
selection = SelectFromModel(et_selection, threshold='mean')
fs_pipe = make_pipeline(ct, selection, logreg)
fs_pipe
```



When we cross-validate the updated Pipeline, the resulting score is 0.815, which is not quite as good.

```
cross_val_score(fs_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8148013307388112

Pipeline accuracy scores:

- Grid search (VC): 0.834
- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (LR with SelectFromModel LR): 0.826
- Baseline (LR with SelectPercentile): 0.819
- Baseline (VC): 0.818
- Baseline (LR with SelectFromModel ET): 0.815
- Baseline (LR): 0.811
- Baseline (RF): 0.811

As I mentioned earlier in this chapter, it's important to tune the feature selection parameters, the transformer parameters, and the model parameters all at the same time. We'll do this using a grid search.

To start, we'll make a copy of our params dictionary called `fs_params`. We want to add a new entry in order to tune the threshold parameter of `SelectFromModel`. For the dictionary key, we specify the step name, which is `selectfrommodel`, followed by two underscores, followed by the parameter name. For the values, we'll pass a list of mean, $1.5 \times \text{mean}$, and negative infinity, which means don't remove any features.

```
fs_params = params.copy()
fs_params['selectfrommodel__threshold'] = ['mean', '1.5*mean', -np.inf]
```

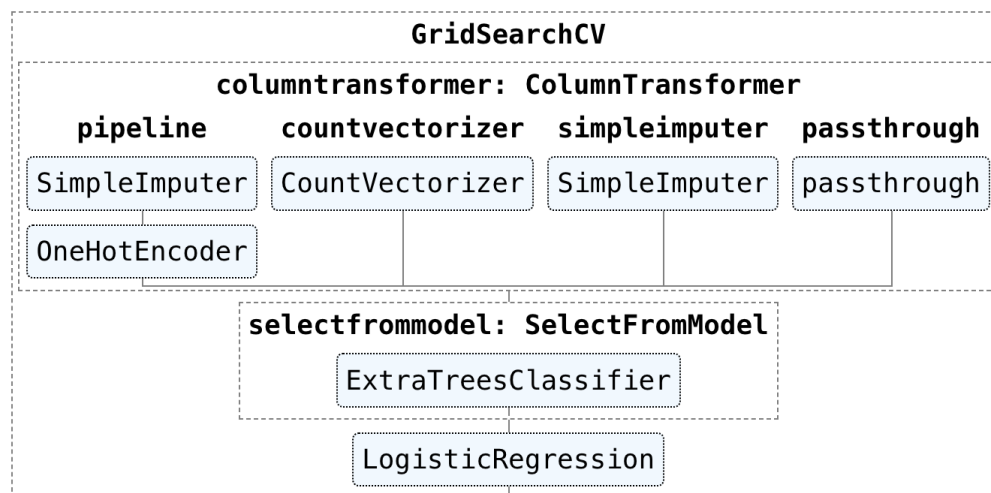
Let's review the `fs_params` dictionary to see everything we're tuning.

```
fs_params
```

```
{'logisticregression__penalty': ['l1', 'l2'],
 'logisticregression__C': [0.1, 1, 10],
 'columntransformer__pipeline__onehotencoder__drop': [None, 'first'],
 'columntransformer__countvectorizer__ngram_range': [(1, 1), (1, 2)],
 'columntransformer__simpleimputer__add_indicator': [False, True],
 'selectfrommodel__threshold': ['mean', '1.5*mean', -inf]}
```

We'll create a new instance of `GridSearchCV` called `fs_grid`, and make sure to pass it the `fs_pipe` and `fs_params` objects. Then we'll run the grid search.

```
fs_grid = GridSearchCV(fs_pipe, fs_params, cv=5, scoring='accuracy',
                       n_jobs=-1)
fs_grid.fit(X, y)
```



This search results in a score of 0.832, which is one of our best scores so far.

```
fs_grid.best_score_
```

0.8316301550436258

Pipeline accuracy scores:

- Grid search (VC): 0.834
- Grid search (LR with `SelectFromModel` ET): 0.832
- Grid search (RF): 0.829
- Grid search (LR): 0.828

- **Baseline (LR with SelectFromModel LR):** 0.826
- **Baseline (LR with SelectPercentile):** 0.819
- **Baseline (VC):** 0.818
- **Baseline (LR with SelectFromModel ET):** 0.815
- **Baseline (LR):** 0.811
- **Baseline (RF):** 0.811

By examining the parameters, we can see that increasing the `SelectFromModel` threshold and thus keeping fewer features helped the model to perform better.

```
fs_grid.best_params_
```

```
{'columntransformer__countvectorizer__ngram_range': (1, 2),
 'columntransformer__pipeline__onehotencoder__drop': None,
 'columntransformer__simpleimputer__add_indicator': False,
 'logisticregression__C': 10,
 'logisticregression__penalty': 'l1',
 'selectfrommodel__threshold': '1.5*mean'}
```

13.5 Filter methods: Summary

To wrap up this section, let's talk about some advantages and disadvantages of filter methods.

The main advantage is that filter methods tend to run very quickly, though it's worth noting that some statistical tests used with `SelectPercentile` and some ensemble methods used with `SelectFromModel` can run quite slowly.

The main disadvantage is that there's a disconnect between how features are being scored and their predictive value. In other words, the chi-squared scores, coefficient values, or feature importance scores are not a perfect measure of whether a particular feature will help a model make more accurate predictions. Thus, it's entirely possible for informative features to receive low scores and be removed from a model, and for uninformative features to receive high scores and be kept in a model. One particular case of note is that the feature importance scores generated by tree-based models will be artificially low for any features which are highly correlated, which may result in important features being removed.

The other disadvantage of filter methods is that scores are calculated only once. This ignores the fact that as you remove certain features, the importance of other features may change. This drawback will be addressed by wrapper methods, which we'll discuss in the next lesson.

Advantages and disadvantages of filter methods:

- **Advantages:**
 - Runs quickly (usually)
- **Disadvantages:**
 - Scores are not always correlated with predictive value

- Scores are calculated only once

13.6 Wrapper methods: Recursive feature elimination

The final type of feature selection we'll cover is wrapper methods.

In contrast to the filter methods we've seen, in which features are scored only once, wrapper methods perform an iterative search in which features are scored multiple times. More specifically, a wrapper method evaluates a subset of features and then uses the results of that evaluation to help it decide which subset to evaluate next, repeating this process until some stopping criteria is met.

Filter methods vs wrapper methods:

- **Filter methods:** Features are scored once
- **Wrapper methods:** Features are scored multiple times

The wrapper method we'll use in this section is Recursive Feature Elimination, which is available in scikit-learn as the `RFE` class. The way RFE starts is the same as `SelectFromModel`:

- You specify a model to use only for feature selection.
- That model is fit on all of the features.
- The coefficients or feature importances of the model are used as scores.

However, this is the point at which `SelectFromModel` and RFE diverge:

- `SelectFromModel` would now pass to your prediction model all of the features that score above a certain threshold.
- RFE, on the other hand, removes the single worst scoring feature, refits the feature selection model, and recalculates the feature scores. It repeats this process, recursively eliminating one feature at a time, until it reaches the number of features that you specify. Those remaining features are the ones that will be passed to the prediction model.

How RFE works:

1. Scores each feature using the model you specify
 - Model is fit on all features
 - Coefficients or feature importances are used as scores
2. Removes the single worst scoring feature
3. Repeats steps 1 and 2 until it reaches the number of features you specify
4. Passes the remaining features to the prediction model

In other words, `SelectFromModel` will always just score your features a single time, whereas RFE will score your features potentially hundreds or thousands of times, depending on how many features you want to eliminate. That is obviously more computationally expensive, though it may better capture the relationships between features.

SelectFromModel vs RFE:

- **SelectFromModel:** Scores your features a single time
- **RFE:** Scores your features many times
 - More computationally expensive
 - May better capture the relationships between features

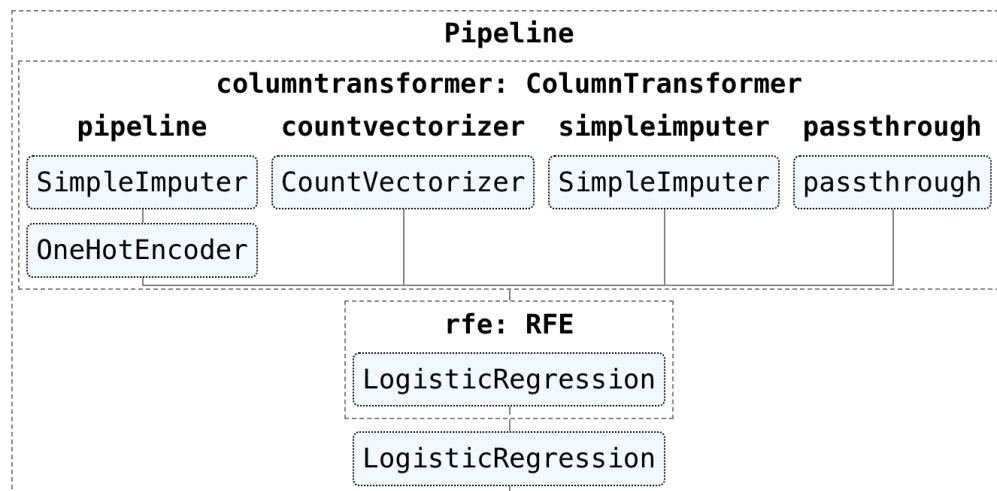
Let's try using RFE. We'll start by importing it from the `feature_selection` module and creating an instance called `selection`. We're actually going to reuse `logreg_selection` (from earlier in the chapter) as our feature selection model.

We're also going to specify a step size of 10. By default, RFE will remove 1 feature at a time, and will stop once it has eliminated half of the features. Since there are about 1500 features, the default settings would require about 750 model fits. By setting a step size of 10, RFE will remove 10 features at a time, which reduces the amount of computation by a factor of 10.

```
from sklearn.feature_selection import RFE
selection = RFE(logreg_selection, step=10)
```

We'll update the `fs_pipe` object to use the new feature selection object.

```
fs_pipe = make_pipeline(ct, selection, logreg)
fs_pipe
```



When we cross-validate it, we see that its score is 0.814, which is barely better than our baseline of 0.811.

```
cross_val_score(fs_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8136965664427847

Pipeline accuracy scores:

- Grid search (VC): 0.834
- Grid search (LR with SelectFromModel ET): 0.832
- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (LR with SelectFromModel LR): 0.826
- Baseline (LR with SelectPercentile): 0.819
- Baseline (VC): 0.818
- Baseline (LR with SelectFromModel ET): 0.815
- Baseline (LR with RFE LR): 0.814
- Baseline (LR): 0.811
- Baseline (RF): 0.811

Of course, it's possible that the accuracy would improve if we tuned the number of features kept by RFE, or if we tried different models with RFE.

It's hard to talk about the advantages and disadvantages of wrapper methods in general because of the diversity of wrapper methods. Instead, let's conclude this section by talking about the advantages and disadvantages of RFE specifically:

- The main advantage of RFE is that it recalculates feature scores as features are removed. This is beneficial because as features are removed, the importance of other features may change, which RFE takes into account (whereas filter methods do not).
- However, RFE has the same disadvantage as filter methods, in that there's a disconnect between how features are being scored and their predictive value. In other words, informative features might be removed and uninformative features might be kept by RFE.
- Another disadvantage of RFE is that it's computationally expensive, especially if you're removing a lot of features.
- A final disadvantage of RFE is that it uses a "greedy" approach to feature selection, which means that it takes whatever action seems best at the time, even if a different action might ultimately lead to better results at the end of the process. There are non-greedy approaches to feature selection, though none of them are currently available in scikit-learn.

Advantages and disadvantages of RFE:

- **Advantages:**
 - Captures the relationships between features
- **Disadvantages:**
 - Scores are not always correlated with predictive value
 - Computationally expensive
 - Does not look ahead when removing features ("greedy" approach)

13.7 Q&A: How do I see which features were selected?

Recall that the `fs_pipe` object has three steps: a `ColumnTransformer`, a feature selector, and a logistic regression model.

```
fs_pipe.named_steps.keys()
```

```
dict_keys(['columntransformer', 'rfe', 'logisticregression'])
```

We can use slicing to select the `ColumnTransformer` step, and then we can run `fit_transform` to see that it outputs 1518 feature columns.

```
fs_pipe[0].fit_transform(X)
```

```
<891x1518 sparse matrix of type '<class 'numpy.float64'>'
  with 7328 stored elements in Compressed Sparse Row format>
```

If we select the first two steps and then run `fit_transform`, we can see that the feature selection step reduced the number of feature columns from 1518 to 759. Note that we have to pass both `X` and `y` to `fit_transform` since the feature selection process requires knowledge of the target values.

```
fs_pipe[0:2].fit_transform(X, y)
```

```
<891x759 sparse matrix of type '<class 'numpy.float64'>'
  with 6008 stored elements in Compressed Sparse Row format>
```

If we then select the feature selection step and run the `get_support` method, it returns a boolean array which includes a `True` for every feature which was kept and a `False` for every feature which was removed.

```
fs_pipe[1].get_support()
```

```
array([ True,  True,  True, ...,  True,  True,  True])
```

As you can see, that array has 1518 elements, and 759 of those elements are `True`.

```
len(fs_pipe[1].get_support())
```

```
1518
```

```
fs_pipe[1].get_support().sum()
```

759

Ideally, you would be able to use this array to filter the list of all features down to a list of the selected features. However, this is not a straightforward process because the `get_feature_names` method of `ColumnTransformer` only works if all of the underlying transformers have a `get_feature_names` method, and that is not the case here.

```
fs_pipe[0].get_feature_names()
```

`AttributeError: Transformer pipeline does not provide get_feature_names`

As such, you would have to inspect the transformers one-by-one to figure out the 1518 column names, as shown previously in lesson 8.4, and then you could filter that list down to the 759 selected features using the boolean array.

Note that starting in scikit-learn version 1.1, the `get_feature_names_out` method should work on this `ColumnTransformer`, since the `get_feature_names_out` method will be available for all transformers.

13.8 Q&A: Are the selected features the “most important” features?

After using any feature selection procedure, it’s hard to say whether the selected feature set is truly the best set of features, or whether there might be another feature set that performs just as well.

For example, one high-performing feature set might include feature A, while another high-performing feature set might exclude feature A but include features B and C (which are highly correlated with A). This is especially likely any time the number of features is much greater than the number of samples.

As such, feature selection is not an optimal tool for determining feature importance.

Feature selection vs feature importance:

- Multiple sets of features may perform similarly
- Especially likely if there are many more features than samples ($p \gg n$)
- Thus, feature selection does not necessarily determine feature importance

13.9 Q&A: Is it okay for feature selection to remove one-hot encoded categories?

The feature selection process examines each feature independently, which means that it does not know that there are groups of feature columns that originated from the same feature. As a result, feature selection might remove some of the columns that resulted from one-hot encoding a feature, and keep others.

However, I don't see this as necessarily being problematic. Each one-hot encoded column can be thought of as independent from all others, since it merely represents the presence or absence of a particular value for a categorical column. If the presence or absence of "Embarked from C" has a relationship with the target but the presence or absence of "Embarked from Q" does not, then I would agree with one of those columns being removed while the other remains.

This is similar to how I think of the features output by `CountVectorizer`: Some text features have a relationship with the target and should be kept, while others do not have a relationship with the target and should be removed.

Feature selection of one-hot encoded categories:

- Feature selection examines each feature column independently (regardless of its "origin")
- Each one-hot encoded column is conceptually independent from the others
- Thus, it's acceptable for feature selection to ignore the origin of each column when removing features

14 Feature standardization

14.1 Standardizing numerical features

Some Machine Learning models benefit from a process called feature standardization. That's because the objective function of those models assumes that all features are centered around zero and have a variance of the same order of magnitude. If that assumption is incorrect, a given feature might dominate the objective function and the model won't be able to learn from all of the features, thus reducing its performance.

Why is feature standardization useful?

- Some models assume that features are centered around zero and have similar variances
- Those models may perform poorly if that assumption is incorrect

In this chapter, we'll experiment with standardizing our features to see if that improves our model performance.

We'll start with the most common approach, which is to use `StandardScaler` and only standardize features that were originally numerical. We'll import it from the `preprocessing` module and create an instance called `scaler` using the default parameters. For each feature, it will subtract the mean and divide by the standard deviation, which centers the data around zero and scales it.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
```

Here's a reminder of our existing `ColumnTransformer`. Note that our numerical features are `Age`, `Fare`, and `Parch`.

```
ct = make_column_transformer(
    (imp_ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp, ['Age', 'Fare']),
    ('passthrough', ['Parch']))
```

To scale our numerical features, we'll make a `Pipeline` of imputation and scaling called `imp_scaler`.

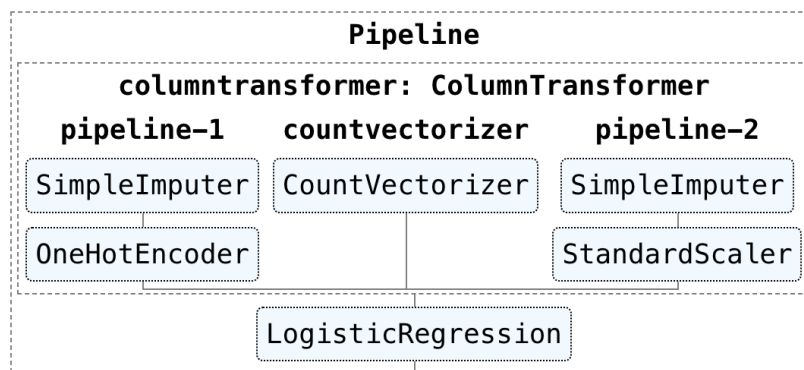
```
imp_scaler = make_pipeline(imp, scaler)
```

Then, we'll replace `imp` with `imp_scaler` in our `ColumnTransformer` and apply it to `Parch` as well, which was previously a passthrough column.

```
ct = make_column_transformer(
    (imp_ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp_scaler, ['Age', 'Fare', 'Parch']))
```

Finally, we'll create a Pipeline called `scaler_pipe` using the updated `ColumnTransformer`.

```
scaler_pipe = make_pipeline(ct, logreg)
scaler_pipe
```



The cross-validated accuracy of this Pipeline is 0.810, which is nearly the same as our baseline accuracy.

```
cross_val_score(scaler_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8103383340656581

Pipeline accuracy scores:

- Grid search (VC): 0.834
- Grid search (LR with `SelectFromModel` ET): 0.832
- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (VC): 0.818
- Baseline (LR): 0.811
- Baseline (RF): 0.811
- Baseline (LR with numerical features standardized): 0.810

That might be surprising, because regularized linear models often benefit from feature standardization. However, our particular logistic regression solver (`liblinear`) happens to be robust to unscaled data, and thus there was no benefit in this case.

The takeaway here is that you shouldn't always assume that standardization of numerical features is necessary.

Why didn't feature standardization help?

- Regularized linear models often benefit from standardization
- However, the liblinear solver is robust to unscaled data

14.2 Standardizing all features

In the previous lesson, we standardized the numerical features only. An alternative approach is to standardize all features after transformation, even if they were not originally numerical. That's what we'll try in this lesson.

Our strategy will be to add standardization as the second step in the Pipeline, in between the ColumnTransformer and the model. However, our ColumnTransformer outputs a sparse matrix, and StandardScaler would destroy the sparseness by centering the data, likely resulting in a memory issue.

Why not use StandardScaler?

- Our ColumnTransformer outputs a sparse matrix
- Centering would cause memory issues by creating a dense matrix

Thus, we're going to use an alternative scaler called MaxAbsScaler. We'll import it from the preprocessing module and create an instance called scaler. MaxAbsScaler divides each feature by its maximum value, which scales each feature to the range -1 to 1. Zeros are never changed, and thus sparsity is preserved.

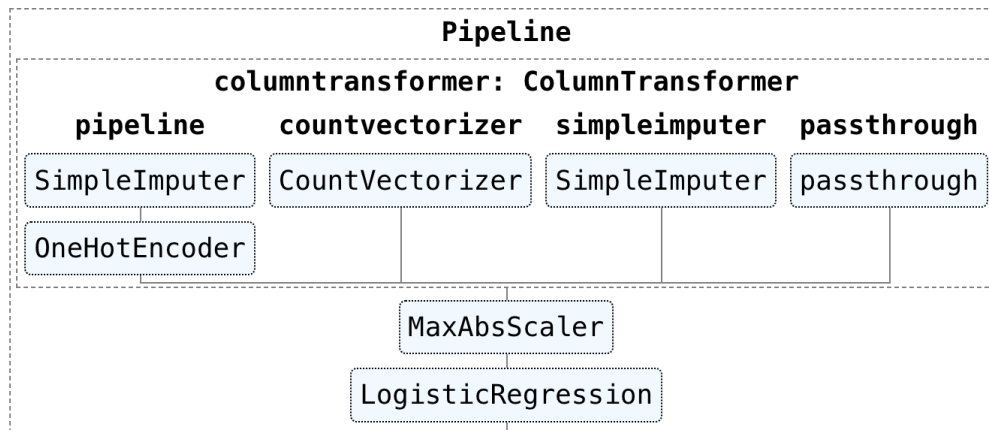
```
from sklearn.preprocessing import MaxAbsScaler
scaler = MaxAbsScaler()
```

First, we'll reset our ColumnTransformer so that it doesn't include the imp_scaler Pipeline.

```
ct = make_column_transformer(
    (imp_ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp, ['Age', 'Fare']),
    ('passthrough', ['Parch']))
```

Then, we'll update the scaler_pipe object to use MaxAbsScaler as the second step.

```
scaler_pipe = make_pipeline(ct, scaler, logreg)
scaler_pipe
```



When we cross-validate it, the accuracy is 0.811, which is exactly the same as our baseline accuracy.

```
cross_val_score(scaler_pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8114556525014123

Pipeline accuracy scores:

- Grid search (VC): 0.834
- Grid search (LR with SelectFromModel ET): 0.832
- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (VC): 0.818
- Baseline (LR): 0.811
- Baseline (LR with all features standardized): 0.811
- Baseline (RF): 0.811
- Baseline (LR with numerical features standardized): 0.810

I'm not surprised at this result, because `MaxAbsScaler` has no effect on the columns output by `OneHotEncoder` and only a tiny effect on the columns output by `CountVectorizer`. As such, our approach in this lesson is mostly just affecting the numerical columns, which is what we did in the previous lesson.

Although we didn't see any benefits from standardization, there are certainly cases in which it will help. If you do want to experiment with standardization, my suggestion is to try out both of the approaches that we used in this chapter and then choose whichever one works better.

14.3 Q&A: How do I see what scaling was applied to each feature?

If you're interested in seeing what scaling was applied to each feature, you can fit the Pipeline and then examine the `scale_` attribute of the `maxabsscaler` step.

For example, the last three entries in the array correspond to the scaling for Age, Fare, and Parch. These are simply the maximum values of Age, Fare, and Parch in X. As a reminder, this is the scaling that will be applied to the features in X_{new} when making predictions.

```
scaler_pipe.fit(X, y)
scaler_pipe.named_steps['maxabsscaler'].scale_
```

```
array([ 1.      ,  1.      ,  1.      , ..., 80.      , 512.3292,  6.      ])
```

As you might expect, the `scale_` attribute is also available when using `StandardScaler`.

14.4 Q&A: How do I turn off feature standardization within a grid search?

Although grid search is usually just used to tune parameter values, you can actually use grid search to turn on and off particular Pipeline steps. Thus you could use a grid search to decide whether or not feature standardization should be included.

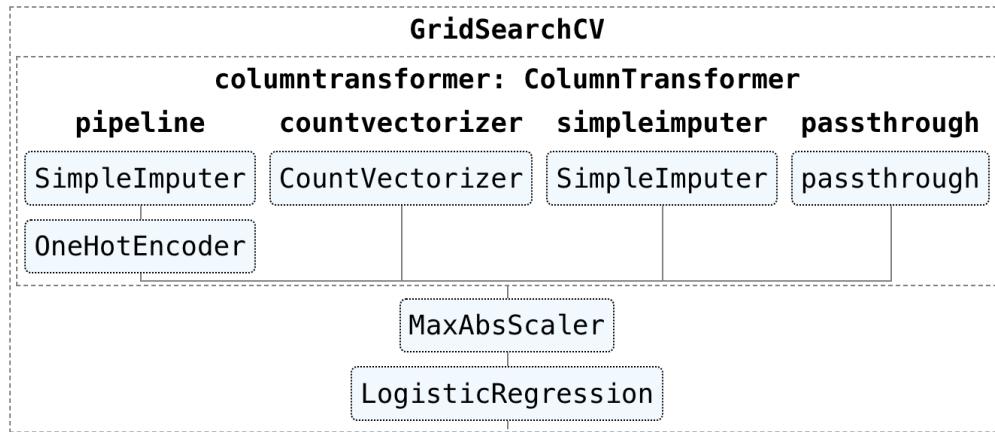
To demonstrate this, let's create a small dictionary called `scaler_params`:

- The first entry tunes the C parameter of logistic regression. The dictionary key is the step name, then two underscores, then the parameter name. The dictionary values are the possible values for that parameter.
- The second entry is different: Rather than tuning the parameter of a Pipeline step, we're tuning the Pipeline step itself. In this case, the dictionary key is simply the step name assigned by `make_pipeline`. The possible values are 'passthrough', which means skip this Pipeline step, or a `MaxAbsScaler` instance, which means keep `MaxAbsScaler` in the Pipeline.

```
scaler_params = {}
scaler_params['logisticregression__C'] = [0.1, 1, 10]
scaler_params['maxabsscaler'] = ['passthrough', MaxAbsScaler()]
```

We'll create `scaler_grid` using the `scaler_pipe` and `scaler_params` objects, and then run the grid search as normal.

```
scaler_grid = GridSearchCV(scaler_pipe, scaler_params, cv=5,
                           scoring='accuracy', n_jobs=-1)
scaler_grid.fit(X, y)
```

As you can see in the results, each C value was tried once with MaxAbsScaler and once without.

```
results = (pd.DataFrame(scaler_grid.cv_results_)
            .filter(regex='param_|mean_test|rank'))
results.columns = results.columns.str.split('__').str[-1]
results
```

	C	param_maxabsscaler	mean_test_score	rank_test_score
0	0.1	passthrough	0.788990	5
1	0.1	MaxAbsScaler()	0.788971	6
2	1	passthrough	0.811462	2
3	1	MaxAbsScaler()	0.811456	3
4	10	passthrough	0.809234	4
5	10	MaxAbsScaler()	0.819308	1

With this limited set of parameters, the best results came from using a C value of 10 and including the MaxAbsScaler step.

```
scaler_grid.best_params_
```

```
{'logisticregression__C': 10, 'maxabsscaler': MaxAbsScaler()}
```

14.5 Q&A: Which models benefit from standardization?

Feature standardization tends to be useful with any model that considers the distance between features, such as K-Nearest Neighbors and Support Vector Machines.

It also tends to be useful with any models that incorporate regularization, such as linear or logistic regression with an L1 or L2 penalty, though we saw earlier in the chapter that this doesn't apply to all solvers.

Notably, feature standardization will not benefit any tree-based models such as random forests.

When is feature standardization likely to be useful?

- **Useful:**
 - Distance-based models (KNN, SVM)
 - Regularized models (linear or logistic regression with L1/L2)
- **Not useful:**
 - Tree-based models (random forests)

15 Feature engineering with custom transformers

15.1 Why not use pandas for feature engineering?

Let's say that you need to create some custom features for your model. This is usually because you believe that your model could learn more from a particular feature if it was represented in a different way or combined with another feature.

Often, feature engineering is done using pandas on the original dataset, and then the updated dataset is passed to scikit-learn. However, you can actually do feature engineering within scikit-learn using custom transformers, and in this chapter I'll show you how.

It's a bit more work to do feature engineering within scikit-learn, but it means that all of your data transformations can be included in a Pipeline, which provides some considerable benefits. Namely, those transformations can be tuned using a grid search, they can be applied to new data without any extra work, and when done correctly, there's no possibility of data leakage.

Options for feature engineering:

- **pandas:** Create features on original dataset, pass updated dataset to scikit-learn
- **scikit-learn:** Create features using custom transformers
 - Requires more work
 - All transformations can be included in a Pipeline

15.2 Transformer 1: Rounding numerical values

To start, we're going to redefine df as a 10-row dataset. This will make it much easier for you to see how custom transformers work.

```
df = pd.read_csv('http://bit.ly/MLtrain', nrows=10)
df
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	3	Bra...	male	22.0	1	0	A/5...	7.2500	NaN	S
1	1	1	Cum...	female	38.0	1	0	PC ...	71....	C85	C
2	1	3	Hei...	female	26.0	0	0	STO...	7.9250	NaN	S
3	1	1	Fut...	female	35.0	1	0	113803	53....	C123	S
4	0	3	All...	male	35.0	0	0	373450	8.0500	NaN	S
5	0	3	Mor...	male	NaN	0	0	330877	8.4583	NaN	Q
6	0	1	McC...	male	54.0	0	0	17463	51....	E46	S
7	0	3	Pal...	male	2.0	3	1	349909	21....	NaN	S
8	1	3	Joh...	female	27.0	0	2	347742	11....	NaN	S
9	1	2	Nas...	female	14.0	1	0	237736	30....	NaN	C

First, let's pretend we believe Fare would be a better feature if we rounded it up to the next integer. We can do this using NumPy's `ceil` function, which is short for ceiling. Notice that each value in Fare has now been rounded up.

In general, NumPy functions are a good choice for custom transformations because both scikit-learn and pandas use NumPy "under the hood".

Note that we passed it a 2D object, namely a one-column DataFrame, and it returned a 2D object, again a one-column DataFrame. This will turn out to be a useful characteristic for custom transformations, as you'll see later in the chapter.

```
np.ceil(df[['Fare']])
```

	Fare
0	8.0
1	72.0
2	8.0
3	54.0
4	9.0
5	9.0
6	52.0
7	22.0
8	12.0
9	31.0

In order to do this transformation within scikit-learn, we need to convert the `ceil` function into a scikit-learn transformer using the `FunctionTransformer` class, which we'll import from the `preprocessing` module.

We simply pass the `ceil` function to `FunctionTransformer`, and it returns a transformer object, which we'll call `ceiling`. (Note that if you're using a version of scikit-learn prior to 0.22, you should also include the argument `validate=False` any time you're using `FunctionTransformer`.)

```
from sklearn.preprocessing import FunctionTransformer
ceiling = FunctionTransformer(np.ceil)
```

Because `ceiling` is a transformer, you can pass `Fare` to its `fit_transform` method, which performs the same transformation as before. This is the simplest example of feature engineering within scikit-learn.

```
ceiling.fit_transform(df[['Fare']])
```

	Fare
0	8.0
1	72.0
2	8.0
3	54.0
4	9.0
5	9.0
6	52.0
7	22.0
8	12.0
9	31.0

Like any transformer, `ceiling` can be included in a `ColumnTransformer`. For the moment, we'll create a `ColumnTransformer` instance that only includes the `ceiling` transformer, though we'll add more transformers throughout the chapter.

Finally, we'll pass `df` to its `fit_transform` method, which confirms that it works. As we've seen throughout the book, `ColumnTransformer` always outputs a NumPy array or a sparse matrix, and in this case it outputs a NumPy array.

```
ct = make_column_transformer(  
    (ceiling, ['Fare']))  
ct.fit_transform(df)
```

```
array([[ 8.],  
       [72.],  
       [ 8.],  
       [54.],  
       [ 9.],  
       [ 9.],  
       [52.],  
       [22.],  
       [12.],  
       [31.]])
```

15.3 Transformer 2: Clipping numerical values

Let's look at the 10-row dataset again.

```
df
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	3	Bra...	male	22.0	1	0	A/5...	7.2500	NaN	S
1	1	1	Cum...	female	38.0	1	0	PC ...	71....	C85	C
2	1	3	Hei...	female	26.0	0	0	STO...	7.9250	NaN	S
3	1	1	Fut...	female	35.0	1	0	113803	53....	C123	S
4	0	3	All...	male	35.0	0	0	373450	8.0500	NaN	S
5	0	3	Mor...	male	NaN	0	0	330877	8.4583	NaN	Q
6	0	1	McC...	male	54.0	0	0	17463	51....	E46	S
7	0	3	Pal...	male	2.0	3	1	349909	21....	NaN	S
8	1	3	Joh...	female	27.0	0	2	347742	11....	NaN	S
9	1	2	Nas...	female	14.0	1	0	237736	30....	NaN	C

For our second transformation, let's pretend we believe Age would be a better feature if we limited the values to the range 5 to 60, such that all values below 5 are rounded up to 5, and all values above 60 are rounded down to 60. We can do this using NumPy's `clip` function. Note that it has two required arguments, `a_min` and `a_max`, which define the limits.

In this case, the only value that changed was row 7, in which 2.0 became 5.0.

```
np.clip(df[['Age']], a_min=5, a_max=60)
```

	Age
0	22.0
1	38.0
2	26.0
3	35.0
4	35.0
5	NaN
6	54.0
7	5.0
8	27.0
9	14.0

We'll convert NumPy's `clip` function to a transformer called `clip`, though this time we need to pass the `a_min` and `a_max` arguments to the `FunctionTransformer`'s `kw_args` parameter.

```
clip = FunctionTransformer(np.clip, kw_args={'a_min':5, 'a_max':60})
```

We'll check that the `clip` transformer works by running the `fit_transform` method and passing it the Age column.

```
clip.fit_transform(df[['Age']])
```

	Age
0	22.0
1	38.0
2	26.0
3	35.0
4	35.0
5	NaN
6	54.0
7	5.0
8	27.0
9	14.0

Finally, we'll add the clip transformer to the ColumnTransformer, and confirm that the ColumnTransformer still works as expected, which it does.

```
ct = make_column_transformer(
    (ceiling, ['Fare']),
    (clip, ['Age']))
ct.fit_transform(df)
```

```
array([[ 8., 22.],
       [72., 38.],
       [ 8., 26.],
       [54., 35.],
       [ 9., 35.],
       [ 9., nan],
       [52., 54.],
       [22.,  5.],
       [12., 27.],
       [31., 14.]])
```

15.4 Transformer 3: Extracting string values

Let's look at the dataset again.

df

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	3	Bra...	male	22.0	1	0	A/5...	7.2500	NaN	S
1	1	1	Cum...	female	38.0	1	0	PC ...	71....	C85	C
2	1	3	Hei...	female	26.0	0	0	STO...	7.9250	NaN	S
3	1	1	Fut...	female	35.0	1	0	113803	53....	C123	S
4	0	3	All...	male	35.0	0	0	373450	8.0500	NaN	S
5	0	3	Mor...	male	NaN	0	0	330877	8.4583	NaN	Q
6	0	1	McC...	male	54.0	0	0	17463	51....	E46	S
7	0	3	Pal...	male	2.0	3	1	349909	21....	NaN	S
8	1	3	Joh...	female	27.0	0	2	347742	11....	NaN	S
9	1	2	Nas...	female	14.0	1	0	237736	30....	NaN	C

For our third transformation, let's assume that the first letter of Cabin indicates the deck they were staying on, which we believe might be predictive. Our first thought might be to use the Series string slice method to extract the first character.

This works, but notice that the output is a 1D object, namely a pandas Series. This is problematic because a function (once transformed) must return 2D output in order to be used in a ColumnTransformer.

```
df['Cabin'].str.slice(0, 1)
```

	Cabin
0	NaN
1	C
2	NaN
3	C
4	NaN
5	NaN
6	E
7	NaN
8	NaN
9	NaN

```
df['Cabin'].str.slice(0, 1).shape
```

(10,)

To resolve this, we'll use the DataFrame apply method with a lambda. It still extracts the first character, but notice that it now returns 2D output. Also notice that it accepts 2D input, which means that it will be able to operate on multiple columns if we like.

```
df[['Cabin']].apply(lambda x: x.str.slice(0, 1))
```

	Cabin
0	NaN
1	C
2	NaN
3	C
4	NaN
5	NaN
6	E
7	NaN
8	NaN
9	NaN

```
df[['Cabin']].apply(lambda x: x.str.slice(0, 1)).shape
```

(10, 1)

Since this doesn't already exist as a standalone function, our next step is to convert this operation into a custom function called `first_letter`.

This function would work, but notice that it requires the input to be a pandas DataFrame, since `apply` is a DataFrame method. That may be problematic since ColumnTransformers accept both DataFrames and NumPy arrays as input. As such, it's better to write a function that will work regardless of whether the input is a DataFrame or a NumPy array.

```
def first_letter(df):  
    return df.apply(lambda x: x.str.slice(0, 1))
```

We can revise the function to work with both DataFrames and NumPy arrays by converting the input to a DataFrame explicitly before using the `apply` method.

```
def first_letter(df):  
    return pd.DataFrame(df).apply(lambda x: x.str.slice(0, 1))
```

We'll check that the function works. Again, note that it accepts 2D input and returns 2D output, which is what we want.

```
first_letter(df[['Cabin']])
```

	Cabin
0	NaN
1	C
2	NaN
3	C
4	NaN
5	NaN
6	E
7	NaN
8	NaN
9	NaN

Next, we'll convert the function to a transformer called `letter`.

```
letter = FunctionTransformer(first_letter)
```

Finally, we'll add the `letter` transformer to the ColumnTransformer and check that it works, which it does.

```
ct = make_column_transformer(  
    (ceiling, ['Fare']),  
    (clip, ['Age']),  
    (letter, ['Cabin']))  
ct.fit_transform(df)
```

```
array([[8.0, 22.0, nan],
       [72.0, 38.0, 'C'],
       [8.0, 26.0, nan],
       [54.0, 35.0, 'C'],
       [9.0, 35.0, nan],
       [9.0, nan, nan],
       [52.0, 54.0, 'E'],
       [22.0, 5.0, nan],
       [12.0, 27.0, nan],
       [31.0, 14.0, nan]], dtype=object)
```

15.5 Rules for transformer functions

We've been talking about input and output shapes, so let me summarize the rules for functions that will be used in a `ColumnTransformer`:

- First, your function isn't required to accept 2D input, but it's better if it accepts 2D input. This enables your function (once transformed) to accept multiple columns in the `ColumnTransformer`.
- Second, your function is required to return 2D output in order to be used in a `ColumnTransformer`. Thus if your function returns a pandas object, it should return a `DataFrame` (not a `Series`). And if your function returns a 1D array, you should reshape the array to be 2D before returning it. We'll see an example of this in the next lesson.

Input and output of transformer functions:

- **Input:**
 - 1D is allowed
 - 2D is preferred (can accept multiple columns)
- **Output:**
 - 2D is required

15.6 Transformer 4: Combining two features

Let's look at the dataset one more time.

```
df
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	3	Bra...	male	22.0	1	0	A/5...	7.2500	NaN	S
1	1	1	Cum...	female	38.0	1	0	PC ...	71....	C85	C
2	1	3	Hei...	female	26.0	0	0	STO...	7.9250	NaN	S
3	1	1	Fut...	female	35.0	1	0	113803	53....	C123	S
4	0	3	All...	male	35.0	0	0	373450	8.0500	NaN	S
5	0	3	Mor...	male	NaN	0	0	330877	8.4583	NaN	Q
6	0	1	McC...	male	54.0	0	0	17463	51....	E46	S
7	0	3	Pal...	male	2.0	3	1	349909	21....	NaN	S
8	1	3	Joh...	female	27.0	0	2	347742	11....	NaN	S
9	1	2	Nas...	female	14.0	1	0	237736	30....	NaN	C

For our fourth transformation, let's say we believe that a passenger's total number of family members aboard, meaning SibSp plus Parch, is more predictive than either feature individually.

To create this feature, we can use the DataFrame's sum method over the 1 axis. However, this outputs a 1D object, which will not work with a ColumnTransformer.

```
df[['SibSp', 'Parch']].sum(axis=1)
```

	0
0	1
1	1
2	0
3	1
4	0
5	0
6	0
7	4
8	2
9	1

```
df[['SibSp', 'Parch']].sum(axis=1).shape
```

```
(10,)
```

To resolve this issue, we can use NumPy's reshape method. Since it's a NumPy method, we need to first convert the DataFrame into a NumPy array, then use NumPy's sum method, then chain the reshape method on the end to convert the output into a 2D object. If you're not familiar with the reshape method, this notation specifies that the second dimension should be 1 and the first dimension should be inferred.

```
np.array(df[['SibSp', 'Parch']]).sum(axis=1).reshape(-1, 1)
```

```
array([[1],
       [1],
       [0],
       [1],
```

```
[0],  
[0],  
[0],  
[4],  
[2],  
[1]])
```

Next, we'll convert this operation into a function called `sum_cols`. This function will work regardless of whether the input is a DataFrame or a NumPy array.

```
def sum_cols(df):  
    return np.array(df).sum(axis=1).reshape(-1, 1)
```

As always, we'll check that the function works.

```
sum_cols(df[['SibSp', 'Parch']])
```

```
array([[1],  
       [1],  
       [0],  
       [1],  
       [0],  
       [0],  
       [0],  
       [4],  
       [2],  
       [1]])
```

Since it does work, we'll convert the function to a transformer called `total`.

```
total = FunctionTransformer(sum_cols)
```

Finally, we'll add the `total` transformer to the `ColumnTransformer`, specify that it should be applied to the `SibSp` and `Parch` columns, and then check that it all works together, which it does.

```
ct = make_column_transformer(  
    (ceiling, ['Fare']),  
    (clip, ['Age']),  
    (letter, ['Cabin']),  
    (total, ['SibSp', 'Parch']))  
ct.fit_transform(df)
```

```
array([[8.0, 22.0, nan, 1],  
       [72.0, 38.0, 'C', 1],  
       [8.0, 26.0, nan, 0],
```

```
[54.0, 35.0, 'C', 1],
[9.0, 35.0, nan, 0],
[9.0, nan, nan, 0],
[52.0, 54.0, 'E', 0],
[22.0, 5.0, nan, 4],
[12.0, 27.0, nan, 2],
[31.0, 14.0, nan, 1]], dtype=object)
```

15.7 Revising the transformers

At this point, we’ve built four custom transformers and tested them out on 10 rows. Now, we want to apply them to our entire dataset, along with all of our other transformations.

Before we start, let’s review our original ColumnTransformer.

```
ct = make_column_transformer(
    (imp_ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp, ['Age', 'Fare']),
    ('passthrough', ['Parch']))
```

Let’s compare that with the ColumnTransformer containing our custom transformations.

```
ct = make_column_transformer(
    (ceiling, ['Fare']),
    (clip, ['Age']),
    (letter, ['Cabin']),
    (total, ['SibSp', 'Parch']))
```

In order to combine the custom transformations with our original transformations, there are three issues we’ll need to handle:

- First, Cabin and SibSp weren’t used in our original ColumnTransformer.
- Second, Fare and Age both have missing values.
- Third, the first letter of Cabin is non-numeric, and it has missing values.

Issues to handle when updating the ColumnTransformer:

1. Cabin and SibSp weren’t originally included
2. Fare and Age have missing values
3. Cabin is non-numeric and has missing values

To start, we’ll define `df` as the entire dataset.

```
df = pd.read_csv('http://bit.ly/MLtrain')
```

Then, we'll add Cabin and SibSp to the list of columns, and update X and X_new to include these columns. That solves issue number one.

```
cols = ['Parch', 'Fare', 'Embarked', 'Sex', 'Name', 'Age', 'Cabin',  
        'SibSp']  
X = df[cols]  
X_new = df_new[cols]
```

Next, let's handle issue number two, which is the missing values in Fare and Age. Remember how we used a Pipeline to combine imputation and one-hot encoding for Embarked and Sex? We'll do something similar for Fare and Age:

- For Fare, we'll create a Pipeline called `imp_ceiling` that does imputation before ceiling.
- For Age, we'll create a Pipeline called `imp_clip` that does imputation before clipping.

```
imp_ceiling = make_pipeline(imp, ceiling)  
imp_clip = make_pipeline(imp, clip)
```

Finally, we'll tackle issue number three, which will turn out to be the most complicated issue. Let's slice the first letter of Cabin and then take the `value_counts` of the result to see why:

- First, it contains missing values, so imputation will be required.
- Second, letters are strings, so one-hot encoding will be required.

```
X['Cabin'].str.slice(0, 1).value_counts(dropna=False)
```

Cabin	
NaN	687
C	59
B	47
D	33
E	32
A	15
F	13
G	4
T	1

In addition, notice that the G and T categories are quite rare, which can cause problems with cross-validation. Why would that be?

For any rare category, it's possible for all values of that category to show up in the same testing fold during cross-validation. If that happens, the rare category won't be learned by the `OneHotEncoder` during the fit step, and will be treated as an unknown category during the transform step. By default, the `OneHotEncoder` will error when it encounters an unknown category, and thus cross-validation will also throw an error.

Why are rare categories problematic for cross-validation?

- Rare category values may all show up in the same testing fold
- The rare category won't be learned during fit and will be treated as an unknown category
- OneHotEncoder will error when it encounters an unknown category

Although the problem is complicated, the solution is simple: We need to set the `handle_unknown` parameter to `'ignore'`, which we learned about back in lesson 3.5.

```
ohe_ignore = OneHotEncoder(handle_unknown='ignore')
```

To resolve all of these issues with the Cabin column, we'll create a three-step Pipeline:

- Step 1 is the letter transformer, which extracts the first letter.
- Step 2 is `imp_constant`, which imputes the constant value "missing".
- Step 3 is `ohe_ignore`, which one-hot encodes the results.

```
letter_imp_ohe = make_pipeline(letter, imp_constant, ohe_ignore)
```

Now that the three big issues in this lesson have been resolved, we're finally ready to update our primary `ColumnTransformer`:

- Embarked, Sex, and Name are transformed exactly as they were previously.
- For Fare, we'll use `imp_ceiling` instead of `imp`.
- For Age, we'll use `imp_clip` instead of `imp`.
- For Cabin, we'll use `letter_imp_ohe`.
- For SibSp and Parch, we'll use `total`.

```
ct = make_column_transformer(  
    (imp_ohe, ['Embarked', 'Sex']),  
    (vect, 'Name'),  
    (imp_ceiling, ['Fare']),  
    (imp_clip, ['Age']),  
    (letter_imp_ohe, ['Cabin']),  
    (total, ['SibSp', 'Parch']))
```

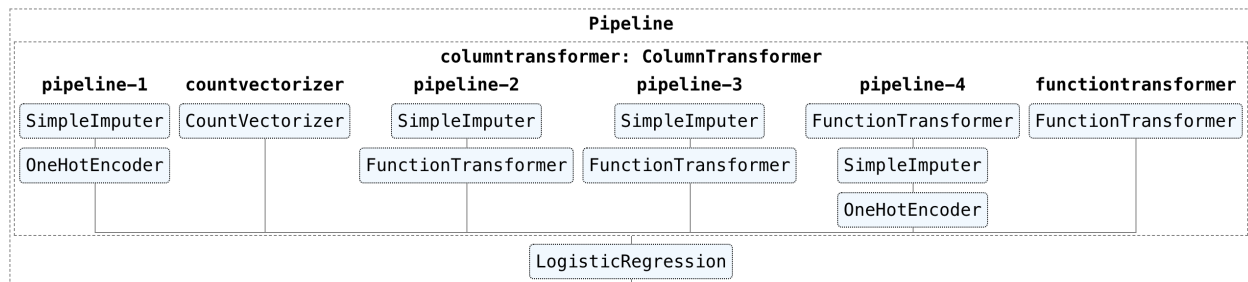
We'll check that it works by passing `X` to the `fit_transform` method.

```
ct.fit_transform(X)
```

```
<891x1527 sparse matrix of type '<class 'numpy.float64'>'  
  with 8360 stored elements in Compressed Sparse Row format>
```

Then we'll update the Pipeline to use the new `ColumnTransformer`.

```
pipe = make_pipeline(ct, logreg)
pipe
```



When we cross-validate the Pipeline, the accuracy is 0.827, which is higher than our baseline accuracy of 0.811. And it's very likely that its accuracy could be further improved through hyperparameter tuning.

```
cross_val_score(pipe, X, y, cv=5, scoring='accuracy').mean()
```

0.8271483271608812

Pipeline accuracy scores:

- Grid search (VC): 0.834
- Grid search (LR with SelectFromModel ET): 0.832
- Grid search (RF): 0.829
- Grid search (LR): 0.828
- Baseline (LR with more features): 0.827
- Baseline (VC): 0.818
- Baseline (LR): 0.811
- Baseline (RF): 0.811

Finally, we'll fit the Pipeline to X and y and use it to make predictions for X_new.

```
pipe.fit(X, y)
pipe.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1,
       1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1,
       1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,
```



```
0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0,
1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1,
0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1,
0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1])
```

15.8 Q&A: How do I fix incorrect data types within a Pipeline?

Let's say that you have the following DataFrame.

```
demo = pd.DataFrame({'A': ['10', '20', '30'],
                      'B': ['40', '50', '60'],
                      'C': [70, 80, 90],
                      'D': ['x', 'y', 'z']})
demo
```

	A	B	C	D
0	10	40	70	x
1	20	50	80	y
2	30	60	90	z

It may look like the first three columns are all integers, but columns A and B are actually object columns because the numbers are being stored as strings, which is a common problem in real datasets.

```
demo.dtypes
```

0	
A	object
B	object
C	int64
D	object

These data types need to be fixed in order for a scikit-learn model to understand them. Within pandas, you can do this using the DataFrame method `astype`.

```
demo[['A', 'B']].astype('int')
```

	A	B
0	10	40
1	20	50
2	30	60

As you can see, this changes the data types of columns A and B to integer.

```
demo[['A', 'B']].astype('int').dtypes
```

0	
A	int64
B	int64

To incorporate this into a scikit-learn Pipeline, we would first define a custom function called `make_integer` that converts DataFrame columns to integers.

```
def make_integer(df):
    return pd.DataFrame(df).astype('int')
```

Then, we can convert the function to a transformer called `integer`, and check that it works.

```
integer = FunctionTransformer(make_integer)
integer.fit_transform(demo[['A', 'B']])
```

	A	B
0	10	40
1	20	50
2	30	60

You can see that the data types are now integer.

```
integer.fit_transform(demo[['A', 'B']]).dtypes
```

0	
A	int64
B	int64

What I've shown so far is a reasonable strategy, and it will work in many cases. However, let's modify the demo DataFrame slightly.

```
demo.loc[2, 'B'] = ''
```

As you can see, the value 60 has been replaced by an empty string.

```
demo
```

	A	B	C	D
0	10	40	70	x
1	20	50	80	y
2	30		90	z

If we try to use our integer transformer on the revised DataFrame, it will error because the `astype` method doesn't know how to handle an empty string.

```
integer.fit_transform(demo[['A', 'B']])
```

```
ValueError: invalid literal for int() with base 10: ''
```

An alternative strategy is to use the pandas function `to_numeric` and apply it to each of the columns. As you can see, `to_numeric` replaced the empty string with `NaN`.

```
demo[['A', 'B']].apply(pd.to_numeric)
```

	A	B
0	10	40.0
1	20	50.0
2	30	NaN

Column A is now an integer column, and column B is now a float column since integer columns don't currently support NumPy's `NaN` value.

```
demo[['A', 'B']].apply(pd.to_numeric).dtypes
```

	0
A	int64
B	float64

If we wanted to use this in the Pipeline, we would create a custom function called `make_number`.

```
def make_number(df):  
    return pd.DataFrame(df).apply(pd.to_numeric)
```

Then we would convert the function to a transformer called `number`, and check that it works.

```
number = FunctionTransformer(make_number)  
number.fit_transform(demo[['A', 'B']])
```

	A	B
0	10	40.0
1	20	50.0
2	30	NaN

You could use either the `integer` or `number` transformers in a `ColumnTransformer`, and both would allow you to fix data types within a `Pipeline`. However, there are some advantages to using the `number` transformer, since the `to_numeric` function includes a few different options for error handling, and it will handle the conversion of both integers and floating point numbers.

15.9 Q&A: How do I create features from datetime data?

To demonstrate how to create date-based features, I'm going to read a tiny dataset of reported UFO sightings into a `DataFrame`.

```
ufo = pd.read_csv('http://bit.ly/ufo-sample', parse_dates=['Date'])
ufo
```

	Date	City	State	Shape
0	2023-08-04	Lexington	SC	Cylinder
1	2023-08-04	Alpharetta	GA	Orb
2	2023-08-05	Baltimore	MD	Light
3	2023-08-05	Helena	MT	Rectangle
4	2023-08-06	Wilmington	NC	Fireball
5	2023-08-06	Brooklyn	NY	Star

Because we parsed the `Date` column during file reading, it uses the special datetime data type.

```
ufo.dtypes
```

	0
Date	datetime64[ns]
City	object
State	object
Shape	object

Because of the datetime data type, we can access various properties of the `Date` column using the `dt` accessor. For example, we can easily access the day of the month using the `day` attribute.

```
ufo['Date'].dt.day
```

	Date
0	4
1	4
2	5
3	5
4	6
5	6

Let's say that you wanted to use the day of the month as a feature. The first step would be to create a custom function like we've done previously called `day_of_month`.

```
def day_of_month(df):  
    return df.apply(lambda x: x.dt.day)
```

We can see that this function works. However, it's not quite optimal because it assumes that the object being passed in is a DataFrame with the datetime data type.

```
day_of_month(ufo[['Date']])
```

Date	
0	4
1	4
2	5
3	5
4	6
5	6

Let's make the function more robust by enabling it to handle NumPy arrays and by doing the datetime conversion within the function.

Here's one option. As you can see, it converts the object to a DataFrame, and then it converts each column to datetime format before accessing the day attribute.

```
def day_of_month(df):  
    return pd.DataFrame(df).apply(lambda x: pd.to_datetime(x).dt.day)
```

Another option is to convert all columns to datetime format within the DataFrame constructor rather than within the `apply` method.

```
def day_of_month(df):  
    return pd.DataFrame(df, dtype=np.datetime64).apply(lambda x:  
                                                         x.dt.day)
```

To check that it works, we'll read in the UFO dataset again, but this time we'll leave the Date column as an object column.

```
ufo = pd.read_csv('http://bit.ly/ufo-sample')  
ufo.dtypes
```

0	
Date	object
City	object
State	object
Shape	object

We can see that the `day_of_month` function still works, even though Date is an object column.

```
day_of_month(ufo[['Date']])
```

Date	
0	4
1	4
2	5
3	5
4	6
5	6

Finally, we'll convert the `day_of_month` function into a transformer called `day`, and check that it works as well.

```
day = FunctionTransformer(day_of_month)
day.fit_transform(ufo[['Date']])
```

Date	
0	4
1	4
2	5
3	5
4	6
5	6

15.10 Q&A: How do I create feature interactions?

When you believe there's an interaction between two or more features, one common technique is to create “interaction terms” or “feature interactions” that your model can learn from. This is generally done by multiplying the values of each pair of features and then using those as new features.

Creating interaction features is useful when the combined impact of a pair of features is different from the impact of the features when considered independently. For example, let's pretend that features A and B each have a small positive impact on the target, but when combined, they have a much larger positive impact on the target than you would expect. In that case, it would be useful to create the interaction feature of $A \times B$.

When are feature interactions useful?

- When the combined impact of features is different from their independent impacts
- **Example:**
 - A and B (individually) each have a small positive impact
 - A and B (combined) have a larger positive impact than expected

Let's see how we can create feature interactions in scikit-learn. We'll assume that we've decided to create interactions between `Fare`, `SibSp`, and `Parch`. Here are the first three and last three rows of each of those features.

```
X[['Fare', 'SibSp', 'Parch']].to_numpy()
```

```
array([[ 7.25 ,  1.    ,  0.    ],
       [71.2833,  1.    ,  0.    ],
       [ 7.925 ,  0.    ,  0.    ],
       ...,
       [23.45 ,  1.    ,  2.    ],
       [30.    ,  0.    ,  0.    ],
       [ 7.75 ,  0.    ,  0.    ]])
```

Our first step is to import the `PolynomialFeatures` class from the `preprocessing` module, and then create an instance called `poly`. We'll set the `include_bias` parameter to `False` to avoid creating a column of ones in the output, and we'll set the `interaction_only` parameter to `True` to avoid creating the square of each feature.

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(include_bias=False, interaction_only=True)
```

When we run the `fit_transform` method and pass it those three columns, it outputs six columns:

- The first three columns of the output are the original three columns: `Fare`, `SibSp`, and `Parch`.
- The next three columns are our interaction terms: `Fare × SibSp`, `Fare × Parch`, and `SibSp × Parch`.

```
poly.fit_transform(X[['Fare', 'SibSp', 'Parch']])
```

```
array([[ 7.25 ,  1.    ,  0.    ,  7.25 ,  0.    ,  0.    ],
       [71.2833,  1.    ,  0.    , 71.2833,  0.    ,  0.    ],
       [ 7.925 ,  0.    ,  0.    ,  0.    ,  0.    ,  0.    ],
       ...,
       [23.45 ,  1.    ,  2.    , 23.45 , 46.9 ,  2.    ],
       [30.    ,  0.    ,  0.    ,  0.    ,  0.    ,  0.    ],
       [ 7.75 ,  0.    ,  0.    ,  0.    ,  0.    ,  0.    ]])
```

Output columns:

1. `Fare`
2. `SibSp`
3. `Parch`
4. `Fare × SibSp`
5. `Fare × Parch`
6. `SibSp × Parch`

If we wanted to include these feature interactions in our model, we would simply include `poly` as one of the transformers in our `ColumnTransformer`.

One obvious question is: How should you choose which feature interactions to create?

- Ideally, you would use expert knowledge of the system in question to guide your decision of which interactions to create.
- If that's not available, then you can explore the data to decide which interactions to create.
- Finally, if you have a small number of features, then you can simply create all possible interactions and then use feature selection to remove the interactions that are not useful.

Keep in mind that if you have a large number of features, it's impractical to create all possible interactions, plus you run an increased risk of false positive feature interactions. These are interactions which appear to have a relationship with the target, but that relationship is actually just occurring due to random chance.

How to choose feature interactions:

- Use expert knowledge
- Explore the data
- Create all possible interactions
 - Impractical with a large number of features
 - Increases risk of false positive interactions

It's also worth noting that tree-based models can learn feature interactions on their own through recursive splitting. That means that if you're using a tree-based model as your prediction model, then you don't need to manually create feature interactions.

Finally, you should be aware that even though linear models can't explicitly learn feature interactions, they can sometimes replace the information supplied by the interaction terms, in which case the interactions are unnecessary. As such, you should always evaluate the model with interactions against the model without interactions, and only include them if they're improving the model's performance.

When are feature interactions not useful?

- Tree-based models can learn feature interactions on their own
- Linear models can sometimes replace the information supplied by interaction terms
- **Conclusion:** Evaluate the model with and without interaction terms

15.11 Q&A: How do I save a Pipeline with custom transformers?

If you save a Pipeline using pickle or joblib, and the Pipeline includes custom transformers, then the saved Pipeline can only be loaded into a new environment if the functions it depends on are defined in the new environment.

For example, let's import pickle and use it to save our current Pipeline.

```
import pickle
with open('pipe.pickle', 'wb') as f:
    pickle.dump(pipe, f)
```


Let's pretend that we're in a brand new environment and we want to make predictions for `X_new` using our saved Pipeline. Because the Pipeline includes custom transformers which use the `first_letter` and `sum_cols` functions, those two functions need to be defined in the new environment.

```
def first_letter(df):  
    return pd.DataFrame(df).apply(lambda x: x.str.slice(0, 1))
```

```
def sum_cols(df):  
    return np.array(df).sum(axis=1).reshape(-1, 1)
```

Because those functions depend on pandas and NumPy, then these libraries also need to be imported into the new environment.

```
import pandas as pd  
import numpy as np
```

Now we can load our saved Pipeline into the `pipe_from_pickle` object.

```
with open('pipe.pickle', 'rb') as f:  
    pipe_from_pickle = pickle.load(f)
```

We also need to create the `X_new` object in our environment.

```
cols = ['Parch', 'Fare', 'Embarked', 'Sex', 'Name', 'Age', 'Cabin',  
        'SibSp']  
df_new = pd.read_csv('http://bit.ly/MLnewdata')  
X_new = df_new[cols]
```

Finally, we can make predictions using the saved Pipeline.

```
pipe_from_pickle.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1,  
       1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,  
       1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,  
       1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1,  
       1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,  
       0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0,  
       0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,  
       1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,  
       0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0,  
       1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,  
       1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
```

```

0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1,
0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1,
0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1])

```

If the process above seems too cumbersome, one alternative is to use a Python library called cloudpickle, which extends the functionality of pickle to allow you to save user-defined functions.

All you have to do is install cloudpickle, import it, and then save the Pipeline using cloudpickle instead of pickle. Notice that the cloudpickle code is exactly the same as the pickle code, except you use the dump function from cloudpickle instead of from pickle.

```

import cloudpickle
with open('pipe.pickle', 'wb') as f:
    cloudpickle.dump(pipe, f)

```

Then, in your new environment, you'll be able to load the saved Pipeline using pickle and use it to make predictions without having to define the custom functions in that environment.

```

with open('pipe.pickle', 'rb') as f:
    pipe_from_pickle = pickle.load(f)

```

```

pipe_from_pickle.predict(X_new)

```

```

array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1,
       1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1,
       1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,
       0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0,
       1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
       1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
       0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1,
       0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
       0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
       1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1])

```

15.12 Q&A: Can FunctionTransformer be used with any transformation?

FunctionTransformer should only be used with stateless transformations. A transformation is considered stateless if it doesn't learn any information during the fit step.

For example, all of our custom transformations in this chapter were stateless: rounding up to the next integer, limiting values to a range, extracting the first letter, and adding two columns. They're considered stateless because they didn't learn anything about the training data that later needed to be applied to testing data. In other words, they work exactly the same on the testing data regardless of what the training data looked like.

Stateless transformations:

- **ceiling:** Rounding up to the next integer
- **clip:** Limiting values to a range
- **letter:** Extracting the first letter
- **total:** Adding two columns

This is in contrast to stateful transformations, which do learn information from the fit step that need to be applied to both training and testing data. We've seen many stateful transformations in this book:

- **OneHotEncoder** learns the categories from the training data, and those same categories need to be applied to the testing data.
- **CountVectorizer** learns the vocabulary from the training data, and that vocabulary needs to be used when building the document-term matrix for the testing data.
- **SimpleImputer** learns the values to impute from the training data, and those values are applied to the testing data.
- **MaxAbsScaler** learns the scale of each feature from the training data, and that scaling is applied to the testing data.

Stateful transformations:

- **OneHotEncoder:** Learns the categories
- **CountVectorizer:** Learns the vocabulary
- **SimpleImputer:** Learns the values to impute
- **MaxAbsScaler:** Learns the scale of each feature

FunctionTransformer should never be used to implement stateful transformations. Depending on the situation, you would either run into an error or you would silently cause data leakage. In that case, you would need to write your own class in order to create a proper transformer.

16 Workflow review #3

16.1 Recap of our workflow

In this chapter, we're going to do one final review of the core workflow that we built throughout this book, including all of the features that we developed in the previous chapter.

We begin by importing pandas and NumPy, the four transformer classes we're using, one modeling class, and two composition functions.

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, FunctionTransformer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline
```

Next, we create a list of the eight columns we're going to select from our data.

```
cols = ['Parch', 'Fare', 'Embarked', 'Sex', 'Name', 'Age', 'Cabin',
        'SibSp']
```

Then, we read in all of our training data and use it to define our X and y.

```
df = pd.read_csv('http://bit.ly/MLtrain')
X = df[cols]
y = df['Survived']
```

We read in all of the new data and use it to define X_new.

```
df_new = pd.read_csv('http://bit.ly/MLnewdata')
X_new = df_new[cols]
```

We create five instances of our transformers, namely two different instances of SimpleImputer, two different instances of OneHotEncoder, and one instance of CountVectorizer.

```

imp = SimpleImputer()
imp_constant = SimpleImputer(strategy='constant', fill_value='missing')
ohe = OneHotEncoder()
ohe_ignore = OneHotEncoder(handle_unknown='ignore')
vect = CountVectorizer()

```

We define two custom functions that will be used for feature engineering.

```

def first_letter(df):
    return pd.DataFrame(df).apply(lambda x: x.str.slice(0, 1))

```

```

def sum_cols(df):
    return np.array(df).sum(axis=1).reshape(-1, 1)

```

We convert two NumPy functions and our two custom functions into transformers.

```

ceiling = FunctionTransformer(np.ceil)
clip = FunctionTransformer(np.clip, kw_args={'a_min':5, 'a_max':60})
letter = FunctionTransformer(first_letter)
total = FunctionTransformer(sum_cols)

```

We create four Pipelines that combine the various transformers.

```

imp_ohe = make_pipeline(imp_constant, ohe)
imp_ceiling = make_pipeline(imp, ceiling)
imp_clip = make_pipeline(imp, clip)
letter_imp_ohe = make_pipeline(letter, imp_constant, ohe_ignore)

```

Then we build the ColumnTransformer, which imputes and one-hot encodes Embarked and Sex, vectorizes Name, imputes and takes the ceiling of Fare, imputes and clips Age, imputes and one-hot encodes the first letter of Cabin, and adds SibSp and Parch.

```

ct = make_column_transformer(
    (imp_ohe, ['Embarked', 'Sex']),
    (vect, 'Name'),
    (imp_ceiling, ['Fare']),
    (imp_clip, ['Age']),
    (letter_imp_ohe, ['Cabin']),
    (total, ['SibSp', 'Parch']))

```

We also create an instance of LogisticRegression.

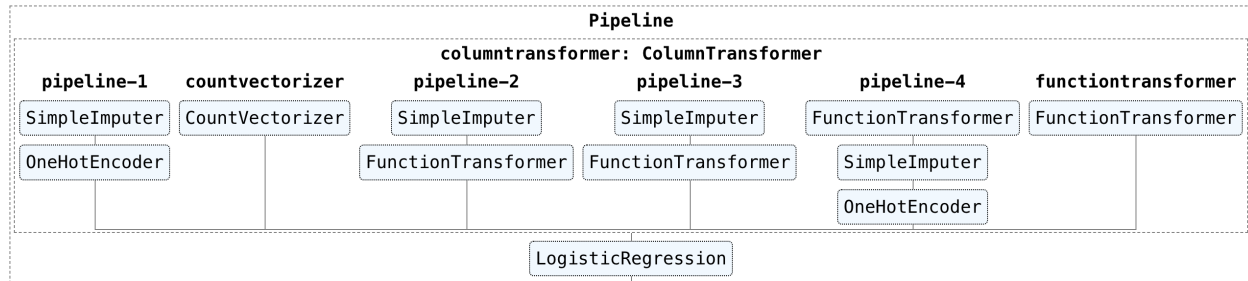
```

logreg = LogisticRegression(solver='liblinear', random_state=1)

```

We create a two-step modeling Pipeline and fit the Pipeline to X and y.

```
pipe = make_pipeline(ct, logreg)
pipe.fit(X, y)
pipe
```



Finally, we use the fitted Pipeline to make predictions on `X_new`.

```
pipe.predict(X_new)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1,
        1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1,
        1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
        1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1,
        1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,
        0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0,
        0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1,
        1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1,
        0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0,
        1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
        1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
        0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0,
        0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1,
        0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
        1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0,
        0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0,
        1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1,
        0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1])
```

Keep in mind that there are many other steps you can incorporate into this workflow in order to potentially improve performance, including hyperparameter tuning, trying a different model, ensembling, feature selection, feature standardization, and additional feature engineering.

16.2 What's the role of pandas?

If we can do all of our data transformations in scikit-learn, then you might be left wondering: What's the role of pandas?

- First is data exploration and visualization. A deep understanding of your dataset will help you with many steps of the Machine Learning workflow, especially selecting which features to use and deciding how you might want to transform those features.
- Second is testing out data transformations for Machine Learning. If I'm thinking about building a custom transformer in scikit-learn, I first create it using pandas to make sure that it works.
- Finally, if your goal is anything other than Machine Learning, then all of your data transformations should still be executed using pandas.

The bottom line is that pandas still has a huge role in the data science workflow. However, if your goal is Machine Learning, then it's best to shift as much of your workflow as possible to scikit-learn.

Uses for pandas in the data science workflow:

- **All projects:** Data exploration and visualization
- **ML projects:** Testing out data transformations
- **Non-ML projects:** Executing data transformations

17 High-cardinality categorical features

17.1 Recap of nominal and ordinal features

Let's talk about categorical features. There are two types of categorical features that we've covered in the book:

- Nominal features have categories that are unordered, such as Embarked and Sex.
- Ordinal features have categories with an inherent logical ordering, such as Pclass.

Types of categorical features:

- **Nominal:** Unordered categories
 - Embarked
 - Sex
- **Ordinal:** Ordered categories
 - Pclass

So far, here's the advice that I've given you for encoding nominal and ordinal features:

- For a nominal feature, you should use `OneHotEncoder`, and it will output one column for each category.
- For an ordinal feature that is already encoded numerically, you should leave it as-is.
- For an ordinal feature that is encoded as strings, you should use `OrdinalEncoder`, and it will output a single column using the category ordering that you define.

Advice for encoding categorical data:

- **Nominal feature:** Use `OneHotEncoder`
- **Ordinal feature stored as numbers:** Leave as-is
- **Ordinal feature stored as strings:** Use `OrdinalEncoder`

Let's do a quick recap as to why `OneHotEncoder` is the preferred approach for a nominal feature, using Embarked as an example.

Embarked has 3 categories, so `OneHotEncoder` would output 3 features. From each of the 3 features, the model can learn the relationship between the target value and whether or not a given passenger embarked at that port. For example, the model might learn from the first feature that passengers who embarked at C have a higher survival rate than passengers who didn't embark at C.

If you were to instead use `OrdinalEncoder` with `Embarked`, it would output 1 feature. This is problematic because it would imply an ordering of the categories that doesn't inherently exist. For example, if passengers who embarked at C and S had high survival rates, and passengers who embarked at Q had low survival rates, there would be no way for a linear model to learn this relationship.

Why use `OneHotEncoder` for `Embarked`?

- `OneHotEncoder`:
 - Outputs 3 features
 - Model can learn the relationship between each feature and the target value
- `OrdinalEncoder`:
 - Outputs 1 feature
 - Implies an ordering that doesn't inherently exist
 - Linear model can't necessarily learn the relationships in the data

In this chapter, we're going to explore whether this advice still holds when you have high-cardinality categorical features, which are categorical features with lots of unique values.

17.2 Preparing the census dataset

We'll use a new dataset for this chapter, namely a dataset of US census data from 1994.

We'll read the dataset into a new `DataFrame` called `census`.

```
census = pd.read_csv('http://bit.ly/censusdataset')
```

We're only going to use the categorical features, which we can explore by using the `DataFrame` `describe` method.

```
census.describe(include='object')
```

	work...	educ...	marital...	occup...	relation...	race	sex	native...	class
count	48842	48842	48842	48842	48842	48842	48842	48842	48842
unique	9	16	7	15	6	5	2	42	2
top	Private	HS-grad	Marri...	Prof-...	Husband	White	Male	Unite...	<=50K
freq	33906	15784	22379	6172	19716	41762	32650	43832	37155

From the row labeled "unique", you can see that education, occupation, and native-country all have more than 10 unique values. There's no hard-and-fast rule for what counts as a high-cardinality feature, but all 3 of these could be considered high-cardinality since they have a lot of unique values.

You can't tell from this display, but the 8 features are all nominal features, with the exception of education since it does have a logical ordering. However, we're going to be treating education as nominal for this experiment.

Categorical features in census dataset:

- **High-cardinality (3 of 8):** education, occupation, native-country
- **Nominal (7 of 8):** All except education

The column labeled “class” is actually our target. This column indicates whether the person has an income of more or less than \$50,000 a year.

We can view the class proportions by normalizing the output of `value_counts`.

```
census['class'].value_counts(normalize=True)
```

	class
<=50K	0.760718
>50K	0.239282

When defining our X DataFrame, which I’m calling `census_X`, we’re only going to use the 8 categorical columns, which I’ve listed out manually. And we’ll use `class` as our y Series, which I’m calling `census_y`.

```
census_cols = ['workclass', 'education', 'marital-status', 'occupation',  
               'relationship', 'race', 'sex', 'native-country']  
census_X = census[census_cols]  
census_y = census['class']
```

17.3 Setting up the encoders

We’re going to be testing the effectiveness of both `OneHotEncoder` and `OrdinalEncoder` with these 8 features. For this experiment, we would normally just create instances using the default arguments.

```
ohe = OneHotEncoder()  
oe = OrdinalEncoder()
```

Notice that we created an instance of `OrdinalEncoder` without defining the category ordering. This is because we’re treating all of the features as nominal, and nominal features have no logical ordering.

As a result, `OrdinalEncoder` would simply learn the categories for each feature in alphabetical order, which we can confirm by fitting the `OrdinalEncoder` and checking the `categories_` attribute.

```
oe.fit(census_X).categories_
```

```
[array(['?', ' Federal-gov', ' Local-gov', ' Never-worked', ' Private',  
       ' Self-emp-inc', ' Self-emp-not-inc', ' State-gov', ' Without-pay'],  
      dtype=object),  
 array([' 10th', ' 11th', ' 12th', ' 1st-4th', ' 5th-6th', ' 7th-8th',
```

```

    ' 9th', ' Assoc-acdm', ' Assoc-voc', ' Bachelors', ' Doctorate',
    ' HS-grad', ' Masters', ' Preschool', ' Prof-school',
    ' Some-college'], dtype=object),
array([' Divorced', ' Married-AF-spouse', ' Married-civ-spouse',
    ' Married-spouse-absent', ' Never-married', ' Separated',
    ' Widowed'], dtype=object),
array([' ?', ' Adm-clerical', ' Armed-Forces', ' Craft-repair',
    ' Exec-managerial', ' Farming-fishing', ' Handlers-cleaners',
    ' Machine-op-inspct', ' Other-service', ' Priv-house-serv',
    ' Prof-specialty', ' Protective-serv', ' Sales', ' Tech-support',
    ' Transport-moving'], dtype=object),
array([' Husband', ' Not-in-family', ' Other-relative', ' Own-child',
    ' Unmarried', ' Wife'], dtype=object),
array([' Amer-Indian-Eskimo', ' Asian-Pac-Islander', ' Black', ' Other',
    ' White'], dtype=object),
array([' Female', ' Male'], dtype=object),
array([' ?', ' Cambodia', ' Canada', ' China', ' Columbia', ' Cuba',
    ' Dominican-Republic', ' Ecuador', ' El-Salvador', ' England',
    ' France', ' Germany', ' Greece', ' Guatemala', ' Haiti',
    ' Holand-Netherlands', ' Honduras', ' Hong', ' Hungary', ' India',
    ' Iran', ' Ireland', ' Italy', ' Jamaica', ' Japan', ' Laos',
    ' Mexico', ' Nicaragua', ' Outlying-US(Guam-USVI-etc)', ' Peru',
    ' Philippines', ' Poland', ' Portugal', ' Puerto-Rico',
    ' Scotland', ' South', ' Taiwan', ' Thailand', ' Trinidad&Tobago',
    ' United-States', ' Vietnam', ' Yugoslavia'], dtype=object)]

```

That being said, we're actually going to run into a problem with encoding due to our highest cardinality feature, native-country. Let's take a look and see why.

```
census_X['native-country'].value_counts()
```

	native-country
United-States	43832
Mexico	951
?	857
Philippines	295
Germany	206
Puerto-Rico	184
Canada	182
El-Salvador	155
India	151
Cuba	138
England	127
China	122
South	115
Jamaica	106
Italy	105
Dominican-Republic	103
Japan	92
Guatemala	88
Poland	87
Vietnam	86
Columbia	85
Haiti	75
Portugal	67
Taiwan	65
Iran	59
Nicaragua	49
Greece	49
Peru	46
Ecuador	45
France	38
Ireland	37
Thailand	30
Hong	30
Cambodia	28
Trinidad&Tobago	27
Outlying-US(Guam-USVI-etc)	23
Yugoslavia	23
Laos	23
Scotland	21
Honduras	20
Hungary	19
Holland-Netherlands	1

You can see that one of the categories appears only once in the dataset. As we talked about in lesson 15.7, rare category values can cause problems with cross-validation.

In this case, it will definitely create a problem, because that sample is guaranteed to appear in the test fold but not a training fold during one of the runs of cross-validation. That will cause an error for both `OneHotEncoder` and `OrdinalEncoder`.

In the case of `OneHotEncoder`, the solution is simply to set the `handle_unknown` parameter to `'ignore'`.

```
ohe_ignore = OneHotEncoder(handle_unknown='ignore')
```

Starting in scikit-learn version 0.24, OrdinalEncoder has a similar `handle_unknown` parameter that could be used for this situation.

But for now, the best solution is to define the categories in advance for each feature using a list comprehension. The list comprehension iterates through the feature columns, extracts the unique values from each column, and stores the result in a list called `cats`.

```
cats = [census_X[col].unique() for col in census_X[census_cols]]
cats
```

```
[array([' Private', ' Local-gov', ' ?', ' Self-emp-not-inc',
        ' Federal-gov', ' State-gov', ' Self-emp-inc', ' Without-pay',
        ' Never-worked'], dtype=object),
array([' 11th', ' HS-grad', ' Assoc-acdm', ' Some-college', ' 10th',
        ' Prof-school', ' 7th-8th', ' Bachelors', ' Masters', ' Doctorate',
        ' 5th-6th', ' Assoc-voc', ' 9th', ' 12th', ' 1st-4th',
        ' Preschool'], dtype=object),
array([' Never-married', ' Married-civ-spouse', ' Widowed', ' Divorced',
        ' Separated', ' Married-spouse-absent', ' Married-AF-spouse'],
      dtype=object),
array([' Machine-op-inspct', ' Farming-fishing', ' Protective-serv', ' ?',
        ' Other-service', ' Prof-specialty', ' Craft-repair',
        ' Adm-clerical', ' Exec-managerial', ' Tech-support', ' Sales',
        ' Priv-house-serv', ' Transport-moving', ' Handlers-cleaners',
        ' Armed-Forces'], dtype=object),
array([' Own-child', ' Husband', ' Not-in-family', ' Unmarried', ' Wife',
        ' Other-relative'], dtype=object),
array([' Black', ' White', ' Asian-Pac-Islander', ' Other',
        ' Amer-Indian-Eskimo'], dtype=object),
array([' Male', ' Female'], dtype=object),
array([' United-States', ' ?', ' Peru', ' Guatemala', ' Mexico',
        ' Dominican-Republic', ' Ireland', ' Germany', ' Philippines',
        ' Thailand', ' Haiti', ' El-Salvador', ' Puerto-Rico', ' Vietnam',
        ' South', ' Columbia', ' Japan', ' India', ' Cambodia', ' Poland',
        ' Laos', ' England', ' Cuba', ' Taiwan', ' Italy', ' Canada',
        ' Portugal', ' China', ' Nicaragua', ' Honduras', ' Iran',
        ' Scotland', ' Jamaica', ' Ecuador', ' Yugoslavia', ' Hungary',
        ' Hong', ' Greece', ' Trinidad&Tobago',
        ' Outlying-US(Guam-USVI-etc)', ' France', ' Holand-Netherlands'],
      dtype=object)]
```

Then, we can pass the `cats` list to the `categories` parameter of `OrdinalEncoder` when creating an instance. This solves our problem since no unknown categories will ever appear during cross-validation.

```
oe_cats = OrdinalEncoder(categories=cats)
```

Resolving the cross-validation error caused by rare categories:

- **OneHotEncoder:** Set `handle_unknown` to `'ignore'`
- **OrdinalEncoder (before version 0.24):** Define the categories in advance
- **OrdinalEncoder (starting in version 0.24):** Set `handle_unknown` to `'use_encoded_value'`

17.4 Encoding nominal features for a linear model

Now that we've set up our `OneHotEncoder`, called `ohe_ignore`, and our `OrdinalEncoder`, called `oe_cats`, let's see what happens when we pass `census_X` to `fit_transform` and then check the shape.

As expected, the `OneHotEncoder` creates a lot of columns due to the high-cardinality features, whereas the `OrdinalEncoder` creates only one column for each of the eight features.

```
ohe_ignore.fit_transform(census_X).shape
```

```
(48842, 102)
```

```
oe_cats.fit_transform(census_X).shape
```

```
(48842, 8)
```

Now let's actually test the advice that I've given, which is that `OneHotEncoder` should be used for nominal features, to see if this advice still holds for high-cardinality features.

The simplest method for doing this is to create two `Pipelines`. One of them uses `OneHotEncoder` and the other uses `OrdinalEncoder`, and both end in a logistic regression model.

```
ohe_logreg = make_pipeline(ohe_ignore, logreg)
oe_logreg = make_pipeline(oe_cats, logreg)
```

We'll cross-validate each `Pipeline` using all features and then compare the accuracies. We'll also time the operations to see if there are significant differences.

```
%time cross_val_score(ohe_logreg, census_X, census_y, cv=5, \
                      scoring='accuracy').mean()
```

```
CPU times: user 548 ms, sys: 5.73 ms, total: 554 ms
Wall time: 554 ms
```

0.8329920571424309

```
%time cross_val_score(oe_logreg, census_X, census_y, cv=5, \
                        scoring='accuracy').mean()
```

CPU times: user 473 ms, sys: 5.21 ms, total: 478 ms
Wall time: 478 ms

0.7547398152859307

The two Pipelines take around the same amount of time to run, but the accuracy of the OneHotEncoder Pipeline is 0.833, which is significantly better than the 0.755 accuracy of the OrdinalEncoder Pipeline. This would suggest that at least for a linear model like logistic regression, OneHotEncoder should be used for nominal features, even when the features have high cardinality.

Pipeline accuracy when encoding nominal features:

- **Linear model:**
 - OneHotEncoder: 0.833
 - OrdinalEncoder: 0.755

17.5 Encoding nominal features for a non-linear model

Let's now do the same comparison as the previous lesson, except this time we'll use random forests, which is a tree-based non-linear model that we used in chapter 11.

First, we'll create two more Pipelines. One uses OneHotEncoder and the other uses OrdinalEncoder, and both end in a random forest model.

```
ohe_rf = make_pipeline(ohe_ignore, rf)
oe_rf = make_pipeline(oe_cats, rf)
```

Then, we'll cross-validate each Pipeline using all features.

```
%time cross_val_score(ohe_rf, census_X, census_y, cv=5, \
                        scoring='accuracy').mean()
```

CPU times: user 38.3 s, sys: 327 ms, total: 38.6 s
Wall time: 5.73 s

0.8260513856992514

```
%time cross_val_score(oe_rf, census_X, census_y, cv=5, \
                        scoring='accuracy').mean()
```

CPU times: user 6.38 s, sys: 351 ms, total: 6.73 s
Wall time: 1.58 s

0.8245362761024548

We can see that the accuracies are about the same for the OneHotEncoder Pipeline (0.826) and the OrdinalEncoder Pipeline (0.825), even though we were using the OrdinalEncoder on nominal features, which would normally be considered improper.

How can this be? Well, because of how decision trees recursively split features, the random forest model can approximately learn the relationships present in categorical features even when they're encoded as single columns with OrdinalEncoder.

It's also worth noting that the OrdinalEncoder Pipeline is significantly faster than the OneHotEncoder Pipeline due to the much smaller feature set created by the OrdinalEncoder.

Pipeline accuracy when encoding nominal features:

- **Linear model:**
 - OneHotEncoder: 0.833
 - OrdinalEncoder: 0.755
- **Non-linear model:**
 - OneHotEncoder: 0.826
 - OrdinalEncoder: 0.825

17.6 Combining the encodings

One final variation that we can try is to use the OneHotEncoder for all features except for education. And since education is actually an ordinal feature, we can use the OrdinalEncoder with it and define the category ordering.

Here are the education categories.

```
census_X['education'].unique()
```

```
array([' 11th', ' HS-grad', ' Assoc-acdm', ' Some-college', ' 10th',  
      ' Prof-school', ' 7th-8th', ' Bachelors', ' Masters', ' Doctorate',  
      ' 5th-6th', ' Assoc-voc', ' 9th', ' 12th', ' 1st-4th',  
      ' Preschool'], dtype=object)
```


We'll manually define the category ordering, from “Preschool” through “Doctorate”, and then create an instance of `OrdinalEncoder` using that ordering.

```
cats = [[' Preschool', ' 1st-4th', ' 5th-6th', ' 7th-8th', ' 9th',
        ' 10th', ' 11th', ' 12th', ' HS-grad', ' Some-college',
        ' Assoc-voc', ' Assoc-acdm', ' Bachelors', ' Masters',
        ' Prof-school', ' Doctorate']]
oe_cats = OrdinalEncoder(categories=cats)
```

Then we'll create a `ColumnTransformer` that applies the `OrdinalEncoder` to education, and applies the `OneHotEncoder` to all other features.

```
ct = make_column_transformer(
    (oe_cats, ['education']),
    remainder=ohe_ignore)
```

When we pass `census_X` to the `fit_transform`, it creates 87 feature columns, compared to the 102 columns that were created when we only used the `OneHotEncoder`.

```
ct.fit_transform(census_X).shape
```

```
(48842, 87)
```

Finally, we'll create two `Pipelines`. Both of them start with the same `ColumnTransformer`, but one ends with logistic regression while the other ends with random forests.

```
oe_ohe_logreg = make_pipeline(ct, logreg)
oe_ohe_rf = make_pipeline(ct, rf)
```

When we cross-validate the first `Pipeline`, the accuracy is 0.832, which is nearly the same as the 0.833 achieved by the logistic regression `Pipeline` that used `OneHotEncoder` for all features.

```
%time cross_val_score(oe_ohe_logreg, census_X, census_y, cv=5, \
    scoring='accuracy').mean()
```

```
CPU times: user 597 ms, sys: 8.85 ms, total: 606 ms
Wall time: 606 ms
```

```
0.8315588308601922
```

When we cross-validate the second `Pipeline`, the accuracy is 0.825, which is nearly the same as the 0.826 achieved by the random forest `Pipeline` that used `OneHotEncoder` for all features.

```
%time cross_val_score(oe_ohe_rf, census_X, census_y, cv=5, \
                        scoring='accuracy').mean()
```

CPU times: user 38.6 s, sys: 299 ms, total: 38.9 s
Wall time: 5.66 s

0.8251300537921482

In summary, encoding the education feature with `OrdinalEncoder` and the 7 other features with `OneHotEncoder` performed basically the same as encoding all 8 features with `OneHotEncoder`. However, it's certainly possible that the `OrdinalEncoder` could provide a benefit under other circumstances.

Pipeline accuracy when encoding nominal features:

- **Linear model:**
 - `OneHotEncoder`: 0.833
 - `OrdinalEncoder`: 0.755
 - `OneHotEncoder (7 features) + OrdinalEncoder (education)`: 0.832
- **Non-linear model:**
 - `OneHotEncoder`: 0.826
 - `OrdinalEncoder`: 0.825
 - `OneHotEncoder (7 features) + OrdinalEncoder (education)`: 0.825

17.7 Best practices for encoding

Let's summarize what we've learned in this chapter:

- If you have nominal features and are using a linear model, you should definitely use `OneHotEncoder`, regardless of whether the features have high cardinality.
- If you have nominal features and are using a non-linear model, you can try using `OneHotEncoder`, and you can try using `OrdinalEncoder` without defining the category ordering, and then see which option performs better.
- If you have ordinal features and are using either type of model, you can try using `OneHotEncoder`, and you can try using `OrdinalEncoder` while defining the category ordering, and then see which option performs better.

In all cases, keep in mind that if the features have high cardinality, `OrdinalEncoder` is likely to run significantly faster than `OneHotEncoder`, which may or may not matter in your particular case.

Summary of best practices:

- **Nominal features, linear model:**

- OneHotEncoder
- **Nominal features, non-linear model:**
 - OneHotEncoder
 - OrdinalEncoder
 - * Don't define category ordering
 - * Much faster than OneHotEncoder (if features have high cardinality)
- **Ordinal features:**
 - OneHotEncoder
 - OrdinalEncoder
 - * Define category ordering
 - * Much faster than OneHotEncoder (if features have high cardinality)

18 Class imbalance

18.1 Introduction to class imbalance

One of the common issues you might run into when working on a classification problem is known as class imbalance. This refers to a situation in which the classes are not equally represented in the dataset.

Class imbalance is inherent to many domains. For example, if we were classifying fraudulent transactions in a credit card dataset, there would naturally be class imbalance since only a tiny fraction of transactions are likely to be fraudulent.

Class imbalance can occur in both binary classification problems, meaning there are two classes, and multiclass problems, meaning there are more than two classes. In binary classification, the class that has more samples is called the “majority class”, and the class that has fewer samples is called the “minority class”.

So why does class imbalance even matter? In brief, Machine Learning models often have a harder time predicting the minority class because there are fewer examples of this class to learn from. In other words, your model won’t be able to learn as much about the patterns in the minority class, and thus it may have a hard time differentiating between the classes.

Keep in mind that some class imbalance is present in most datasets. In the Titanic dataset, for example, the majority class represents 62% of the samples and the minority class represents 38%. There’s even more imbalance in the census dataset, with a split of 76% to 24%.

```
y.value_counts(normalize=True)
```

Survived	
0	0.616162
1	0.383838

```
census_y.value_counts(normalize=True)
```

class	
<=50K	0.760718
>50K	0.239282

A small amount of class imbalance (like in the Titanic dataset) tends not to matter, and you should just use all of the usual techniques that you’ve learned in this book. But as the amount of class imbalance increases, more specialized techniques need to be applied to the problem, and those techniques are the focus of this chapter.

Overview of class imbalance:

- “Class imbalance” is when classes are not equally represented
- Inherent to many domains
- Can occur in binary and multiclass problems
- Binary problems have a “majority class” and a “minority class”
- Makes it harder for the model to learn patterns in the minority class
- Most datasets have some class imbalance
- Greater class imbalance requires more specialized techniques

18.2 Preparing the mammography dataset

For this chapter, we’re going to use a dataset of mammography scans that were designed to detect the presence of cancer.

We’ll read in the dataset using pandas. You can see that there are 6 features as well as a target column called “class”. Each sample represents an object of interest that was extracted from a scan, and each object was translated into these features using a computer vision algorithm.

The class column has two possible values, -1 and 1. -1 means the object did not indicate the presence of cancer, and 1 means the object did indicate the presence of cancer. These labels were assigned by a human expert and thus represent the “ground truth” labels for the dataset.

```
scan = pd.read_csv('http://bit.ly/scanrecords')
scan.head()
```

	attr1	attr2	attr3	attr4	attr5	attr6	class
0	0.230020	5.072578	-0.276061	0.832444	-0.377866	0.480322	'-1'
1	0.155491	-0.169390	0.670652	-0.859553	-0.377866	-0.945723	'-1'
2	-0.784415	-0.443654	5.674705	-0.859553	-0.377866	-0.945723	'-1'
3	0.546088	0.131415	-0.456387	-0.859553	-0.377866	-0.945723	'-1'
4	-0.102987	-0.394994	-0.140816	0.979703	-0.377866	1.013566	'-1'

As you might expect, -1 (meaning “non-cancerous”) is the majority class, and 1 (meaning “cancerous”) is the minority class. You would call this severe class imbalance because the minority class makes up only 2% of the dataset.

```
scan['class'].value_counts(normalize=True)
```

	class
'-1'	0.97675
'1'	0.02325

Although scikit-learn doesn’t require it, we’re going to map the strings in the class column to the integers 0 and 1, in which 0 means “non-cancerous” and 1 means “cancerous”.

```
scan['class'] = scan['class'].map({'-1':0, '1':1})
scan.head()
```

	attr1	attr2	attr3	attr4	attr5	attr6	class
0	0.230020	5.072578	-0.276061	0.832444	-0.377866	0.480322	0
1	0.155491	-0.169390	0.670652	-0.859553	-0.377866	-0.945723	0
2	-0.784415	-0.443654	5.674705	-0.859553	-0.377866	-0.945723	0
3	0.546088	0.131415	-0.456387	-0.859553	-0.377866	-0.945723	0
4	-0.102987	-0.394994	-0.140816	0.979703	-0.377866	1.013566	0

Now that the dataset is ready, we're going to define two objects, `scan_X` and `scan_y`, to represent the feature matrix and the target.

```
scan_X = scan.drop('class', axis='columns')
scan_y = scan['class']
```

18.3 Evaluating a model with train/test split

Throughout the book, we've been using cross-validation as the model evaluation procedure. But in this chapter, we're going to use train/test split instead, since that will help you to better understand the metrics and techniques I'll be outlining. However, we'll return to cross-validation in the next chapter.

Before I get into the code, here's a quick overview of how we use train/test split for model evaluation:

1. You split the dataset into training and testing sets.
2. You fit the model on the training set.
3. You use the fitted model to make predictions on the testing set and evaluate those predictions.

Steps of train/test split:

1. Split rows into training and testing sets
2. Train model on training set
3. Make predictions on testing set and evaluate predictions

When we use K-fold cross-validation, we're really just running train/test split "K" times in a very systematic way and then averaging the results. Cross-validation is superior because it outputs a lower variance estimate of model performance, but again, we're going to start with train/test split to help you better understand the concepts in this chapter.

Cross-validation vs train/test split:

- Cross-validation is just repeated train/test split
- Cross-validation outputs a lower variance estimate of performance

Let's start by splitting the dataset into training and testing sets. We'll use a split of 75% of the rows for training and 25% of the rows for testing, and we'll set a `random_state` for reproducibility.

```
X_train, X_test, y_train, y_test = train_test_split(scan_X, scan_y,
                                                    test_size=0.25,
                                                    random_state=1,
                                                    stratify=scan_y)
```

We're using stratified sampling so that the class proportions will be approximately equal in the training and testing sets. This is especially important when you have severe class imbalance, otherwise you might have insufficient examples of the minority class in either the training or testing set.

Stratified sampling:

- Ensures that each set is representative of the dataset
- Especially important when there is severe class imbalance

Now we'll fit our logistic regression model on the training set, and use the fitted model to make predictions on the testing set. The `predict` method outputs class predictions of 0 or 1, which we'll store as `y_pred`.

As an aside, we're not using a Pipeline here because the features are entirely numeric and don't need any preprocessing. Though if you were using a Pipeline, you would just use `pipe.fit` and `pipe.predict` instead.

```
logreg.fit(X_train, y_train)
y_pred = logreg.predict(X_test)
y_pred
```

```
array([0, 0, 0, ..., 0, 0, 0])
```

Finally, let's evaluate the model's predictions. We've used accuracy as our evaluation metric throughout the book, so we'll try that here. We have to import the `accuracy_score` function from the `metrics` module, and then we pass it the true values followed by the predicted values.

It outputs an accuracy of 98%, which sounds great. But as you might recall, about 98% of the target values are 0, so an uninformed model could achieve 98% accuracy by always predicting class 0.

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

```
0.9831902718168812
```

Because the accuracy value isn't telling us much about how the model is actually performing, it's not a particularly useful evaluation metric in this case, or in any case in which there's severe class imbalance.

18.4 Exploring the results with a confusion matrix

Before we choose an evaluation metric other than accuracy, let's first use a confusion matrix to get a better understanding of our results.

We'll import it from the `metrics` module, and then pass it the true values followed by the predicted values. (Note that the order is important, because your confusion matrix will be incorrect if you pass the predicted values first instead of second.)

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, y_pred)
```

```
array([[2721,  10],
       [  37,  28]])
```

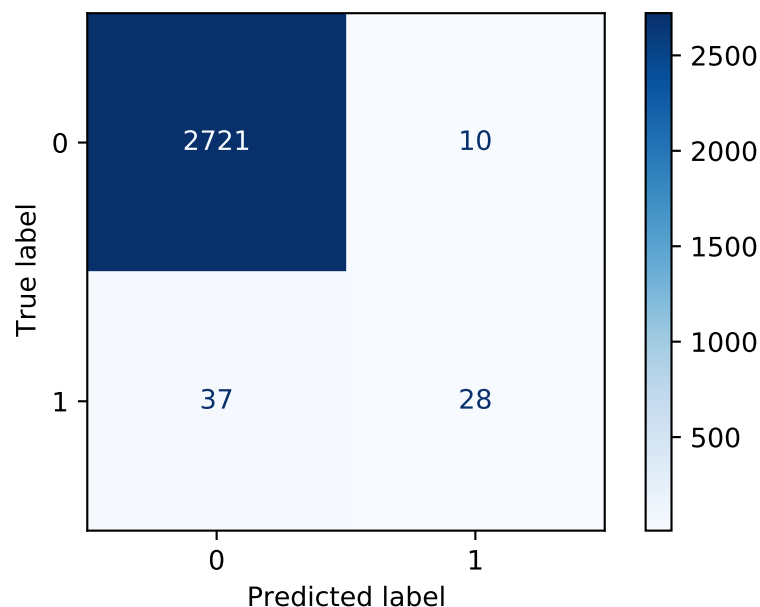
It can be hard to remember what each of the four boxes represents, so a nice alternative is to use the `plot_confusion_matrix` function, which was introduced in scikit-learn version 0.22.

After importing it, we pass it a fitted model, `X_test`, and `y_test`. It makes predictions for `X_test`, and outputs a confusion matrix by comparing the results to `y_test`. Thus it duplicates some of the work we've already done, though it ends up with the same result. (Starting in scikit-learn 1.0, there is a different method for plotting a confusion matrix that allows you to directly pass in `y_pred`.)

The confusion matrix is now labeled, with the true labels as the rows and the predicted labels as the columns. You can see that:

- In 2721 cases, the model predicted 0 and that was correct. These are called True Negatives.
- In 28 cases, it predicted 1 and that was correct. These are True Positives.
- In 37 cases, it predicted 0 and that was incorrect. These are False Negatives.
- In 10 cases, it predicted 1 and that was incorrect. These are False Positives.

```
from sklearn.metrics import plot_confusion_matrix
disp = plot_confusion_matrix(logreg, X_test, y_test, cmap='Blues')
```

	Predicted 0	Predicted 1
True label 0	True Negatives	False Positives
True label 1	False Negatives	True Positives

This should illustrate how the confusion matrix helps us to understand the performance of our classifier much better than the accuracy score.

In particular, it highlights a troubling issue, which we can see by examining the bottom row. There were 65 samples in which cancer was present in the testing set, and it was detected in only 28 of those cases. Given the context, in which the model is attempting to detect the presence of cancer, you could imagine that it would be highly problematic to miss 37 out of 65 cases. This is an issue that we will address by the end of this chapter.

18.5 Calculating rates from a confusion matrix

Before discussing a solution to the problem from the previous lesson, let's first calculate a few rates from the confusion matrix to help us quantify what we want to improve and what tradeoffs we'll be making. Here's another look at the confusion matrix.

```
confusion_matrix(y_test, y_pred)
```

```
array([[2721, 10],
       [ 37, 28]])
```

True Positive Rate answers the question: When cancer is present, how often is that correctly predicted? We divide the True Positives by the entire bottom row, and we get 43.1%. True Positive Rate is also known as recall.

$$28 / (37 + 28)$$

0.4307692307692308

True Negative Rate answers the question: When cancer is not present, how often is that correctly predicted? We divide the True Negatives by the entire top row, and we get 99.6%.

$$2721 / (2721 + 10)$$

0.9963383376052728

False Positive Rate answers the question: When cancer is not present, how often is that incorrectly predicted? We divide the False Positives by the entire top row, and we get 0.4%. As you might have figured out, the False Positive Rate is 1 minus the True Negative Rate.

$$10 / (2721 + 10)$$

0.003661662394727206

Calculated rates:

- **True Positive Rate:** 0.431
 - Recall for class 1
- **True Negative Rate:** 0.996
 - Recall for class 0
- **False Positive Rate:** 0.004
 - 1 minus True Negative Rate

Another way to see these results without doing the calculations yourself is with the classification report. After importing it and passing it the true and predicted values, we can see the following:

- The True Positive Rate is the recall for class 1, which is 43%.
- The True Negative Rate is the recall for class 0, which has been rounded to 100%.
- The False Positive Rate is not directly shown, but it's 1 minus the True Negative Rate, thus it rounds to 0%.
- The overall accuracy is also shown, which is 98%, though that's not our focus here.

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.99	1.00	0.99	2731
1	0.74	0.43	0.54	65
accuracy			0.98	2796
macro avg	0.86	0.71	0.77	2796
weighted avg	0.98	0.98	0.98	2796

Because the True Positive Rate of 43% seems quite problematic, you might think that the solution is just to work on maximizing that metric alone. However, the danger of only focusing on True Positive Rate is that you might end up with a confusion matrix similar to this.

```
confusion_matrix(y_test, np.ones_like(y_pred))
```

```
array([[ 0, 2731],
       [ 0, 65]])
```

The True Positive Rate in this case is 100%, but that's because we're always predicting the positive class. As such, the False Positive Rate would also be 100%. I think you would agree that this is not a useful solution.

18.6 Using AUC as the evaluation metric

Now that I've gone through the nature of the problem, let's start talking about a solution.

The first part of the solution is to choose a different evaluation metric to focus on when tuning our model. We're going to use AUC, which is short for Area Under the ROC Curve. At a high level, AUC is a measure of how well the model separates the classes by predicting higher probabilities for the class 1 samples than for the class 0 samples.

To explain this further, let's have the fitted model output predicted probabilities (rather than class predictions) by using the `predict_proba` method. We'll store these in `y_score` and print them out.

The first number in the array indicates that for the first sample in the test set, the model predicts a 0.18% likelihood that it belongs to the positive class. The second number indicates that for the second sample, the model predicts a 0.26% likelihood that it belongs to the positive class. And so on.

```
y_score = logreg.predict_proba(X_test)[: , 1]
y_score
```

```
array([0.00179856, 0.00255211, 0.00181612, ..., 0.0015053 , 0.02641707,  
       0.00073432])
```

AUC is a measure of how well the model succeeds at assigning higher probabilities to class 1 samples than class 0 samples. In other words, AUC doesn't care about the actual predicted probability values, rather it cares only about the rank ordering of the values. As such, it can be used with any classifier that outputs predicted probabilities, regardless of whether those probabilities are well-calibrated.

Area Under the ROC Curve (AUC):

- Measures how well the model separates the classes
- Rewards the model for assigning higher probabilities to class 1 samples than class 0 samples
- Can be used with any classifier that outputs predicted probabilities

Let's actually calculate the AUC for our model. We'll import the `roc_auc_score` function, and then pass it the true values followed by the predicted probabilities (not the class predictions). It outputs an AUC of 0.93, which is quite good.

```
from sklearn.metrics import roc_auc_score  
roc_auc_score(y_test, y_score)
```

```
0.9339323437455991
```

To put this in context, a perfect model would achieve an AUC of 1.0, whereas a completely uninformed model would achieve an AUC of 0.5.

A more formal way to interpret this value is as follows: If you randomly chose a class 1 sample and then randomly chose a class 0 sample, there's a 93% probability that the model assigned a higher predicted probability to the class 1 sample than the class 0 sample.

AUC scores:

- **Perfect model:** 1.0
- **Uninformed model:** 0.5

A natural follow-up question might be: Why is the AUC so high (0.93), when the True Positive Rate is so low (0.43)?

To answer that question, we have to talk about the decision threshold. The decision threshold is the predicted probability value (between 0 and 1) above which a model will predict the positive class. The threshold is 0.5 by default, which means that if a model predicts a probability of 0.7, it predicts class 1. Whereas if a model predicts a probability of 0.2, it predicts class 0.

Decision threshold:

- Predicted probability value above which a model will predict the positive class
- **Default threshold is 0.5:**

- Probability of 0.7 → predicts class 1
- Probability of 0.2 → predicts class 0

From looking at the right column of the confusion matrix, we can see that there were only 38 samples in the testing set for which the model predicted class 1.

```
confusion_matrix(y_test, y_pred)
```

```
array([[2721,  10],  
       [  37,  28]])
```

Equivalently, there were only 38 samples for which the model predicted a probability greater than 0.5. (This code works by creating a boolean array and then counting the number of True values in it.)

```
sum(y_score > 0.5)
```

38

With all of that in mind, we can infer two things from the fact that the AUC is high but the True Positive Rate is low:

- First, we know from the high AUC that the model is already doing a good job of separating the classes.
- Second, we know from the low True Positive Rate that the default decision threshold is not serving us well.

What can we infer so far?

- **High AUC:** Model is already doing a good job separating the classes
- **Low TPR:** Default decision threshold is not serving us well

To understand this second issue better, we'll plot the ROC curve using the `plot_roc_curve` function, which was introduced in version 0.22. It uses an API similar to `plot_confusion_matrix`, in that we pass it the fitted model, `X_test`, and `y_test`, though the API has also changed starting in scikit-learn 1.0.

So what are we looking at? The ROC curve is a plot of the True Positive Rate (on the y-axis) versus the False Positive Rate (on the x-axis) for all possible decision thresholds.

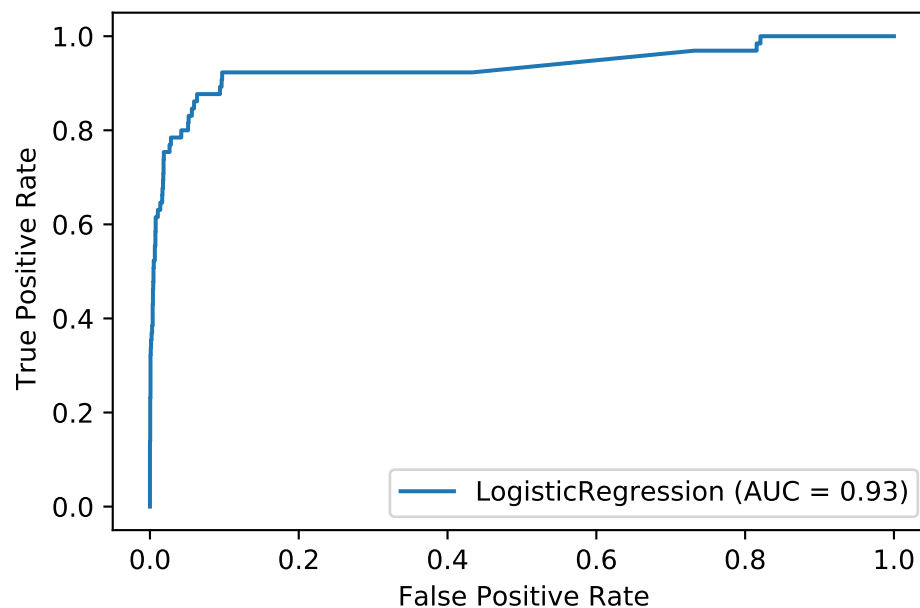
For example, there is a single point on the curve that represents the default decision threshold of 0.5, for which the True Positive Rate is 43% and the False Positive Rate is nearly 0.

Changing the decision threshold simply moves you to another point on the curve. For example, you could move to a point that has a True Positive Rate of about 90% and a False Positive Rate of about 10% just by changing the threshold. To be clear, the threshold itself is not shown on this plot, rather

the plot only shows the True Positive Rate and False Positive Rate pairs that result from all possible thresholds.

One final thing I want to note is that the Area Under the Curve is literally the percentage of the plot that is underneath the curve. So because the AUC is 0.93, we know that 93% of the plot is underneath the curve. If we can increase the AUC, we're moving the curve further into the upper left corner, which means that higher True Positive Rates and lower False Positive Rates will be available to us.

```
from sklearn.metrics import plot_roc_curve
disp = plot_roc_curve(logreg, X_test, y_test)
```



Interpreting the ROC curve:

- Plot of the TPR vs FPR for all possible decision thresholds
- Move to another point on the curve by changing the threshold
 - Threshold is not shown on the plot
- AUC is the percentage of the plot underneath the curve

Given what we've learned so far, I want to chart our path forward. In lesson 18.7, we're going to try to improve the AUC of our model. Then in lesson 18.8, we're going to explore alternative decision thresholds in order to balance the True Positive Rate and the False Positive Rate to our liking.

Next steps:

1. Improve the model's AUC
2. Explore alternative decision thresholds

18.7 Cost-sensitive learning

Now that we know that our next step is to improve our model's AUC, how do we actually do that? The good news is that we can use any technique I've covered in this book, such as hyperparameter tuning, feature selection, trying non-linear models, and so on. All of those techniques have the potential to improve the model's AUC.

In this lesson, I want to focus on one particular technique that we haven't covered in the book that is particularly useful in cases of class imbalance. This technique is called cost-sensitive learning.

The insight behind cost-sensitive learning is that not all prediction errors have the same "cost". This can refer to an actual dollar cost of one type of error versus another, or in our case, the real-world implications of a certain type of error.

When there's severe class imbalance, it's usually the case that False Negatives, in which positive samples are identified as negative, have a higher cost than False Positives, in which negative samples are identified as positive. This makes sense because the positive samples are rare occurrences, and thus we're more interested in locating the positive samples than the negative samples. In simple terms, we would prefer a False Positive to a False Negative.

How to improve the model's AUC?

- **Any technique covered in this book:**
 - Hyperparameter tuning
 - Feature selection
 - Trying non-linear models
 - ...
- **Cost-sensitive learning:**
 - Particularly useful when there's class imbalance
 - **Insight:** Not all prediction errors have the same "cost"
 - **If positive samples are rare:**
 - * False Negatives have a higher cost
 - * False Positives have a lower cost

So how does cost-sensitive learning actually work? In scikit-learn, this is implemented using the `class_weight` parameter of some models, such as logistic regression and random forests.

By setting `class_weight` to 'balanced', scikit-learn will give more "weight" to the samples from the minority class than those from the majority class. More specifically, the model is penalized more for making mistakes on the minority class (meaning False Negatives) than it is for making mistakes on the majority class (meaning False Positives). And because the model's goal is to minimize the total cost, the model may show increased bias toward predicting the minority class.

How does cost-sensitive learning work?

- Gives more "weight" to samples from the minority class
 - Model is penalized more for False Negatives than False Positives

- Model's goal is to minimize total cost
 - Model may be biased toward predicting the minority class

Let's try this out by creating a logistic regression instance that uses `class_weight='balanced'`. This specifies a class weighting that is inversely proportional to the class frequencies in the input data, though you can specify custom weights for each class if you like.

```
logreg_cost = LogisticRegression(solver='liblinear',
                                class_weight='balanced', random_state=1)
```

We'll fit our logistic regression model on the training set, and use the fitted model to make class predictions and predicted probabilities on the testing set. Then we'll calculate the AUC, and it has increased from 0.93 to 0.94 simply by setting class weights.

Keep in mind that class weighting is not guaranteed to improve your AUC, and thus it should be tuned like any other parameter, as we'll see in the next chapter.

```
logreg_cost.fit(X_train, y_train)
y_pred = logreg_cost.predict(X_test)
y_score = logreg_cost.predict_proba(X_test)[:, 1]
roc_auc_score(y_test, y_score)
```

0.9404163028476467

Let's take a look at the classification report to see how our rates have changed:

- The True Positive Rate increased from 43% to 88%.
- The True Negative Rate decreased from about 100% to 89%, which means that the False Positive Rate increased from about 0% to 11%.

I do want to point out that even though this model might match our priorities better, its accuracy has decreased from 98% to 89%. This illustrates how sometimes a useful classifier has a lower accuracy than null accuracy.

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	0.89	0.94	2731
1	0.16	0.88	0.27	65
accuracy			0.89	2796
macro avg	0.58	0.88	0.61	2796
weighted avg	0.98	0.89	0.93	2796

Changes due to cost-sensitive learning:

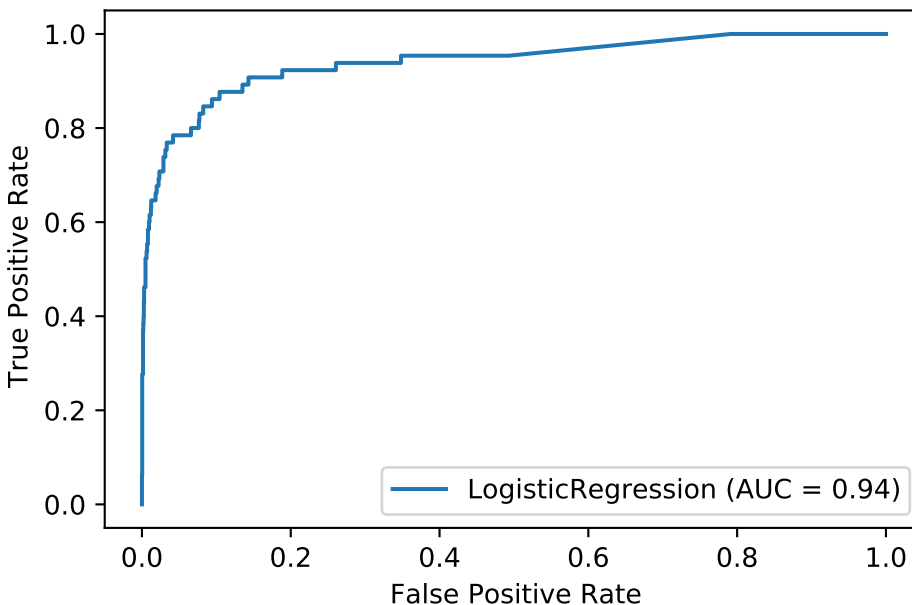
- TPR: 0.43 → 0.88
- FPR: 0.00 → 0.11

18.8 Tuning the decision threshold

At this point, we could continue to tune various aspects of our model to increase the AUC, but instead we're just going to proceed to our final step, which is tuning the decision threshold.

Let's take a look at the ROC curve for our class-weighted logistic regression model. Using the default threshold of 0.5, the model achieved a True Positive Rate of 88% and a False Positive Rate of 11%, which is represented by a single point on the curve. If we wanted to move somewhere else on this curve, in order to better match our priorities, we would simply change the threshold.

```
disp = plot_roc_curve(logreg_cost, X_test, y_test)
```



Before tuning the threshold, it's useful to examine the current confusion matrix, which results from the default threshold of 0.5. You might notice that there are a lot more True Positives and False Positives than before.

```
confusion_matrix(y_test, y_pred)
```

```
array([[2436, 295],  
       [ 8, 57]])
```

More specifically, the model is predicting the positive class 352 times, whereas previously it predicted the positive class only 38 times.

```
sum(y_score > 0.5)
```

352

Let's pretend that we're still uncomfortable with having 8 False Negatives, and thus we want to reduce those even further. By decreasing the threshold to 0.25, we find out that the model will predict the positive class 870 times.

```
sum(y_score > 0.25)
```

870

The boolean array created by this condition can be converted to class predictions simply by multiplying it by 1.

```
(y_score > 0.25) * 1
```

```
array([0, 0, 0, ..., 0, 1, 0])
```

In fact, the boolean array can be passed directly to the confusion matrix function, and it will do the conversion automatically.

Thus we can see that by decreasing the threshold to 0.25, the number of False Negatives has been reduced from 8 to 4, but the number of False Positives has increased from 295 to 809. More generally, decreasing the threshold moves samples from the left column of the confusion matrix to the right column.

```
confusion_matrix(y_test, y_score > 0.25)
```

```
array([[1922, 809],  
       [  4,  61]])
```

Looking at the classification report, the True Positive Rate has increased to 94%, but the False Positive Rate has increased to 30%. That moves us to a new spot on the ROC curve.

```
print(classification_report(y_test, y_score > 0.25))
```

	precision	recall	f1-score	support
0	1.00	0.70	0.83	2731
1	0.07	0.94	0.13	65
accuracy			0.71	2796
macro avg	0.53	0.82	0.48	2796
weighted avg	0.98	0.71	0.81	2796

Changes due to decreasing the threshold:

- **TPR:** 0.88 → 0.94
- **FPR:** 0.11 → 0.30

Now, let's pretend that we think the original threshold resulted in too many False Positives. In that case, we could increase the threshold to 0.75, which moves samples from the right column to the left column.

```
confusion_matrix(y_test, y_score > 0.75)
```

```
array([[2632,  99],
       [ 15,  50]])
```

We can see from the classification report that the True Positive Rate has decreased to 77%, but the False Positive Rate has decreased to 4%. Again, that moves us to a new spot on the ROC curve.

```
print(classification_report(y_test, y_score > 0.75))
```

	precision	recall	f1-score	support
0	0.99	0.96	0.98	2731
1	0.34	0.77	0.47	65
accuracy			0.96	2796
macro avg	0.66	0.87	0.72	2796
weighted avg	0.98	0.96	0.97	2796

Changes due to increasing the threshold:

- **TPR:** 0.88 → 0.77
- **FPR:** 0.11 → 0.04

Keep in mind that as we change the threshold, it's not actually changing the model itself. Instead, changing the threshold is just a way to make tradeoffs between two different types of errors, namely False Positives and False Negatives.

To be clear, there is no correct threshold that we are trying to find. Instead, the threshold you should choose is the one that best matches your priorities. A method does exist for finding the point on the ROC curve that is closest to the upper left corner and calling that the correct threshold, but again, that threshold is only useful if it matches your priorities.

19 Class imbalance walkthrough

19.1 Best practices for class imbalance

We covered a lot of concepts in the previous chapter that may have been new to you: class imbalance, the confusion matrix and all the rates you can calculate from it, the classification report, ROC curves and AUC, the decision threshold, and cost-sensitive learning.

New concepts from chapter 18:

- Class imbalance
- Confusion matrix, TPR, TNR, FPR
- Classification report
- ROC curve and AUC
- Decision threshold
- Cost-sensitive learning

In order to properly demonstrate these concepts, I modified my actual workflow in two important ways that are slightly less than optimal. In this chapter, I'm going to walk through my real workflow for class imbalance problems so that you can see the best practices.

So what are the two modifications that I previously made?

1. I used train/test split for model evaluation instead of cross-validation. Cross-validation is better than train/test split because it's easier to use and it provides more reliable estimates of model performance.
2. I tuned the decision threshold using the same dataset that I used to optimize the model. It's actually better to tune the threshold using different data than you used to optimize the model, because it has been shown experimentally to lead to more reliable estimates of True Positive Rate and False Positive Rate.

Workflow improvements in chapter 19:

1. Use cross-validation instead of train/test split
 - Easier to use
 - More reliable performance estimates
2. Use new data to tune the decision threshold
 - More reliable TPR/FPR estimates

In this chapter, I'm going to walk through my entire workflow, start to finish, while making these two improvements.

19.2 Step 1: Splitting the dataset

Step 1 of my real workflow is to split the dataset into training and testing sets. This might be a confusing way to start, because I just said that we're going to use cross-validation in this chapter rather than train/test split.

That is absolutely true, but we're actually using the `train_test_split` function differently than we did in chapter 18:

- In chapter 18, we used train/test split for model evaluation.
- In this chapter, we're using it to set aside independent data for tuning the decision threshold.

For a longer discussion of this concept, you can review lesson [10.12](#).

Different uses of train/test split:

- **Chapter 18:** Set aside data for model evaluation
- **Chapter 19:** Set aside data for tuning the decision threshold

Anyway, our `train_test_split` code is identical to the code that we used in lesson [18.3](#), including the use of stratified sampling.

```
X_train, X_test, y_train, y_test = train_test_split(scan_X, scan_y,
                                                    test_size=0.25,
                                                    random_state=1,
                                                    stratify=scan_y)
```

19.3 Step 2: Optimizing the model on the training set

Step 2 is to optimize your model (or Pipeline) using the training set only.

As you can see, I'm using cross-validation as the evaluation procedure, and AUC as the evaluation metric to optimize. But notice that I'm passing only the training set to `cross_val_score`. I'm not touching the testing set during this step so that it can function as an independent dataset for the next step.

```
cross_val_score(logreg, X_train, y_train, cv=5, scoring='roc_auc').mean()
```

```
0.9137689754365026
```

To optimize the model, we'll use a grid search. Normally this would be a grid search of all Pipeline steps, but in this case it will be a grid search of just the model.

Since cost-sensitive learning is useful when there's class imbalance, we're going to include the `class_weight` parameter in our search. There are four options we'll try:

- None is the default, and it means don't use cost-sensitive learning.
- 'balanced' is what we used previously, and it specifies class weights that are inversely proportional to the class frequencies in the input data. Since our input data is about 98% class 0 and 2% class 1, it would apply a weight of 2 to class 0 and a weight of 98 to class 1.
- The third option is custom weights, which we designate with a dictionary. Using {0:1, 1:99} specifies a weight of 1 for class 0 and a weight of 99 for class 1. In other words, we're applying an even higher weight to class 1 than the 'balanced' option.
- The fourth option is another set of custom weights, namely {0:3, 1:97}, in which we're applying a slightly lower weight to class 1 than the 'balanced' option. Note that the weights can be any numbers and don't actually have to add up to 100 like I'm doing here.

```
im_params = {}
im_params['penalty'] = ['l1', 'l2']
im_params['C'] = [0.1, 1, 10]
im_params['class_weight'] = [None, 'balanced', {0:1, 1:99}, {0:3, 1:97}]
```

Now that we've set up the parameters to be searched, we can pass them to `GridSearchCV` and use AUC as the metric. When doing the search, again we only pass it the training set.

The search results in an AUC of 0.92, which is higher than the 0.91 from the unoptimized model.

```
training_grid = GridSearchCV(logreg, im_params, cv=5, scoring='roc_auc',
                             n_jobs=-1)
training_grid.fit(X_train, y_train)
training_grid.best_score_
```

0.9227187648199522

Here's the best set of parameters it found, which actually uses one of the custom class weights.

```
training_grid.best_params_
```

```
{'C': 1, 'class_weight': {0: 1, 1: 99}, 'penalty': 'l1'}
```

Now that we've found the best set of parameters, we'll save the model with those parameters as an object called `best_model`.

```
best_model = training_grid.best_estimator_
```

We could try to further optimize the model by creating more features, using feature selection, trying a different type of model, and so on, but instead we're just going to move on to the next step.

19.4 Step 3: Evaluating the model on the testing set

Step 3 is to use our best model to make predictions for the testing set and evaluate those predictions. Again, we did not touch the testing set during step 2, so the model has never seen this data.

We'll use the `predict` method to generate class predictions, and the `predict_proba` method to generate predicted probabilities.

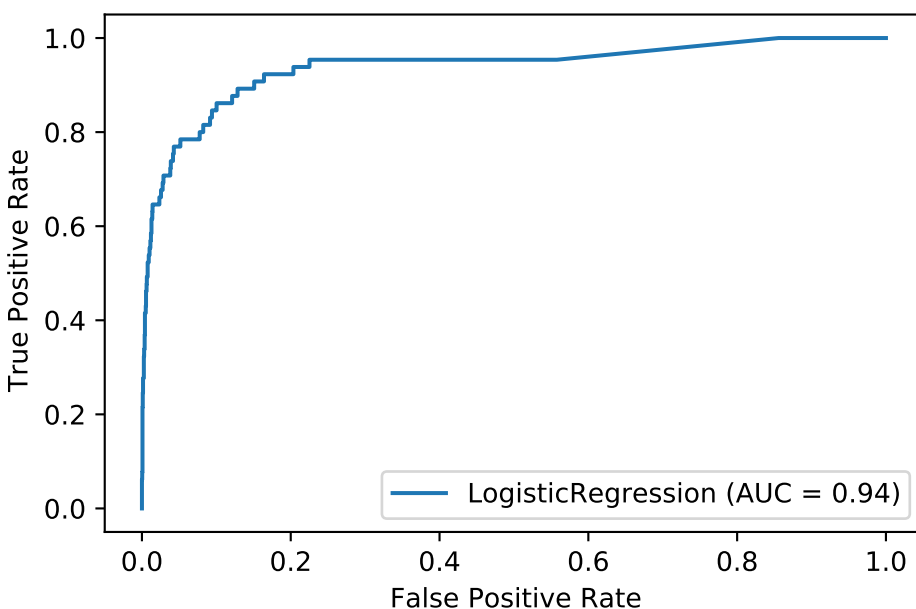
```
y_pred = best_model.predict(X_test)
y_score = best_model.predict_proba(X_test)[:, 1]
```

We'll evaluate the predicted probabilities using AUC and the ROC curve. The AUC is 0.94, which is our best estimate of how well the trained model will perform on truly new data.

```
roc_auc_score(y_test, y_score)
```

```
0.9375151395656705
```

```
disp = plot_roc_curve(best_model, X_test, y_test)
```



We'll evaluate the class predictions using a confusion matrix and the classification report. Notice that the True Positive Rate (class 1 recall) is 95%, and the False Positive Rate (1 minus class 0 recall) is 24%.

```
confusion_matrix(y_test, y_pred)
```

```
array([[2082, 649],
       [  3,  62]])
```



```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	0.76	0.86	2731
1	0.09	0.95	0.16	65
accuracy			0.77	2796
macro avg	0.54	0.86	0.51	2796
weighted avg	0.98	0.77	0.85	2796

19.5 Step 4: Tuning the decision threshold

Step 4 is to tune the decision threshold based on our priorities, meaning our tolerance of False Negatives versus False Positives.

This is the same process you saw in lesson [18.8](#), except this time we're tuning the threshold using data that the model never saw when it was being optimized. Again, this is important because it will lead to a more reliable estimate of the True Positive and False Positive Rates.

Let's pretend that we would prefer a slightly lower False Positive Rate, and are willing to tolerate a lower True Positive Rate in order to achieve it. As such, we'll increase the decision threshold slightly to 0.55.

```
confusion_matrix(y_test, y_score > 0.55)
```

```
array([[2179,  552],
       [   5,   60]])
```

```
print(classification_report(y_test, y_score > 0.55))
```

	precision	recall	f1-score	support
0	1.00	0.80	0.89	2731
1	0.10	0.92	0.18	65
accuracy			0.80	2796
macro avg	0.55	0.86	0.53	2796
weighted avg	0.98	0.80	0.87	2796

The True Positive Rate has decreased from 95% to 92%, and the False Positive Rate has decreased from 24% to 20%. Let's assume we're happy with these numbers, and we'll move on to the final step.

Changes due to increasing the threshold:

- TPR: 0.95 → 0.92
- FPR: 0.24 → 0.20

19.6 Step 5: Retraining the model and making predictions

Step 5 is to use our chosen decision threshold when making predictions on new data.

Before making predictions, it's critical that we train our best model on all of our data, meaning the entirety of `scan_X` and `scan_y`, otherwise we're throwing away valuable data. In other words, we're using the hyperparameters that were chosen from the training data during step 2, and fitting that model with all of the data.

```
best_model.fit(scan_X, scan_y)
```

LogisticRegression

```
LogisticRegression(C=1, class_weight={0: 1, 1: 99}, penalty='l1',  
                    random_state=1, solver='liblinear')
```

We'll use that model to make predictions on new data, meaning data for which you don't know the actual class labels. I don't have access to any new data, so I'm just going to create some random data for demonstration purposes.

I'll set NumPy's random seed for reproducibility, and then use the `randint` function to create a 4 by 6 array with random integers between 0 and 2. In other words, this is a simulation of 4 samples of new data, each of which has 6 features.

```
np.random.seed(1)  
scan_X_new = np.random.randint(0, 3, (4, 6))  
scan_X_new
```

```
array([[1, 0, 0, 1, 1, 0],  
       [0, 1, 0, 1, 0, 2],  
       [1, 2, 0, 2, 1, 2],  
       [0, 0, 2, 0, 1, 2]])
```

To make predictions, we'll pass the new data to the `predict_proba` method and save the output.

```
scan_y_new_score = best_model.predict_proba(scan_X_new)[: , 1]
```

Finally, we'll predict class 1 any time the predicted probability is greater than our decision threshold of 0.55, otherwise we'll predict class 0. These are our class predictions for the 4 new samples.

```
(scan_y_new_score > 0.55) * 1
```

```
array([1, 0, 1, 0])
```

19.7 Q&A: Should I use an ROC curve or a precision-recall curve?

One alternative to ROC curves that you might have heard about is the precision-recall curve. In this lesson, I'll explain the precision-recall curve and then compare it to the ROC curve.

To start, I want to take another look at the confusion matrix for our best model. We'll fit the model on `X_train` and `y_train`, and then generate class predictions and predicted probabilities for `X_test`.

```
best_model.fit(X_train, y_train)
y_pred = best_model.predict(X_test)
y_score = best_model.predict_proba(X_test)[: , 1]
```

Then, we can create the confusion matrix by comparing the true values with the predicted values. There are two rates that we'll calculate from the confusion matrix this time.

```
confusion_matrix(y_test, y_pred)
```

```
array([[2082, 649],
       [ 3, 62]])
```

The first rate is recall, which is just another name for True Positive Rate. It answers the question: When cancer is present, how often is that correctly predicted? We divide the True Positives by the entire bottom row, and we get 95%.

```
62 / (3 + 62)
```

```
0.9538461538461539
```

The second rate is precision. It answers the question: When cancer is predicted, how often is that prediction correct? We divide the True Positives by the entire right column, and we get 9%. Notice that unlike all of our other calculations, the denominator for precision is a column total rather than a row total.

```
62 / (649 + 62)
```

```
0.08720112517580872
```

Calculated rates:

- **Recall:** 0.95
- **Precision:** 0.09

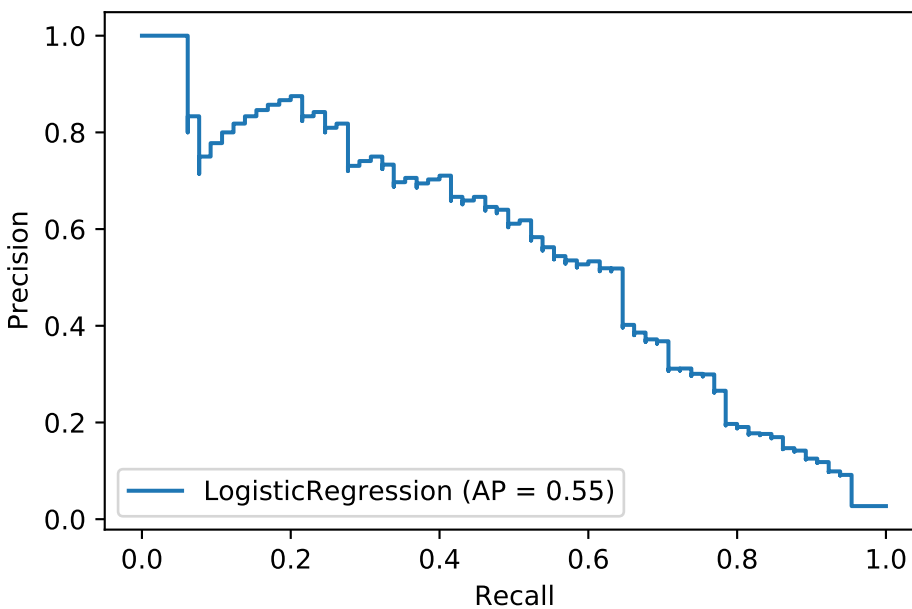
Both precision and recall are listed in the classification report. We just calculated the precision and recall for class 1, as you can see here.

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	0.76	0.86	2731
1	0.09	0.95	0.16	65
accuracy			0.77	2796
macro avg	0.54	0.86	0.51	2796
weighted avg	0.98	0.77	0.85	2796

Now that you understand precision and recall, let's plot the precision-recall curve using the `plot_precision_recall_curve` function, which was introduced in version 0.22. It uses an API similar to `plot_roc_curve` in that we pass it the fitted model, `X_test`, and `y_test`, though the API has changed starting in scikit-learn 1.0.

```
from sklearn.metrics import plot_precision_recall_curve
disp = plot_precision_recall_curve(best_model, X_test, y_test)
```



So what are we looking at? The precision-recall curve is a plot of precision (on the y-axis) versus recall (on the x-axis) for all possible decision thresholds. This is very similar to an ROC curve in that it can help you to tune the decision threshold of your model based on your priorities.

In the same way that you can summarize an ROC curve by calculating the area underneath it, you can summarize a precision-recall curve by calculating the area underneath it. There are multiple ways to do this calculation, but the metric used in scikit-learn is called average precision.

Interpreting the precision-recall curve:

- Plot of precision vs recall for all possible decision thresholds
- Move to another point on the curve by changing the threshold
- Average precision is the percentage of the plot underneath the curve

To calculate average precision, we'll import `average_precision_score` and pass it the true values and the predicted probabilities. It outputs a score of 0.55.

```
from sklearn.metrics import average_precision_score
average_precision_score(y_test, y_score)
```

0.5475069395983533

To put this in context, a perfect model would achieve a score of 1.0, whereas a completely uninformed model would achieve a score equivalent to the fraction of positive samples, which in this case is about 0.02.

As a reminder, the AUC of an uninformed model is 0.5, which means that AUC and average precision often have very different baseline scores for the same problem.

Precision-recall scores:

- **Perfect model:** 1.0
- **Uninformed model:** 0.02 in this case (fraction of positive samples)

In this chapter, I recommended using AUC as your primary evaluation metric in cases of class imbalance. However, there are many people who recommend using average precision instead. Which metric should you use?

First, let me share the most common critique of AUC, and then I'll share my response. Referencing our confusion matrix might be helpful to you during this discussion, so I'll display it here.

```
confusion_matrix(y_test, y_pred)
```

```
array([[2082,  649],
       [   3,   62]])
```

Let's quickly run through the rates again:

- True Positive Rate (or recall) is 62 out of 65, which is 95%.
- False Positive Rate is 649 out of 2731, which is 24%.
- Precision is 62 out of 711, which is 9%.

Calculated rates:

- **TPR / Recall:** 0.95
- **FPR:** 0.24
- **Precision:** 0.09

With that in mind, I'll do my best to summarize the most common critique of AUC in cases of class imbalance, which is as follows:

In cases of severe class imbalance, there will be a very large number of True Negatives. As such, the False Positive Rate will be artificially low, and thus the AUC will be artificially high, and so AUC will no longer provide a realistic estimate of the model's performance.

Critique of AUC in cases of class imbalance:

- **Results of severe class imbalance:**
 - Very large number of True Negatives
 - FPR will be artificially low
 - AUC will be artificially high (and no longer realistic)
- **Example:**
 - Increase True Negatives from 2082 to 200000
 - FPR would drop from 0.24 to 0.003
 - AUC would increase
 - Precision would still be 0.09
- **Proposed solution:**
 - Use precision-recall curve and average precision
 - More realistic because it ignores the number of True Negatives

As an example of this, imagine that we increased the number of True Negatives from 2082 to 200000 but left all the other values the same, as shown below. The False Positive Rate would drop from 24% to 0.3%, and the AUC would increase (though we can't say by how much, since you can't calculate AUC from a confusion matrix). Thus the model would look great based on the AUC, even though the model's precision is still only 9%.

The solution, according to this critique, is to use the precision-recall curve and average precision, since they will provide a more realistic estimate of the model's performance by ignoring the number of True Negatives.

	Predicted 0	Predicted 1
True label 0	200000	649
True label 1	3	62

I have three responses to this critique.

My first response is that neither AUC nor average precision is inherently better. Instead, it depends on what you're trying to measure:

- AUC focuses on both classes. In the context of this problem, AUC measures how skillfully the model finds cancer when it's present, balanced against how skillfully the model doesn't find cancer when it's not present.
- Average precision, on the other hand, focuses only on the positive class. Because both precision and recall ignore the number of True Negatives, the average precision will not change regardless of whether there are two thousand or two million True Negatives.

Ultimately, the choice between AUC and average precision is as follows:

- If you're interested in a model's performance across both classes, then AUC is the better choice.
- If you're only interested in how well the model locates the positive class, then average precision is the better choice.

In our particular case of detecting cancer, my judgment is that the performance on both classes is relevant, and thus AUC is the better choice.

My second response to this critique is that in cases of severe class imbalance, even if you think the False Positive Rate is artificially low (which makes the model look really good), it's just as fair to call the precision artificially low (which makes the model look really bad).

For example, I would judge the model shown in this latest (hypothetical) confusion matrix to be very good:

- If someone doesn't have cancer, they only have a 0.3% chance of being told incorrectly that they do. ($649 / 200649 = 0.003$)
- If someone does have cancer, they only have a 5% chance of being told incorrectly that they don't. ($3 / 65 = 0.05$)

Despite these characteristics, the model still has a precision of just 9%. Even if we moved all 3 False Negatives to the True Positive box, resulting in a True Positive Rate of 100%, the precision would still be 9%, making it sound like a very poor model!

My third response to this critique is that the actual AUC score is irrelevant, and thus it doesn't matter if the AUC is artificially high.

The only reason we're using any evaluation metric during model tuning is that we need some relevant metric to maximize in order to choose between models. AUC fits this purpose well because it measures how skillfully the model is separating the classes. Once you've maximized AUC, then you can change the decision threshold to balance the True Positive Rate and False Positive Rate according to your priorities. But the AUC score itself is never your business objective, so it doesn't matter if the AUC is artificially high.

My responses to this critique:

- **Neither metric is inherently better:**
 - AUC focuses on both classes
 - Average precision focuses only on the positive class
 - Both classes are relevant in this case
- **FPR and precision are both artificially low:**
 - Excellent model can still have a low precision
 - **Example:** Model can have FPR of 0.003 and TPR of 1.0, but precision of 0.09
- **AUC score itself is irrelevant:**
 - Our goal is to choose between models
 - Maximizing AUC helps you choose the most skillful model
 - AUC score itself is never your business objective

The bottom line is as follows:

- Both AUC and average precision are reasonable metrics to maximize even in cases of class imbalance.
- Neither metric is a perfect representation of a model's performance.
- Choose AUC if you're interested in the model's performance across both classes, and choose average precision if you're only interested in how well the model locates the positive class.

Summary of AUC vs average precision:

- Both are reasonable metrics to maximize (even with class imbalance)
- Neither metric perfectly captures model performance
- AUC focuses on both classes, average precision focuses only on the positive class

19.8 Q&A: Can I use a different metric such as F1 score?

There are many other metrics that are popular to use in cases of class imbalance, such as F1 score or F-beta score, balanced accuracy, Cohen's kappa, and Matthews correlation coefficient.

Alternative metrics in cases of class imbalance:

- F1 score or F-beta score
- Balanced accuracy
- Cohen's kappa
- Matthews correlation coefficient

However, all of these metrics require you to choose a decision threshold, whereas AUC and average precision capture the performance of a classifier across all possible thresholds. Thus by using AUC or

average precision as your evaluation metric, you can first maximize the overall performance of your classifier during model tuning, and then alter the decision threshold to match your priorities.

Advantage of AUC and average precision:

- They don't require you to choose a decision threshold
- You can first maximize classifier performance, then alter the decision threshold

If you were instead trying to maximize F1 score (for example) during model tuning, you'll be optimizing the model's hyperparameters based on the default decision threshold of 0.5, but that threshold might be far from optimal for your given problem. In other words, you might miss out on a more optimal model because you were tuning it based on a non-optimal decision threshold.

Disadvantage of other metrics during model tuning:

- You're optimizing hyperparameters based on a decision threshold of 0.5
- An alternative decision threshold might lead to a more optimal model

If you do want to use any of these other metrics, I would recommend using them only to help you choose between different decision thresholds, after your model has already been optimized for either AUC or average precision.

19.9 Q&A: Should I use resampling to fix class imbalance?

In cases of class imbalance, there's a set of techniques collectively known as resampling that is often used. Resampling refers to any technique that transforms the training data in order to achieve more balance between the classes. In other words, resampling attempts to fix the class imbalance at the dataset level rather than working around it, which is what we've done in this chapter.

What is resampling?

- Transforming the training data in order to balance the classes
- Fixes class imbalance at the dataset level

Here are the two most common resampling approaches:

- Undersampling (or downsampling) is the process of deleting samples from the majority class.
- Oversampling (or upsampling) is the process of creating new samples from the minority class, either by duplicating existing samples or by simulating new samples. One popular oversampling method that simulates new samples is SMOTE.

Both of these approaches can be done in either a directed, strategic fashion or in a random fashion. Or they can be done together, in which you both undersample and oversample.

Regardless of the specific approach, keep in mind that the act of resampling risks deleting important samples and/or adding meaningless new samples.

Common resampling approaches:

- **Undersampling (downsampling):**
 - Deleting samples from the majority class
 - Risk of deleting important samples
- **Oversampling (upsampling):**
 - Creating new samples of the minority class
 - Done through duplication or simulation (SMOTE)
 - Risk of adding meaningless new samples

All of that being said, is resampling actually helpful? Experimental results show that resampling methods can be helpful, but are not always helpful. And while there are dozens of different resampling algorithms, no one algorithm works best across all datasets and models, meaning that it's hard to give practical advice for which one to use.

Is resampling helpful?

- Sometimes helpful, sometimes not
- No one algorithm works best across all datasets and models

If you decide to pursue resampling, keep in mind that it's not yet supported by scikit-learn, though it may eventually be available. In the meantime, you can use the imbalanced-learn library, which is supposed to be fully compatible with scikit-learn. Personally, I tend not to use this approach in order to avoid adding additional complexity or project dependencies.

How to implement resampling:

- Not yet supported by scikit-learn
- Use imbalanced-learn library (compatible with scikit-learn)

Here are two guidelines for the proper use of resampling:

- First, you should treat resampling like any other preprocessing technique. Namely, it should be included in a Pipeline in order to avoid data leakage.
- Second, the resampling technique should only ever be applied to training data, and not testing data. The model should always be tested on the natural, imbalanced data so that it can output a realistic estimate of model performance.

Advice for proper resampling:

- Treat like any other preprocessing technique
 - Include in a Pipeline to avoid data leakage
- Only apply to training data
 - Model should be tested on natural, imbalanced data

20 Going further

20.1 Q&A: How do I read the scikit-learn documentation?

In order to become truly proficient with scikit-learn and go beyond what I’ve covered in this book, you need to be able to read the [documentation](#). In this lesson, I’m going to walk through the five main pages and page types that you need to be familiar with.

The first page is the [API Reference](#), which you can get to by clicking on API from the top navigation bar. The API Reference gives you a high-level view of everything available in scikit-learn. Namely, it lists all of the classes and functions, organized by module.

For example, let’s click on the [compose module](#). There’s a brief description of the module, a link to the relevant section of the User Guide, and descriptions of the two classes and two functions contained in the module. Let’s pretend we wanted to learn more about `ColumnTransformer`, so we’ll click on it and be taken to the class documentation.

The [class documentation](#) is the second type of page you need to be familiar with. It gives you a detailed view of a class, in this case the `ColumnTransformer` class. At the top is the class signature, which lists the parameters and their default values. Starting in version 0.23, everything after the `*` needs to be passed as a keyword argument, also known as a named argument, rather than by position. There’s also a link to the source code for the class.

Below that is the class description, a link to the relevant section of the User Guide, and sometimes the version number in which the class was introduced. Next you’ll see the same parameters from the class signature, except here it provides a detailed description of each parameter and the expected data type. Below that are the attributes. As a reminder, attributes end with an underscore if they are learned or estimated from the data during the fit step.

Next are links to any related functions or classes, any important notes, and some simple usage examples. The next big section is a list of the methods. It starts with a description of each method, and below that you can see the parameters and return values for each method. Again, you can click on “source” to view the source code for any method. At the bottom of the page, there are links to examples that use this class.

Let’s scroll back to the top of this page, and then click over to the [User Guide](#) for `ColumnTransformer`. The User Guide is the third type of page you need to be familiar with, and you’ll know you’re in the User Guide when you see the numbered sections. The User Guide is more like a tutorial, because it explains why you might want to use a particular class and advice for how to use that class properly. It often discusses related functions and classes, and includes additional usage examples.

Let’s go back to the class documentation for `ColumnTransformer`, scroll to the bottom, and click on an [example](#). Examples are the fourth type of page you need to be familiar with. Examples vary in structure and length, but their distinguishing feature is that they demonstrate how to solve a

particular problem from start to finish. You'll always see imports at the top, and buttons on the right to run the example online or download the example to your computer.

Let's go back to the class documentation for `ColumnTransformer` one more time, and then scroll up to the top of the parameters section. Sometimes in the documentation, you'll see linked terms that are not classes or functions, such as “fit” and “transform” in this case. If you click on one of those terms, you'll be taken to the [Glossary](#), which is the fifth page you need to be familiar with. The Glossary defines almost all of the important terms used in the scikit-learn documentation. If you just want to browse through it, you can access it through the More menu in the top navigation bar.

I'll now summarize the five pages and page types we walked through, and describe how I use each of them:

- The first page is the [API Reference](#). I go here when I want to see what classes or functions are in a particular module. For example, if I wanted to review all of the classes and functions that are available for preprocessing, the API Reference is the fastest way to do this.
- The second page type is the [class documentation](#). I go here any time I need to understand a particular class in-depth, especially all of its parameters and attributes.
- The third page type is the [User Guide](#). I go here when I need more context about a particular class or function, or advice about how to use it properly.
- The fourth page type is [Examples](#). I go here when I need to see a more complex usage example of a particular class, since the examples in the class documentation and User Guide are purposefully simple.
- The fifth page is the [Glossary](#). I go here when I need to understand a particular term that is not already explained in the User Guide.

Key pages in the documentation:

- **API Reference:** List of classes and functions in each module
- **Class documentation:** Detailed view of a class
- **User Guide:** Advice for proper usage of a class or function
- **Examples:** More complex usage examples
- **Glossary:** Definitions of important terms

If you're ever unsure where to start, you can click on User Guide in the top navigation bar and just start browsing the section that seems most relevant, or you can do a search in the search box.

20.2 Q&A: How do I stay up-to-date with new scikit-learn features?

After every major release of scikit-learn, I recommend reviewing the [Release Highlights](#), which are linked from the top of the home page. This page summarizes a small subset of updates that the scikit-learn developers have judged to be especially important or exciting.

I also recommend that you review the [detailed release notes](#), which are linked from the Release Highlights and are also linked from the More menu under “Release History”. This will give you the most comprehensive look at all of the new features, enhancements, and API changes. Even if you

aren't upgrading right away, it can help you to decide whether to upgrade and warn you about future API changes so that you can start preparing your code now.

Reading this page might seem like an overwhelming task given its length, but my recommendation is to skip past any modules you never import and only read through the modules you actually use.

While reading about a particular change, you can often click through to the class documentation for more details. If you still have questions after reading the class documentation, you can click on the number to the right of each entry to read through the GitHub pull request that introduced this feature or change. Often, the pull request will include a long conversation about the reasons for the change and why it's designed in a certain way. These kinds of details are not always captured in the documentation, and they can help you to build a more in-depth knowledge of scikit-learn.

Pages to review after each release:

- **Release Highlights:** Most important or exciting changes
- **Detailed release notes:** All new features, enhancements, and API changes
 - Only review the modules you use
 - Read the class documentation for more details
 - Read the pull request for further context

20.3 Q&A: How do I improve my Machine Learning skills?

If you want to get better at Machine Learning, my top recommendation is to practice what you've learned in this book as much as possible. I recommend choosing different types of problems and datasets so that you can learn how to adapt these skills to all different types of situations. During that process, you are likely to learn about other topics and scikit-learn modules that we didn't cover in the book, which will expand your Machine Learning and scikit-learn fluency.

Practice what you've learned:

- Choose different types of problems and datasets
- Learn about other topics and modules we didn't cover

One particular area that might benefit you is to study Machine Learning models in more depth. This will help you in a number of ways:

- It will help you to choose which models are worth trying for a given problem.
- It will help you to efficiently and properly tune those models.
- It will help you to better interpret the results of those models.

There are many resources on this topic, but my favorite resource is the book, [An Introduction to Statistical Learning](#). It will help you to gain a practical understanding of many of the most important and widely used Machine Learning models. It's available in hardcover, or you can download it as a free PDF. Although initially the book was written for the R language, there is now a second version that uses Python.

Study Machine Learning models:

- **Benefits:**
 - Learn which models are worth trying
 - Learn how to tune those models
 - Learn how to interpret the results of those models
- **Resource:**
 - *An Introduction to Statistical Learning* (free book)

20.4 Q&A: How do I learn Deep Learning?

As I mentioned at the beginning of this book, there are some specialized problems for which a Deep Learning library will provide results that are superior to what you could achieve with scikit-learn. As such, you may decide that you'd like to spend your time learning how to use Deep Learning.

Deep Learning is a fast-moving field, but one course that has stayed up-to-date and is taught in an approachable way is [Practical Deep Learning for Coders](#). It's available for free and is complemented by a book, a Deep Learning library called fastai, and a huge online community.

Although Deep Learning has a higher learning curve than Machine Learning, it's based on many of the same fundamental principles that you've learned in this book, and so you're well-positioned to begin learning Deep Learning.

Learn Deep Learning:

- **Benefit:**
 - Deep Learning will provide superior results for some specialized problems
- **Resource:**
 - *Practical Deep Learning for Coders* (free course)

Keep learning from Kevin

I hope you enjoyed our journey together! If you'd like to keep learning from me, please [check out my online Data Science courses](#).

I write about Python, Machine Learning, and Artificial Intelligence in my email newsletter. [Join 25,000+ readers](#) and receive practical tips as well as announcements of my upcoming books and courses.

Connect with Kevin

- [YouTube](#) (250,000+)
- [LinkedIn](#) (50,000+)
- [X/Twitter](#) (20,000+)
- [GitHub](#) (8,000+)
- [Facebook](#) (6,000+)
- [Bluesky](#) (900+)